



SAPIENZA
UNIVERSITÀ DI ROMA

Modellazione

MIP

Modelli di programmazione lineare intera
per l'Ingegneria Gestionale

Dispensa didattica di modellazione con variabili intere e binarie: logica e variabili binarie, legami fra variabili, big-M, bound duali del rilassamento LP, euristiche costruttive; tre famiglie di problemi e un capitolo di modelli misti — assegnamento e scheduling, localizzazione e copertura, produzione e lotti, selezione e zaino, partizione e bilanciamento, logistica — con esercizi risolti, domande di modellazione aggiuntive e implementazioni complete in Python/Gurobi.

Fabio Furini

sites.google.com/view/fabiofurini

Dipartimento di Ingegneria informatica, automatica e gestionale — Sapienza Università di Roma

Versione online e materiale scaricabile:
fabiofurini.github.io/modellazione-mip/
github.com/fabiofurini/modellazione-mip

Anno accademico 2026–2027

Indice

I	Modellazione	5
1	Che cos'è un modello MIP	6
1.1	Dati, variabili, obiettivo, vincoli	6
1.2	Perché l'interesse conta	9
1.3	Il rilassamento LP e il sandwich dei bound	11
1.4	Tre pattern che tornano ovunque	13
1.5	Che cosa fa il solver con i due bound	13
1.6	Quello che questo capitolo lascia aperto	14
2	Logica e variabili binarie	15
2.1	Proposizioni e variabili binarie	15
2.2	Letterali, clausole, forma normale congiuntiva	16
2.3	Da CNF a vincoli lineari	17
2.4	Implicazioni logiche	18
2.5	Scindere un'implicazione: quando si può	19
2.6	Contare: al più uno, almeno uno, esattamente uno	19
2.7	Operatori logici e operatori insiemistici	20
2.8	Verificare la traduzione, non fidarsene	20
2.9	Cinque esercizi	21
2.10	I vincoli logici dentro un modello di ottimizzazione	23
2.11	Che cosa resta al capitolo successivo	25
3	Legami fra variabili	26
3.1	Attivazione: forma aggregata e forma disaggregata	27
3.2	Costo fisso, capacità e flusso continuo	28
3.3	Lotto minimo e variabile semicontinua	29
3.4	Conteggi interi, capacità multipla, arrotondamento all'intero superiore	30
3.5	L'ausiliaria di massimo	30
3.6	Min-max, max-min e differenza fra il massimo e il minimo	31
3.7	Il valore assoluto	32
3.8	Big-M: vincoli condizionati e disgiunzioni	33

3.9	Precedenze e sequenziamento	34
3.10	«Se e solo se»	35
3.11	Contare i tipi diversi	36
3.12	Alldiff ed espansione binaria	37
3.13	Vincoli violabili, deviazioni e penalità	38
3.14	Funzioni lineari a tratti	39
3.15	La mappa delle tecniche	40
4	Rilassamenti, dualità e bound	42
4.1	Che cos'è un rilassamento	42
4.2	La tabella di conversione primale/duale	43
4.3	Dualità debole, dualità forte	43
4.4	Tre ricette per costruire a mano una soluzione duale	44
4.5	Un problema di minimo, per esteso	45
4.6	Un problema di massimo: i ruoli si scambiano	47
4.7	Disuguaglianze valide e vincoli che preservano l'ottimalità	49
4.8	Formulazioni più forti: che cosa vuol dire, e come si dimostra	50
4.9	Quello che dice il solver	51
4.10	I duali dell'LP non sono i prezzi marginali del MILP	51
4.11	Il protocollo dei bound del corso	52
5	Euristiche costruttive	53
5.1	Che cos'è un'euristica costruttiva	53
5.2	Le regole di inserimento in contenitori	53
5.3	La regola del meno carico	54
5.4	La regola del rapporto migliore	55
5.5	La regola del completamento più economico	55
5.6	Quando un'euristica costruttiva fallisce	56
5.7	Che cosa dà, e che cosa non dà	56
6	Dal modello a Python/Gurobi	58
6.1	Le quattro classi di variabili	58
6.2	Il modello, una famiglia per blocco	58
6.3	Leggere i risultati	59
6.4	Le tolleranze	60
6.5	Il rilassamento con <code>relax()</code>	61
6.6	Il protocollo del corso, dall'inizio alla fine	61
6.7	Eseguire il codice	62
II	I problemi	64
	Quindici modelli numerici	65

7	Assegnamento e scheduling	105
	La famiglia	105
7.1	Assegnamento a costo minimo con disponibilità	106
7.2	Macchine con costo fisso di utilizzo	111
7.3	Selezione di lavori con ricavo e macchine a costo fisso	118
7.4	Lavori in parallelo: il tempo di lavorazione come massimo	124
7.5	Una macchina, classi di lavori con setup	129
7.6	Classi con premio di completamento e riduzione «se e solo se»	134
7.7	Ritardo totale su una macchina: sequenziamento con big-M	140
8	Localizzazione e copertura	147
	La famiglia	147
8.1	Localizzazione capacitata	148
8.2	p-mediana: al più k sedi	153
8.3	Copertura del segnale con interferenza	158
8.4	Localizzazione di hub con costo massimo	163
9	Pianificazione della produzione	170
	La famiglia	170
9.1	Produzione e lotti con costo fisso di lancio	171
9.2	Produzione e manodopera: due formulazioni equivalenti	179
9.3	Veicoli: lotto minimo e premio per la varietà	186
10	Modelli misti	193
	Perché un capitolo di modelli misti	193
10.1	Premi acquistabili con due modalità	194
10.2	Asta combinatoria	199
10.3	Dieta con conteggio dei cibi e lotto minimo	204
10.4	Alberi di Natale e scatole di luci	209
10.5	Spedizioni in scatole	214
10.6	Campi estivi	219
10.7	Suddivisione antitrust	224
10.8	Distribuzione dei brani sui CD	229
10.9	Libri sugli scaffali	234
10.10	Due problemi da modellare	239
III	Il corso	242
11	Organizzazione del corso	243
11.1	Percorso in tre parti	243
11.2	Il formato di ogni esercizio (e dell'esame)	243
11.3	Criteri di valutazione	244
11.4	Domande tipiche di discussione	244

11.5 Gli errori più comuni	244
11.6 Riproducibilità	244

Parte I

Modellazione



Che cos'è un modello MIP

Script: `python/cap01_modelli.py`

[Pagina online](#)

1.1 Dati, variabili, obiettivo, vincoli

Un modello di *programmazione matematica* traduce una decisione in quattro ingredienti, sempre nello stesso ordine:

1. i **dati**: i numeri che descrivono il problema, noti prima di decidere (costi, capacità, domande, tempi...);
2. le **variabili**: le quantità che il decisore controlla;
3. l'**obiettivo**: una funzione delle variabili da minimizzare o massimizzare;
4. i **vincoli**: le equazioni e disequazioni che le variabili devono soddisfare perché la soluzione sia realizzabile.

Un problema di *ottimizzazione* chiede la soluzione ammissibile (che soddisfa tutti i vincoli) di valore migliore per l'obiettivo.

Le classi di modelli

- **Le sigle**: LP dritto è la *classe* di problemi, *LP* in corsivo è *un* problema di quella classe — quello di cui si scrivono $\tilde{x}(LP)$ e $z(LP)$. Lo stesso per ILP, BIP e MILP.
- **La taglia e i dati**: n variabili, con $j \in \{1, 2, \dots, n\}$, e m vincoli, con $i \in \{1, 2, \dots, m\}$; i costi c_j , i coefficienti a_{ij} e i termini noti b_i .

Che cosa può contenere un modello

- una **funzione obiettivo lineare** nelle variabili, da minimizzare o da massimizzare;
- **vincoli lineari** di tre tipi: di verso $\geq$, per una richiesta da soddisfare; di verso $\leq$, per una capacità da rispettare; di uguaglianza, per un bilancio o un'assegnazione esatta;
- **variabili**, in una o più famiglie, ciascuna con il proprio dominio, dichiarato in fondo al modello.

Nessuna delle tre cose ha una forma obbligata: quanti vincoli, di quale tipo e con quante famiglie di variabili lo decide il problema.

Che cosa distingue le quattro classi: i domini

- **LP**: tutte le variabili sono continue, $x_j \in \mathbb{R}$, di solito non negative. In questo corso un modello di LP non si scrive mai da zero: arriva come *rilassamento* di un modello MIP, quando si lasciano cadere i vincoli di interezza.
- **ILP**: tutte le variabili sono intere, $x_j \in \mathbb{Z}$.
- **BIP**: tutte le variabili sono binarie, $x_j \in \{0, 1\}$.
- **MILP**: alcune variabili sono intere o binarie e altre continue.

È lì che vive la differenza di difficoltà: un LP si risolve in tempo polinomiale, un ILP e un MILP in generale no.

Un modello di ILP

Quante unità comprare da due fornitori, x_1 e x_2 , a lotti interi.

$$\begin{array}{ll} \min & 4x_1 + 7x_2 \\ \text{soggetto a} & 2x_1 + 3x_2 \geq 12 \\ & x_1 + x_2 \leq 5 \\ & x_1, x_2 \in \mathbb{Z}_{\geq 0} \end{array}$$

L'obiettivo minimizza la spesa. Il primo vincolo, di verso $\geq$, impone di coprire la domanda di 12 unità; il secondo, di verso $\leq$, limita a cinque i lotti totali. L'ultima riga è il dominio: tutte le variabili sono intere, e il modello è un ILP.

Un modello di BIP

Quali progetti finanziare: $y_k = 1$ se il progetto k si finanzia, 0 altrimenti.

$$\begin{array}{ll} \max & 5y_1 + 4y_2 + 6y_3 \\ \text{soggetto a} & y_1 + y_2 + y_3 \leq 2 \\ & y_1 + y_3 \geq 1 \\ & y_1, y_2, y_3 \in \{0, 1\} \end{array}$$

L'obiettivo massimizza il valore dei progetti scelti. Il primo vincolo, di verso $\leq$, dice che se ne possono finanziare al più due; il secondo, di verso $\geq$, che almeno uno fra il primo e il terzo va finanziato. L'ultima riga è il dominio: tutte le variabili sono binarie, e il modello è un BIP.

Un modello di MILP

Due prodotti da fabbricare, con x_1 e x_2 le quantità prodotte, e un impianto da attivare o no, con $y = 1$ se lo si attiva e $y = 0$ altrimenti.

$$\begin{array}{rcll}
 \max & 3x_1 & + & 8x_2 & - & 10y \\
 \text{soggetto a} & x_1 & + & x_2 & & = & 6 \\
 & & & x_2 & - & 4y & \leq & 0 \\
 & x_1 & & & & \geq & 1 \\
 & x_1, & x_2 & & & \geq & 0 \\
 & & & & y & \in & \{0, 1\}
 \end{array}$$

L'obiettivo massimizza il ricavo dei due prodotti meno il costo dell'impianto. Il primo vincolo, di uguaglianza, impone di produrre esattamente le sei unità richieste; il secondo, di verso $\leq$, dice che del secondo prodotto se ne possono fare al più quattro unità e soltanto a impianto attivo, perché con $y = 0$ resta $x_2 \leq 0$; il terzo, di verso $\geq$, chiede almeno un'unità del primo prodotto. Le ultime due righe sono i domini: due variabili continue e una binaria, e il modello è un MILP.

Ogni dato, ogni insieme e ogni simbolo è *definito prima di essere usato*. In ogni modello della dispensa:

1. le variabili sono introdotte esplicitamente prima della formulazione;
2. obiettivo e ciascuna famiglia di vincoli hanno un numero d'equazione;
3. i vincoli di dominio, che dichiarano le variabili, chiudono il modello;
4. dopo il modello un elenco spiega obiettivo e vincoli famiglia per famiglia.

Notazione dei valori ottimi e dei bound

- I **valori ottimi** si scrivono indicando sempre di quale modello si parla: $z(MILP)$ è l'ottimo di un modello di MILP, e allo stesso modo si scriveranno quelli dei rilassamenti e dei duali che il capitolo introduce più avanti. Il modello che si sta ottimizzando è sempre esplicito nel simbolo.
- Le soluzioni **ammissibili** portano la barra — comunque siano state costruite — e quelle **ottime** la tilde:

$$(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n) \quad \text{e} \quad (\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n).$$

- I **bound** si chiamano LB e UB :

$$LB \leq z(MILP) \leq UB.$$

Quando serve dire da quale soluzione arriva un bound, lo si scrive fra parentesi: una

soluzione ammissibile $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ di un modello di minimo dà

$$UB(\bar{x}) = \sum_{j=1}^n c_j \bar{x}_j,$$

e in un modello di massimo la stessa somma è $LB(\bar{x})$. Quale dei due venga da una soluzione ammissibile e quale da un rilassamento dipende dunque dal verso dell'obiettivo: in un problema di minimo una soluzione ammissibile dà UB e il rilassamento dà LB , in un problema di massimo è il contrario.

Questo corso lavora quasi solo con MILP: variabili binarie o intere per le decisioni «sì/no» o di conteggio, variabili continue per quantità che si misurano (tempo, denaro, flusso). Il capitolo 3 insegna a collegarle correttamente.

1.2 Perché l'interezza conta

Un ILP o un MILP non si risolve «arrotondando» la soluzione del suo rilassamento continuo. Un controesempio minimo lo mostra.

L'arrotondamento fallisce

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{soggetto a} \quad & 2x_1 + 2x_2 \leq 3 \\ & x_1, x_2 \in \{0, 1\} \end{aligned} \tag{1.1}$$

Il rilassamento LP sostituisce il dominio binario con

$$0 \leq x_1 \leq 1, \quad 0 \leq x_2 \leq 1,$$

e ha valore ottimo

$$z(LP^+) = \frac{3}{2}.$$

Su questo valore vanno notate tre cose.

- È raggiunto da *infinite* soluzioni ottime: tutti i punti del segmento

$$x_1 + x_2 = \frac{3}{2}, \quad 0 \leq x_1 \leq 1, \quad 0 \leq x_2 \leq 1,$$

fra cui $(3/4, 3/4)$, $(1, 1/2)$ e $(1/2, 1)$.

- Quale di queste restituisca il solver dipende dall'algoritmo, non dal modello: nessuna è «la» soluzione del rilassamento. Sulla nostra installazione Gurobi restituisce il vertice $(1/2, 1)$.
- Arrotondando si ottengono esiti diversi a seconda del punto di partenza *e* del verso:
 - da $(3/4, 3/4)$, per eccesso si ottiene $(1, 1)$, che viola il vincolo perché $2 + 2 = 4 > 3$; per difetto si ottiene $(0, 0)$, ammissibile ma di valore 0. Da questo punto *nessuno* dei due versi trova l'ottimo;

- da $(1, 1/2)$, per eccesso si ottiene ancora $(1, 1)$, non ammissibile; per difetto si ottiene $(1, 0)$, ammissibile e di valore 1, che è proprio l'ottimo intero $z(MILP) = 1$.

Non c'è dunque un verso «giusto»: su questa istanza funziona il difetto, e solo a partire da uno dei due punti. Nulla, prima di risolvere, diceva quale dei due il solver avrebbe restituito.

Le lezioni sono due, distinte.

- L'arrotondamento può produrre punti *non ammissibili*, e quando ne produce di ammissibili non c'è alcuna garanzia sul loro valore.

- Il divario

$$\frac{3}{2} - 1 = \frac{1}{2}$$

non è colpa dell'arrotondamento: *nessun* punto intero ammissibile vale più di 1.

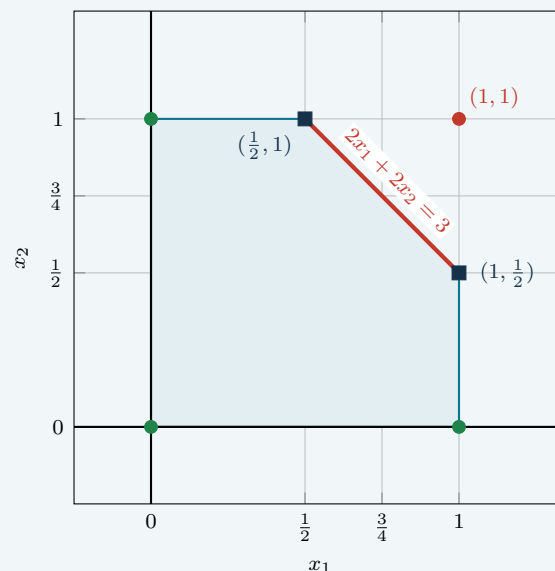


Figura 1.1: L'esempio (1.1). In azzurro la regione ammissibile del rilassamento LP^+ ; in verde i tre punti interi ammissibili, in rosso $(1, 1)$ che viola il vincolo. I due quadrati blu sono due delle infinite soluzioni ottime del rilassamento, tutte di valore $3/2$.

La lezione: il rilassamento continuo dà un'indicazione (un bound, si veda sotto), non una soluzione. I MILP si risolvono con algoritmi dedicati, non con euristiche ingenue sul continuo.

1.3 Il rilassamento LP e il sandwich dei bound

Rilassamento LP: due versioni da non confondere

Il **rilassamento LP** di un MILP si ottiene togliendo i vincoli di interezza e binarietà e tenendo tutto il resto. Ci sono due modi di farlo, e il corso li usa entrambi:

- il **rilassamento con i bound**, $z(LP^+)$: $x \in \{0,1\}$ diventa $0 \leq x \leq 1$, $x \in \mathbb{Z}_{\geq 0}$ diventa $x \geq 0$. È quello che risolve Gurobi con `relax()` ed è il primo rilassamento che il solver risolve;
- il **rilassamento senza i bound**, $z(LP)$: $x \in \{0,1\}$ diventa il solo $x \geq 0$, senza riportare il limite superiore. È più debole (o uguale) del precedente, ma ha un duale più compatto: è di questo che negli esercizi si scrive il duale a mano.

Il rilassamento senza i bound è il più debole dei due, cioè il bound *più lontano* dall'ottimo intero. I due coincidono quando gli altri vincoli del modello implicano già $x \leq 1$: succede, per esempio, quando c'è un vincolo di assegnamento $\sum_m x_{jm} = 1$. Da che parte stiano rispetto a $z(MILP)$ lo dice il riquadro qui sotto.

Il rilassamento è un bound: da che parte, e perché

Il rilassamento *toglie* vincoli, quindi ogni soluzione ammissibile del MILP resta ammissibile per il rilassamento, mentre il viceversa è falso: il rilassamento ottimizza su un insieme di soluzioni più grande. Ottimizzare su un insieme più grande non può dare un valore peggiore, quindi:

$$\text{massimo: } z(LP) \geq z(LP^+) \geq z(MILP);$$

$$\text{minimo: } z(LP) \leq z(LP^+) \leq z(MILP).$$

In un massimo il rilassamento è un *upper bound*, in un minimo un *lower bound*. In entrambi i casi è un bound **ottimistico**: promette almeno quanto l'ottimo vero, e in generale di più. Non è mai una soluzione, e il valore promesso può essere irraggiungibile.

La regola è questa, e vale in tutto il corso.

- I **bound primali** vengono dalle *soluzioni intere ammissibili*: una soluzione che esiste davvero, costruita da un'euristica (capitolo 5) o trovata dal solver. Sono il lato pessimistico.
- I **bound duali** vengono o dalla *soluzione ottima del rilassamento LP*, o da una qualunque *soluzione ammissibile del suo duale*. Sono il lato ottimistico.

Il duale del rilassamento LP, dunque, *non* dà un bound «dall'altro lato»: per dualità debole, in un problema di minimo ogni soluzione duale ammissibile vale al più $z(LP)$, quindi al più $z(MILP)$, e sta dalla *stessa* parte del rilassamento — in generale più debole. Riassumendo: sia $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ una soluzione ammissibile del MILP, di valore $\sum_{j=1}^n c_j \bar{x}_j$, e $(\bar{\pi}_1, \bar{\pi}_2, \dots, \bar{\pi}_m)$ una soluzione ammissibile del duale del rilassamento, di valore $\sum_{i=1}^m b_i \bar{\pi}_i$.

In un problema di **minimo** la prima dà $UB(\bar{x})$ e la seconda $LB(\bar{\pi})$:

$$LB(\bar{\pi}) \leq z(D(LP)) = z(LP) \leq z(LP^+) \leq z(MILP) \leq UB(\bar{x}),$$

in un problema di **massimo** i ruoli si scambiano, la prima dà $LB(\bar{x})$ e la seconda $UB(\bar{\pi})$:

$$LB(\bar{x}) \leq z(MILP) \leq z(LP^+) \leq z(LP) = z(D(LP)) \leq UB(\bar{\pi}).$$

L'uguaglianza in mezzo è la dualità forte: il duale del rilassamento vale quanto il rilassamento, e la disuguaglianza vera è quella fra il *valore di una soluzione duale ammissibile* e quel valore comune.

I due bound si incontrano nel *sandwich* $LB \leq z(MILP) \leq UB$ che percorre tutto il corso: il capitolo 5 costruisce il lato pessimistico, il capitolo 4 quello ottimistico.

Tre «gap» da non confondere

La parola *gap* indica tre quantità diverse. Fissiamo che il denominatore sia sempre $|z(MILP)|$ oppure $|UB|$ come indicato, che il numeratore sia in valore assoluto e che, se il denominatore è nullo, il gap relativo non si scriva (si riporta la differenza assoluta).

1. **Gap dell'euristica.** Se $\bar{x}$ è la soluzione costruita dall'euristica, il suo valore $\sum_{j=1}^n c_j \bar{x}_j$ è il bound $UB(\bar{x})$ in un minimo (e $LB(\bar{x})$ in un massimo), e il gap è

$$\frac{|UB(\bar{x}) - z(MILP)|}{|z(MILP)|}$$

quando l'ottimo è noto. È il gap riportato nelle tabelle degli esercizi, perché le istanze del corso si risolvono all'ottimo. Quando l'ottimo *non* è noto lo si calcola rispetto a un bound duale, che di $z(MILP)$ prende il posto: il numero che se ne ottiene è più grande, ed è l'unico che si possa dichiarare.

2. **Divario certificato** fra due bound noti, senza conoscere $z(MILP)$: $(UB - LB)/|UB|$ per un minimo con $UB > 0$. È quello che si può dichiarare quando l'ottimo non è stato dimostrato: garantisce che $z(MILP)$ stia nell'intervallo $[LB, UB]$, non che sia vicino a uno dei due estremi.
3. **MIPGap del solver:** la stessa idea, calcolata da Gurobi con i suoi UB (`ObjVal` dell'incumbent) e LB (`ObjBound`), con la sua convenzione $|UB - LB|/|UB|$ e le sue tolleranze. Vale 0 quando l'ottimalità è dimostrata.

Le formule dei punti 2 e 3 richiedono che l'incumbent esista e che il denominatore non sia nullo: sono convenzioni, non identità matematiche, e vanno dichiarate ogni volta che si riporta un numero.

A che cosa servono davvero i due bound

Negli esercizi di questo corso le istanze sono piccole apposta: Gurobi trova $z(MILP)$ in una frazione di secondo, e LB e UB servono a *capire* da dove viene un bound e a verificare il ragionamento sull'euristica e sul duale. Nella pratica, però, un MILP reale può avere

migliaia di variabili e il solver può non terminare in tempo utile: lo si interrompe, e resta la migliore soluzione trovata finora e con il miglior bound dimostrato al momento dello stop. Il divario certificato fra i due è tutto quello che si può dire con sicurezza sulla distanza dall'ottimo, *senza* conoscere $z(MILP)$. È per questo che il capitolo 5 insegna a costruire in fretta una soluzione ammissibile e il capitolo 4 a costruire a mano un bound ottimistico: sono gli strumenti che restano quando l'ottimo esatto non è raggiungibile, e sono anche quelli che permettono di giudicare i numeri che il solver riporta.

1.4 Tre pattern che tornano ovunque

Molti vincoli MILP con variabili binarie di selezione (indicizzate da un insieme I) appartengono a una di tre famiglie, a seconda del verso:

Nome	Vincolo	Significato
set partitioning	$\sum_{i \in I} x_i = 1$	esattamente un elemento di I è scelto
set packing	$\sum_{i \in I} x_i \leq 1$	al più un elemento di I è scelto
set covering	$\sum_{i \in I} x_i \geq 1$	almeno un elemento di I è scelto

Il problema 7.1 (assegnamento) usa un *set partitioning* per ogni lavoro; il problema 7.3 (selezione) un *set packing*; il capitolo 2 mostra il *set covering* come traduzione diretta di una clausola OR. Riconoscere il pattern dice subito come si comporta il vincolo: un *covering* ammette che più elementi siano scelti insieme, un *packing* no, un *partitioning* impone l'uno e l'altro insieme.

1.5 Che cosa fa il solver con i due bound

Un MILP con insieme ammissibile limitato si risolve con algoritmi dedicati. Come funzionano — come si esplora lo spazio delle soluzioni, come si separano i tagli, come si sceglie su che cosa ramificare — non è argomento di questo corso: le tecniche risolutive sono materia di un corso a parte. Qui interessa una cosa sola, ed è bene dirla subito: i due bound che questo corso insegna a costruire a mano sono esattamente quelli su cui il solver lavora.

- Il **bound primale** è una soluzione ammissibile, e dà un valore da battere: ogni parte dello spazio che non può fare meglio viene scartata senza esplorarla.
- Il **bound duale** viene dal rilassamento, e dice quanto si può sperare al massimo: più è vicino all'ottimo intero — cioè più la formulazione è stretta, si veda il capitolo 3 — meno lavoro resta da fare.

Il divario fra i due è quello che il solver riporta come **MIPGap**, ed è anche l'unica cosa che si può affermare con sicurezza quando l'ottimo non si raggiunge. È per questo che il capitolo 5 insegna a costruire in fretta una soluzione ammissibile e il capitolo 4 a costruire a mano un bound ottimistico: scrivere il modello con una formulazione stretta e saper produrre i due bound è ciò che, di questo corso, serve al momento di risolvere.

1.6 Quello che questo capitolo lascia aperto

Domanda	Dove si risponde
Come si traducono le condizioni logiche in vincoli lineari?	Capitolo 2
Come si legano fra loro famiglie di variabili diverse?	Capitolo 3
Come si costruisce a mano un bound ottimistico, e come si dimostra che una formulazione è più forte di un'altra?	Capitolo 4
Come si costruisce in fretta una soluzione ammissibile?	Capitolo 5
Come si scrive tutto questo in Python/Gurobi e come si leggono i risultati?	Capitolo 6

Questa tabella è una mappa di orientamento, non una dipendenza: i capitoli si leggono nell'ordine in cui sono scritti.

Logica e variabili binarie

Script: `python/cap02_logica.py`, `python/booleane.py`

[Pagina online](#)

Una variabile binaria è una risposta «sì/no». Questo capitolo mostra come tradurre in *vincoli lineari* le condizioni logiche fra quelle risposte, e — soprattutto — come *dimostrare* che la traduzione è esatta.

2.1 Proposizioni e variabili binarie

Una proposizione, una variabile

Una **funzione booleana** restituisce uno dei due valori `TRUE` e `FALSE`; la si rappresenta con una variabile binaria

$$x \in \{0, 1\}, \quad x = 1 \iff \text{la proposizione è TRUE}, \quad x = 0 \iff \text{è FALSE}.$$

Un'**espressione booleana** si costruisce con variabili binarie, i tre operatori `AND` (congiunzione, $\wedge$), `OR` (disgiunzione, $\vee$) e `NOT` (negazione, $\neg$), e le parentesi.

Due espressioni booleane

$$\begin{aligned} &\text{NOT } x_a \text{ OR } ((x_b \text{ OR } x_c) \text{ AND } (x_d \text{ OR } x_e)), \\ &(x_a \text{ OR } x_b) \text{ AND } (x_c \text{ OR } x_d) \text{ AND NOT } x_e. \end{aligned}$$

Soddisfacibilità

Il **problema di soddisfacibilità booleana** chiede se esiste un'assegnazione di valori logici alle variabili che renda vera l'espressione. Se esiste, l'espressione è **soddisfacibile**; altrimenti è **insoddisfacibile**.

Soddisfacibili e no

- `NOT  $x_a$  OR  $((x_b \text{ OR } x_c) \text{ AND } (x_d \text{ OR } x_e))$` è soddisfatta, per esempio, da $x_a = 0$ (le altre qualsiasi).
- `$(x_a \text{ OR } x_b) \text{ AND } (x_c \text{ OR } x_d) \text{ AND NOT } x_e$` è soddisfatta da $x_a = 1, x_c = 1$,

$$x_b = x_d = x_e = 0.$$

- $(x_a \text{ OR } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR } \text{NOT } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } \text{NOT } x_b)$ è insoddisfacibile: le quattro assegnazioni possibili di (x_a, x_b) falsificano, ciascuna, una delle quattro clausole.

2.2 Letterali, clausole, forma normale congiuntiva

Le tre parole

Un **letterale** è una variabile (x_a , letterale positivo) o la sua negazione ($\text{NOT } x_a$, letterale negativo). Una **clausola** è una disgiunzione (**OR**) di letterali, o un singolo letterale. Un'espressione è in **forma normale congiuntiva** (CNF) se è una congiunzione (**AND**) di clausole, o una singola clausola.

Delle due espressioni dell'esempio precedente, la seconda è in CNF e la prima no. Ogni espressione booleana *può* essere riscritta in CNF, usando le equivalenze che seguono.

Le equivalenze che servono

Per ogni terna di variabili binarie $x_a, x_b, x_c \in \{0, 1\}$:

$$x_a \text{ AND } (x_b \text{ OR } x_c) \iff (x_a \text{ AND } x_b) \text{ OR } (x_a \text{ AND } x_c) \quad (2.1)$$

$$x_a \text{ OR } (x_b \text{ AND } x_c) \iff (x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR } x_c) \quad (2.2)$$

$$\text{NOT } (x_a \text{ OR } x_b) \iff \text{NOT } x_a \text{ AND } \text{NOT } x_b \quad (2.3)$$

$$\text{NOT } (x_a \text{ AND } x_b) \iff \text{NOT } x_a \text{ OR } \text{NOT } x_b \quad (2.4)$$

$$x_a \text{ OR } (x_a \text{ AND } x_b) \iff x_a \quad (2.5)$$

$$x_a \text{ AND } (x_a \text{ OR } x_b) \iff x_a \quad (2.6)$$

$$\text{NOT } (\text{NOT } x_a) \iff x_a \quad (2.7)$$

Le (2.1)–(2.2) sono le **distributive**, le (2.3)–(2.4) sono le leggi di **De Morgan**, le (2.5)–(2.6) sono l'**assorbimento**. Valgono anche con più di due termini fra parentesi e con espressioni al posto delle variabili.

Ogni equivalenza si dimostra per casi: con due variabili ci sono 4 assegnazioni, con tre ce ne sono 8, e si verifica che i due membri hanno lo stesso valore su ciascuna. Lo script `cap02_logica.py` esegue questo controllo su tutte e sette le equivalenze.

Raccogliere: la (2.1) letta al contrario

Letta da destra a sinistra, la distributiva (2.1) permette di *raccogliere* due congiunzioni che condividono un letterale:

$$(x_a \text{ AND } x_b) \text{ OR } (x_a \text{ AND } x_c) \iff x_a \text{ AND } (x_b \text{ OR } x_c).$$

Serve quando si porta in CNF una disgiunzione di più congiunzioni con letterali in comune: riduce, passo dopo passo, il numero di termini da distribuire.

Due trasformazioni in CNF

- $\text{NOT } x_a \text{ OR } ((x_b \text{ OR } x_c) \text{ AND } (x_d \text{ OR } x_e))$: basta la (2.2) e diventa

$$(\text{NOT } x_a \text{ OR } x_b \text{ OR } x_c) \text{ AND } (\text{NOT } x_a \text{ OR } x_d \text{ OR } x_e).$$

- $(x_a \text{ AND } x_b) \text{ OR } (x_a \text{ AND } x_c) \text{ OR } (x_b \text{ AND } x_c)$ — «almeno due fra a, b, c ». Si raccoglie x_a nelle prime due, $[x_a \text{ AND } (x_b \text{ OR } x_c)] \text{ OR } (x_b \text{ AND } x_c)$; si applica la (2.2), ottenendo i due fattori $x_a \text{ OR } (x_b \text{ AND } x_c)$ e $(x_b \text{ OR } x_c) \text{ OR } (x_b \text{ AND } x_c)$; il secondo si riduce a $(x_b \text{ OR } x_c)$ per assorbimento (2.5), e il primo si distribuisce di nuovo. In tutto:

$$(x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR } x_c) \text{ AND } (x_b \text{ OR } x_c).$$

Sarà la chiave dell'esercizio 2.4.

2.3 Da CNF a vincoli lineari

La traduzione, in tre regole

Un'espressione booleana in CNF è equivalente a un sistema di vincoli lineari sulle stesse variabili binarie:

1. ogni **clausola** diventa un vincolo di disuguaglianza ≥ 1 ;
2. ogni **OR** dentro la clausola diventa un $+$;
3. ogni letterale negativo $\text{NOT } x$ diventa $1 - x$.

Perché è esatta

Una clausola con letterali positivi P e negativi N diventa

$$\sum_{i \in P} x_i + \sum_{i \in N} (1 - x_i) \geq 1 \quad \Longleftrightarrow \quad \sum_{i \in P} x_i - \sum_{i \in N} x_i \geq 1 - |N|.$$

Su una variabile binaria, $x_i = 1$ contribuisce 1 al membro sinistro se il letterale è positivo e soddisfatto, e $1 - x_i = 1$ contribuisce 1 se il letterale è negativo e soddisfatto. La somma conta *quanti letterali della clausola sono veri*: chiedere che sia ≥ 1 è esattamente chiedere che la clausola sia vera. La congiunzione di clausole diventa il sistema di tutti i vincoli, perché tutte devono valere insieme.

Con due o più letterali negativi si preferisce, per leggibilità, la forma equivalente ottenuta moltiplicando per -1 : per esempio $1 - x_1 + 1 - x_6 + x_7 \geq 1$ si scrive $x_1 + x_6 - x_7 \leq 1$. È lo stesso vincolo. Quello che non cambia mai è il numero di vincoli: *uno per clausola*, e le clausole si contano dopo la semplificazione (via le tautologie $x \text{ OR } \text{NOT } x$ e le clausole assorbite da una loro sottoclausola).

Tre traduzioni

$$(\text{NOT } x_a \text{ OR } x_b \text{ OR } x_c) \text{ AND } (\text{NOT } x_a \text{ OR } x_d \text{ OR } x_e) \iff \begin{cases} -x_a + x_b + x_c \geq 0, \\ -x_a + x_d + x_e \geq 0; \end{cases}$$

$$(x_a \text{ OR } x_b) \text{ AND } (x_c \text{ OR } x_d) \text{ AND NOT } x_e \iff \begin{cases} x_a + x_b \geq 1, \\ x_c + x_d \geq 1, \\ x_e \leq 0; \end{cases}$$

$$(x_a \text{ OR } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR NOT } x_b) \text{ AND } (\text{NOT } x_a \text{ OR NOT } x_b) \\ \iff \begin{cases} x_a + x_b \geq 1, \\ -x_a + x_b \geq 0, \\ x_a - x_b \geq 0, \\ x_a + x_b \leq 1. \end{cases}$$

L'ultimo sistema è privo di soluzioni binarie, come l'espressione era insoddisfacibile: i primi tre vincoli danno $x_a = x_b = 1$, che viola il quarto.

2.4 Implicazioni logiche

L'implicazione fra due variabili

L'implicazione logica $x_a \Rightarrow x_b$ («se a allora b ») equivale all'espressione booleana

$$\text{NOT } x_a \text{ OR } x_b,$$

già in CNF, e quindi al vincolo lineare

$$1 - x_a + x_b \geq 1 \iff x_b - x_a \geq 0.$$

La contronominale è la stessa cosa

Per contrapposizione, $x_a \Rightarrow x_b$ implica anche $\text{NOT } x_b \Rightarrow \text{NOT } x_a$. Non è un secondo vincolo: la sua espressione booleana è $\text{NOT}(\text{NOT } x_b) \text{ OR } \text{NOT } x_a$, che per doppia negazione (2.7) è la stessa $\text{NOT } x_a \text{ OR } x_b$. Un solo vincolo impone entrambe le letture — e nella descrizione di un modello conviene scrivere quella che si legge meglio nel contesto applicativo.

$x_b - x_a \geq 0$ dice «se a allora b », non « a se e solo se b »: ammette $x_a = 0, x_b = 1$. Per il «se e solo se» servono *due* implicazioni, cioè $x_a = x_b$, oppure una implicazione più un argomento di ottimalità: è il tema della sezione 3.10.

La stessa ricetta vale quando antecedente e conseguente sono espressioni booleane qualsiasi: si scrive $\text{NOT}(\text{antecedente}) \text{ OR } (\text{conseguente})$, si porta in CNF e si traduce.

Le quattro combinazioni di base

Implicazione	Espressione in CNF	Vincoli	#
$x_a \text{ AND } x_b \Rightarrow x_c$	$\text{NOT } x_a \text{ OR } \text{NOT } x_b \text{ OR } x_c$	$x_a + x_b - x_c \leq 1$	1
$x_a \text{ OR } x_b \Rightarrow x_c$	$(\text{NOT } x_a \text{ OR } x_c) \text{ AND } (\text{NOT } x_b \text{ OR } x_c)$	$x_c - x_a \geq 0, x_c - x_b \geq 0$	2
$x_a \Rightarrow x_b \text{ AND } x_c$	$(\text{NOT } x_a \text{ OR } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } x_c)$	$x_b - x_a \geq 0, x_c - x_a \geq 0$	2
$x_a \Rightarrow x_b \text{ OR } x_c$	$\text{NOT } x_a \text{ OR } x_b \text{ OR } x_c$	$x_b + x_c - x_a \geq 0$	1

Si legga la tabella così: una *disgiunzione nell'antecedente* e una *congiunzione nel conseguente* costano due vincoli; una *congiunzione nell'antecedente* e una *disgiunzione nel conseguente* ne costano uno. È il motivo per cui il vincolo aggregato $\sum_j x_{jm} \leq k y_m$ e quello disaggregato $x_{jm} \leq y_m$ hanno costi diversi in numero di righe — e forze diverse nel rilassamento (sezione 3.1).

Un antecedente con tre letterali

$$x_a \text{ AND } \text{NOT } x_b \text{ AND } x_c \Rightarrow \text{NOT } x_d \iff \text{NOT } x_a \text{ OR } x_b \text{ OR } \text{NOT } x_c \text{ OR } \text{NOT } x_d$$

$$\iff x_a - x_b + x_c + x_d \leq 2.$$

2.5 Scindere un'implicazione: quando si può

Due scissioni valide e una no

$$(x_a \text{ OR } x_b) \Rightarrow x_c \iff (x_a \Rightarrow x_c) \text{ AND } (x_b \Rightarrow x_c) \quad (2.8)$$

$$x_a \Rightarrow (x_b \text{ AND } x_c) \iff (x_a \Rightarrow x_b) \text{ AND } (x_a \Rightarrow x_c) \quad (2.9)$$

La scissione analoga con l'antecedente **congiunzione** *non* è valida:

$$(x_a \text{ AND } x_b) \Rightarrow x_c \text{ non } \iff (x_a \Rightarrow x_c) \text{ AND } (x_b \Rightarrow x_c).$$

Con $x_a = 1, x_b = 0, x_c = 0$ l'implicazione $(x_a \text{ AND } x_b) \Rightarrow x_c$ è *vera* — il suo antecedente è **FALSE** — ma $x_a \Rightarrow x_c$ è *falsa*. La congiunzione delle due implicazioni scisse è quindi **strettamente più forte** dell'originale: taglia soluzioni che il problema ammette. Sbagliare qui non produce un modello «un po' diverso», produce un modello che può escludere l'ottimo.

Le due scissioni valide sono spesso la strada più corta: nella tabella precedente, le righe con due vincoli si ottengono direttamente da (2.8) o (2.9), senza passare da De Morgan e dalla distributiva.

2.6 Contare: al più uno, almeno uno, esattamente uno

Le condizioni di cardinalità su un insieme I di variabili binarie sono già lineari e non richiedono la CNF.

Condizione	Vincolo	Nota
almeno uno	$\sum_{i \in I} x_i \geq 1$	è la clausola $\bigvee_{i \in I} x_i$: <i>set covering</i>
al più uno	$\sum_{i \in I} x_i \leq 1$	<i>set packing</i> ; equivale alle $\binom{ I }{2}$ clausole $\text{NOT } x_i \text{ OR } \text{NOT } x_j$, ma in un solo vincolo e con un rilassamento più stretto
esattamente uno	$\sum_{i \in I} x_i = 1$	<i>set partitioning</i> : le due precedenti insieme
almeno p	$\sum_{i \in I} x_i \geq p$	in CNF servirebbero $\binom{ I }{p+1}$ clausole
al più p	$\sum_{i \in I} x_i \leq p$	in CNF servirebbero $\binom{ I }{p+1}$ clausole
nessuno	$\sum_{i \in I} x_i \leq 0$	cioè $x_i = 0$ per ogni i

Un vincolo di cardinalità val più di molte clausole

«Al più uno fra tre» si può scrivere come tre clausole, $\text{NOT } x_1 \text{ OR } \text{NOT } x_2, \text{NOT } x_1 \text{ OR } \text{NOT } x_3, \text{NOT } x_2 \text{ OR } \text{NOT } x_3$, cioè $x_1 + x_2 \leq 1, x_1 + x_3 \leq 1, x_2 + x_3 \leq 1$; oppure come il solo $x_1 + x_2 + x_3 \leq 1$. I due sistemi hanno le *stesse* soluzioni binarie, ma non lo stesso rilassamento: $x_1 = x_2 = x_3 = 1/2$ soddisfa le tre clausole e viola il vincolo aggregato. La versione compatta è più corta e più forte: un caso in cui non c'è niente da scambiare. Quando si può contare, si conta; la CNF serve quando la condizione non è una cardinalità.

2.7 Operatori logici e operatori insiemistici

Associando a ogni variabile binaria un sottoinsieme di un universo U — $x_a = 1$ significa «l'elemento scelto sta in A » — gli operatori logici diventano operatori insiemistici:

Logica	Insiami
$x_a \text{ AND } x_b$	$A \cap B$ intersezione
$x_a \text{ OR } x_b$	$A \cup B$ unione
$\text{NOT } x_a$	$\overline{A}$ complemento

Le equivalenze della sezione precedente diventano le identità insiemistiche $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$, $\overline{A \cap B} = \overline{A} \cup \overline{B}$, $\overline{A \cup B} = \overline{A} \cap \overline{B}$, $A \cup (A \cap B) = A$ e $A \cap (A \cup B) = A$. Chi preferisce i diagrammi di Venn ha così una seconda dimostrazione delle stesse regole; chi preferisce l'enumerazione ha la prima. Le due si controllano a vicenda.

2.8 Verificare la traduzione, non fidarsene

Il criterio operativo

Una traduzione in vincoli lineari è corretta quando, per *ogni* assegnazione binaria delle variabili coinvolte, l'espressione è vera se e solo se tutti i vincoli sono soddisfatti. Con poche variabili questa verifica è una enumerazione di 2^n casi, che si fa in un istante: è la dimostrazione, per casi, del risultato. Il modulo `python/booleane.py` la esegue — e tutte le traduzioni della sezione seguente sono verificate così.

Il modulo fa anche il resto del lavoro del capitolo: elimina le implicazioni ($A \Rightarrow B$ diventa $\text{NOT } A \text{ OR } B$), spinge le negazioni sui letterali con De Morgan, distribuisce l' `OR` sull' `AND`,

scarta le clausole tautologiche e quelle assorbite, e infine applica le tre regole di traduzione. Non è un solutore SAT: è la trascrizione, riga per riga, delle regole di questo capitolo.

2.9 Cinque esercizi

In tutti e cinque, i progetti sono numerati e per ciascun progetto p la variabile binaria è

$$x_p = \begin{cases} 1 & \text{se il progetto } p \text{ è scelto,} \\ 0 & \text{altrimenti.} \end{cases}$$

Le soluzioni — espressione in CNF, vincoli lineari, conteggio dei vincoli e numero di assegnazioni su cui l'equivalenza è verificata per enumerazione — stanno nel documento delle soluzioni, riservato ai docenti.

Esercizio 2.1 — implicazioni dirette

Dieci progetti. Si modellino come vincoli lineari:

1. se si sceglie il progetto 2 allora si sceglie il 3;
2. se si sceglie il 2 allora non si sceglie il 4;
3. se si scelgono l'1 e il 6 allora si sceglie il 7;
4. se si sceglie l'1 oppure il 6 allora si sceglie l'8;
5. se si scelgono il 2 e il 3 allora non si sceglie il 9;
6. se si sceglie il 2 oppure il 3 allora non si sceglie il 10.

Esercizio 2.2 — antecedenti e conseguenti negati

Dieci progetti. Si modellino:

1. se non si sceglie il 3 allora si sceglie il 2;
2. se non si sceglie il 4 allora non si sceglie il 2;
3. se si sceglie il 7 allora si scelgono l'1 e il 6;
4. se si sceglie l'8 allora si sceglie l'1 oppure il 6;
5. se non si sceglie il 9 allora si scelgono il 2 e il 3;
6. se non si sceglie il 10 allora si sceglie il 2 oppure il 3.

Esercizio 2.3 — antecedenti e conseguenti composti

Otto progetti. Si modellino:

1. $x_7 \text{ OR } x_3 \Rightarrow x_1 \text{ AND } x_2$;
2. $x_1 \text{ AND } x_6 \text{ AND } x_7 \Rightarrow x_8$;
3. $x_5 \text{ AND } x_2 \text{ AND NOT } x_4 \Rightarrow \text{NOT } x_3$;

4. $(x_1 \text{ OR } x_4) \text{ AND } x_6 \Rightarrow x_2 \text{ AND } (x_5 \text{ OR } x_7)$;
5. $(x_2 \text{ OR } x_5) \text{ AND } \text{NOT } x_8 \Rightarrow x_3 \text{ OR } \text{NOT } x_6$;
6. $(x_1 \text{ OR } x_4) \text{ AND } (x_2 \text{ OR } x_5) \text{ AND } \text{NOT } x_8 \Rightarrow x_3 \text{ AND } (\text{NOT } x_6 \text{ OR } x_7)$.

Esercizio 2.4 — «almeno due fra»

Nove progetti. Si modellino:

1. se si sceglie il 4 allora almeno due fra 1, 2, 3;
2. se almeno due fra 6, 7, 8 allora si sceglie il 5;
3. se non si sceglie il 4 allora almeno due fra 1, 2, 3, 9;
4. se si sceglie l'8 allora (1 e 6) oppure (1 e 7) oppure (2 e 6);
5. se almeno due fra 1, 3, 5 allora non si sceglie il 9;
6. se (1 e 2) oppure (3 e 4) allora si sceglie il 5.

La condizione «almeno due fra 1, 2, 3» è anche $x_1 + x_2 + x_3 \geq 2$. Come conseguente dell'implicazione governata da x_4 diventa

$$x_1 + x_2 + x_3 \geq 2x_4 :$$

un vincolo solo, contro le tre clausole del punto 1. I due sistemi hanno le stesse 16 soluzioni binarie — si verifica per enumerazione — ma non lo stesso rilassamento, e questa volta è la forma contata a essere **più forte**. Sommando le tre clausole $x_i + x_j \geq x_4$ si ottiene infatti solo $x_1 + x_2 + x_3 \geq \frac{3}{2}x_4$, più debole di $\geq 2x_4$; viceversa, con $0 \leq x_i \leq 1$, la forma contata implica ciascuna clausola. Su $\max x_1 + x_2 + x_3 + 3x_4$ con $x_1 + x_2 + x_3 + 2x_4 \leq 3$ e $z(MILP) = 3$, le tre clausole danno $z(LP^+) = 27/7 \approx 3,86$ e il vincolo contato $z(LP^+) = 15/4 = 3,75$: più vicino all'ottimo intero.

La morale non è «contare è sempre meglio»: nella sezione precedente «al più uno» contato era più forte delle clausole a coppie, qui «almeno due» contato è di nuovo più forte, ma il vincolo di attivazione aggregato del capitolo 3 sarà più *debole* di quello disaggregato. La morale è che due formulazioni con lo stesso insieme intero vanno confrontate sul rilassamento, con i criteri del capitolo 4, e non a occhio.

Esercizio 2.5 — scissioni

Dieci progetti. Si modellino:

1. $x_1 \text{ OR } x_2 \Rightarrow x_3$;
2. $x_4 \Rightarrow x_5 \text{ AND } x_6$;
3. $x_1 \text{ OR } x_2 \Rightarrow x_3 \text{ AND } x_4$;
4. $x_1 \text{ AND } x_2 \Rightarrow x_3$;

5. x_5 OR $x_6 \Rightarrow \text{NOT } x_7$;
 6. $\text{NOT } x_8$ OR $\text{NOT } x_9 \Rightarrow x_{10}$.

2.10 I vincoli logici dentro un modello di ottimizzazione

I vincoli di questo capitolo raramente vivono da soli: si aggiungono a un modello che ottimizza qualcosa. Con gli $n = 10$ progetti dell'esercizio 2.9, di ricavo r_p e costo b_p

progetto p	1	2	3	4	5	6	7	8	9	10
ricavo r_p	9	7	4	8	3	6	2	5	7	6
costo b_p	4	3	2	4	2	3	1	3	4	3

e budget $B = 14$, il modello è

Selezione di progetti con vincoli logici

$$\begin{aligned}
 & \max \sum_{p=1}^n r_p x_p \\
 & \text{soggetto a } \sum_{p=1}^n b_p x_p \leq B, \\
 & \text{gli 8 vincoli dell'esercizio 2.9,} \\
 & x_p \in \{0, 1\}, \quad \forall p \in \{1, 2, \dots, n\}.
 \end{aligned}$$

Descrizione. L'obiettivo somma i ricavi dei progetti scelti. Il primo vincolo, uno solo, è il budget. Gli otto vincoli richiamati sono le traduzioni lineari delle sei implicazioni dell'esercizio 2.9: non aggiungono variabili, solo righe. I domini sono le n condizioni di binarietà, una per progetto.

Senza i vincoli logici l'ottimo è 30. Con i vincoli logici l'ottimo scende a $z(MILP) = 28$, ottenuto scegliendo i progetti 1, 2, 3, 5, 8, di costo complessivo 14: il budget è saturo. Il rilassamento $z(LP^+)$ vale 29. Le sei implicazioni sono soddisfatte: si sceglie il 2 e infatti si sceglie anche il 3; non si sceglie il 4; non si scelgono insieme 1 e 6; si sceglie l'1 e infatti si sceglie l'8; si scelgono 2 e 3 e infatti non si sceglie il 9; si sceglie il 2 e infatti non si sceglie il 10.

Quanto tagliano sei implicazioni

Le $2^{10} = 1024$ assegnazioni delle dieci binarie si riducono a 234 quando si impongono tutte e sei le implicazioni dell'esercizio 2.9: meno di un quarto. La figura 2.1 mostra quante ne sopravvivono a ciascuna implicazione presa da sola e quante quando si accumulano. Nessuna di queste implicazioni, da sola, taglia più della metà dello spazio; insieme lo riducono di tre quarti. È il motivo per cui un modello si scrive un vincolo alla volta, ma si valuta tutto intero.

Il modello in gurobipy

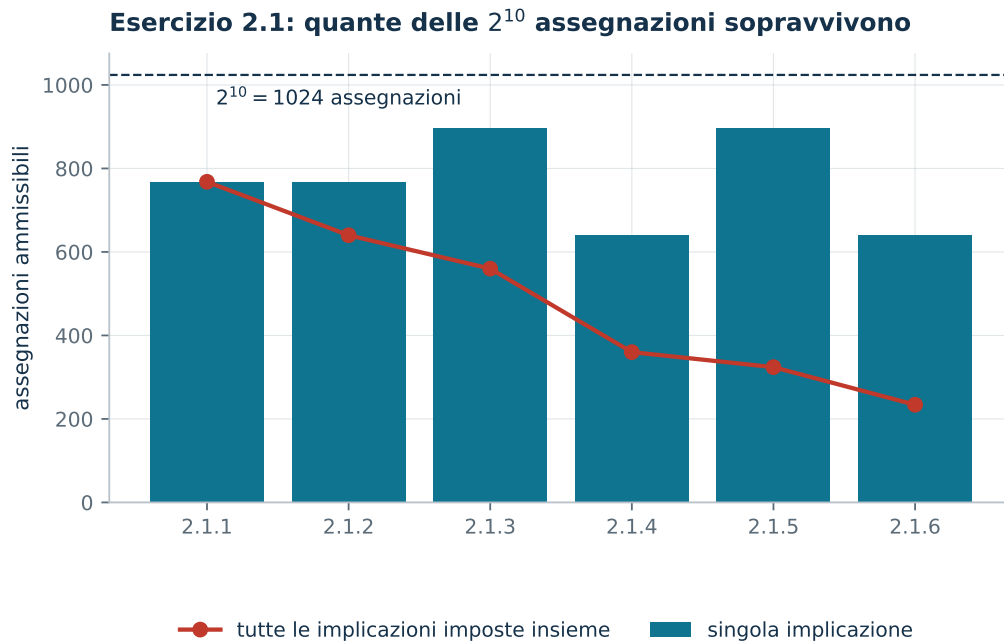


Figura 2.1: Esercizio 2.9: assegnazioni delle dieci binarie che sopravvivono a ciascuna implicazione presa da sola (barre) e a tutte quelle imposte fino a quel punto (linea).

```

1 from booleane import cnf, vincolo, IMP, AND, OR, NOT, V
2
3 x = {p: V(f"x{p}") for p in range(1, 11)}
4 implicazioni = [IMP(x[2], x[3]), IMP(x[2], NOT(x[4])),
5                 IMP(AND(x[1], x[6]), x[7]), IMP(OR(x[1], x[6]), x[8]),
6                 IMP(AND(x[2], x[3]), NOT(x[9])), IMP(OR(x[2], x[3]),
7                 NOT(x[10]))]
8
9 m = gp.Model("selezione_progetti"); m.Params.OutputFlag = 0
10 xv = m.addVars(range(1, 11), vtype=GRB.BINARY, name="x")
11 m.setObjective(gp.quicksum(r[p] * xv[p] for p in range(1, 11)), GRB.
12     MAXIMIZE)
13 m.addConstr(gp.quicksum(b[p] * xv[p] for p in range(1, 11)) <= budget,
14     name="budget")
15 for i, formula in enumerate(implicazioni, 1):
16     un vincolo
17     for j, clausola in enumerate(cnf(formula), 1):
18         coef, verso, rhs = vincolo(clausola)
19         lhs = gp.quicksum(k * xv[int(n[1:])] for n, k in coef.items())
20         m.addConstr(lhs <= rhs if verso == "<=" else lhs >= rhs, name=f
21             "logica{i}_{j}")
22 m.optimize()

```

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

2.11 Che cosa resta al capitolo successivo

Questo capitolo tratta legami fra variabili *tutte binarie*. Quando una delle due famiglie è continua o intera — «se la macchina non è accesa allora non produce niente», «questa variabile vale il massimo di quelle» — la CNF non basta più: servono i coefficienti, i big-M e gli argomenti di ottimalità del capitolo 3.

Legami fra variabili

Script: `python/cap03_legami.py`

[Pagina online](#)

Il capitolo 2 collega variabili *tutte binarie*. Qui si collegano famiglie *diverse*: binarie con continue, binarie con intere, continue fra loro. Sono quattordici tecniche, e sono il vero contenuto della modellazione MIP: quasi ogni modello di questo corso è una combinazione di queste.

Come si legge ogni sezione

Ogni tecnica è presentata in cinque passi:

- (a) il **legame in parole**: che cosa si vuole dire;
- (b) i **vincoli lineari** che lo dicono, con il conteggio;
- (c) la **dimostrazione**: nei due versi se entrambi sono imposti dai vincoli; con un argomento di scambio se uno segue dall'ottimalità; con un *controesempio* quando una converso è falsa;
- (d) la **forza del rilassamento**, su un'istanza minima risolta dallo script;
- (e) la **riga gurobipy** e i rimandi ai problemi della Parte II in cui la tecnica si rivede.

Il passo (c) è quello che distingue un modello corretto da uno plausibile. Il passo (d) è quello che distingue un modello che si risolve da uno che non si risolve.

Per tutto il capitolo distinguiamo:

- una proprietà **imposta dai vincoli**: vale per *ogni* soluzione ammissibile, ottima o no. Si dimostra guardando solo i vincoli;
- una proprietà **di ottimalità**: vale solo nelle soluzioni ottime. Si dimostra con un *argomento di scambio*: si prende una soluzione ammissibile che la viola, si costruisce una soluzione modificata, si verifica che resta ammissibile e si confronta il valore. Se il valore migliora *strettamente*, la conclusione è «in ogni ottimo»; se non peggiora, la conclusione è «esiste un ottimo», che è più debole.

Scrivere «in ogni ottimo» quando il coefficiente è solo non negativo è l'errore più comune di questo capitolo.

3.1 Attivazione: forma aggregata e forma disaggregata

(a) **Il legame in parole.** Una risorsa (una macchina, una sede, un impianto) va *attivata* prima di essere usata. Le binarie x_{ij} dicono «l'oggetto i usa la risorsa j », le binarie y_j dicono «la risorsa j è attivata». Si vuole: se qualcuno usa j , allora j è attivata.

(b) **I vincoli.** Due modi, con n oggetti e m risorse:

$$\text{disaggregato: } x_{ij} \leq y_j, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\} \quad (nm \text{ vincoli}), \quad (3.1)$$

$$\text{aggregato: } \sum_{i=1}^n x_{ij} \leq n y_j, \quad \forall j \in \{1, 2, \dots, m\} \quad (m \text{ vincoli}). \quad (3.2)$$

Quando la risorsa ha una capacità k_j (al più k_j oggetti), la forma aggregata si scrive con quel coefficiente, $\sum_i x_{ij} \leq k_j y_j$, e allora impone *anche* la capacità: due condizioni in un vincolo solo.

(c) **La dimostrazione.** Su punti binari le due forme sono **equivalenti**.

- *Disaggregato* $\Rightarrow$ *aggregato*: sommando (3.1) su i si ottiene $\sum_i x_{ij} \leq n y_j$.
- *Aggregato* $\Rightarrow$ *disaggregato*, su punti binari: se $y_j = 0$, (3.2) dà $\sum_i x_{ij} \leq 0$ e quindi $x_{ij} = 0 \leq y_j$ per ogni i ; se $y_j = 1$, allora $x_{ij} \leq 1 = y_j$ per ogni i perché le x sono binarie.

Il secondo verso usa la binarietà di x e y : *fuori* dai punti binari non vale, ed è esattamente questo che rende le due forme diverse nel rilassamento.

Il verso opposto non è imposto. «Se j è attivata allora qualcuno la usa», cioè $y_j = 1 \Rightarrow \sum_i x_{ij} \geq 1$, **non** segue dai vincoli: $y_j = 1$ con tutte le $x_{ij} = 0$ è ammissibile in entrambe le forme. Segue dall'ottimalità quando attivare costa: se il costo di attivazione f_j è *strettamente* positivo, data una soluzione ottima con $y_j = 1$ e nessun $x_{ij} = 1$, porre $y_j = 0$ lascia tutti i vincoli soddisfatti e riduce il costo di $f_j > 0$ — contraddizione. Quindi «in ogni ottimo». Se invece $f_j = 0$ è ammesso, la conclusione corretta è la più debole «esiste un ottimo in cui $y_j = 0$ ».

(d) **La forza del rilassamento.** Istanza: $n = 3$ clienti, $m = 2$ sedi, costi di attivazione $f = (8, 6)$, costi di servizio

$$c = \begin{pmatrix} 2 & 5 \\ 4 & 1 \\ 3 & 3 \end{pmatrix}.$$

Formulazione	vincoli di link	$z(LP^+)$
aggregata (3.2)	$m = 2$	$38/3 \approx 12,67$
disaggregata (3.1)	$nm = 6$	15

L'ottimo intero è $z(MILP) = 15$: la forma disaggregata lo raggiunge già nel rilassamento, quella aggregata si ferma a $38/3$. Più righe, rilassamento più stretto: è il compromesso tipico di questa tecnica. Su istanze grandi la scelta non è ovvia — nm righe possono essere molte — ma su istanze difficili la forma disaggregata quasi sempre paga.

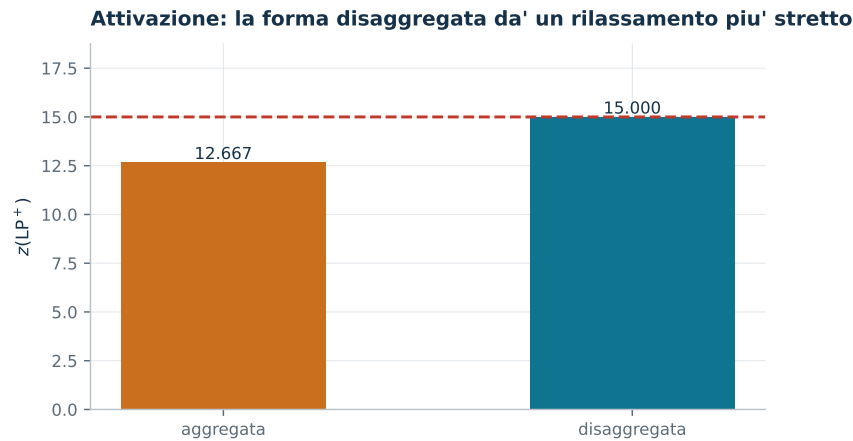


Figura 3.1: Il rilassamento delle due forme di attivazione sull'istanza a tre clienti e due sedi. La linea rossa è l'ottimo intero.

(e) In gurobipy, e dove si rivede.

```

1 m.addConstrs((x[i, j] <= y[j] for i in range(n) for j in range(mm)),
   name="link") # disaggregato
2 m.addConstrs((x.sum("*", j) <= n * y[j] for j in range(mm)), name="link
   ") # aggregato

```

Si rivede nei problemi 7.2 (macchine con costo fisso), 7.3 (selezione), 7.5 (classi con setup) e 8.4 (hub), dove il confronto fra le due forme è una domanda di modellazione.

3.2 Costo fisso, capacità e flusso continuo

(a) **Il legame in parole.** Come sopra, ma la quantità usata è *continua*: $q_j \geq 0$ è quanto si produce nell'impianto j , e y_j dice se l'impianto è aperto. Un impianto chiuso non produce; uno aperto produce al più la sua capacità c_j .

(b) **I vincoli.**

$$q_j \leq c_j y_j, \quad \forall j \in \{1, 2, \dots, m\} \quad (m \text{ vincoli}). \quad (3.3)$$

Un solo vincolo per impianto, con il coefficiente *giusto*: la capacità, non un numero grande a caso.

(c) **La dimostrazione.** Se $y_j = 0$ il vincolo dà $q_j \leq 0$ e, con $q_j \geq 0$, forza $q_j = 0$: l'implicazione «chiuso $\Rightarrow$ non produce» è imposta dal vincolo, e con essa la sua contronominale «produce $\Rightarrow$ aperto». Se $y_j = 1$ il vincolo dà $q_j \leq c_j$: la capacità. Il verso «aperto $\Rightarrow$ produce» non è imposto e segue dall'ottimalità solo se $f_j > 0$, come nella sezione precedente.

(d) **La forza del rilassamento.** Istanza: due impianti, costi fissi $f = (10, 14)$, costi unitari $c = (3, 2)$, capacità $C = (6, 7)$, domanda $D = 9$. L'ottimo è $z(MILP) = 44$ (entrambi aperti, $q = (2, 7)$).

Coefficiente in (3.3)	$z(LP^+)$
la capacità, c_j	$112/3 \approx 37,33$
un big-M = 100 (più il vincolo $q_j \leq c_j$ a parte)	$1059/50 = 21,18$

Stesso insieme intero, stesso ottimo, rilassamenti lontanissimi. La regola: **il coefficiente di una binaria di attivazione è il più piccolo valore che la variabile continua può assumere quando l'attivazione vale 1**, e va ricavato dai dati.

(e) In gurobipy, e dove si rivede.

```
1 m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="link")
```

Si rivede nel problema 8.1 (localizzazione capacità) e in tutta la famiglia 9.

3.3 Lotto minimo e variabile semicontinua

(a) **Il legame in parole.** «Se si produce, si produce almeno ℓ »: la quantità q_j vale zero oppure sta fra una soglia ℓ e la capacità c_j . Non è un intervallo: è l'unione di un punto e di un intervallo. Una variabile con questo dominio si chiama **semicontinua**.

(b) **I vincoli.** Con la stessa binaria y_j della sezione precedente,

$$\begin{aligned} \ell y_j &\leq q_j, & \forall j \in \{1, 2, \dots, m\} & \quad (m \text{ vincoli}), \\ q_j &\leq c_j y_j, & \forall j \in \{1, 2, \dots, m\} & \quad (m \text{ vincoli}). \end{aligned} \tag{3.4}$$

(c) **La dimostrazione.** Entrambi i versi sono imposti dai vincoli. Se $y_j = 0$: $0 \leq q_j \leq 0$, cioè $q_j = 0$. Se $y_j = 1$: $\ell \leq q_j \leq c_j$. Quindi

$$q_j \in \{0\} \cup [\ell, c_j],$$

esattamente il dominio voluto. Serve $\ell \leq c_j$, altrimenti $y_j = 1$ è inammissibile e la variabile è costretta a zero: un errore di dati che il solver segnala come inammissibilità solo se y_j è forzata a 1 da altri vincoli.

(d) **La forza del rilassamento.** Sulla stessa istanza della sezione precedente con $\ell = 5$: l'ottimo passa da 44 a $z(MILP) = 49$, con $q = (5, 5)$ invece di $(2, 7)$ — la soglia costringe a produrre 5 nel secondo impianto anche se il primo è più caro. Ma $z(LP^+)$ resta $112/3$, identico al caso senza soglia.

Nel rilassamento y_j è libera in $[0, 1]$, e il vincolo $q_j \geq \ell y_j$ si soddisfa abbassando y_j : basta $y_j \leq q_j/\ell$. Il vincolo di soglia *non morde mai* sul continuo. Tutto il suo effetto si scarica sull'interezza — ed è per questo che i modelli con lotto minimo sono tipicamente più difficili di quelli senza, a parità di dimensione.

(e) In gurobipy, e dove si rivede.

```
1 m.addConstrs((q[j] >= ell * y[j] for j in range(mm)), name="lotto")
2 m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="capacita")
```

Gurobi ha anche il tipo `GRB.SEMICONT`, che dichiara direttamente il dominio; in questo corso si scrive la formulazione a mano, perché è quella che si deve saper dimostrare. Si rivede nella domanda 7.2.2 e negli esercizi 9.1 e 9.3.

3.4 Conteggi interi, capacità multipla, arrotondamento all'intero superiore

(a) **Il legame in parole.** «Quanti contenitori servono?» La variabile non è binaria ma *intera*: $w \in \mathbb{Z}_{\geq 0}$ conta oggetti indivisibili (scatole, camion, turni, operai) e ciascuno porta una capacità c .

(b) **I vincoli.**

$$\sum_i a_i x_i \leq c w, \quad (1 \text{ vincolo}),$$

$$w \in \mathbb{Z}_{\geq 0}. \quad (1 \text{ variabile intera})$$
(3.5)

(c) **La dimostrazione.** Posto $q = \sum_i a_i x_i$, il vincolo impone $w \geq q/c$ e l'interezza impone $w \geq \lceil q/c \rceil$. Insieme a un obiettivo che minimizza w (o che paga w), in ogni ottimo w vale esattamente quel tetto: se valesse di più, abbassarlo di 1 resterebbe ammissibile e ridurrebbe il costo. Con costo nullo per w la conclusione si indebolisce in «esiste un ottimo».

$\lceil t \rceil$ non è una funzione lineare: non si può scrivere dentro un vincolo. La coppia «disuguaglianza $\leq cw$ + dichiarazione w intera» la realizza *implicitamente*, e va letta così quando si spiega il modello.

(d) **La forza del rilassamento.** 17 pezzi unitari, capienza $c = 5$: il rilassamento dà $w \geq 17/5 = 3,4$ e l'ottimo intero è $z(MILP) = 4$. Il gap $4 - 17/5 = 3/5$ viene tutto dall'interezza: nessun taglio lineare sulle sole x lo chiude, serve una disuguaglianza che usi w intera.

(e) In gurobipy, e dove si rivede.

```
1 w = m.addVar(vtype=GRB.INTEGER, name="w")
2 m.addConstr(gp.quicksum(a[i] * x[i] for i in range(n)) <= c * w, name="capienza")
```

Si rivede negli esercizi 10.4 (scatole di luci), 10.5 (spedizioni) e 9.2 (manodopera).

3.5 L'ausiliaria di massimo

(a) **Il legame in parole.** Una variabile z deve valere il *massimo* di certe quantità: il tempo dell'attività più lunga, il costo di connessione più alto, il carico della macchina più carica.

(b) **I vincoli.** Se le quantità sono $t_j x_j$ con x_j binaria,

$$\begin{aligned} z &\geq t_j x_j, \quad \forall j \in \{1, 2, \dots, n\} & (n \text{ vincoli}), \\ z &\geq 0. \end{aligned} \quad (3.6)$$

(c) **La dimostrazione, in tre passi.**

1. *Imposto dal vincolo:* per ogni j con $x_j = 1$ si ha $z \geq t_j$, quindi $z \geq \max_{j:x_j=1} t_j$. Vale in ogni soluzione ammissibile.
2. *Dall'ottimalità:* se z compare nell'obiettivo con coefficiente strettamente positivo in un minimo (o strettamente negativo in un massimo) e in nessun altro vincolo, allora in ogni ottimo z assume il valore più piccolo ammissibile. Infatti, data una soluzione ottima con $z > \max_{j:x_j=1} t_j$, abbassare z fino a quel massimo lascia tutti i vincoli (3.6) soddisfatti e migliora strettamente l'obiettivo.
3. *Sintesi:* in ogni ottimo, $z = \max_{j:x_j=1} t_j$ esattamente (con la convenzione che il massimo sull'insieme vuoto è 0, garantita da $z \geq 0$).

Se z compare in un altro vincolo che la vuole *grande* — per esempio $z \geq$ qualcos'altro, oppure un vincolo $z \geq \tau_j$ come nella domanda 7.7.2 — l'argomento di scambio non funziona più: abbassare z può violare quel vincolo. In quel caso $z \geq \max$ resta vero, ma $z = \max$ no. È l'errore che la Parte II segnala nella variante makespan del problema 7.4.

(d) **La forza del rilassamento.** $\min z$ con $z \geq t_j x_j$, $t = (4, 7, 3)$ e $\sum_j x_j \geq 2$: l'ottimo intero è $z(MILP) = 4$ (si scelgono i lavori 1 e 3, e il massimo è 4). Il rilassamento vale $168/61 \approx 2,75$: la soluzione frazionaria $x_j = z/t_j$ spalma la scelta su tutti e tre i lavori e abbassa il massimo. Il legame di massimo dà rilassamenti *deboli*: è una delle ragioni per cui i problemi di makespan sono difficili.

(e) **In gurobipy, e dove si rivede.**

```
1 z = m.addVar(name="z")
2 m.addConstrs((z >= t[j] * x[j] for j in range(n)), name="massimo")
```

Si rivede nei problemi 7.4, 7.7, 8.4 e 10.9 (libri sugli scaffali).

3.6 Min-max, max-min e differenza fra il massimo e il minimo

(a) **Il legame in parole.** Tre obiettivi di equità, spesso confusi: *min-max* (minimizzare il carico più alto), *max-min* (massimizzare il carico più basso) e *differenza* (minimizzare lo scarto fra il più alto e il più basso).

(b) **I vincoli.** Con ℓ_k il carico della risorsa k ($k \in \{1, 2, \dots, m\}$):

$$\text{min-max:} \quad \min z \quad \text{con} \quad z \geq \ell_k, \quad \forall k \in \{1, 2, \dots, m\} \quad (m \text{ vincoli}), \quad (3.7)$$

$$\text{max-min:} \quad \max u \quad \text{con} \quad u \leq \ell_k, \quad \forall k \in \{1, 2, \dots, m\} \quad (m \text{ vincoli}), \quad (3.8)$$

$$\text{differenza:} \quad \min (z - u) \quad \text{con entrambi} \quad (2m \text{ vincoli}). \quad (3.9)$$

(c) **La dimostrazione.** Ciascuna è l'ausiliaria di massimo (o di minimo) della sezione precedente, con l'argomento di scambio nel verso giusto: in un $\min z$ la z scende al massimo dei carichi; in un $\max u$ la u sale al minimo dei carichi. Nella differenza entrambe le spinte sono presenti contemporaneamente e le due conclusioni valgono insieme.

Sull'istanza a cinque pesi $p = (3, 5, 2, 4, 7)$ da ripartire su due operai (totale 21), le tre versioni scelgono la *stessa* ripartizione (11, 10) — la migliore possibile, perché il totale è dispari — ma i loro valori ottimi sono 11, 10 e 1. Sono tre numeri diversi che descrivono la stessa soluzione. Confrontare « $z = 11$ » di un min-max con « $z = 1$ » di una differenza non significa niente: prima di confrontare due modelli bisogna che ottimizzino la stessa cosa. E le soluzioni ottime possono anche non coincidere: con più di due risorse, min-max e max-min in generale scelgono ripartizioni diverse.

(d) **La forza del rilassamento.** Il min-max su quell'istanza dà $z(LP^+) = 21/2 = 10,5$ contro $z(MILP) = 11$: il rilassamento distribuisce i pesi a metà esatta, cosa che l'interezza non permette.

(e) **In gurobipy, e dove si rivede.**

```
1 z = m.addVar(name="z")
2 m.addConstrs((z >= carico[k] for k in range(len(carico))), name="max")
3 m.setObjective(z, GRB.MINIMIZE)
```

Si rivede nella domanda 7.4.1 (makespan), nell'esercizio 10.7 (suddivisione antitrust) e nell'10.8 (distribuzione dei brani sui CD).

3.7 Il valore assoluto

(a) **Il legame in parole.** Si vuole $|u - v|$: uno scarto, un errore, uno sbilanciamento. Il caso in obiettivo e il caso in vincolo si comportano in modo radicalmente diverso.

(b) **I vincoli.**

- **In obiettivo, minimizzando:** una variabile $d \geq 0$ e due vincoli,

$$\begin{aligned} d &\geq u - v, & (1 \text{ vincolo}), \\ d &\geq v - u, & (1 \text{ vincolo}), \end{aligned} \tag{3.10}$$

con d nell'obiettivo da minimizzare. Nessuna binaria.

- **Come vincolo $\leq$:** $|u - v| \leq k$ è semplicemente $u - v \leq k$ e $v - u \leq k$ (2 vincoli, nessuna binaria).
- **Come vincolo $\geq$:** $|u - v| \geq k$ **non** si scrive senza binarie. È la disgiunzione « $u - v \geq k$ oppure $v - u \geq k$ », e richiede una binaria b e un big-M:

$$\begin{aligned} u - v &\geq k - M(1 - b), & (1 \text{ vincolo}), \\ v - u &\geq k - Mb. & (1 \text{ vincolo}, 1 \text{ binaria}) \end{aligned} \tag{3.11}$$

(c) **La dimostrazione.** Nel primo caso i due vincoli (3.10) impongono $d \geq |u - v|$ (uno dei due membri destri è $|u - v|$); l'obiettivo, che minimizza d e in cui d non compare altrove, la porta all'uguaglianza con l'argomento di scambio della sezione 3.5. Nel terzo caso, $b = 1$ disattiva il secondo vincolo (se $M \geq k + \max(v - u)$) e lascia il primo, e viceversa: è una disgiunzione, non una congiunzione, e senza binaria si imporrebbero entrambi — cioè $0 \geq 2k$, inammissibile per $k > 0$.

$|u - v| \leq k$ è l'intersezione di due semipiani: un insieme convesso, che si scrive con due vincoli lineari. $|u - v| \geq k$ è il complemento di una striscia: *non* è convesso, e nessun sistema di vincoli lineari senza variabili intere può descriverlo. La binaria non è un trucco: è necessaria.

(d) **La forza del rilassamento.** Sull'istanza dei cinque pesi, il problema $\min |\ell_1 - \ell_2|$ ha ottimo $z(MILP) = 1$ e rilassamento $z(LP^+) = 0$: il continuo divide 21 in due metà uguali e azzerava lo scarto. Il rilassamento di un obiettivo di valore assoluto è tipicamente 0, cioè inutile.

(e) **In gurobipy, e dove si rivede.**

```
1 d = m.addVar(name="d")
2 m.addConstr(d >= u - v, name="abs_piu")
3 m.addConstr(d >= v - u, name="abs_meno")
```

Si rivede negli esercizi 10.8 (CD) e 10.7 (antitrust), e nella sezione 3.13 in una forma equivalente con due deviazioni.

3.8 Big-M: vincoli condizionati e disgiunzioni

(a) **Il legame in parole.** «Se $y = 1$ allora vale il vincolo $a'x \leq b$; se $y = 0$ il vincolo non c'è». Un vincolo che si accende e si spegne.

(b) **I vincoli.**

$$a'x \leq b + M(1 - y) \quad (1 \text{ vincolo}). \quad (3.12)$$

(c) **La dimostrazione, e come si sceglie M .** Con $y = 1$ il vincolo è $a'x \leq b$: acceso. Con $y = 0$ diventa $a'x \leq b + M$: spento *purché* M sia abbastanza grande da non tagliare nulla, cioè

$$M \geq \max\{a'x : x \text{ ammissibile per gli altri vincoli}\} - b. \quad (3.13)$$

Si distinguono tre cose:

- un **M valido**: soddisfa (3.13), quindi il modello ha l'insieme intero giusto;
- un **M migliorabile**: valido ma più grande del necessario; l'insieme intero è lo stesso, il rilassamento è più debole;
- il **minimo M dimostrato**: il più piccolo valore per cui si è *dimostrata* la validità, tipicamente calcolando il massimo in (3.13) sui soli bound delle variabili. Non è necessariamente il minimo assoluto, ed è onesto dirlo.

Con $a = (3, 4, 5)$, $b = 6$ e x binarie, il massimo di $a'x$ è 12, quindi $M = 12 - 6 = 6$ è valido. Su $\max x_1 + x_2 + x_3 + y$ l'ottimo è $z(MILP) = 3$ (con $y = 0$ e tutte le x a 1). Con $M = 5$ il vincolo con $y = 0$ resta $3x_1 + 4x_2 + 5x_3 \leq 11$, che esclude $x = (1, 1, 1)$: l'ottimo scende a 2. Il modello non risponde più alla domanda posta.

(d) **La forza del rilassamento.** Sulla stessa istanza:

M	$z(LP^+)$	
6 (il minimo dimostrato)	3	coincide con $z(MILP)$
20	$37/10 = 3,7$	
1000	$1997/500 \approx 3,994$	quasi il massimo possibile, 4

Il degrado è monotono e rapido. La regola operativa: **calcolare M dai dati, sempre, e scriverlo nel testo subito dopo il modello.**

(e) **In gurobipy, e dove si rivede.**

```

1 M = sum(max(a[j], 0) for j in range(n)) - b          # calcolato dai dati,
    non scelto a occhio
2 m.addConstr(gp.quicksum(a[j] * x[j] for j in range(n)) <= b + M * (1 -
    y), name="cond")

```

Si rivede nel problema 7.7 (ritardo totale) e nella sezione 3.9.

3.9 Precedenze e sequenziamento

(a) **Il legame in parole.** Su una macchina che esegue un lavoro alla volta, per ogni coppia di lavori uno dei due precede l'altro. È una disgiunzione per coppia, non una precedenza fissata dai dati.

(b) **I vincoli.** Con $\kappa_j \geq 0$ l'istante di completamento del lavoro j , t_j la sua durata e $s_{ij} \in \{0, 1\}$ che vale 1 se j precede i :

$$s_{ij} + s_{ji} = 1, \quad \forall i, j \in \{1, 2, \dots, n\}, i < j \quad \left(\binom{n}{2} \text{ vincoli}\right), \quad (3.14)$$

$$\kappa_i \geq \kappa_j + t_i - M(1 - s_{ij}), \quad \forall i, j \in \{1, 2, \dots, n\}, i \neq j \quad (n(n-1) \text{ vincoli}), \quad (3.15)$$

$$\kappa_j \geq t_j, \quad \forall j \in \{1, 2, \dots, n\} \quad (n \text{ vincoli}). \quad (3.16)$$

(c) **La dimostrazione, e il minimo M .** Il vincolo (3.14) impone che esattamente uno dei due ordini sia scelto. Se $s_{ij} = 1$, (3.15) diventa $\kappa_i \geq \kappa_j + t_i$: il lavoro i finisce almeno t_i dopo la fine di j , cioè non si sovrappongono. Se $s_{ij} = 0$ diventa $\kappa_i \geq \kappa_j + t_i - M$, che deve essere sempre soddisfatto. Poiché $\kappa_i \geq t_i$ e $\kappa_j \leq \sum_h t_h$ in ogni soluzione sensata, basta

$$M = \sum_{h=1}^n t_h,$$

perché allora $\kappa_j + t_i - M \leq \sum_h t_h + t_i - \sum_h t_h = t_i \leq \kappa_i$. L'orizzonte temporale $\sum_h t_h$ va *dichiarato*: senza un limite superiore sui κ_j , nessun M finito è valido.

(d) **La forza del rilassamento.** Tre lavori di durata (3, 2, 4) su una macchina, obiettivo makespan: $z(MILP) = 9 = \sum_h t_h$ (ovvio: una sola macchina), con completamenti (3, 5, 9). Il rilassamento vale 4: con $s_{ij} = 1/2$ tutti i vincoli di precedenza si spengono a metà e i lavori possono sovrapporsi. È il rilassamento più debole di tutto il capitolo, e spiega perché i modelli di sequenziamento big-M scalano male.

(e) **In gurobipy, e dove si rivede.**

```

1 M = sum(t)                                     # l'orizzonte, dichiarato
2 for i in range(n):
3     for j in range(i):
4         m.addConstr(s[i, j] + s[j, i] == 1, name=f"disg{i}{j}")
5         m.addConstr(kappa[i] >= kappa[j] + t[i] - M * (1 - s[i, j]),
6                     name=f"prec{i}{j}")
7         m.addConstr(kappa[j] >= kappa[i] + t[j] - M * (1 - s[j, i]),
8                     name=f"prec{j}{i}")

```

Si rivede nel problema 7.7, dove le date di rilascio permettono di ridurre M .

3.10 «Se e solo se»

(a) **Il legame in parole.** Una binaria y deve valere 1 *esattamente quando* una condizione è vera — per esempio «tutti i lavori della classe sono eseguiti». Non basta un'implicazione: ne servono due.

(b) **I vincoli.** Con $x_1, \dots, x_p$ binarie e la condizione «tutte valgono 1»:

$$y \leq x_j, \quad \forall j \in \{1, 2, \dots, p\} \quad (p \text{ vincoli}), \quad (3.17)$$

$$y \geq \sum_{j=1}^p x_j - (p - 1) \quad (1 \text{ vincolo}). \quad (3.18)$$

(c) **La dimostrazione, nei due versi.**

- (3.17) dà $y = 1 \Rightarrow x_j = 1$ per ogni j ; equivalentemente, se anche uno solo degli x_j è zero allora $y = 0$.
- (3.18) dà il verso opposto: se tutti gli $x_j = 1$, il membro destro vale $p - (p - 1) = 1$, quindi $y \geq 1$ e dunque $y = 1$. Se almeno uno è zero, il membro destro è ≤ 0 e il vincolo non dice nulla.

Insieme, i due impongono $y = 1 \iff \sum_j x_j = p$ in *ogni* soluzione ammissibile.

Quando basta un verso solo. Se y compare nell'obiettivo di un massimo con un premio $v > 0$ e in nessun altro vincolo, il verso (3.18) si può omettere: in ogni ottimo, se tutti gli $x_j = 1$ e $y = 0$, alzare y a 1 resta ammissibile per (3.17) e aumenta l'obiettivo di $v > 0$. È un argomento di ottimalità, e vale solo all'ottimo.

Sull'istanza a tre lavori, ricavi (2, 2, 2), capacità 3:

premio v	vincoli	$z(MILP)$	y, x all'ottimo	y dice «classe completa»?
9	solo (3.17)	15	$y = 1, x = (1, 1, 1)$	sì
9	entrambi	15	$y = 1, x = (1, 1, 1)$	sì
0	solo (3.17)	6	$y = 0, x = (1, 1, 1)$	no
0	entrambi	6	$y = 1, x = (1, 1, 1)$	sì

Con $v = 0$ il valore ottimo è lo stesso nei due modelli, ma nel terzo caso y non è più un indicatore fedele. Se y serve solo a pagare un premio, il verso mancante è inutile; se y compare in *altri* vincoli — e questo è il caso non appena si aggiunge una condizione sul numero di classi complete — va scritto.

(d) **La forza del rilassamento.** $z(LP^+) = 15 = z(MILP)$ su questa istanza: entrambi i versi imposti danno un rilassamento esatto. Il vincolo (3.18) è debole nel rilassamento (con $x_j = 1/2$ il membro destro è negativo), ma qui non serve.

(e) **In gurobipy, e dove si rivede.**

```
1 m.addConstrs((y <= x[j] for j in range(p)), name="iff_su")
2 m.addConstr(y >= gp.quicksum(x[j] for j in range(p)) - (p - 1), name="iff_giu")
```

Si rivede nel problema 7.6 (classi con premio) e nell'esercizio 9.3 (premio per varietà).

3.11 Contare i tipi diversi

(a) **Il legame in parole.** «Si devono produrre almeno due tipi diversi», «al più tre alimenti in dieta», «non più di quattro configurazioni». Si conta *quanti* elementi sono attivi, non quanto se ne produce.

(b) **I vincoli.** Con $q_j \geq 0$ la quantità e y_j l'indicatore:

$$q_j \leq c_j y_j, \quad \forall j \in \{1, 2, \dots, n\} \quad (n \text{ vincoli}), \quad (3.19)$$

$$q_j \geq \ell y_j, \quad \forall j \in \{1, 2, \dots, n\} \quad (n \text{ vincoli}), \quad (3.20)$$

$$\sum_{j=1}^n y_j \geq p \quad (1 \text{ vincolo}). \quad (3.21)$$

(c) **La dimostrazione, e perché servono entrambe le soglie.** (3.19) dà « $y_j = 0 \Rightarrow q_j = 0$ »; (3.20) dà il verso mancante « $y_j = 1 \Rightarrow q_j \geq \ell$ ». Senza (3.20), una soluzione con $y_j = 1$ e $q_j = 0$ è ammissibile: il vincolo di conteggio (3.21) si soddisfa accendendo indicatori vuoti, e la condizione «almeno p tipi diversi» non dice più niente. **Il conteggio dei tipi funziona solo se ogni tipo acceso produce davvero qualcosa**, e la soglia ℓ è ciò che lo garantisce.

(d) **La forza del rilassamento.** Tre tipi, ricavi unitari (4, 3, 5), risorsa 12, capacità 10 ciascuno, soglia $\ell = 3$, almeno due tipi: $z(MILP) = 57$, con $q = (3, 0, 9)$. Il rilassamento vale anch'esso 57: qui il conteggio non introduce alcun gap.

(e) In gurobiipy, e dove si rivede.

```
1 m.addConstrs((q[j] <= c[j] * y[j] for j in range(n)), name="attiva")
2 m.addConstrs((q[j] >= ell * y[j] for j in range(n)), name="lotto")
3 m.addConstr(y.sum() >= p, name="almeno_p_tipi")
```

Si rivede negli esercizi 9.3 (veicoli), 10.3 (dieta) e 10.4 (alberi).

3.12 Alldiff ed espansione binaria

(a) **Il legame in parole.** Due esigenze diverse ma imparentate: dare a n oggetti n valori *tutti distinti* (*alldiff*), e rappresentare una variabile intera limitata con variabili binarie (*espansione binaria*).

(b) **I vincoli.** *Alldiff*, con $p_{iv} = 1$ se l'oggetto i riceve il valore v :

$$\sum_{v=1}^n p_{iv} = 1, \quad \forall i \in \{1, 2, \dots, n\} \quad (n \text{ vincoli}), \quad (3.22)$$

$$\sum_{i=1}^n p_{iv} = 1, \quad \forall v \in \{1, 2, \dots, n\} \quad (n \text{ vincoli}). \quad (3.23)$$

Espansione binaria di $v \in \{0, 1, \dots, 2^p - 1\}$:

$$v = \sum_{k=0}^{p-1} 2^k b_k, \quad (1 \text{ vincolo}), \quad (3.24)$$

$$b_k \in \{0, 1\}, \quad \forall k \in \{0, 1, \dots, p-1\} \quad (p \text{ binarie}).$$

(c) **La dimostrazione.** L'alldiff è un doppio *set partitioning*: il primo gruppo dà «ogni oggetto un valore», il secondo «ogni valore a un solo oggetto». Insieme impongono una biiezione, cioè valori tutti distinti. L'espansione binaria è la rappresentazione in base 2, unica per ogni intero in quell'intervallo: la corrispondenza fra v e $(b_0, \dots, b_{p-1})$ è biunivoca.

L'alldiff ha rilassamento esatto, l'espansione no

La matrice del doppio partitioning è quella del problema di assegnamento: è totalmente unimodulare, quindi tutti i vertici del rilassamento sono interi e $z(LP^+) = z(MILP)$. Sull'istanza 3×3 dello script entrambi valgono 7: l'interezza è gratis. L'espansione binaria, al contrario, non aggiunge forza: $\sum_k 2^k b_k$ con $b_k \in [0, 1]$ copre tutto $[0, 2^p - 1]$ in modo continuo, esattamente come $v \geq 0$, $v \leq 2^p - 1$. Serve a *reformulare*, non a rafforzare — per esempio quando un'altra parte del modello ha bisogno di indicatori binari e non di una variabile intera.

(d) **L'esempio.** $v \in \{0, \dots, 7\}$ con $v \geq 5$, $\min v$: il modello dà $v = 5 = 1 + 4$, cioè $(b_0, b_1, b_2) = (1, 0, 1)$.

(e) In gurobiipy, e dove si rivede.

```
1 m.addConstrs((p.sum(i, "*") == 1 for i in range(n)), name="un_valore")
2 m.addConstrs((p.sum("v", v) == 1 for v in range(n)), name="alldiff")
3 m.addConstr(v == gp.quicksum(2 ** k * b[k] for k in range(p)), name="
   espansione")
```

L'alldiff si rivede nel modello numerico delle regine (EX 9) e nell'orario della scuola di musica (EX 15).

3.13 Vincoli violabili, deviazioni e penalità

(a) **Il legame in parole.** Un vincolo che si *preferirebbe* rispettare, ma che può essere violato pagando: la domanda di un periodo, una preferenza di orario, un obiettivo di servizio. Trasformare un vincolo rigido in uno morbido è spesso la differenza fra un modello inammissibile e uno utile.

(b) **I vincoli.** Al posto di $a'x = \beta$ si scrive

$$\begin{aligned} a'x + s^- - s^+ &= \beta, & (1 \text{ vincolo}), \\ s^- &\geq 0, & (1 \text{ variabile continua}), \\ s^+ &\geq 0, & (1 \text{ variabile continua}), \end{aligned} \quad (3.25)$$

e nell'obiettivo si aggiunge $\pi^- s^- + \pi^+ s^+$ con penalità $\pi^-, \pi^+ > 0$. La variabile s^- misura di quanto si sta *sotto* il target, s^+ di quanto si sta *sopra*.

(c) **La dimostrazione.** Il vincolo, da solo, è sempre soddisfacibile: qualunque $a'x$ si può compensare con una delle due deviazioni. La proprietà interessante è che **in ogni ottimo almeno una delle due deviazioni è nulla**: se $s^- = \sigma > 0$ e $s^+ = \tau > 0$, sottrarre $\min(\sigma, \tau)$ da entrambe lascia (3.25) soddisfatto (la differenza $s^- - s^+$ non cambia) e riduce l'obiettivo di $(\pi^- + \pi^+) \min(\sigma, \tau) > 0$. Quindi s^- e s^+ sono davvero le parti positiva e negativa dello scarto, e $s^- + s^+ = |a'x - \beta|$ in ogni ottimo: la stessa cosa della sezione 3.7, scritta con un'uguaglianza invece che con due disuguaglianze.

Se $\pi^- = 0$, l'argomento di scambio non è più stretto e una soluzione ottima può avere entrambe le deviazioni positive: la lettura « s^- è la sotto-copertura» non è più garantita. Vale la stessa regola di tutto il capitolo: la forza della conclusione dipende dal segno del coefficiente nell'obiettivo.

(d) **L'esempio.** Domanda 6 in ciascuno di tre periodi, disponibilità totale $15 < 18$, penalità $\pi^+ = 3$ e $\pi^- = 2$. Il modello rigido sarebbe inammissibile; quello morbido dà $z(MILP) = 6$, con $q = (3, 6, 6)$ e una sotto-copertura di 3 concentrata sul primo periodo. Nulla, nei dati, dice che vada concentrata: qualunque ripartizione della sotto-copertura totale 3 ha lo stesso costo, e il solver ne restituisce una.

(e) **In gurobipy, e dove si rivede.**

```
1 sm = m.addVars(T, name="s_meno"); sp = m.addVars(T, name="s_piu")
2 m.addConstrs((q[t] + sm[t] - sp[t] == domanda[t] for t in range(T)),
3              name="target")
4 m.setObjective(costo + gp.quicksum(pen_giu * sm[t] + pen_su * sp[t] for
5                                   t in range(T)),
6              GRB.MINIMIZE)
```

Si rivede nel modello numerico dell'orario della scuola di musica (EX 15) e nell'esercizio 9.1.

3.14 Funzioni lineari a tratti

(a) **Il legame in parole.** Il costo non è proporzionale alla quantità: cambia a scaglioni — sconti sopra una soglia, sovrapprezzi oltre una capacità, tariffe a fasce. La funzione $g(q)$ è continua e lineare a tratti, con nodi $q_0 < q_1 < \dots < q_K$ e valori $g_0, \dots, g_K$.

(b) **I vincoli.** Con $\lambda_k \geq 0$ i pesi della combinazione convessa e $w_t \in \{0, 1\}$ l'indicatore del tratto t (fra q_{t-1} e q_t):

$$\sum_{k=0}^K \lambda_k = 1, \quad (1 \text{ vincolo}), \quad (3.26)$$

$$q = \sum_{k=0}^K q_k \lambda_k, \quad (1 \text{ vincolo}), \quad (3.27)$$

$$g(q) = \sum_{k=0}^K g_k \lambda_k, \quad (1 \text{ vincolo}), \quad (3.28)$$

$$\sum_{t=1}^K w_t = 1, \quad (1 \text{ vincolo}), \quad (3.29)$$

$$\lambda_k \leq \sum_{t: k \in \{t-1, t\}} w_t, \quad \forall k \in \{0, 1, \dots, K\} \quad (K+1 \text{ vincoli}). \quad (3.30)$$

(c) **La dimostrazione.** Il vincolo (3.30) è l'**adiacenza**: λ_k può essere positiva solo se il tratto scelto ha q_k come estremo. Con (3.29) il tratto è uno solo, quindi al più due λ consecutive sono positive, e allora $(q, g(q))$ è un punto del segmento fra (q_{t-1}, g_{t-1}) e (q_t, g_t) : cioè un punto *del grafico*. Senza l'adiacenza si potrebbero mescolare nodi non consecutivi, e (q, g) finirebbe nell'**inviluppo convesso inferiore** del grafico — sotto la funzione.

Quando g è convessa e si *minimizza*, l'inviluppo convesso inferiore coincide con la funzione e le λ ottime risultano automaticamente adiacenti: il modello senza binarie è già corretto. Quando g non è convessa — il caso degli sconti di quantità, in cui il costo marginale *scende* — la differenza si vede subito. Con nodi $(0, 4, 10, 16)$ e valori $(0, 12, 30, 36)$ e domanda $q \geq 13$:

Formulazione	valore ottimo
senza adiacenza (combinazione convessa libera)	$117/4 = 29,25$
con adiacenza (3.30)	33

La prima mescola i nodi 0 e 3 (le uniche λ non nulle) e restituisce un costo che la funzione *non assume in nessun punto*: è il valore dell'inviluppo, non del grafico. La seconda mescola i nodi 2 e 3, adiacenti, e dà il valore esatto $g(13) = 30 + 6 \cdot \frac{3}{6} = 33$.

(d) **La forza del rilassamento.** Attenzione a non confondere due cose: l'adiacenza cambia l'*insieme intero* (i due modelli hanno ottimi diversi, 29,25 e 33), ma non la forza del rilassamento — con w_t frazionarie il vincolo (3.30) non morde e i due modelli hanno lo stesso $z(LP^+) = 117/4$.

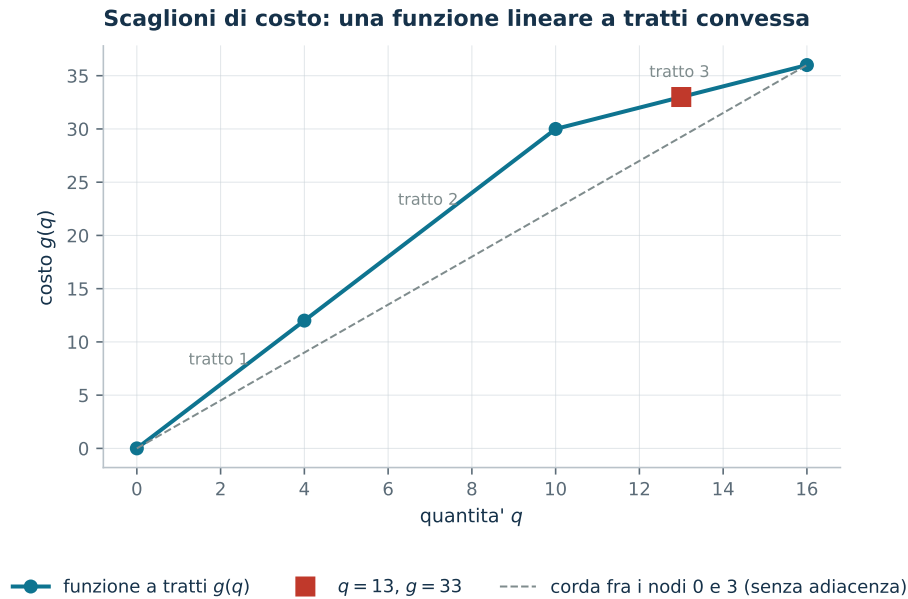


Figura 3.2: La funzione a tratti con sconto di quantità. La corda tratteggiata fra i nodi estremi è ciò che la formulazione senza adiacenza permette di raggiungere: sta *sotto* il grafico.

(e) In gurobipy, e dove si rivede.

```

1 m.addConstr(lam.sum() == 1, name="convessa")
2 m.addConstr(q == gp.quicksum(nodi[k] * lam[k] for k in range(K + 1)),
   name="ascissa")
3 m.addConstr(w.sum() == 1, name="un_tratto")
4 for k in range(K + 1):                                     # adiacenza: lambda_k solo
   sui tratti che toccano k
5     m.addConstr(lam[k] <= gp.quicksum(w[t] for t in (k - 1, k) if 0 <=
       t < K),
6               name=f"adiacenza{k}")
  
```

Gurobi offre anche i vincoli `addGenConstrPWL` e i tipi `SOS2`, che fanno lo stesso lavoro internamente; qui la formulazione manuale resta il materiale principale, perché è quella di cui si deve saper dimostrare la correttezza. Si rivede nell'esercizio 10.1 (premi con due modalità).

3.15 La mappa delle tecniche

La tabella seguente è pensata per la consultazione: si legge dalla prima colonna («che cosa voglio dire») alla seconda («come si scrive»), e le ultime due ricordano che cosa va dichiarato e dove la tecnica si rivede all'opera.

Tecnica	Formulazione	Da dichiarare	Si rivede in
3.1 attivazione	$x_{ij} \leq y_j$ (disagg.) oppure $\sum_i x_{ij} \leq k_j y_j$ (agg.)	quale forma, e perché; segno di f_j per il verso di ottimalità	7.2, 7.3, 7.5, 8.4
3.2 costo fisso	$q_j \leq c_j y_j$	che c_j è la capacità, non un big-M	8.1, cap. 9
3.3 lotto minimo	$\ell y_j \leq q_j \leq c_j y_j$	che $\ell \leq c_j$; che il rilassamento non ne risente	7.2.2, 9.1, 9.3
3.4 conteggio intero	$\sum_i a_i x_i \leq Kw$, w intera	che l'interrezza realizza il tetto	9.2, 10.4, 10.5
3.5 ausiliaria di massimo	$z \geq t_j x_j$, $z \geq 0$	che z non compare altrove; segno nell'obiettivo	7.4, 7.7, 8.4, 10.9
3.6 min-max / max-min	$T \geq L_k$ e $\min T$; $U \leq L_k$ e $\max U$	quale dei tre obiettivi, e che non si confrontano	7.4.1, 10.7, 10.8
3.7 valore assoluto	$d \geq \pm(u - v)$ in obiettivo; disgiunzione se $\geq k$	se è obiettivo o vincolo, e in quale verso	10.7, 10.8
3.8 big-M	$a'x \leq b + M(1 - y)$	il valore di M calcolato dai dati	7.7, 3.9
3.9 precedenze	$s_{ij} + s_{ji} = 1$, $\kappa_i \geq \kappa_j + t_i - M(1 - s_{ij})$	l'orizzonte e $M = \sum_h t_h$	7.7
3.10 se e solo se	$y \leq x_j$ e $y \geq \sum_j x_j - (p - 1)$	se il secondo verso serve o segue dall'ottimalità	7.6, 9.3
3.11 conteggio dei tipi	$\ell y_j \leq q_j \leq c_j y_j$, $\sum_j y_j \geq p$	che senza la soglia ℓ il conteggio è vuoto	9.3, 10.3, 10.4
3.12 alldiff / espansione	doppio partitioning; $v = \sum_k 2^k b_k$	che l'alldiff ha rilassamento esatto	EX 9, EX 15
3.13 vincoli violabili	$a'x + s^- - s^+ = \beta$ con penalità	i due segni delle penalità	9.1, EX 15
3.14 funzione a tratti	combinazione convessa + adiacenza	se g è convessa; altrimenti l'adiacenza è obbligatoria	10.1

Le tre domande da farsi davanti a un legame nuovo

1. **Quale verso è imposto dai vincoli e quale no?** Si risponde guardando solo i vincoli, un caso per valore della binaria.
2. **Il verso mancante segue dall'ottimalità?** Si risponde con l'argomento di scambio, e la risposta dipende dal *segno* del coefficiente nell'obiettivo: strettamente diverso da zero dà «in ogni ottimo», nullo dà al più «esiste un ottimo».
3. **Quanto è forte il rilassamento?** Si risponde confrontando $z(LP^+)$ con $z(MILP)$ su un'istanza piccola, e — quando ci sono due formulazioni — confrontandole fra loro. Le due formulazioni vanno prima dimostrate equivalenti sui punti interi: senza quel passaggio non si stanno confrontando due modelli dello stesso problema.

Rilassamenti, dualità e bound

Script: `python/cap04_bound.py`

[Pagina online](#)

Questo capitolo insegna a produrre, *a mano*, un numero che sta certamente da una parte dell'ottimo intero. Serve a tre cose: capire quanto vale un modello, capire quanto vale un'euristica, e leggere con giudizio i numeri che un solver riporta quando non ha finito.

4.1 Che cos'è un rilassamento

Definizione operativa

Un **rilassamento** di un problema di minimo con insieme ammissibile X è un problema con lo stesso obiettivo e un insieme ammissibile più grande, $X \subseteq \hat{X}$: l'unica cosa che si rilassa sono i vincoli. Vale allora

$$\min_{x \in \hat{X}} \sum_{j=1}^n c_j x_j \leq \min_{x \in X} \sum_{j=1}^n c_j x_j,$$

perché il minimo su un insieme più grande non può essere più alto. In un massimo la disuguaglianza si rovescia.

I tre rilassamenti che il corso usa, in ordine di forza crescente:

Nome	Che cosa si toglie	Nota
$z(LP)$, rilassamento senza i bound	$x \in \{0, 1\}$ diventa $x \geq 0$	è quello di cui si scrive il duale a mano: ha meno vincoli e quindi un duale con meno variabili
$z(LP^+)$, con i bound	$x \in \{0, 1\}$ diventa $0 \leq x \leq 1$	è <code>relax()</code> di Gurobi, il primo rilassamento che il solver risolve
$z(LP^{++})$, con i bound e le disuguaglianze valide	come sopra, più disuguaglianze valide	la sezione 4.7 spiega quali

In un problema di minimo $z(LP) \leq z(LP^+) \leq z(LP^{++}) \leq z(MILP)$.

Se il modello contiene un vincolo di assegnamento $\sum_m x_{jm} = 1$ con $x \geq 0$, allora $x_{jm} \leq 1$ è già implicato e i due rilassamenti sono *uguali*. Nella tabella dei bound del capitolo 7 succede nei problemi 1, 4, 6 e 7. Quando invece i due differiscono, la differenza va riportata:

è un'informazione sul modello.

4.2 La tabella di conversione primale/duale

Da un primale di minimo al suo duale

Sia il primale un minimo, con m vincoli indicizzati da i e n variabili indicizzate da j .

Nel primale (min)	Nel duale (max)
vincolo i di verso $\geq$	variabile $\pi_i \geq 0$
vincolo i di verso $\leq$	variabile $\pi_i \leq 0$
vincolo i di uguaglianza	variabile π_i libera
variabile $x_j \geq 0$	vincolo j di verso $\leq c_j$
variabile x_j libera	vincolo j di uguaglianza $= c_j$

L'obiettivo duale è $\max \sum_i b_i \pi_i$ e il vincolo duale j è $\sum_i a_{ij} \pi_i \leq c_j$ con il verso dato dalla tabella. Se il primale è di **massimo**, tutti i versi della tabella si rovesciano e il duale è di minimo.

Come si legge il duale

Il vincolo duale j dice: «il valore che attribuisco alle risorse consumate dall'attività j non può superare il suo costo». Con questa lettura, ogni ricetta per costruire una soluzione duale ammissibile ha un significato economico, ed è quello che la rende ricordabile: la sezione 4.4 ne dà tre.

4.3 Dualità debole, dualità forte

I due teoremi, e a che cosa servono qui

Siano $(\bar{x}_1, \dots, \bar{x}_n)$ una soluzione ammissibile per un modello di LP di minimo e $(\bar{\pi}_1, \dots, \bar{\pi}_m)$ una soluzione ammissibile per il suo duale.

- **Dualità debole:** $\sum_{i=1}^m b_i \bar{\pi}_i \leq \sum_{j=1}^n c_j \bar{x}_j$. *Sempre*, senza ipotesi. È questa che ci serve: dà un lower bound da *qualunque* soluzione duale ammissibile, anche costruita a mano e lontana dall'ottimo.
- **Dualità forte:** se l'LP ha ottimo finito, allora $z(D(LP)) = z(LP)$. Serve come *controllo*: l'ottimo del duale scritto a mano deve coincidere con $z(LP)$; se non coincide, il duale è scritto male. Lo script del corso lo verifica con un **assert** in `due_rilassamenti`.

E poi il passaggio all'intero, che è banale ma va detto: siccome ogni soluzione ammissibile del MILP è ammissibile anche per il rilassamento,

$$\sum_{i=1}^m b_i \bar{\pi}_i \leq z(LP) \leq z(MILP).$$

Un bound duale dell'LP è un bound anche per il MILP, dallo stesso lato.

Il duale che si scrive è quello del *rilassamento*, non del MILP. Il MILP non ha un duale lineare, e la dualità forte fra MILP e un qualsiasi programma lineare in generale non vale: il salto $z(MILP) - z(LP)$ è precisamente ciò che manca. Quando in questo corso si dice «il duale», si intende sempre il duale del rilassamento indicato.

4.4 Tre ricette per costruire a mano una soluzione duale

Le ricette

1. **Azzerare e saturare.** Si pongono a zero tutte le variabili duali tranne una famiglia, e si portano quelle al valore più grande (in un duale di massimo) compatibile con i vincoli duali. Esempio: nel problema 7.1 si pone $\bar{\pi} = 0$ e si ottiene $\bar{\mu}_j = \min_m c_{jm}$, cioè «ogni lavoro costa almeno il suo costo minimo».
2. **Euristica costruttiva sui vincoli.** Si scorrono i vincoli primali uno alla volta, si alza la variabile duale corrispondente fino a saturare il primo vincolo duale che si oppone, e si aggiornano i residui. È la ricetta usata qui sotto per la copertura.
3. **Il rapporto migliore.** Con un solo vincolo di capacità in un massimo, la variabile duale $\bar{v} = \max_j p_j/w_j$ è ammissibile, e dà il bound $b\bar{v}$: «tutta la capacità impiegata all'efficienza migliore».

Qualunque ricetta si usi, la soluzione costruita va *verificata* ammissibile per il duale — è l'unica cosa che rende valido il bound — e il suo valore va confrontato con $z(LP)$ per vedere quanto si è lasciato sul tavolo.

4.5 Un problema di minimo, per esteso

Copertura di zone a costo minimo

Un comune ha $n \in \mathbb{Z}_{\geq 1}$ distretti; ciascuno ha una squadra di pronto intervento, e attivare la squadra del distretto $j \in \{1, 2, \dots, n\}$ costa $c_j \in \mathbb{Q}_{>0}$ migliaia di euro. Ci sono $m \in \mathbb{Z}_{\geq 1}$ zone sensibili: la zona $i \in \{1, 2, \dots, m\}$ è coperta se almeno una delle squadre dell'insieme S_i dei distretti che la confinano è attiva. Si vogliono coprire tutte le zone a costo minimo.

I dati dell'istanza sono $n = 4$ distretti, di costi

$$c_1 = 4, \quad c_2 = 3, \quad c_3 = 5, \quad c_4 = 3,$$

e $m = 6$ zone, che corrispondono alle sei coppie di distretti confinanti:

zona i	1	2	3	4	5	6
squadre che la coprono, S_i	$\{1, 2\}$	$\{2, 3\}$	$\{1, 3\}$	$\{1, 4\}$	$\{2, 4\}$	$\{3, 4\}$

Si introducono le seguenti n variabili binarie:

$$x_j = \begin{cases} 1 & \text{se la squadra del distretto } j \text{ viene attivata,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, \dots, n\}.$$

Copertura a costo minimo

$$\min \sum_{j=1}^n c_j x_j \quad (4.1a)$$

$$\text{soggetto a } \sum_{j \in S_i} x_j \geq 1, \quad \forall i \in \{1, 2, \dots, m\}, \quad (4.1b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}. \quad (4.1c)$$

Descrizione. L'obiettivo (4.1a) somma il costo delle squadre attivate. Gli m vincoli (4.1b), uno per zona, dicono che almeno una delle due squadre confinanti con quella zona è attiva: è la struttura del *set covering*. I domini (4.1c) sono le n condizioni di binarietà, una per squadra.

Il duale del rilassamento senza i bound. Rilassando (4.1c) a $x_j \geq 0$ e associando a ciascun vincolo (4.1b) una variabile duale $\pi_i \geq 0$ (verso $\geq$ in un minimo, per la tabella di conversione):

Duale del rilassamento LP

$$\max \sum_{i=1}^m \pi_i \quad (4.2a)$$

$$\text{soggetto a } \sum_{i: j \in S_i} \pi_i \leq c_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (4.2b)$$

$$\pi_i \geq 0, \quad \forall i \in \{1, 2, \dots, m\}. \quad (4.2c)$$

Descrizione. La variabile π_i è il prezzo che si attribuisce alla zona i . L'obiettivo (4.2a) somma i prezzi delle m zone. I vincoli (4.2b) sono le colonne delle x_j , uno per squadra: le zone che la squadra j copre, valutate a quei prezzi, non possono valere più di quanto costa attivarla, cioè c_j . È la lettura economica della sezione precedente: nessuna squadra deve risultare un affare.

Scritti per esteso sui dati dell'istanza, il primale e il duale sono i seguenti.

Copertura sull'istanza — primale

$$\begin{aligned} \min \quad & 4x_1 + 3x_2 + 5x_3 + 3x_4 \\ & x_1 + x_2 \geq 1 \\ & x_2 + x_3 \geq 1 \\ & x_1 + x_3 \geq 1 \\ & x_1 + x_4 \geq 1 \\ & x_2 + x_4 \geq 1 \\ & x_3 + x_4 \geq 1 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{aligned}$$

Copertura sull'istanza — duale del rilassamento LP

$$\begin{aligned} \max \quad & \pi_1 + \pi_2 + \pi_3 + \pi_4 + \pi_5 + \pi_6 \\ & \pi_1 + \pi_3 + \pi_4 \leq 4 \\ & \pi_1 + \pi_2 + \pi_5 \leq 3 \\ & \pi_2 + \pi_3 + \pi_6 \leq 5 \\ & \pi_4 + \pi_5 + \pi_6 \leq 3 \\ & \pi_1, \pi_2, \pi_3, \pi_4, \pi_5, \pi_6 \geq 0 \end{aligned}$$

Ogni squadra copre tre zone, e infatti ogni vincolo duale ha tre termini: la colonna della variabile x_j nel primale diventa il vincolo j -esimo del duale.

Una soluzione duale a mano (ricetta 2). Si scorrono le zone in ordine; per la zona i si alza π_i del massimo consentito dai residui dei vincoli duali delle squadre in S_i , e si aggiornano i residui.

- **Zona 1** ($S_1 = \{1, 2\}$): residui (4, 3, 5, 3), il minimo fra quelli delle squadre 1 e 2 è 3. Si pone $\bar{\pi}_1 = 3$; residui (1, 0, 5, 3).
- **Zona 2** ($\{2, 3\}$): il residuo della squadra 2 è 0, quindi $\bar{\pi}_2 = 0$.
- **Zona 3** ($\{1, 3\}$): minimo fra 1 e 5, cioè 1. Si pone $\bar{\pi}_3 = 1$; residui (0, 0, 4, 3).
- **Zone 4 e 5** ($\{1, 4\}$ e $\{2, 4\}$): le squadre 1 e 2 hanno residuo nullo, quindi $\bar{\pi}_4 = \bar{\pi}_5 = 0$.
- **Zona 6** ($\{3, 4\}$): minimo fra 4 e 3, cioè 3. Si pone $\bar{\pi}_6 = 3$; residui (0, 0, 1, 0).

Tutti i vincoli duali sono soddisfatti per costruzione, e il valore è

$$LB = \bar{\pi}_1 + \dots + \bar{\pi}_6 = 3 + 0 + 1 + 0 + 0 + 3 = 7.$$

Un upper bound primale. L'euristica euristica costruttiva di copertura sceglie ogni volta la squadra col miglior rapporto fra costo e zone ancora scoperte: prende le squadre 1, 2 e 4, di costo $4 + 3 + 3 = 10$. La soluzione è ammissibile e *intera*, quindi $UB = 10$.

Il sandwich, e cosa dice il solver. $z(LP) = z(LP^+) = 15/2$ (i due rilassamenti coincidono, e l'ottimo del duale li conferma: dualità forte) e $z(MILP) = 10$, ottenuto con le squadre 1, 2, 4 — la stessa soluzione dell'euristica, che qui è ottima.

UB (euristica costruttiva)	LB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
10	7	15/2	10	0,0%

Da leggere così: il bound a mano vale 7, il rilassamento 7,5, l'ottimo 10. Il divario certificato fra i nostri due bound costruiti a mano è $(10 - 7)/10 = 30\%$: senza risolvere il MILP sapremmo soltanto che l'ottimo sta fra 7 e 10. L'euristica era già ottima, ma non lo si può sapere dai bound.

4.6 Un problema di massimo: i ruoli si scambiano

Zaino

Ci sono $n \in \mathbb{Z}_{\geq 1}$ oggetti: l'oggetto $j \in \{1, 2, \dots, n\}$ ha valore $p_j \in \mathbb{Q}_{>0}$ e peso $w_j \in \mathbb{Q}_{>0}$, e lo zaino ha capacità $b \in \mathbb{Q}_{>0}$. Si scelga il sottoinsieme di valore massimo che ci sta.

I dati dell'istanza sono $n = 4$ oggetti, di valore $p = (10, 7, 6, 4)$ e peso $w = (5, 4, 3, 3)$, e uno zaino di capacità $b = 9$.

Si introducono le seguenti n variabili binarie:

$$x_j = \begin{cases} 1 & \text{se l'oggetto } j \text{ viene messo nello zaino,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, \dots, n\}.$$

Zaino binario

$$\max \sum_{j=1}^n p_j x_j \quad (4.3a)$$

$$\text{soggetto a } \sum_{j=1}^n w_j x_j \leq b, \quad (4.3b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}. \quad (4.3c)$$

Descrizione. L'obiettivo (4.3a) somma il valore degli oggetti scelti. Il vincolo (4.3b) è la capacità dello zaino: è uno solo, e dice che il peso complessivo degli oggetti scelti non supera b . I domini (4.3c) sono le n condizioni di binarietà, una per oggetto: un oggetto o si prende tutto, o non si prende.

Il duale del rilassamento senza i bound. Si rilassa il dominio (4.3c) a $x_j \geq 0$. Il primale è di massimo e ha un solo vincolo, di verso $\leq$: la tabella di conversione dà una sola variabile duale $v \geq 0$, associata alla capacità, e un vincolo duale per ciascuna delle n variabili primali.

Duale del rilassamento LP del modello dello zaino

$$\min \quad b v \quad (4.4a)$$

$$\text{soggetto a} \quad w_j v \geq p_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (4.4b)$$

$$v \geq 0. \quad (4.4c)$$

Descrizione. La variabile v è il prezzo di una unità di capacità. L'obiettivo (4.4a) è il valore attribuito a tutta la capacità disponibile. Gli n vincoli (4.4b), uno per oggetto, dicono che la capacità consumata dall'oggetto j , valutata al prezzo v , non vale meno del profitto che quell'oggetto porta.

Scritti per esteso sui dati dell'istanza, il primale e il duale sono i seguenti.

Zaino sull'istanza — primale

$$\begin{aligned} \max \quad & 10x_1 + 7x_2 + 6x_3 + 4x_4 \\ & 5x_1 + 4x_2 + 3x_3 + 3x_4 \leq 9 \\ & x_1, \quad x_2, \quad x_3, \quad x_4 \in \{0, 1\} \end{aligned}$$

Zaino sull'istanza — duale del rilassamento LP

$$\begin{aligned} \min \quad & 9v \\ & 5v \geq 10 \\ & 4v \geq 7 \\ & 3v \geq 6 \\ & 3v \geq 4 \\ & v \geq 0 \end{aligned}$$

Ogni vincolo duale ha un solo termine, perché ogni oggetto consuma una sola risorsa: la colonna della variabile x_j nel primale ha una sola riga.

I due bound.

- **Euristica** (euristica costruttiva per rapporto p_j/w_j): i rapporti sono 2, 7/4, 2, 4/3; si prendono l'oggetto 1 ($w = 5$) e poi il 3 ($w = 3$), per un peso di $8 \leq 9$; l'oggetto 2 non ci sta più. Valore $10 + 6 = 16$. In un **massimo** l'euristica dà un **lower** bound: $LB = 16 \leq z(MILP)$.
- **Duale a mano** (ricetta 3): $\bar{v} = \max_j p_j/w_j = 2$ soddisfa $w_j \bar{v} \geq p_j$ per ogni j , quindi è ammissibile, e il suo valore è $b\bar{v} = 9 \cdot 2 = 18$. In un **massimo** il duale dà un **upper** bound: $z(MILP) \leq UB = 18$.

Il sandwich. $z(MILP) = 17$ (oggetti 1 e 2, peso esattamente 9), $z(LP) = 18$ e $z(LP^+) = 71/4 = 17,75$:

$$16 \leq z(MILP) = 17 \leq z(LP^+) = \frac{71}{4} \leq z(LP) = 18 = UB.$$

LB (euristica costruttiva)	$z(MILP)$	$z(LP^+)$	$z(LP)$	UB (duale a mano)	gap euristica
16	17	71/4	18	18	5,9%

Qui il duale a mano è *ottimo* per il rilassamento senza i bound ($UB = z(LP) = 18$), e il rilassamento con i bound è strettamente migliore ($71/4 < 18$): il vincolo $x_j \leq 1$ morde, perché senza di esso l'LP prende 9/5 unità dell'oggetto 1.

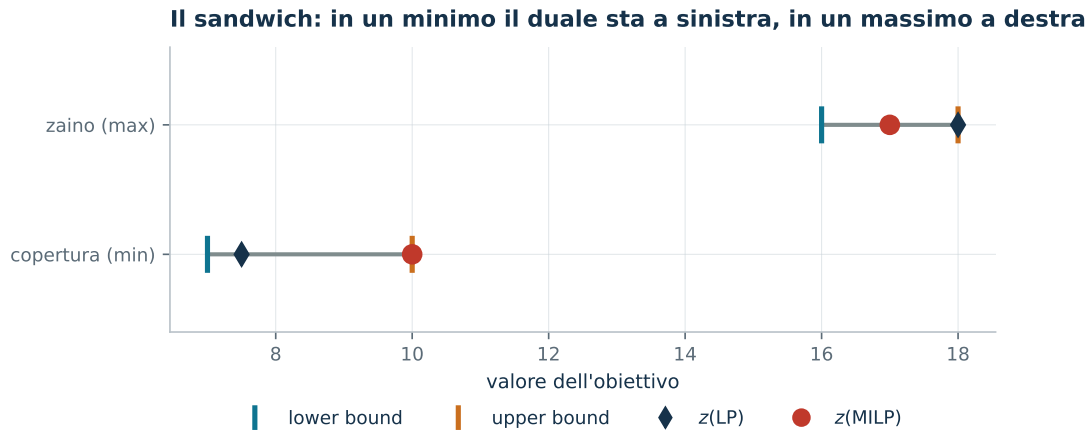


Figura 4.1: I due esempi. In un minimo il bound duale sta a sinistra dell'ottimo e l'euristica a destra; in un massimo i ruoli si scambiano.

Il sandwich scritto una volta per tutte

minimo: $LB(\bar{\pi}) \leq z(D(LP)) = z(LP) \leq z(LP^+) \leq z(MILP) \leq UB(\bar{x})$

massimo: $LB(\bar{x}) \leq z(MILP) \leq z(LP^+) \leq z(LP) = z(D(LP)) \leq UB(\bar{\pi})$

dove $(\bar{\pi}_1, \bar{\pi}_2, \dots, \bar{\pi}_m)$ è una soluzione duale ammissibile del rilassamento, di valore $\sum_{i=1}^m b_i \bar{\pi}_i$, e $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ una soluzione ammissibile del MILP, di valore $\sum_{j=1}^n c_j \bar{x}_j$. Le catene si leggono così: il *lato del rilassamento* è ottimistico e contiene tutti i bound duali; il *lato dell'euristica* è pessimistico e contiene tutte le soluzioni ammissibili. Il nome (LB o UB) dipende dal verso dell'obiettivo, il ruolo no.

4.7 Disuguaglianze valide e vincoli che preservano l'ottimalità

Due nozioni diverse

- Una **disuguaglianza valida** per un MILP è soddisfatta da *tutte* le soluzioni ammissibili intere. Aggiungerla non cambia $z(MILP)$; può ridurre $z(LP^+)$, e allora si chiama **taglio**.
- Un **vincolo che preserva l'ottimalità** taglia alcune soluzioni ammissibili, ma non

tutte quelle ottime: dopo l'aggiunta esiste ancora un ottimo, e $z(MILP)$ non cambia. Non è una disuguaglianza valida, e va dichiarato come tale. Esempio: $z_j \leq M_j y_j$ nel problema 8.4.

La differenza conta: la prima si può aggiungere sempre; la seconda soltanto se si è dimostrato che almeno un ottimo sopravvive, e mai se poi si vuole enumerare *tutte* le soluzioni ottime.

Il taglio di copertura per lo zaino

Un insieme S di oggetti è una **copertura** se $\sum_{j \in S} w_j > b$: non ci stanno tutti insieme. Allora

$$\sum_{j \in S} x_j \leq |S| - 1$$

è una disuguaglianza valida. La copertura è **minimale** se nessun suo sottoinsieme proprio è già una copertura; i tagli delle coperture minimali sono i più forti.

Sull'istanza dello zaino ($w = (5, 4, 3, 3)$, $b = 9$) le coperture minimali sono le quattro terne $\{1, 2, 3\}$, $\{1, 2, 4\}$, $\{1, 3, 4\}$, $\{2, 3, 4\}$, ciascuna di peso 12, 12, 11, 10: tutte > 9 , e nessuna coppia lo è. I quattro tagli sono $\sum_{j \in S} x_j \leq 2$.

La soluzione ottima del rilassamento $z(LP^+) = 71/4$ è $\tilde{x} = (1, 1/4, 1, 0)$. Su di essa:

Copertura S	$\sum_{j \in S} \tilde{x}_j$	$ S - 1$	
$\{1, 2, 3\}$	9/4	2	violato: il taglio serve
$\{1, 2, 4\}$	5/4	2	soddisfatto
$\{1, 3, 4\}$	2	2	soddisfatto (all'uguaglianza)
$\{2, 3, 4\}$	5/4	2	soddisfatto

Aggiungendo tutti e quattro i tagli, $z(LP^+)$ scende da $71/4 = 17,75$ a $69/4 = 17,25$, e $z(MILP)$ resta 17: il rilassamento si è avvicinato all'ottimo intero senza cambiarlo. È il meccanismo con cui un solver moderno chiude i gap.

4.8 Formulazioni più forti: che cosa vuol dire, e come si dimostra

Il criterio

Due formulazioni A e B dello stesso problema si confrontano in due passi.

1. **Stesso insieme intero:** $X_A \cap \mathbb{Z}^n = X_B \cap \mathbb{Z}^n$. Senza questo passaggio non si sta confrontando nulla. Si dimostra nei due versi, usando la binarietà o l'interezza dove serve.
2. **Rilassamento:** B è *più forte* di A se $X_B \subseteq X_A$ come poliedri, e allora $z_B(LP^+)$ è almeno buono quanto $z_A(LP^+)$. Se l'inclusione è stretta e si esibisce un punto di $X_A \setminus X_B$ ottimo per il rilassamento di A , il miglioramento è dimostrato su quell'istanza.

Il caso di riferimento è l'attivazione della sezione 3.1: $x_{ij} \leq y_j$ implica $\sum_i x_{ij} \leq n y_j$, quindi il disaggregato è più forte; su tre clienti e due sedi il rilassamento passa da $38/3$ a 15, che è già l'ottimo intero.

Una formulazione più forte ha meno nodi ma righe in più, e ogni nodo costa di più. Il bilancio dipende dall'istanza. Quello che si può *dimostrare* è la forza del rilassamento; la velocità si *misura*.

4.9 Quello che dice il solver

I quattro attributi da leggere, e in quale ordine

1. **Status**: 2 è OPTIMAL, 3 INFEASIBLE, 9 TIME_LIMIT. Va letto *prima* di ogni altra cosa.
2. **SolCount**: quante soluzioni intere sono state trovate. Se è 0, **ObjVal** non esiste e leggerlo è un errore.
3. **ObjVal** e **ObjBound**: la migliore soluzione trovata e il miglior bound dimostrato. In un minimo, $\text{ObjBound} \leq z(\text{MILP}) \leq \text{ObjVal}$.
4. **MIPGap** e **NodeCount**: il divario certificato fra i due e quanti nodi sono stati esplorati.

Sull'istanza di copertura, il rilassamento del modello *come lo abbiamo scritto* vale $15/2$ e l'ottimo intero 10. Eppure Gurobi riporta **ObjBound** = 10 e **NodeCount** = 0: ha chiuso il gap nella radice, con presolve, tagli propri ed euristiche, senza mai ramificare. Spegnerdoli (**Presolve** = **Cuts** = **Heuristics** = 0) lo stesso modello dà lo stesso ottimo 10 ma con 5 nodi. Due conseguenze:

- «quanto è difficile un modello» non è una proprietà del solo modello: dipende anche da che cosa il solver mette in campo;
- il rilassamento di cui parliamo nei bound a mano è quello del modello scritto, e va ottenuto con **relax()**: leggere **ObjBound** a fine risoluzione dà l'ottimo, non il rilassamento.

4.10 I duali dell'LP non sono i prezzi marginali del MILP

L'affermazione, e il controesempio

In un LP, il duale $\tilde{\pi}_i$ di un vincolo di risorsa è il suo *prezzo marginale*: aumentando b_i di una piccola quantità, l'ottimo varia di $\tilde{\pi}_i$ per unità (finché la base non cambia). **Per un MILP questo è falso**, e non per approssimazione.

Sullo zaino, al variare della capacità b :

b	$z(MILP)$	$z(LP^+)$	duale dell'LP	variazione vera di $z(MILP)$
8	16	16	2	—
9	17	71/4	7/4	+1
10	17	39/2	7/4	0
11	20	85/4	7/4	+3
12	23	23	7/4	+3

Il duale dell'LP è il rapporto p_j/w_j dell'oggetto «critico»: vale 2 quando la capacità si esaurisce sull'oggetto 1 e 7/4 quando avanza spazio per il 2. La variazione vera dell'ottimo intero è a scatti: passando da $b = 9$ a $b = 10$ non cambia *affatto* (nessun sottoinsieme sfrutta l'unità in più), mentre il duale promette 7/4; passando da 10 a 11 salta di 3.

Del duale dell'LP resta vero l'unico uso che questo corso ne fa: è un **bound**. $\tilde{\pi}_i$ dice quanto vale la risorsa *nel rilassamento*, e la somma $b'\tilde{u}$ è un bound valido per il MILP. Come indicazione gestionale («conviene comprare un'unità in più?») va invece verificata risolvendo di nuovo il MILP: la differenza $z(MILP)(b_i + 1) - z(MILP)(b_i)$ è l'unica risposta corretta, e non c'è una formula chiusa che la dia.

4.11 Il protocollo dei bound del corso

Che cosa produce ogni esercizio della Parte II

1. una soluzione **ammissibile e intera** da un'euristica costruttiva, verificata su vincoli, bound e interezza (il controllo di interezza è quello che rende il valore un bound del MILP e non del solo rilassamento);
2. il **duale del rilassamento senza i bound**, scritto in forma generale e per l'istanza, con i versi presi dalla tabella di conversione;
3. una **soluzione duale ammissibile costruita a mano**, con la ricetta dichiarata, verificata ammissibile e valutata;
4. i due **rilassamenti** $z(LP)$ e $z(LP^+)$ dal solver, con l'annotazione se coincidono e perché;
5. l'**ottimo** $z(MILP)$ e la tabella $UB \cdot LB \cdot z(LP) \cdot z(LP^+) \cdot z(MILP) \cdot \text{gap}$, con il gap definito come nel capitolo 1;
6. le **considerazioni aggiuntive**: disuguaglianze valide con il loro conteggio, vincoli che preservano l'ottimalità dichiarati come tali, confronto dei rilassamenti fra formulazioni alternative.

Ogni numero della tabella esiste in un CSV prodotto dallo script del problema, e un **assert** in `verifica_numeri.py` lo confronta con il valore citato nel testo.

Euristiche costruttive

5.1 Che cos'è un'euristica costruttiva

Un modello MILP si può risolvere all'ottimo, ma prima di risolverlo conviene avere in mano una soluzione ammissibile. Serve a due cose:

- dà un **bound primale**: un numero che l'ottimo non può superare (in un problema di minimo) o non può stare sotto (in un massimo);
- dà un punto di partenza al solver, e un termine di confronto per capire se il modello dice cose sensate.

Un'euristica costruttiva costruisce una soluzione aggiungendo un elemento per volta, senza mai tornare indietro. Non garantisce l'ottimo, e in generale non lo trova; e non garantisce nemmeno di arrivare a una soluzione ammissibile: può bloccarsi a metà, con un elemento che non entra da nessuna parte — proprio perché indietro non torna. Quello che garantisce è di fermarsi in fretta. Quando finisce con una soluzione ammissibile, il valore di quella soluzione è un bound primale; quando fallisce, bound primale non ce n'è.

I tre ingredienti

Ogni euristica costruttiva si descrive dicendo tre cose, e solo tre:

1. un **ordine** in cui si guardano gli elementi da collocare;
2. una **regola di inserimento**, che dice dove va l'elemento corrente;
3. un **controllo di ammissibilità**, che dice quando un inserimento non si può fare.

Cambiando uno solo dei tre si ottiene un'euristica diversa, con un valore diverso. Per questo, quando si presenta un'euristica, i tre ingredienti vanno scritti esplicitamente.

5.2 Le regole di inserimento in contenitori

La situazione più comune è: degli *oggetti* vanno messi in dei *contenitori* di capacità limitata. Sia n il numero di oggetti, indicizzati da $j \in \{1, 2, \dots, n\}$, e m il numero di contenitori, indicizzati da $i \in \{1, 2, \dots, m\}$; sia $w_j \in \mathbb{Q}_{>0}$ la dimensione dell'oggetto j e $c \in \mathbb{Q}_{>0}$ la capacità di ogni contenitore.

Tre regole classiche, in ordine di quanto guardano prima di decidere:

- **next-fit**: si tiene un contenitore «corrente». Se l'oggetto ci entra, ci va; altrimenti si apre il contenitore successivo e non si torna più su quelli precedenti.
- **first-fit**: si scorrono i contenitori dal primo e l'oggetto va nel primo in cui entra.
- **best-fit**: fra tutti i contenitori in cui l'oggetto entra si sceglie quello che lascia il residuo più piccolo.

Le tre regole su cinque oggetti

Cinque oggetti di dimensioni

$$w_1 = 4, \quad w_2 = 3, \quad w_3 = 5, \quad w_4 = 2, \quad w_5 = 4,$$

contenitori di capacità $c = 8$, oggetti guardati nell'ordine naturale.

regola	contenuto dei contenitori	contenitori usati
next-fit	{1, 2}, {3, 4}, {5}	3
first-fit	{1, 2}, {3, 4}, {5}	3
best-fit	{1, 2}, {3, 4}, {5}	3

Sulle dimensioni di questo esempio le tre regole coincidono. Non è sempre così: è proprio quando *non* coincidono che vale la pena calcolarle tutte e tre e tenere la migliore, cosa che costa poco.

Con n oggetti e m contenitori: next-fit guarda un contenitore per oggetto, first-fit al più m , best-fit tutti e m . I tre costi sono dunque

$$O(n), \quad O(nm), \quad O(nm).$$

Sono tutte e tre economiche: il costo di un'euristica costruttiva non è mai il problema, e non è un buon motivo per preferirne una.

5.3 La regola del meno carico

Quando i contenitori non hanno una capacità ma un *carico* che si vuole tenere basso o pareggiato, la regola naturale è: l'oggetto corrente va nel contenitore che al momento è meno carico.

Da sola questa regola dipende molto dall'ordine. L'ordine che funziona meglio, in pratica, è quello per **dimensione decrescente**: si collocano prima gli oggetti grandi, quando c'è ancora spazio per accomodarli, e si usano quelli piccoli per pareggiare alla fine.

Lo stesso problema, due ordini

Quattro oggetti di dimensioni

$$w_1 = 3, \quad w_2 = 4, \quad w_3 = 5, \quad w_4 = 6,$$

due contenitori, regola del meno carico.

ordine	carichi finali	carico massimo
naturale (1, 2, 3, 4)	$3 + 5 = 8$ e $4 + 6 = 10$	10
decrescente (4, 3, 2, 1)	$6 + 3 = 9$ e $5 + 4 = 9$	9

La regola di inserimento è la stessa; cambia solo l'ordine, e il risultato cambia. La somma delle dimensioni è 18: con due contenitori il carico massimo non può stare sotto 9, quindi il secondo ordine trova l'ottimo — ma questo lo sappiamo solo perché abbiamo anche un bound, non perché l'euristica ce lo dica.

5.4 La regola del rapporto migliore

Quando gli oggetti hanno un *valore* oltre a una dimensione, e la risorsa è una sola, la regola naturale guarda il rapporto fra i due. Sia $p_j \in \mathbb{Q}_{>0}$ il valore dell'oggetto j e $w_j \in \mathbb{Q}_{>0}$ la risorsa che consuma: si scandiscono gli oggetti per

$$\frac{p_j}{w_j}$$

decrescente, e si prende ciascuno se la risorsa residua lo consente.

Il rapporto non basta

Tre oggetti, risorsa disponibile 10:

$$(p_1, w_1) = (10, 6), \quad (p_2, w_2) = (7, 5), \quad (p_3, w_3) = (6, 5).$$

I rapporti sono $10/6 \approx 1,67$, $7/5 = 1,4$ e $6/5 = 1,2$: l'ordine è 1, 2, 3. Si prende l'oggetto 1 (residuo 4), poi né il 2 né il 3 ci entrano: valore 10. La scelta $\{2, 3\}$ vale però 13 e usa esattamente 10. La regola del rapporto migliore è ragionevole e veloce, e in generale non è ottima.

5.5 La regola del completamento più economico

Quando si devono *coprire* dei requisiti scegliendo degli insiemi, la regola naturale è: si aggiunge l'insieme con il rapporto

$$\frac{\text{costo dell'insieme}}{\text{requisiti ancora scoperti che copre}}$$

più basso, e si ricalcola il rapporto a ogni passo, perché i requisiti scoperti cambiano.

Copertura di cinque requisiti

Cinque requisiti $\{1, 2, 3, 4, 5\}$ e quattro insiemi disponibili, tutti di costo 1:

$$S_1 = \{1, 2, 3\}, \quad S_2 = \{3, 4\}, \quad S_3 = \{4, 5\}, \quad S_4 = \{5\}.$$

Primo passo: i rapporti sono $1/3$, $1/2$, $1/2$, $1/1$, e si prende S_1 . Secondo passo: restano scoperti 4 e 5; i rapporti diventano $1/1$ per S_2 , $1/2$ per S_3 , $1/1$ per S_4 , e si prende S_3 . Tutto è coperto con due insiemi, che è anche l'ottimo. Il punto da notare è il *ricalcolo*: al secondo passo S_2 non vale più $1/2$ ma $1/1$, perché il requisito 3 è già coperto.

5.6 Quando un'euristica costruttiva fallisce

Un'euristica costruttiva può arrivare a un punto in cui l'elemento corrente non si può collocare da nessuna parte. Non è un errore di programmazione: è la conseguenza del non tornare mai indietro.

Un fallimento in tre mosse

Tre oggetti di dimensioni

$$w_1 = 3, \quad w_2 = 3, \quad w_3 = 4,$$

due contenitori di capacità 5, ordine naturale, regola first-fit. L'oggetto 1 va nel primo contenitore, il 2 nel secondo, e il 3 non entra da nessuna parte: residui 2 e 2. Eppure una soluzione ammissibile esiste — basta mettere 1 e nulla accanto, 2 e nulla accanto, e 3 da solo, se i contenitori fossero tre; con due contenitori il problema è *davvero* inammissibile, perché $3 + 3 + 4 = 10 = 2 \cdot 5$ e nessuna coppia somma a 5.

Se l'euristica non restituisce una soluzione, non si è ottenuto niente: non c'è alcun bound primale. Le due cose da fare sono, nell'ordine:

1. cambiare l'ordine — l'ordine per dimensione decrescente fallisce molto meno spesso di quello naturale, perché sistema per primi gli oggetti difficili;
2. cambiare la regola di inserimento.

Se anche così non si trova nulla, la domanda diventa se il problema sia ammissibile, e questa è una domanda per il modello, non per l'euristica.

5.7 Che cosa dà, e che cosa non dà

Il conto esatto di che cosa si sa

Sia $\bar{z}$ il valore della soluzione trovata da un'euristica costruttiva.

- In un problema di **minimo** si sa che

$$z(MILP) \leq \bar{z},$$

cioè $\bar{z}$ è un upper bound: $UB = \bar{z}$.

- In un problema di **massimo** si sa che

$$z(MILP) \geq \bar{z},$$

cioè $\bar{z}$ è un lower bound: $LB = \bar{z}$.

- In nessuno dei due casi si sa *quanto* $\bar{z}$ sia lontano dall'ottimo. Per saperlo serve un bound dall'altro lato, e quello non viene da una soluzione ammissibile: viene da un rilassamento o da un argomento combinatorio (capitolo 4).

Con entrambi i bound in mano si può scrivere il **gap certificato**

$$\frac{|UB - LB|}{|z(MILP)|},$$

che è l'unica misura di qualità disponibile prima di risolvere il MILP.

Perché costruire l'euristica a mano, se c'è il solver

Un solver moderno trova soluzioni ammissibili da solo, spesso migliori. Costruire un'euristica a mano serve comunque, per tre ragioni:

- obbliga a capire che cosa rende una soluzione ammissibile, e questo è esattamente il contenuto dei vincoli;
- dà un valore di riferimento indipendente dal solver: se il solver restituisce di peggio, c'è un errore nel modello;
- nei problemi grandi il solver non arriva all'ottimo, e la qualità della soluzione finale dipende da quella iniziale.

Dal modello a Python/Gurobi

Script: `python/cap06_gurobi.py`

[Pagina online](#)

Il corso usa **un solo solver**, Gurobi da Python. Questo capitolo mostra come si scrive un modello — una famiglia di vincoli per blocco, con i nomi del modello matematico — e soprattutto come si *leggono* i risultati, incluso il caso in cui il solver non ha finito.

6.1 Le quattro classi di variabili

Che cosa	vtype	Dominio	Uso tipico
decisione sì/no	GRB.BINARY	$\{0, 1\}$	selezione, attivazione, assegnamento
conteggio quantità misurabile	GRB.INTEGER (default)	$\mathbb{Z}$ fra lb e ub $[lb, ub] \subseteq \mathbb{R}$	scatole, turni, operai tempo, denaro, flusso
variabile libera	(default) con lb=-GRB.INFINITY	$\mathbb{R}$	duali di uguaglianze

GRB.BINARY implica già $lb = 0$ e $ub = 1$: non serve ripeterli. Una variabile continua ha $lb = 0$ per default: una variabile che deve poter essere negativa — tipicamente un duale di uguaglianza, o una deviazione con segno — va dichiarata con $lb = -GRB.INFINITY$, altrimenti il modello è silenziosamente sbagliato.

6.2 Il modello, una famiglia per blocco

```

1 def modello(t, c, a):
2     """Il problema 7.1: una addConstrs per famiglia, con il nome dell'
3     etichetta."""
4     m = gp.Model("assegnamento"); m.Params.OutputFlag = 0
5     x = m.addVars(n, k, vtype=GRB.BINARY, name="x") # dati ->
6     # variabili
7     m.setObjective(gp.quicksum(c[j][h] * x[j, h] for j in range(n)
8                               for h in range(k)), GRB.MINIMIZE)
9     m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assegna")

```

```

8     m.addConstrs((gp.quicksum(t[j][h] * x[j, h] for j in range(n)) <= a
9         [h]
10         for h in range(k)), name="disponibilita")
    return m, x

```

Le tre regole di scrittura del corso

1. Una `addConstrs` per famiglia, nell'ordine del modello matematico, e il `name` è l'etichetta della famiglia ("assegna", "disponibilita"). I nomi generati sono `assegna[0]`, `assegna[1]`, ...: si ritrovano nei duali e nei messaggi del solver.
2. Le variabili si dichiarano tutte insieme, con `addVars` e gli indici del modello. `x.sum(j, "*")` è la somma su tutti gli indici della seconda dimensione: è la scrittura dell'*indice muto* del modello.
3. I dati arrivano come argomenti, non come variabili globali: la stessa funzione serve per l'istanza base e per tutte le varianti delle domande di modellazione.

Il modello scritto ha 9 variabili, 6 vincoli e 18 coefficienti non nulli. Per controllare che sia quello che si è scritto sulla carta, `m.write("modello.lp")` produce il formato LP leggibile:

```

Minimize
  5 x[0,0] + 10 x[0,1] + 2 x[0,2] + 5 x[1,0] + 4 x[1,1] + 6 x[1,2]
  + 5 x[2,0] + 4 x[2,1] + 6 x[2,2]
Subject To
  assegna[0]: x[0,0] + x[0,1] + x[0,2] = 1
  ...

```

È il modo più rapido per accorgersi di un coefficiente sbagliato: il tabulare dell'istanza nella dispensa e questo output devono coincidere riga per riga.

6.3 Leggere i risultati

L'ordine di lettura, che non si cambia

1. Status. Sempre per primo.
2. SolCount: se vale 0, *non esiste* una soluzione e `ObjVal` non si può leggere.
3. ObjVal e ObjBound: la migliore soluzione trovata e il miglior bound dimostrato.
4. MIPGap, NodeCount, Runtime: quanto resta incerto, e quanto è costato.

Status	valore	SolCount	Che cosa si può dire
OPTIMAL	2	≥ 1	$z(MILP) = \text{ObjVal}$, dimostrato
INFEASIBLE	3	0	il modello non ha soluzioni ammissibili
UNBOUNDED	5	0	manca un vincolo, o un bound su una variabile
TIME_LIMIT	9	0	niente: né una soluzione né, in generale, un bound utile
TIME_LIMIT	9	≥ 1	$\text{ObjBound} \leq z(MILP) \leq \text{ObjVal}$
SOLUTION_LIMIT	10	≥ 1	come sopra

I quattro casi sull'istanza del problema 7.1

- **Risoluzione normale.** Status = 2, SolCount = 2, ObjVal = ObjBound = 11, MIPGap = 0, NodeCount = 0: Gurobi ha chiuso tutto nella radice.
- **Inammissibile.** Con disponibilità $a = (1, 1, 1)$ nessun assegnamento è possibile: Status = 3, SolCount = 0.
- **Fermato subito.** Con TimeLimit = 0: Status = 9, SolCount = 0 e ObjBound = $-\infty$. Non c'è nulla da riportare: né una soluzione né un bound.
- **Fermato alla prima soluzione.** Con SolutionLimit = 1: Status = 10, SolCount = 1, ObjVal = 12, ObjBound = 10, MIPGap = 0,1667. Questo è il caso in cui si riporta un *intervallo*: l'ottimo sta fra 10 e 12, e il 16,7% ne misura l'ampiezza. Dire «l'ottimo è 12» sarebbe falso.

Non si scrive «l'ottimo è ObjVal» se Status non è OPTIMAL. Non si scrive un gap se SolCount è 0. Non si confronta il Runtime di due modelli risolti con impostazioni diverse. E non si legge ObjBound a fine risoluzione pensando di leggere il rilassamento della radice: per quello serve `relax()`, come nella sezione 4.9.

6.4 Le tolleranze

Parametro	Default	Significato
IntFeasTol	10^{-5}	quanto una variabile intera può distare da un intero
FeasibilityTol	10^{-6}	violazione ammessa su un vincolo lineare
OptimalityTol	10^{-6}	tolleranza sui costi ridotti
MIPGap	10^{-4}	gap relativo sotto il quale il solver si ferma

Una binaria può tornare come 0,999999997. Nel testo si scrive 1: i valori si arrotondano *quando si riportano*, e i confronti si fanno sempre con una tolleranza — nel corso, 10^{-6} , la costante TOL di `python/mip.py`. Scrivere `if x.X == 1` è un errore; si scrive `if x.X > 0.5`. Il default MIPGap = 10^{-4} significa anche che il solver può fermarsi *prima* dell'ottimo esatto,

dichiarando OPTIMAL: su istanze con valori grandi vale la pena abbassarlo.

6.5 Il rilassamento con relax()

```

1 m.update()                # relax() copia il modello: le modifiche pendenti
    vanno applicate prima
2 r = m.relax()             # le binarie diventano 0 <= x <= 1, le intere x
    >= lb
3 r.Params.OutputFlag = 0
4 r.optimize()
5 zlp = r.ObjVal
6 duali = {c.ConstrName: c.Pi for c in r.getConstrs()}

```

Sull'istanza del problema 7.1, $z(LP^+) = z(LP) = 53/5$ — i due rilassamenti coincidono, perché i vincoli di assegnamento implicano già $x_{jm} \leq 1$ — e i duali non nulli sono

$$\tilde{\mu} = (2, 4, 8, 5), \quad \tilde{\pi}_2 = -0,2,$$

cioè le tre variabili duali dei vincoli **assegna** e quella del vincolo **disponibilita[1]**: la macchina 2 è la sola risorsa stretta.

Il rilassamento senza i bound, quando serve

La funzione `rilassamento(m, rafforzato=False)` di `python/mip.py` toglie anche il limite superiore delle binarie: è il rilassamento di cui la dispensa scrive il duale a mano. Le due versioni si riportano entrambe, con l'annotazione se coincidono e perché.

6.6 Il protocollo del corso, dall'inizio alla fine

La sequenza di ogni esercizio della Parte II

dati → modello → euristica e verifica → LP e duale
→ MIP → tabella → figure e notebook

```

1 m, x = modello(t, c, a)                # (1) dati,
    (2) modello
2
3 e = best_fit(t, a, lambda j, h, ra: c[j][h], "costo")    # (3)
    euristica
4 ub = sum(c[j][h] for (j, h) in e.x)
5 assert ammissibile(m, {f"x[{j},{h}]" : 1 for (j, h) in e.x})    # vincoli
    , bound E interezza
6
7 d = duale(t, c, a)                    # (4) duale
    scritto a mano
8 mano = {f"mu[{j}]" : min(c[j]) for j in range(n)}
9 lb, viol = valuta(d, mano); assert viol <= 1e-9
10 zlp, zlp_raff, pi = due_rilassamenti(m, d)                #
    controlla la dualita' forte

```

```

11
12 z = risolvi(m)                                     # (5) il MIP
13 riga = registra_bound("7.1 assegnamento", ub, lb, zlp, zlp_raff, z) #
      (6) la tabella
14 salva_dati(pd.DataFrame([riga]), "sched1_bound")    #      -> dati/
      sched1_bound.csv

```

Sull'istanza del problema 7.1 il protocollo produce

$$LB = 10, \quad z(LP) = z(LP^+) = \frac{53}{5}, \quad z(MILP) = 11, \quad UB = 11,$$

e la riga finisce in `dati/sched1_bound.csv`. Da lì la leggono la dispensa, la pagina del sito e `verifica_numeri.py`: un solo posto in cui il numero esiste.

I quattro numeri che ogni esercizio della Parte II produce

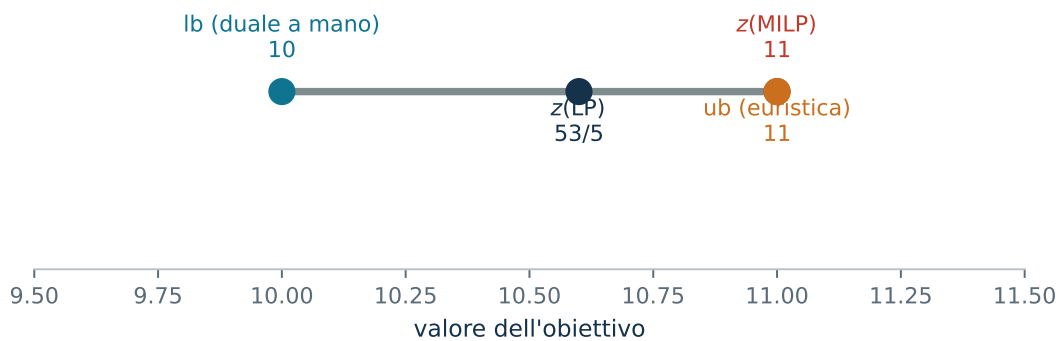


Figura 6.1: I quattro numeri che ogni esercizio della Parte II produce, sull'istanza del problema 7.1.

6.7 Eseguire il codice

In locale

Dalla cartella `python/`:

```

python3 -m venv .venv && source .venv/bin/activate
pip install gurobipy pandas matplotlib
python3 fam07_1_assegnamento.py      # un capitolo
python3 esegui_tutti.py              # tutti, piu' i notebook
python3 verifica_numeri.py           # gli assert su ogni numero citato

```

Su Colab

Ogni capitolo ha un notebook che si apre dal badge sulla pagina del sito. La prima cella installa `gurobipy` e scarica i tre moduli comuni (`stile.py`, `mip.py`, `euristiche.py`) se non sono già presenti: sul computer non serve installare niente.

La licenza *size-limited* del pacchetto pip permette modelli fino a **2000 variabili e 2000 vincoli**. Le istanze di questo corso sono piccolissime — il modello più grande della Parte II ha poche decine di variabili — e ci stanno tutte con enorme margine. Per istanze più grandi si attiva gratuitamente la licenza accademica da portal.gurobi.com. Se un modello supera il limite, Gurobi lo segnala con un errore esplicito all'atto di `optimize()`: non produce silenziosamente un risultato sbagliato.

Parte II

I problemi

Quindici modelli numerici

Questo capitolo raccoglie i quindici modelli numerici del corso, dal NUM 1 al NUM 15. Sono gli esempi più facili: dati espliciti, poche variabili, un solo passaggio per ciascuna tecnica. Vengono prima delle sei famiglie di problemi proprio per questo — si leggono per prendere le misure, e poi si affrontano i problemi generali dei capitoli 7–10.

Il formato è ridotto ma conserva sempre gli stessi cinque pezzi:

- l'enunciato, con i dati dell'istanza;
- il **modello simbolico**, con le sue variabili e i suoi vincoli;
- il **modello dell'istanza**, primale e duale, nel box viola;
- una soluzione ammissibile costruita a mano, che dà il bound primale;
- una soluzione duale costruita a mano, che dà il bound duale, e il confronto con l'ottimo del solver.

Ogni modello ha il suo script e il suo notebook, uno per modello, elencati all'inizio di ciascuna sezione.

EX 1 — Il furgone da otto posti

Script: `python/ex01_furgone.py`

[Notebook in Colab](#)

EX 1 — quali gruppi di turisti accettare

Un tour operator noleggia un furgone da otto posti per accompagnare i turisti in una località panoramica. In una giornata si presentano quattro gruppi, di 2, 3, 4 e 5 persone. Poiché il furgone non può ospitarli tutti insieme, l'operatore chiede a ogni gruppo (i cui membri non vogliono essere separati) di fare un'offerta: i quattro gruppi offrono 30, 50, 80 e 70 euro. Per motivi organizzativi si possono accettare al più due gruppi; inoltre, se si accetta il gruppo 2 si deve accettare anche il gruppo 4. Si vuole massimizzare l'incasso.

Le variabili

I gruppi sono quattro: serve una variabile binaria per gruppo.

$$x_j = \begin{cases} 1 & \text{se il gruppo } j \text{ viene accettato,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, 3, 4\}.$$

Il modello dell'istanza

Il modello si scrive direttamente sui dati: quattro colonne, una per variabile.

EX 1 sull'istanza — primale

$$\begin{array}{ll}
 \max & 30x_1 + 50x_2 + 80x_3 + 70x_4 \\
 \text{soggetto a} & 2x_1 + 3x_2 + 4x_3 + 5x_4 \leq 8 \\
 & x_1 + x_2 + x_3 + x_4 \leq 2 \\
 & x_2 - x_4 \leq 0 \\
 & x_1, x_2, x_3, x_4 \in \{0, 1\}
 \end{array}$$

Una soluzione ammissibile: il bound primale

Si scandiscono i gruppi per offerta decrescente e si accetta un gruppo solo se tutti i vincoli restano soddisfatti, implicazione compresa.

Sull'istanza l'ordine per offerta decrescente è 3, 4, 2, 1:

- gruppo 3 (offerta 80): accettato, 4 posti occupati, 1 gruppo;
- gruppo 4 (offerta 70): scartato, servirebbero 9 posti su 8;
- gruppo 2 (offerta 50): scartato, obbligherebbe ad accettare il gruppo 4, e i gruppi diventerebbero tre;
- gruppo 1 (offerta 30): accettato, 6 posti occupati, 2 gruppi.

La soluzione ammissibile costruita è

$$\bar{x}_1 = 1, \quad \bar{x}_2 = 0, \quad \bar{x}_3 = 1, \quad \bar{x}_4 = 0,$$

di valore

$$LB = 30 + 80 = 110.$$

Il rilassamento LP e il suo duale

Il rilassamento LP si ottiene sostituendo il dominio binario delle variabili con la sola non negatività. Il duale del rilassamento LP associa una variabile non negativa a ciascuno dei tre vincoli: α ai posti, β al numero di gruppi, γ all'implicazione. Le quattro colonne del primale danno quattro vincoli duali.

EX 1 sull'istanza — duale del rilassamento LP

$$\begin{aligned}
& \min 8\alpha + 2\beta \\
& \text{soggetto a } 2\alpha + \beta \geq 30 \\
& \quad 3\alpha + \beta + \gamma \geq 50 \\
& \quad 4\alpha + \beta \geq 80 \\
& \quad 5\alpha + \beta - \gamma \geq 70 \\
& \quad \alpha \geq 0 \\
& \quad \beta \geq 0 \\
& \quad \gamma \geq 0
\end{aligned}$$

Una soluzione duale: il bound duale

Per esempio si può procedere così: si valuta il solo posto, e si azzerano gli altri due moltiplicatori,

$$\bar{\beta} = 0, \quad \bar{\gamma} = 0.$$

I quattro vincoli duali diventano

$$2\alpha \geq 30, \quad 3\alpha \geq 50, \quad 4\alpha \geq 80, \quad 5\alpha \geq 70,$$

e il più piccolo α che li soddisfa tutti è

$$\bar{\alpha} = \max\left(\frac{30}{2}, \frac{50}{3}, \frac{80}{4}, \frac{70}{5}\right) = 20.$$

Il valore duale è

$$UB = 8 \cdot 20 = 160.$$

La lettura è «un posto vale quanto il gruppo che lo paga meglio».

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. L'ottimo accetta i gruppi 2 e 4, riempiendo esattamente gli otto posti:

$$\begin{aligned}
\tilde{x}_1 &= 0, & \tilde{x}_2 &= 1, & \tilde{x}_3 &= 0, & \tilde{x}_4 &= 1, \\
z(MILP) &= 50 + 70 = 120.
\end{aligned}$$

Il confronto finale

	valore	che cos'è
LB	110	soluzione euristica (LB)
UB	160	bound duale (UB)
$z(LP)$	160	rilassamento senza i bound
$z(LP^+)$	140	rilassamento con i bound
$z(MILP)$	120	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|110 - 120|}{120} = 8,3\%.$$

- Gap certificato:

$$\frac{|110 - 160|}{120} = 41,7\%.$$

- La soluzione duale costruita a mano è *ottima* per il rilassamento senza i bound. Il rilassamento con i bound, che aggiunge $x_j \leq 1$, scende a 140 e si avvicina all'ottimo.

Il contributo di ciascun vincolo

Togliendo un vincolo per volta si vede quale morde:

modello	$z(MILP)$	gruppi accettati
completo	120	2, 4
senza il limite sul numero di gruppi	120	2, 4
senza l'implicazione	130	2, 3
senza entrambi	130	2, 3

L'implicazione costa 10 euro; il limite di due gruppi non costa nulla, perché è la capacità in posti a mordere.

EX 2 — Linee di autobus

Script: `python/ex02_linee.py`

[Notebook in Colab](#)

EX 2 — assegnamento con capacità in numero di linee

Un comune valuta le offerte di tre compagnie di autobus per gestire quattro linee. Ogni compagnia ha indicato il costo con cui gestirebbe ciascuna linea:

Compagnia	Linea 1	Linea 2	Linea 3	Linea 4
1	10	4	9	7
2	1	2	3	10
3	8	9	10	1

Ogni linea va assegnata a esattamente una compagnia e ogni compagnia può gestire al più due linee. Si vuole minimizzare il costo totale.

Le variabili

Le linee sono quattro e le compagnie tre: servono dodici variabili binarie, una per ogni coppia compagnia–linea.

$$x_{ij} = \begin{cases} 1 & \text{se la compagnia } i \text{ gestisce la linea } j, \\ 0 & \text{altrimenti,} \end{cases}$$

$$\forall i \in \{1, 2, 3\}, \forall j \in \{1, 2, 3, 4\}.$$

Il numero massimo di linee per compagnia è 2.

Il modello dell'istanza

Dodici colonne, una per ogni coppia compagnia–linea.

EX 2 sull'istanza — primale

$$\begin{array}{llllllllllll}
\min & 10x_{11} + 4x_{12} + 9x_{13} + 7x_{14} + x_{21} + 2x_{22} + 3x_{23} + 10x_{24} + 8x_{31} + 9x_{32} + 10x_{33} + x_{34} \\
\text{soggetto a} & x_{11} & & & +x_{21} & & & +x_{31} & & & & = 1 \\
& & x_{12} & & & +x_{22} & & & +x_{32} & & & = 1 \\
& & & x_{13} & & & +x_{23} & & & +x_{33} & & = 1 \\
& & & & x_{14} & & & +x_{24} & & & +x_{34} & = 1 \\
& x_{11} & +x_{12} & +x_{13} & +x_{14} & & & & & & & \leq 2 \\
& & & & & x_{21} & +x_{22} & +x_{23} & +x_{24} & & & \leq 2 \\
& & & & & & & & & x_{31} & +x_{32} & +x_{33} +x_{34} \leq 2 \\
& x_{11}, & x_{12}, & x_{13}, & x_{14}, & x_{21}, & x_{22}, & x_{23}, & x_{24}, & x_{31}, & x_{32}, & x_{33}, & x_{34} \in \{0, 1\}
\end{array}$$

Una soluzione ammissibile: il bound primale

Si scandiscono le linee nell'ordine dato e si assegna ogni linea alla compagnia più economica fra quelle che hanno ancora posti liberi.

Sull'istanza:

- linea 1: costi 10, 1, 8; la più economica è la compagnia 2;
- linea 2: costi 4, 2, 9; di nuovo la compagnia 2, che ora è satura;
- linea 3: restano le compagnie 1 e 3, con costi 9 e 10; si prende la compagnia 1;
- linea 4: restano le compagnie 1 e 3, con costi 7 e 1; si prende la compagnia 3.

La soluzione ammissibile costruita è

$$\bar{x}_{21} = 1, \quad \bar{x}_{22} = 1, \quad \bar{x}_{13} = 1, \quad \bar{x}_{34} = 1,$$

e tutte le altre variabili valgono zero. Il suo valore è

$$UB = 1 + 2 + 9 + 1 = 13.$$

Il rilassamento LP e il suo duale

Il rilassamento LP si ottiene sostituendo il dominio binario delle variabili con la sola non negatività. Il duale del rilassamento LP ha una variabile libera α_j per ogni vincolo di linea e una variabile non positiva π_i per ogni vincolo di capacità; le dodici colonne del primale danno dodici vincoli duali.

EX 2 sull'istanza — duale del rilassamento LP

$$\begin{array}{llll}
\max & \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4 + 2\beta_1 + 2\beta_2 + 2\beta_3 \\
\text{soggetto a} & \alpha_1 & +\beta_1 & \leq 10 \\
& \alpha_2 & +\beta_1 & \leq 4 \\
& \alpha_3 & +\beta_1 & \leq 9 \\
& \alpha_4 & +\beta_1 & \leq 7 \\
& \alpha_1 & & +\beta_2 \leq 1 \\
& \alpha_2 & & +\beta_2 \leq 2 \\
& \alpha_3 & & +\beta_2 \leq 3 \\
& \alpha_4 & & +\beta_2 \leq 10 \\
& \alpha_1 & & +\beta_3 \leq 8 \\
& \alpha_2 & & +\beta_3 \leq 9 \\
& \alpha_3 & & +\beta_3 \leq 10 \\
& \alpha_4 & & +\beta_3 \leq 1 \\
& \alpha_1, \alpha_2, \alpha_3, \alpha_4 & & \geq 0 \\
& \beta_1, \beta_2, \beta_3 & & \geq 0
\end{array}$$

Una soluzione duale: il bound duale

Per esempio si può procedere così: si azzerano i tre moltiplicatori della capacità,

$$\bar{\pi}_1 = 0, \quad \bar{\pi}_2 = 0, \quad \bar{\pi}_3 = 0,$$

e per ogni linea si prende il più grande valore ammissibile, cioè il costo più basso fra le tre compagnie su quella linea.

$$\bar{\alpha}_1 = 1, \quad \bar{\alpha}_2 = 2, \quad \bar{\alpha}_3 = 3, \quad \bar{\alpha}_4 = 1,$$

e il valore duale è

$$LB = 1 + 2 + 3 + 1 = 7.$$

La lettura è «ogni linea costa almeno quanto la sua compagnia più economica».

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. L'ottimo assegna la linea 1 alla compagnia 2, la linea 2 alla compagnia 1, la linea 3 alla compagnia 2 e la linea 4 alla compagnia 3:

$$\tilde{x}_{21} = 1, \quad \tilde{x}_{12} = 1, \quad \tilde{x}_{23} = 1, \quad \tilde{x}_{34} = 1,$$

e tutte le altre variabili valgono zero. Il valore è

$$z(MILP) = 1 + 4 + 3 + 1 = 9.$$

Il confronto finale

	valore	che cos'è
UB	13	soluzione euristica (UB)
LB	7	bound duale (LB)
$z(LP)$	9	rilassamento LP
$z(MILP)$	9	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|13 - 9|}{9} = 44,4\%.$$

- Gap certificato:

$$\frac{|13 - 7|}{9} = 66,7\%.$$

- Il rilassamento LP è *esatto* ($z(LP) = z(MILP) = 9$), ma la ricetta duale non lo raggiunge: azzerare i moltiplicatori della capacità butta via proprio l'informazione che rende il problema difficile, cioè che la compagnia più economica non può prendersi tutte le linee.

EX 3 — Staffetta

Script: `python/ex03_staffetta.py`

[Notebook in Colab](#)

EX 3 — assegnamento con più risorse che compiti

Un allenatore deve formare una staffetta scegliendo tre nuotatori fra quattro. La gara ha tre frazioni da 50 metri: dorso, rana e farfalla. I tempi registrati (secondi) sono:

Stile	Carlo	Paolo	Fabio	Marco
Dorso	37	32	33	37
Rana	43	33	42	34
Farfalla	33	28	38	30

Si vuole minimizzare il tempo totale della staffetta.

Le variabili

Le frazioni sono tre e i nuotatori quattro: servono dodici variabili binarie, una per ogni coppia frazione–nuotatore.

$$x_{sn} = \begin{cases} 1 & \text{se la frazione } s \text{ è nuotata dal nuotatore } n, \\ 0 & \text{altrimenti,} \end{cases}$$

$$\forall s \in \{1, 2, 3\}, \forall n \in \{1, 2, 3, 4\}.$$

Il modello dell'istanza

Dodici colonne, una per ogni coppia frazione–nuotatore: la prima riga è l'obiettivo, le tre successive impongono che ogni frazione sia nuotata da esattamente un nuotatore, le quattro dopo che ogni nuotatore nuoti al più una frazione.

EX 3 sull'istanza — primale

$$\begin{aligned}
 & \min 37x_{11} + 32x_{12} + 33x_{13} + 37x_{14} + 43x_{21} + 33x_{22} + 42x_{23} + 34x_{24} + 33x_{31} + 28x_{32} + 38x_{33} + 30x_{34} \\
 \text{soggetto a } & x_{11} + x_{12} + x_{13} + x_{14} = 1 \\
 & x_{21} + x_{22} + x_{23} + x_{24} = 1 \\
 & x_{31} + x_{32} + x_{33} + x_{34} = 1 \\
 & x_{11} + x_{21} + x_{31} \leq 1 \\
 & x_{12} + x_{22} + x_{32} \leq 1 \\
 & x_{13} + x_{23} + x_{33} \leq 1 \\
 & x_{14} + x_{24} + x_{34} \leq 1 \\
 & x_{11}, x_{12}, x_{13}, x_{14}, x_{21}, x_{22}, x_{23}, x_{24}, x_{31}, x_{32}, x_{33}, x_{34} \in \{0, 1\}
 \end{aligned}$$

Una soluzione ammissibile: il bound primale

Si scandiscono le frazioni nell'ordine dato e si assegna ciascuna al nuotatore più veloce fra quelli ancora liberi.

Sull'istanza:

- dorso: il più veloce dei quattro è Paolo, con 32;
- rana: fra i tre rimasti il più veloce è Marco, con 34;
- farfalla: fra i due rimasti il più veloce è Carlo, con 33.

La soluzione ammissibile costruita è

$$\bar{x}_{12} = 1, \quad \bar{x}_{24} = 1, \quad \bar{x}_{31} = 1,$$

e tutte le altre variabili valgono zero. Il tempo totale è

$$UB = 32 + 34 + 33 = 99 \text{ secondi.}$$

Il rilassamento LP e il suo duale

Il rilassamento LP si ottiene sostituendo il dominio binario delle variabili con la sola non negatività. Il duale del rilassamento LP ha una variabile libera α_s per ogni frazione e una variabile non positiva β_n per ogni nuotatore.

EX 3 sull'istanza — duale del rilassamento LP

$$\begin{array}{llll}
\max & \alpha_1 + \alpha_2 + \alpha_3 + \beta_1 + \beta_2 + \beta_3 + \beta_4 & & \\
\text{soggetto a} & \alpha_1 & + \beta_1 & \leq 37 \\
& \alpha_1 & + \beta_2 & \leq 32 \\
& \alpha_1 & + \beta_3 & \leq 33 \\
& \alpha_1 & + \beta_4 & \leq 37 \\
& \alpha_2 & + \beta_1 & \leq 43 \\
& \alpha_2 & + \beta_2 & \leq 33 \\
& \alpha_2 & + \beta_3 & \leq 42 \\
& \alpha_2 & + \beta_4 & \leq 34 \\
& \alpha_3 + \beta_1 & & \leq 33 \\
& \alpha_3 & + \beta_2 & \leq 28 \\
& \alpha_3 & + \beta_3 & \leq 38 \\
& \alpha_3 & + \beta_4 & \leq 30 \\
& \alpha_1, \alpha_2, \alpha_3 & & \geq 0 \\
& \beta_1, \beta_2, \beta_3, \beta_4 & & \geq 0
\end{array}$$

Una soluzione duale: il bound duale

Per esempio si può usare la stessa di EX 2 con capacità unitaria: si pone $\bar{\beta}_n = 0$ per ogni nuotatore e si prende il più grande α_s ammissibile,

$$\bar{\alpha}_s = \min_n t_{sn}.$$

Sull'istanza

$$\bar{\alpha}_1 = 32, \quad \bar{\alpha}_2 = 33, \quad \bar{\alpha}_3 = 28,$$

e il valore duale è

$$LB = 32 + 33 + 28 = 93.$$

Il significato è «ogni frazione costa almeno il tempo del suo migliore specialista». Il bound è debole proprio quando due frazioni hanno lo stesso specialista migliore: qui Paolo è il più veloce sia nel dorso sia nella farfalla, e il bound non è raggiungibile.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. L'ottimo assegna il dorso a Fabio (33), la rana a Marco (34) e la farfalla a Paolo (28); Carlo resta fuori.

$$\tilde{x}_{13} = 1, \quad \tilde{x}_{24} = 1, \quad \tilde{x}_{32} = 1,$$

e tutte le altre variabili valgono zero. Il tempo totale è

$$z(MILP) = 33 + 34 + 28 = 95 \text{ secondi.}$$

Il confronto finale

	valore	che cos'è
UB	99	soluzione euristica (UB)
LB	93	bound duale (LB)
$z(LP)$	95	rilassamento LP
$z(MILP)$	95	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|99 - 95|}{95} = 4,2\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|99 - 93|}{95} = 6,3\%.$$

Perché qui il rilassamento è esatto

La matrice del modello è quella del problema di assegnamento: totalmente unimodulare. Tutti i vertici del rilassamento sono interi, quindi $z(LP) = z(MILP)$ e l'interezza non costa nulla — lo stesso fenomeno dell'alldiff nella sezione 3.12. Non è così, invece, appena si aggiunge un vincolo di capacità con coefficienti diversi da 0 e 1.

EX 3: la staffetta ottima ($z = 95$ s); resta fuori Carlo

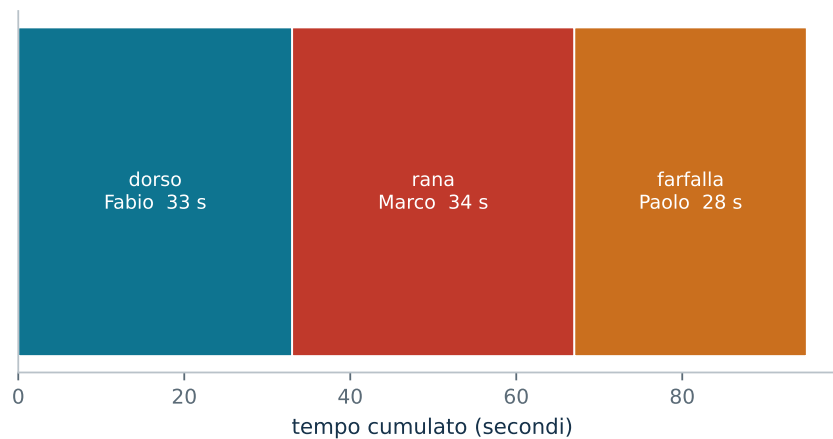


Figura 6.2: EX 3: la staffetta ottima, frazione per frazione.

EX 4 — Scarpe: produzione, scorte e assunzioni

Script: `python/ex04_scarpe.py`

[Notebook in Colab](#)

EX 4 — produzione e manodopera su tre mesi

Nei prossimi tre mesi un calzaturificio deve soddisfare domande mensili di 3000, 5000 e 7000 paia di scarpe. All'inizio del primo mese ha 500 paia in magazzino e impiega 100 operai.

I mesi sono tre. Servono quattro famiglie di variabili, una per decisione.

$$x_t = \text{paia prodotte nel mese } t, \quad \forall t \in \{1, 2, 3\},$$
$$y_t = \text{operai in servizio nel mese } t, \quad \forall t \in \{1, 2, 3\},$$
$$z_t = \text{operai assunti all'inizio del mese } t, \quad \forall t \in \{1, 2, 3\}.$$

Il modello dell'istanza

EX 4 sull'istanza

Il rilassamento LP si ottiene sostituendo l'interezza di y_t e z_t con la sola non negatività. Il duale ha una variabile libera α_t per ogni bilancio delle scorte, una variabile non negativa β_t per ogni vincolo delle ore e una variabile libera γ_t per ogni bilancio dell'organico; le undici colonne del primale danno undici vincoli duali.

Duale di EX 4 sull'istanza

$$\begin{array}{rcll}
\max & 2500\alpha_1 + 5000\alpha_2 + 7000\alpha_3 & + 100\gamma_1 & \\
\text{soggetto a} & \alpha_1 & -4\beta_1 & \leq 15 \\
& \alpha_2 & -4\beta_2 & \leq 15 \\
& \alpha_3 & -4\beta_3 & \leq 15 \\
& -\alpha_1 & +\alpha_2 & \leq 3 \\
& -\alpha_2 & +\alpha_3 & \leq 3 \\
& & 160\beta_1 & +\gamma_1 - \gamma_2 \leq 1500 \\
& & 160\beta_2 & +\gamma_2 - \gamma_3 \leq 1500 \\
& & 160\beta_3 & +\gamma_3 \leq 1500 \\
& & & -\gamma_1 \leq 100 \\
& & & -\gamma_2 \leq 100 \\
& & & -\gamma_3 \leq 100 \\
& \alpha_1, & \alpha_2, & \alpha_3 \geq 0 \\
& & \beta_1, & \beta_2, & \beta_3 \geq 0 \\
& & & \gamma_1, & \gamma_2, & \gamma_3 \geq 0
\end{array}$$

Una soluzione ammissibile: il bound primale

Produzione «just in time»: ogni mese si produce la domanda netta, senza scorte, assumendo quanti operai servono. Mese 1: 10 000 ore, 63 operai, ne bastano i 100 già in servizio. Mese 2: 20 000 ore, 125 operai, se ne assumono 25. Mese 3: 28 000 ore, 175 operai, se ne assumono altri 50. Il costo è

$$z(MILP) \leq UB = 825\,000.$$

Una soluzione duale: il bound duale

Con α_t libera sui bilanci della domanda, $\beta_t \geq 0$ sulle ore e γ_t libera sui bilanci dell'organico, la ricetta è: $\gamma = 0$, e un'ora di lavoro vale quanto davvero costa,

$$\bar{\beta}_t = \frac{w}{r} = \frac{1500}{160} = \frac{75}{8},$$

da cui un paio vale al più

$$\bar{\alpha}_t = 15 + 4 \cdot \frac{75}{8} = \frac{105}{2} = 52,5.$$

Si verifica che tutte le colonne duali sono soddisfatte, e

$$LB = \frac{105}{2} \cdot 14\,500 = 761\,250.$$

Il confronto finale

	UB	LB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 4 scarpe	825 000	761 250	773 500	773 500	774 180

Il piano ottimo produce 3980, 4000 e 6520 paia con 100, 100 e 163 operai: si tiene 1480 paia in magazzino a fine primo mese e 480 a fine secondo, e si assumono 63 operai una volta sola, nel

terzo mese. Anticipare la produzione costa 3 euro al paio e fa risparmiare stipendi: alzando il costo di magazzino a 20 euro le scorte scendono a 1000 e 0 (ottimo 807 500), a 60 euro spariscono del tutto e si ricade sull'euristica (825 000).

EX 5 — Veicoli con quantità minima

Script: `python/ex05_veicoli.py`

[Notebook in Colab](#)

EX 5 — cinque tipi di veicolo

Un'azienda valuta la produzione di cinque tipi di veicolo: utilitaria, berlina, vettura grande, monovolume e monovolume grande. Dispone di 1000 tonnellate di acciaio e 2000 ore di lavoro. La tabella riporta, per ciascun tipo, l'acciaio per unità, le ore per unità, il profitto unitario e la quantità minima da produrre se il tipo viene scelto:

Tipo	acciaio	ore	profitto	minimo
utilitaria	2	30	200	10
berlina	3	25	250	10
vettura grande	5	40	300	10
monovolume	6	45	550	5
monovolume grande	8	55	700	5

Si vuole il piano di profitto massimo.

Le variabili

I tipi di veicolo sono cinque e le risorse due. Servono due famiglie di variabili: una quantità per tipo e un indicatore per tipo.

$$x_j = \text{unità prodotte del tipo } j, \quad \forall j \in \{1, 2, 3, 4, 5\},$$

$$y_j = \begin{cases} 1 & \text{se il tipo } j \text{ viene prodotto,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, 3, 4, 5\}.$$

Il massimo producibile di un solo tipo, che serve come coefficiente nei vincoli di attivazione, si legge dai dati: vale 66, 80, 50, 44 e 36 unità per i cinque tipi, sempre limitato dalle ore di lavoro.

Il modello simbolico

EX 5 — veicoli con quantità minima

$$\max \sum_{j=1}^s p_j x_j \quad (6.1a)$$

$$\text{soggetto a } \sum_{j=1}^s a_{ij} x_j \leq b_i, \quad \forall i \in \{1, 2, \dots, k\}, \quad (6.1b)$$

$$x_j - q_j y_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (6.1c)$$

$$x_j - M_j y_j \leq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (6.1d)$$

$$x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (6.1e)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, s\}. \quad (6.1f)$$

Descrizione. I vincoli (6.1c) e (6.1d) insieme rendono x_j una variabile *semicontinua*, come mostra la tavola di verità:

y_j	che cosa impongono i due vincoli	x_j ammesse
0	$x_j \geq 0$ e $x_j \leq 0$	solo $x_j = 0$
1	$x_j \geq q_j$ e $x_j \leq M_j$	$q_j \leq x_j \leq M_j$

Nessuno dei due basta da solo: senza (6.1d) un tipo potrebbe essere prodotto con $y_j = 0$; senza (6.1c) la quantità minima sarebbe vuota.

Il modello dell'istanza

Sui dati dell'istanza i big-M valgono $M = (66, 80, 50, 44, 36)$, tutti limitati dalle ore di lavoro.

EX 5 sull'istanza — primale

$$\begin{array}{llllll} \max & 200x_1 + 250x_2 + 300x_3 + 550x_4 + 700x_5 \\ \text{soggetto a} & 2x_1 + 3x_2 + 5x_3 + 6x_4 + 8x_5 & \leq & 1000 \\ & 30x_1 + 25x_2 + 40x_3 + 45x_4 + 55x_5 & \leq & 2000 \\ & x_1 & -10y_1 & \geq & 0 \\ & & x_2 & -10y_2 & \geq & 0 \\ & & & x_3 & -10y_3 & \geq & 0 \\ & & & & x_4 & -5y_4 & \geq & 0 \\ & & & & & x_5 & -5y_5 & \geq & 0 \\ & x_1 & & & -66y_1 & \leq & 0 \\ & & x_2 & & -80y_2 & \leq & 0 \\ & & & x_3 & & -50y_3 & \leq & 0 \\ & & & & x_4 & & -44y_4 & \leq & 0 \\ & & & & & x_5 & & -36y_5 & \leq & 0 \\ & x_1, & x_2, & x_3, & x_4, & x_5 & \in & \mathbb{Z}_{\geq 0} \\ & & & & & & y_1, & y_2, & y_3, & y_4, & y_5 \in \{0, 1\} \end{array}$$

Il duale del rilassamento LP associa una variabile non negativa π_i a ciascuna risorsa, una variabile non positiva a ciascun vincolo di quantità minima — che scriviamo $-\ell_j$ con $\ell_j \geq 0$ — e una variabile non negativa μ_j a ciascun vincolo di attivazione.

Duale del rilassamento LP del modello EX 5

$$\begin{aligned}
 & \min \sum_{i=1}^k b_i \pi_i \\
 \text{soggetto a } & \sum_{i=1}^k a_{ij} \pi_i - \ell_j + \mu_j \geq p_j, & \forall j \in \{1, 2, \dots, s\}, \\
 & q_j \ell_j - M_j \mu_j \geq 0, & \forall j \in \{1, 2, \dots, s\}, \\
 & \pi_i \geq 0, & \forall i \in \{1, 2, \dots, k\}, \\
 & \ell_j \geq 0, & \forall j \in \{1, 2, \dots, s\}, \\
 & \mu_j \geq 0, & \forall j \in \{1, 2, \dots, s\}.
 \end{aligned}$$

Descrizione. La variabile π_i è il prezzo di una unità della risorsa i , mentre ℓ_j e μ_j sono i prezzi dei due vincoli che rendono x_j semicontinua. L'obiettivo valuta a quei prezzi tutte le risorse disponibili. Il primo gruppo di vincoli è fatto dalle colonne delle x_j : le risorse consumate da una unità del tipo j , corrette dai due vincoli di semicontinuità, devono coprire il profitto p_j . Il secondo gruppo è fatto dalle colonne delle y_j : accendere il tipo j obbliga a produrne almeno q_j e ne concede al più M_j , e il saldo dei due prezzi deve restare non negativo.

EX 5 sull'istanza — duale

$$\begin{array}{llllllllll}
 \min & 1000\pi_1 + 2000\pi_2 & & & & & & & & \\
 \text{soggetto a} & 2\pi_1 & +30\pi_2 & +lam_1 & & & +\mu_1 & & & \geq 200 \\
 & 3\pi_1 & +25\pi_2 & & +lam_2 & & & +\mu_2 & & \geq 250 \\
 & 5\pi_1 & +40\pi_2 & & & +lam_3 & & & +\mu_3 & \geq 300 \\
 & 6\pi_1 & +45\pi_2 & & & & +lam_4 & & & +\mu_4 \geq 550 \\
 & 8\pi_1 & +55\pi_2 & & & & & +lam_5 & & +\mu_5 \geq 700 \\
 & & & -10lam_1 & & & & & -66\mu_1 & \geq 0 \\
 & & & & -10lam_2 & & & & & -80\mu_2 \geq 0 \\
 & & & & & -10lam_3 & & & & -50\mu_3 \geq 0 \\
 & & & & & & -5lam_4 & & & -44\mu_4 \geq 0 \\
 & & & & & & & -5lam_5 & & -36\mu_5 \geq 0 \\
 & \pi_1, & \pi_2 & & & & & & & \geq 0 \\
 & & & lam_1, & lam_2, & lam_3, & lam_4, & lam_5 & & \geq 0 \\
 & & & & & & & & \mu_1, & \mu_2, & \mu_3, & \mu_4, & \mu_5 \geq 0
 \end{array}$$

Una soluzione ammissibile: il bound primale

Si scandiscono gli elementi in ordine di profitto per ora di lavoro: 20/3, 10, 15/2, 110/9 e 140/11 per i cinque tipi. Si parte dal monovolume grande (140/11), se ne producono 36 (il

massimo consentito dalle ore) e restano 712 tonnellate di acciaio ma solo 20 ore: nessun altro tipo raggiunge la propria quantità minima e l'euristica costruttiva si ferma.

$$z(MILP) \geq LB = 25\,200.$$

Una soluzione duale: il bound duale

Con $\ell_j = \beta_j = 0$ (il lotto minimo non si valuta) e una sola risorsa valutata al prezzo unitario più alto fra i tipi:

$$\begin{aligned}\bar{\pi}_{\text{acciaio}} &= \max_j \frac{\bar{p}_j}{a_{1j}} = 100 \Rightarrow 1000 \cdot 100 = 100\,000, \\ \bar{\pi}_{\text{ore}} &= \max_j \frac{\bar{p}_j}{a_{2j}} = \frac{140}{11} \Rightarrow 2000 \cdot \frac{140}{11} = \frac{280\,000}{11} \approx 25\,454,5.\end{aligned}$$

Il bound migliore viene dalle ore:

$$z(MILP) \leq UB = \frac{280\,000}{11}.$$

Il confronto finale

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 5 veicoli (max)	25 200	280000/11	280000/11	229000/9	25 250

L'ottimo produce 9 monovolume e 29 monovolume grandi, usando tutte le 2000 ore e 286 tonnellate di acciaio su 1000. Il bound duale è *ottimo* per il rilassamento senza i bound, e l'euristica sbaglia di appena lo 0,2%. Senza quantità minime l'ottimo sarebbe 25 300: il lotto minimo costa 50 euro di profitto. Raddoppiando le quantità minime si scende a 25 200, e raddoppiando le ore di lavoro si arriva a 50 750 con le ore ancora sature.

EX 6 — Hub-and-spoke

Script: `python/ex06_hub.py`

[Notebook in Colab](#)

EX 6 — il minimo numero di hub

Una compagnia aerea organizza la rete con un sistema *hub-and-spoke*: un hub serve direttamente tutte le città entro 1000 miglia. La compagnia opera su otto città; per ciascuna, le città entro 1000 miglia sono:

Città	Città entro 1000 miglia
Atlanta	Atlanta, Chicago, Houston, New York
Chicago	Atlanta, Chicago, New York
Denver	Denver, Los Angeles
Houston	Atlanta, Houston
Los Angeles	Denver, Los Angeles, San Francisco
New York	Atlanta, Chicago, New York
San Francisco	Los Angeles, San Francisco, Seattle
Seattle	San Francisco, Seattle

Si vuole il numero minimo di hub tale che ogni città sia entro 1000 miglia da almeno un hub.

Le variabili

Le città sono otto: serve una variabile binaria per città.

$$y_j = \begin{cases} 1 & \text{se la città } j \text{ diventa un hub,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, \dots, 8\}.$$

Numerando le città nell'ordine Atlanta, Chicago, Denver, Houston, Los Angeles, New York, San Francisco, Seattle, il dato è l'insieme

$$S_i \subseteq \{1, 2, \dots, 8\}$$

delle città che stanno entro 1000 miglia dalla città i , cioè quelle che possono coprirla:

città i	S_i
1 Atlanta	$\{1, 2, 4, 6\}$
2 Chicago	$\{1, 2, 6\}$
3 Denver	$\{3, 5\}$
4 Houston	$\{1, 4\}$
5 Los Angeles	$\{3, 5, 7\}$
6 New York	$\{1, 2, 6\}$
7 San Francisco	$\{5, 7, 8\}$
8 Seattle	$\{7, 8\}$

Il modello dell'istanza

Otto colonne, una per città.

EX 6 sull'istanza — primale

$$\begin{aligned}
 &\min y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7 + y_8 \\
 \text{soggetto a } &y_1 + y_2 + y_4 + y_6 \geq 1 \\
 &y_1 + y_2 + y_6 \geq 1 \\
 &y_3 + y_5 \geq 1 \\
 &y_1 + y_4 \geq 1 \\
 &y_3 + y_5 + y_7 \geq 1 \\
 &y_1 + y_2 + y_6 \geq 1 \\
 &y_5 + y_7 + y_8 \geq 1 \\
 &y_7 + y_8 \geq 1 \\
 &y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8 \in \{0, 1\}
 \end{aligned}$$

Il duale del rilassamento LP associa una variabile non negativa π_i a ciascun vincolo di copertura: la colonna della variabile y_j raccoglie le città che j è in grado di coprire.

EX 6 sull'istanza — duale del rilassamento LP

$$\begin{array}{llll}
\max & u_1 + u_2 + u_3 + u_4 + u_5 + u_6 + u_7 + u_8 \\
\text{soggetto a} & u_1 + u_2 & + u_4 & + u_6 \leq 1 \\
& u_1 + u_2 & & + u_6 \leq 1 \\
& & u_3 & + u_5 \leq 1 \\
& u_1 & & + u_4 \leq 1 \\
& & u_3 & + u_5 + u_7 \leq 1 \\
& u_1 + u_2 & & + u_6 \leq 1 \\
& & & u_5 + u_7 + u_8 \leq 1 \\
& & & u_7 + u_8 \leq 1 \\
& u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8 \geq 0
\end{array}$$

Una soluzione ammissibile: il bound primale

Euristica costruttiva di copertura (sezione 5), con il criterio «costo per città nuova coperta».

- **Passo 1.** Rapporti 1/4 per Atlanta, 1/3 per Chicago, New York e San Francisco, 1/2 per le altre: si sceglie **Atlanta**, che copre Atlanta, Chicago, Houston e New York.
- **Passo 2.** Restano Denver, Los Angeles, San Francisco e Seattle; rapporti 1/2 per Denver, 1/3 per Los Angeles e San Francisco, 1/2 per Seattle: si sceglie **Los Angeles**, che copre Denver, Los Angeles e San Francisco.
- **Passo 3.** Resta Seattle; rapporto 1/1 per San Francisco e per Seattle: si sceglie **San Francisco**.

Tre hub, quindi $UB = 3$.

Una soluzione duale: il bound duale

Con $n = 8$ città, rilassando a $y_j \geq 0$ e con $\pi_i \geq 0$ per ogni vincolo di copertura (verso $\geq$ in un minimo):

Duale del rilassamento LP di EX 6

$$\begin{array}{ll}
\max & \sum_{i=1}^n \pi_i \\
\text{soggetto a} & \sum_{i: j \in S_i} \pi_i \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \\
& \pi_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}.
\end{array}$$

Descrizione. La variabile π_i è il prezzo che si attribuisce alla città i . L'obiettivo somma i prezzi delle n città. I vincoli sono le colonne delle x_j , uno per hub: le città che l'hub j copre, valutate a quei prezzi, non possono valere più di quanto costa aprirlo, cioè 1.

Euristica costruttiva duale sulle città, nell'ordine: si alza π_i fino a saturare il primo vincolo duale che si oppone, e si aggiornano i residui.

- **Atlanta** è coperta da quattro hub, tutti con residuo 1: $\bar{\pi}_1 = 1$, e i residui di Atlanta, Chicago, Houston e New York vanno a zero.
- **Chicago, Houston e New York** hanno tutti i loro hub a residuo zero: $\bar{\pi} = 0$.
- **Denver** è coperta da Denver e Los Angeles, entrambi con residuo 1: $\bar{\pi}_3 = 1$, e i loro residui vanno a zero.
- **Los Angeles e San Francisco** hanno un hub a residuo zero: $\bar{\pi} = 0$.
- **Seattle** è coperta da San Francisco e Seattle, entrambi con residuo 1: $\bar{\pi}_8 = 1$.

Valore $LB = 1 + 1 + 1 = 3$.

Che cosa dimostra questo bound, in parole

Le tre città che hanno ricevuto $\pi_i = 1$ — Atlanta, Denver, Seattle — sono a due a due «lontane»: nessuna città può fare da hub per due di esse. Quindi servono almeno tre hub. Il duale ha formalizzato un argomento che si può raccontare senza scriverlo: è il senso della ricetta euristica costruttiva.

Il confronto finale

$z(MILP) = 3$, con gli hub ad Atlanta, Los Angeles e San Francisco — la stessa soluzione della euristica costruttiva.

UB (euristica costruttiva)	LB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
3	3	3	3	0,0%

I due bound a mano coincidono: l'ottimalità è dimostrata senza il solver.

EX 6: i 3 hub scelti (in teal) e quante città copre ciascuna sede

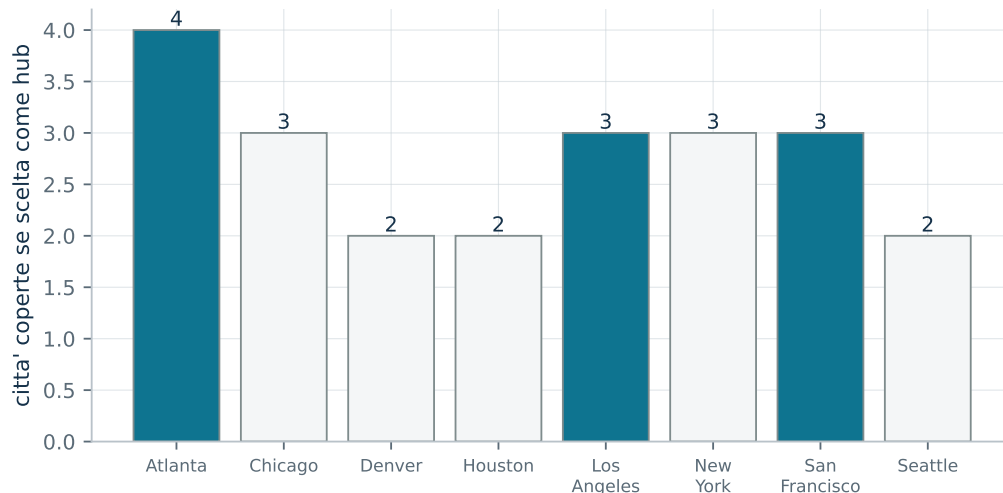


Figura 6.3: EX 6: per ogni città, quante ne coprirebbe come hub; in teal le tre scelte.

EX 7 — Aerei su commessa con costo fisso

Script: `python/ex07_aerei.py`

[Notebook in Colab](#)

EX 7 — quali commesse accettare

Un'azienda costruisce aerei su misura; prima di produrre per un cliente occorre sostenere un costo fisso di attrezzaggio. Tre clienti hanno ordinato, rispettivamente, 3, 2 e 5 aerei. La capacità produttiva è di 1000 ore. L'azienda decide quali ordini accettare e quante unità produrne (anche meno di quante richieste). Costi di attrezzaggio, profitti unitari e ore per aereo:

	cliente 1	cliente 2	cliente 3
attrezzaggio (milioni)	3	2	0
profitto unitario (milioni)	2	3	1
ore per aereo	200	400	200

Si vuole massimizzare il profitto totale al netto dei costi di attrezzaggio.

Il modello.

I dati sono $n = 3$ clienti, di profitto unitario p_j , costo di attrezzaggio f_j , ore per aereo h_j e ordine massimo M_j , più la capacità produttiva $H = 1000$ ore. Con x_j aerei prodotti per il cliente j (intero) e y_j indicatore di commessa accettata:

EX 7

$$\begin{aligned}
 & \max \sum_{j=1}^n (p_j x_j - f_j y_j) \\
 & \text{soggetto a } \sum_{j=1}^n h_j x_j \leq H, \\
 & x_j - M_j y_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \\
 & x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \dots, n\}, \\
 & y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}.
 \end{aligned}$$

EX 7 sull'istanza — primale

$$\begin{aligned}
 & \max \quad 2x_1 + 3x_2 + x_3 - 3y_1 - 2y_2 \\
 & \text{soggetto a } 200x_1 + 400x_2 + 200x_3 \leq 1000 \\
 & x_1 - 3y_1 \leq 0 \\
 & x_2 - 2y_2 \leq 0 \\
 & x_3 - 5y_3 \leq 0 \\
 & x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0} \\
 & y_1, y_2, y_3 \in \{0, 1\}
 \end{aligned}$$

Descrizione. Un solo vincolo per cliente fa due cose insieme: limita la quantità al numero di aerei ordinati e fa pagare l'attrezzaggio. Il cliente 3 ha attrezzaggio nullo, quindi y_3 non compare nell'obiettivo e vale 1 in ogni soluzione ottima con $x_3 > 0$.

Una soluzione ammissibile: il bound primale

Si scandiscono gli elementi in ordine di profitto netto per ora, calcolato sulla commessa intera: $3/600$, $4/800$ e $5/1000$, cioè $1/200$ per tutti e tre. A parità di rapporto si scorre nell'ordine naturale: si accetta il cliente 1 con 3 aerei (600 ore, profitto netto 3), poi il cliente 2 con un solo aereo (400 ore, profitto netto $3 - 2 = 1$), e le ore finiscono.

$$z(MILP) \geq LB = 4.$$

Una soluzione duale: il bound duale

Con $\alpha \geq 0$ sulla capacità e $\beta_j \geq 0$ sui legami:

$$\min 1000\alpha \quad \text{s.a.} \quad h_j\alpha + \beta_j \geq p_j, \quad M_j\beta_j \leq f_j.$$

La ricetta: $\bar{\beta}_j = f_j/M_j$, cioè l'attrezzaggio spalmato sugli aerei ordinati (1, 1 e 0), da cui $\alpha \geq (p_j - \bar{\beta}_j)/h_j$ per ogni j , cioè $\alpha \geq 1/200$ in tutti e tre i casi. Con $\bar{\alpha} = 1/200$,

$$z(MILP) \leq UB = 1000 \cdot \frac{1}{200} = 5.$$

Il confronto finale

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 7 aerei (max)	4	5	5	5	5

L'ottimo vale 5 ed è raggiunto da *tre* piani diversi: cinque aerei al cliente 3; due al cliente 2 e uno al terzo; tre al cliente 1 e due al terzo. Tutti e tre usano esattamente le 1000 ore. Il bound duale è esatto, e in questo caso il certificato chiude il problema da solo: al solver non resta niente da fare. Senza costi di attrezzaggio l'ottimo salirebbe a 9; con 1400 ore disponibili a 7.

EX 8 — Seminari

Script: `python/ex08_seminari.py`

[Notebook in Colab](#)

EX 8 — selezione con cardinalità esatta e non-adiacenza

Uno studente deve seguire *esattamente due* sessioni di una scuola estiva. Ci sono tre seminari, ciascuno offerto in quattro fasce orarie (9–10, 10–11, 11–12, 12–13); ogni seminario si segue al più una volta e due sessioni non possono occupare la stessa fascia. Il gradimento di ciascuna sessione è:

	9–10	10–11	11–12	12–13
Seminario 1	8	6	5	3
Seminario 2	7	9	4	6
Seminario 3	5	7	8	9

Lo studente non vuole *mai due ore consecutive* di lezione. Si massimizza il gradimento totale.

Il modello.

I dati sono $n = 3$ seminari, $m = 4$ fasce orarie, $q = 2$ sessioni da seguire e il gradimento p_{sk} . Con $x_{sk} = 1$ se si segue il seminario s nella fascia k ($n m = 12$ binarie):

EX 8

$$\begin{aligned}
& \max \sum_{s=1}^n \sum_{k=1}^m p_{sk} x_{sk} \\
& \text{soggetto a} \quad \sum_{s=1}^n x_{sk} \leq 1, & \forall k \in \{1, 2, \dots, m\}, \\
& \quad \sum_{k=1}^m x_{sk} \leq 1, & \forall s \in \{1, 2, \dots, n\}, \\
& \quad \sum_{s=1}^n \sum_{k=1}^m x_{sk} = q, \\
& \quad \sum_{s=1}^n (x_{sk} + x_{s,k+1}) \leq 1, & \forall k \in \{1, 2, \dots, m-1\}, \\
& \quad x_{sk} \in \{0, 1\}, & \forall s \in \{1, 2, \dots, n\}, \forall k \in \{1, 2, \dots, m\}.
\end{aligned}$$

EX 8 sull'istanza — primale

$$\begin{aligned}
& \max 8x_{11} + 6x_{12} + 5x_{13} + 3x_{14} + 7x_{21} + 9x_{22} + 4x_{23} + 6x_{24} + 5x_{31} + 7x_{32} + 8x_{33} + 9x_{34} \\
& \text{soggetto a} \quad x_{11} \quad \quad \quad +x_{21} \quad \quad \quad +x_{31} \quad \quad \quad \leq 1 \\
& \quad \quad x_{12} \quad \quad \quad +x_{22} \quad \quad \quad +x_{32} \quad \quad \quad \leq 1 \\
& \quad \quad \quad x_{13} \quad \quad \quad +x_{23} \quad \quad \quad +x_{33} \quad \quad \quad \leq 1 \\
& \quad \quad \quad \quad x_{14} \quad \quad \quad +x_{24} \quad \quad \quad +x_{34} \leq 1 \\
& \quad x_{11} + x_{12} + x_{13} + x_{14} \quad \quad \quad \leq 1 \\
& \quad \quad \quad \quad x_{21} + x_{22} + x_{23} + x_{24} \quad \quad \quad \leq 1 \\
& \quad \quad \quad \quad \quad \quad \quad x_{31} + x_{32} + x_{33} + x_{34} \leq 1 \\
& \quad x_{11} + x_{12} + x_{13} + x_{14} + x_{21} + x_{22} + x_{23} + x_{24} + x_{31} + x_{32} + x_{33} + x_{34} = 2 \\
& \quad x_{11} + x_{12} \quad \quad \quad +x_{21} + x_{22} \quad \quad \quad +x_{31} + x_{32} \quad \quad \quad \leq 1 \\
& \quad \quad x_{12} + x_{13} \quad \quad \quad +x_{22} + x_{23} \quad \quad \quad +x_{32} + x_{33} \quad \quad \quad \leq 1 \\
& \quad \quad \quad x_{13} + x_{14} \quad \quad \quad +x_{23} + x_{24} \quad \quad \quad +x_{33} + x_{34} \leq 1 \\
& \quad x_{11}, \quad x_{12}, \quad x_{13}, \quad x_{14}, \quad x_{21}, \quad x_{22}, \quad x_{23}, \quad x_{24}, \quad x_{31}, \quad x_{32}, \quad x_{33}, \quad x_{34} \in \{0, 1\}
\end{aligned}$$

Descrizione. con $q = 2$: quattro vincoli di fascia, tre di seminario, uno di cardinalità *esatta* e tre di non-adiacenza fra fasce consecutive.

Una soluzione ammissibile: il bound primale

Euristica costruttiva: si prende la sessione ammissibile di gradimento più alto, poi la migliore compatibile. Passo 1: seminario 3 nella fascia 4 (gradimento 9). Passo 2: fra le sessioni che non usano il seminario 3, non occupano la fascia 4 e non sono adiacenti ad essa, la migliore è il seminario 2 nella fascia 2 (gradimento 9). In un **massimo** il valore è un *lower bound*: $LB = 18$.

Duale e upper bound.

Con $\alpha_k \geq 0$ per le fasce, $\beta_s \geq 0$ per i seminari, γ libera per la cardinalità esatta e $\delta_k \geq 0$ per la non-adiacenza:

Duale del rilassamento LP di EX 8

$$\begin{aligned}
 \min \quad & \sum_{k=1}^m \alpha_k + \sum_{s=1}^n \beta_s + q\gamma + \sum_{k=1}^{m-1} \delta_k \\
 \text{soggetto a} \quad & \alpha_k + \beta_s + \gamma + \sum_{k': k \in \{k', k'+1\}} \delta_{k'} \geq p_{sk}, \quad \forall s \in \{1, 2, \dots, n\}, \forall k \in \{1, 2, \dots, m\}, \\
 & \alpha_k \geq 0, \quad \forall k \in \{1, 2, \dots, m\}, \\
 & \beta_s \geq 0, \quad \forall s \in \{1, 2, \dots, n\}, \\
 & \gamma \geq 0, \\
 & \delta_k \geq 0, \quad \forall k \in \{1, 2, \dots, m-1\}.
 \end{aligned}$$

Descrizione. La variabile α_k è il prezzo della fascia oraria k , β_s quello del seminario s , γ il prezzo del vincolo di cardinalità e δ_k quello del vincolo di non-adiacenza fra le fasce k e $k+1$. L'obiettivo somma i termini noti di quei vincoli a quei prezzi. I vincoli duali sono le colonne delle x_{sk} : seguire il seminario s nella fascia k occupa la fascia, occupa il seminario, consuma una unità di cardinalità e tocca i vincoli di non-adiacenza che contengono k ; il totale deve coprire la preferenza p_{sk} .

La ricetta più corta: tutto a zero tranne $\bar{\gamma} = \max_{s,k} p_{sk} = 9$. Ogni vincolo duale diventa $9 \geq p_{sk}$, vero per costruzione, e il valore è $q\bar{\gamma} = 2 \cdot 9 = 18$. Si legge: «si seguono due sessioni, e nessuna vale più della migliore».

Ricetta duale	UB
tutto a zero tranne $\gamma = \max_{s,k} p_{sk}$	18
$\gamma = 0$, $\alpha_k = \max_s p_{sk}$ («ogni fascia vale al più il suo massimo»)	34

La prima è ottima per il rilassamento; la seconda, apparentemente più naturale, è quasi il doppio. La ricetta conta.

Il confronto finale

L'ottimo è $z(MILP) = 18$: seminario 2 nella fascia 2 e seminario 3 nella fascia 4 — una pausa fra le due, come richiesto.

LB (euristica costruttiva)	$z(MILP)$	$z(LP)$	UB (duale a mano)	gap euristica
18	18	18	18	0,0%

Qui i due bound costruiti a mano *chiudono* il problema: l'euristica vale 18, il duale 18, quindi $z(MILP) = 18$ è dimostrato senza il solver.

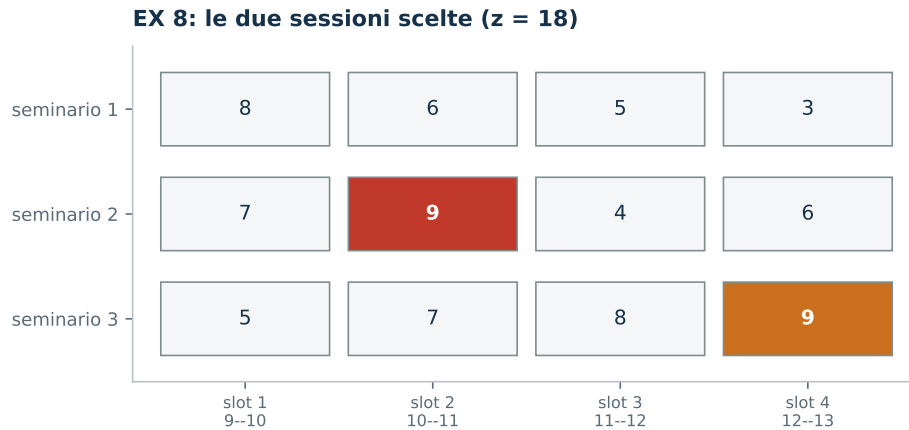


Figura 6.4: EX 8: la griglia dei gradimenti; in teal le due sessioni scelte.

EX 9 — Le otto regine

Script: `python/ex09_regine.py`

[Notebook in Colab](#)

EX 9 — il massimo numero di regine

Su una scacchiera 8×8 si vuole collocare il massimo numero di regine in modo che nessuna possa catturarne un'altra in una sola mossa. Negli scacchi la regina si muove di un numero qualsiasi di caselle lungo la stessa riga, la stessa colonna o una delle due diagonali.

Il modello.

La scacchiera è $n \times n$, con $n = 8$. Con $x_{ij} = 1$ se sulla casella (i, j) c'è una regina ($n^2 = 64$ binarie):

EX 9

$$\begin{aligned}
 & \max \sum_{i=1}^n \sum_{j=1}^n x_{ij} \\
 \text{soggetto a } & \sum_{j=1}^n x_{ij} \leq 1, & \forall i \in \{1, 2, \dots, n\}, \\
 & \sum_{i=1}^n x_{ij} \leq 1, & \forall j \in \{1, 2, \dots, n\}, \\
 & \sum_{i-j=k} x_{ij} \leq 1, & \forall k \in \{-(n-1), -(n-2), \dots, n-1\}, \\
 & \sum_{i+j=k} x_{ij} \leq 1, & \forall k \in \{2, 3, \dots, 2n\}, \\
 & x_{ij} \in \{0, 1\}, & \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, n\}.
 \end{aligned}$$

Descrizione. Un set packing su quattro famiglie di rette: 8 righe, 8 colonne e 15+15 diagonali, in tutto 46 vincoli. Il trucco di indicizzazione è tutto qui: le diagonali «discendenti» sono gli insiemi a $i - j$ costante, quelle «ascendenti» gli insiemi a $i + j$ costante.

Una soluzione ammissibile: il bound primale

Euristica costruttiva riga per riga: nella riga i si sceglie la prima colonna libera che non sia attaccata dalle regine già piazzate; se non ce n'è, la riga resta vuota. Non si torna mai indietro. Sull'istanza il euristica costruttiva piazza le regine in (1, 1), (2, 3), (3, 5), (4, 2), (5, 4) e poi si blocca: le righe 6, 7 e 8 non hanno più caselle libere.

$$z(MILP) \geq LB = 5.$$

Una soluzione duale: il bound duale

Il duale è un set covering:

$$\begin{aligned} \min \quad & \sum_{i=1}^8 \alpha_i + \sum_{j=1}^8 \beta_j + \sum_{k=-7}^7 \gamma_k + \sum_{k=2}^{16} \delta_k \\ \text{s.a.} \quad & \alpha_i + \beta_j + \gamma_{i-j} + \delta_{i+j} \geq 1, \quad \forall i \in \{1, 2, \dots, 8\}, \forall j \in \{1, 2, \dots, 8\}. \end{aligned}$$

Per esempio si può usare di una riga sola: $\bar{\alpha}_i = 1$ per ogni riga, tutto il resto zero. Ogni casella (i, j) ha $\alpha_i = 1 \geq 1$, quindi la soluzione è ammissibile, e vale 8. È la traduzione duale della frase «in ogni riga sta al più una regina».

$$z(MILP) \leq UB = 8.$$

Il confronto finale

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 9 regine (max)	5	8	8	8	8

Il bound duale è raggiunto: la soluzione trovata è ottima, e lo si sa senza fidarsi del solver. L'euristica costruttiva invece si ferma a 5: è l'euristica, non il bound, a lasciare il divario del 37,5%.

Due varianti.

Su scacchiere $n \times n$ con $n = 4, 5, 6$ l'ottimo vale sempre n , e il certificato $\bar{\alpha}_i = 1$ lo dimostra. Per $n = 2$ e $n = 3$ invece l'ottimo vale 1 e 2: il bound n resta valido ma non è raggiungibile, ed è il primo caso in cui questa ricetta duale non chiude il problema. Togliendo i vincoli sulle diagonali si ottengono le *torri* invece delle regine: il modello diventa un assegnamento, la matrice è totalmente unimodulare e il rilassamento LP dà già un valore intero.

EX 10 — Utensili di una macchina CNC

Script: `python/ex10_utensili.py`

[Notebook in Colab](#)

EX 10 — selezione con corredo di utensili

Una macchina CNC può eseguire cinque operazioni; ciascuna richiede un sottoinsieme di sei utensili e, se eseguita, rende un profitto:

Operazione	Utensili richiesti	Profitto (€)
1	1, 3, 4, 5	2000
2	1, 2, 6	1500
3	1, 2, 3, 5	1800
4	2, 4, 5	1700
5	5, 6	800

Il magazzino della macchina tiene al più $k = 4$ utensili alla volta. Si vuole la configurazione di profitto massimo.

Il modello.

I dati sono $n = 5$ operazioni, di profitto p_i , $m = 6$ utensili e un magazzino da $k = 4$ utensili; T_i è l'insieme degli utensili richiesti dall'operazione i . Con $x_i = 1$ se l'operazione i è eseguita (n binarie) e $y_j = 1$ se l'utensile j è caricato (m binarie):

EX 10

$$\begin{aligned}
 & \max \sum_{i=1}^n p_i x_i \\
 & \text{soggetto a } \sum_{j=1}^m y_j \leq k, \\
 & x_i - y_j \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in T_i, \\
 & x_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \\
 & y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, m\}.
 \end{aligned}$$

EX 10 sull'istanza — primale

$$\begin{array}{ll}
 \max & 2000x_1 + 1500x_2 + 1800x_3 + 1700x_4 + 800x_5 \\
 \text{soggetto a} & y_1 + y_2 + y_3 + y_4 + y_5 + y_6 \leq 4 \\
 & x_1 - y_1 \leq 0 \\
 & x_1 - y_3 \leq 0 \\
 & x_1 - y_4 \leq 0 \\
 & x_1 - y_5 \leq 0 \\
 & x_2 - y_1 \leq 0 \\
 & x_2 - y_2 \leq 0 \\
 & x_2 - y_6 \leq 0 \\
 & x_3 - y_1 \leq 0 \\
 & x_3 - y_2 \leq 0 \\
 & x_3 - y_3 \leq 0 \\
 & x_3 - y_5 \leq 0 \\
 & x_4 - y_2 \leq 0 \\
 & x_4 - y_4 \leq 0 \\
 & x_4 - y_5 \leq 0 \\
 & x_5 - y_5 \leq 0 \\
 & x_5 - y_6 \leq 0 \\
 & x_1, x_2, x_3, x_4, x_5 \in \{0, 1\} \\
 & y_1, y_2, y_3, y_4, y_5, y_6 \in \{0, 1\}
 \end{array}$$

Descrizione. Un vincolo di magazzino e $\sum_i |T_i| = 4 + 3 + 4 + 3 + 2 = 16$ vincoli di link. È l'attivazione *disaggregata* della sezione 3.1 letta al rovescio: qui l'operazione è l'oggetto e gli utensili sono le risorse, e l'operazione richiede *tutti* i suoi utensili — una congiunzione nel conseguente, che per la tabella del capitolo 2 costa un vincolo per utensile.

Una soluzione ammissibile: il bound primale

Euristica costruttiva per profitto decrescente: si scorre l'elenco delle operazioni dalla più redditizia e si carica il corredo di ciascuna, se il magazzino lo permette.

- **Operazione 1** (2000, utensili 1, 3, 4, 5): servono 4 utensili nuovi, il magazzino arriva esattamente a 4: si esegue.
- **Operazioni 3, 4, 2, 5**: ciascuna richiederebbe almeno un utensile in più, e il magazzino è pieno: si scartano.

In un **massimo** il valore è un *lower bound*: $LB = 2000$.

Duale e upper bound.

Rilassando a $x_i, y_j \geq 0$, con $\alpha \geq 0$ per il vincolo di magazzino e $\beta_{ij} \geq 0$ per ciascun vincolo di link:

Duale del rilassamento LP di EX 10

$$\begin{aligned}
& \min \quad k \alpha \\
& \text{soggetto a} \quad \sum_{j \in T_i} \beta_{ij} \geq p_i, & \forall i \in \{1, 2, \dots, n\}, \\
& \sum_{i: j \in T_i} \beta_{ij} \leq \alpha, & \forall j \in \{1, 2, \dots, m\}, \\
& \alpha \geq 0, \\
& \beta_{ij} \geq 0, & \forall i \in \{1, 2, \dots, n\}, \forall j \in T_i.
\end{aligned}$$

Descrizione. Si legge così: β_{ij} è la parte del profitto dell'operazione i che si attribuisce all'utensile j ; il primo gruppo chiede che le parti coprano tutto il profitto, il secondo che nessun utensile riceva più di α ; l'obiettivo paga α per ciascuno dei k posti in magazzino.

La ricetta. Si spalma il profitto di ogni operazione in parti uguali sugli utensili che le servono, $\bar{\beta}_{ij} = p_i/|T_i|$: il primo gruppo è soddisfatto con l'uguaglianza per costruzione. Poi si prende $\bar{\alpha} = \max_j \sum_{i: j \in T_i} \bar{\beta}_{ij}$, cioè il carico dell'utensile più richiesto.

$\bar{\beta}_{ij} = p_i/ T_i $	carico dell'utensile					
	1	2	3	4	5	6
op. 1: $2000/4 = 500$	500		500	500	500	
op. 2: $1500/3 = 500$	500	500				500
op. 3: $1800/4 = 450$	450	450	450		450	
op. 4: $1700/3 = 1700/3$		1700/3		1700/3	1700/3	
op. 5: $800/2 = 400$					400	400
totale	1450	4550/3	950	3200/3	5750/3	900

L'utensile critico è il 5, richiesto da quattro operazioni su cinque: quindi $\bar{\alpha} = 5750/3$ e

$$UB = k \bar{\alpha} = 4 \cdot \frac{5750}{3} = \frac{23000}{3} \approx 7666,7.$$

Una scelta che viene naturale è $\bar{\alpha} = 2000$ con tutti i $\bar{\beta}_{ij} = 900$: i vincoli sulle operazioni sono soddisfatti ($2 \cdot 900 = 1800 \geq 800$ è il caso più stretto) e il valore sarebbe $4 \cdot 2000 = 8000$. **Ma quella soluzione non è ammissibile.** Il secondo gruppo di vincoli chiede $\sum_{i: j \in T_i} \beta_{ij} \leq \alpha$, e l'utensile 5 serve a *quattro* operazioni: riceverebbe $4 \cdot 900 = 3600 > 2000$. La violazione è di 1600. Il numero 8000 non è un bound: non discende da nessuna soluzione duale ammissibile.

È la ragione per cui il protocollo del corso non chiede di *proporre* una soluzione duale ma di *verificarla*: lo script controlla ogni vincolo, e un **assert** fallisce se la violazione supera la tolleranza. Con la ricetta corretta il bound è $23000/3 \approx 7666,7$, cioè *migliore* di quello che si sperava di ottenere sbagliando.

Il confronto finale

$z(MILP) = 2500$: si caricano gli utensili 2, 4, 5, 6 e si eseguono le operazioni 4 (1700) e 5 (800). Nessuna configurazione da quattro utensili rende di più: con $\{1, 3, 4, 5\}$ si esegue solo l'operazione 1 (2000), con $\{1, 2, 3, 5\}$ solo la 3 (1800).

LB (euristica costruttiva)	$z(MILP)$	$z(LP)$	$z(LP^+)$	UB (duale a mano)	gap euristica
2000	2500	5200	5200	23000/3	20,0%

Perché qui il rilassamento è così debole

$z(LP) = 5200$ contro $z(MILP) = 2500$: più del doppio. Nel continuo si può caricare «un po'» di ogni utensile — per esempio $y_j = 2/3$ per sei utensili, che rispetta $\sum_j y_j \leq 4$ — ed eseguire una frazione di *ogni* operazione. L'inezienza del magazzino è tutto: è il caso in cui i bound a mano, per quanto ben costruiti, non bastano, e il lavoro lo deve fare il solver. Un modo per rafforzare è aggiungere le disuguaglianze valide $\sum_{j \in T_i} y_j \geq |T_i| x_i$ (la forma aggregata dei link), oppure osservare che ogni operazione con $|T_i| > k$ può essere fissata a zero.

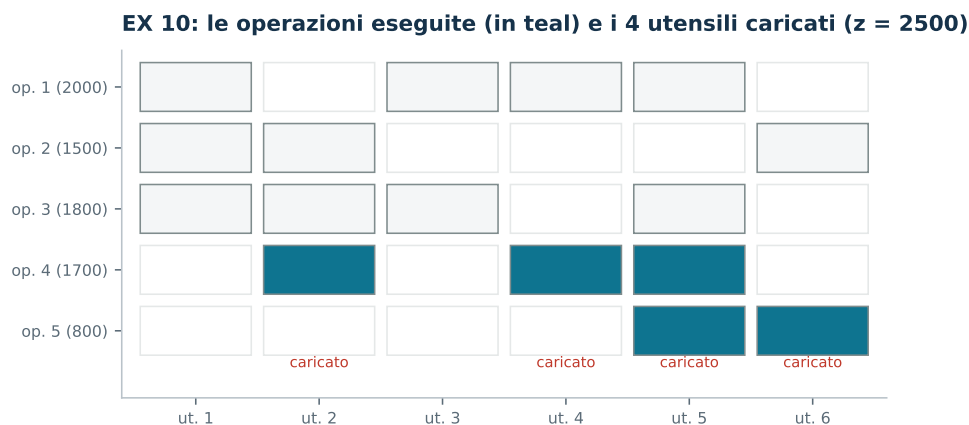


Figura 6.5: EX 10: la matrice operazioni-utensili. In teal le operazioni eseguite; le colonne annotate sono i quattro utensili caricati.

EX 11 — Bilanciamento fra due operai

Script: `python/ex11_bilanciamento.py`

[Notebook in Colab](#)

EX 11 — carichi il più possibile bilanciati

Quattro lavori indivisibili di durata 2, 3, 6 e 7 vanno assegnati a due operai in modo che i loro carichi siano il più possibile bilanciati.

«Carichi bilanciati» non è un obiettivo: è una parola. Le due traduzioni lineari sono quelle della sezione 3.6: minimizzare il *massimo* dei carichi, oppure minimizzare la loro *differenza*.

Poiché il totale $q = 18$ è costante, vale l'identità

$$\max(w_1, w_2) = \frac{q}{2} + \frac{|w_1 - w_2|}{2},$$

quindi i due modelli hanno le *stesse* soluzioni ottime — ma valori ottimi diversi: 9 il primo, 0 il secondo. Riportare «9» come «differenza fra i carichi» è un errore, e riportare «0» come «carico massimo» pure.

Il modello (min-max).

I dati sono $n = 4$ lavori, di durata d_j , e il carico totale $q = \sum_{j=1}^n d_j = 18$. Con $x_j = 1$ se il lavoro j va all'operaio 1 e $z \geq 0$ il carico massimo:

EX 11, versione min-max

$$\begin{aligned} & \min \quad z \\ \text{soggetto a} \quad & \sum_{j=1}^n d_j x_j - z \leq 0, \\ & -\sum_{j=1}^n d_j x_j - z \leq -q, \quad (\text{carico dell'operaio 2, pari a } q - \sum_j d_j x_j) \\ & x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \\ & z \geq 0. \end{aligned}$$

EX 11 sull'istanza — primale

$$\begin{aligned} & \min \quad z \\ \text{soggetto a} \quad & 2x_1 + 3x_2 + 6x_3 + 7x_4 - z \leq 0 \\ & -2x_1 - 3x_2 - 6x_3 - 7x_4 - z \leq -18 \\ & x_1, \quad x_2, \quad x_3, \quad x_4 \in \{0, 1\} \\ & z \geq 0 \end{aligned}$$

Descrizione. L'obiettivo è il carico dell'operaio più carico. Il primo vincolo dice che z non è inferiore al carico dell'operaio 1; il secondo che non è inferiore a quello dell'operaio 2, che vale $q - \sum_j d_j x_j$. Insieme impongono $z \geq \max(w_1, w_2)$, e il minimo trasforma la disuguaglianza in un'uguaglianza sul più carico dei due.

Una soluzione ammissibile: il bound primale

LPT (sezione 5): i lavori in ordine di durata decrescente, ciascuno all'operaio meno carico. Lavoro 4 (7) all'operaio 1; lavoro 3 (6) all'operaio 2; lavoro 2 (3) all'operaio 2, che arriva a 9; lavoro 1 (2) all'operaio 1, che arriva a 9. Carichi (9, 9), quindi $UB = 9$.

Una soluzione duale: il bound duale

Rilassando a $x_j \geq 0$ e usando la tabella di conversione — primale di *minimo* con vincoli $\leq$, quindi variabili duali $\pi_1, \pi_2 \leq 0$:

Duale del rilassamento LP di EX 11

$$\begin{aligned} \max \quad & -q \pi_2 \\ \text{soggetto a} \quad & d_j (\pi_1 - \pi_2) \leq 0, & \forall j \in \{1, 2, \dots, n\}, \\ & -\pi_1 - \pi_2 \leq 1, & (\text{dalla colonna di } z) \\ & \pi_1, \pi_2 \leq 0. \end{aligned}$$

Descrizione. Le variabili π_1 e π_2 sono i prezzi (non positivi) dei due vincoli di min-max. L'obiettivo valuta il carico totale q al prezzo π_2 . Il primo gruppo di vincoli è fatto dalle colonne delle x_j : spostare il lavoro j da un operaio all'altro sposta di d_j i due carichi, e il saldo non può essere positivo perché le x_j non compaiono nell'obiettivo. L'ultimo vincolo è la colonna di z , che compare in entrambi i vincoli con coefficiente -1 e nell'obiettivo primale con costo 1.

Poiché $d_j > 0$, i primi vincoli dicono $\pi_1 \leq \pi_2$. Ponendo $\pi_1 = \pi_2 = t$, il vincolo su z dà $-2t \leq 1$, cioè $t \geq -1/2$; l'obiettivo $-qt$ è massimo per $t = -1/2$ e vale $LB = q/2 = 9$. Significato: «i due carichi sommano a q , quindi il maggiore vale almeno $q/2$ ».

La stessa cosa con variabili duali non negative

Molte trattazioni preferiscono variabili duali ≥ 0 . Basta porre $\alpha = -\pi_1$ e $\beta = -\pi_2$: il duale diventa $\max q\beta$ con $\alpha \leq \beta$ e $\alpha + \beta \leq 1$, e $\alpha = \beta = 1/2$ dà di nuovo 9. È la *stessa* soluzione scritta con l'altro segno; quello che non si può fare è dichiarare variabili ≥ 0 e poi usarne una negativa.

Il confronto finale

$z(MILP) = 9$ con i lavori $\{1, 4\}$ al primo operaio e $\{2, 3\}$ al secondo: carichi $(9, 9)$, perfettamente bilanciati.

UB (LPT)	LB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
9	9	9	9	0,0%

Tutti i numeri coincidono: qui i due bound costruiti a mano si toccano, e l'ottimalità è dimostrata senza risolvere il MILP. Succede perché q è pari e i lavori si possono spezzare esattamente a metà; con q dispari il bound diventerebbe $\lceil q/2 \rceil$ e servirebbe un argomento in più.

EX 12 — Scarpe con soglia minima di produzione

Script: `python/ex12_scarpe_soglia.py`

[Notebook in Colab](#)

EX 12 — tre tipi di scarpa con soglia

Un calzaturificio produce scarponi, mocassini e scarpe da passeggio. La tabella riporta le risorse per un paio, la quantità minima da produrre se il tipo viene prodotto e il prezzo di

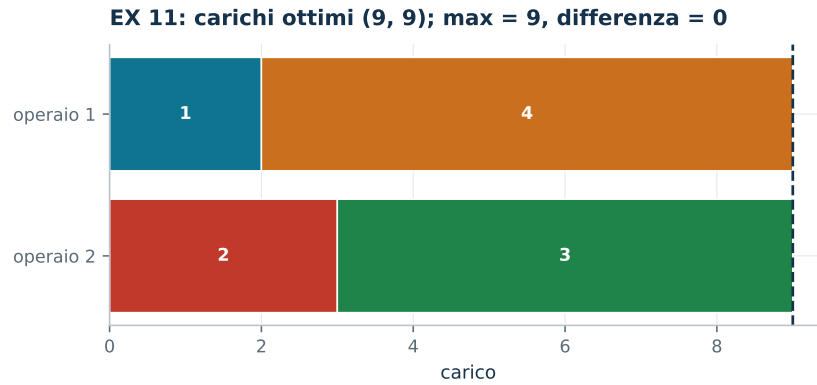


Figura 6.6: EX 11: i carichi ottimi. La linea tratteggiata è $q/2$, il bound duale.

vendita:

Tipo	pelle (g)	ore	chiodi	minimo	prezzo
scarponi	850	3	20	100	150
mocassini	600	2	15	200	120
scarpe da passeggio	700	2,5	20	150	130

Sono disponibili 120 000 grammi di pelle, 7000 ore macchina e 40 000 chiodi. Si vuole massimizzare il ricavo.

Il modello.

È di nuovo il modello 9.3 senza premio, con tre risorse. I big-M letti dai dati sono $M = (141, 200, 171)$, tutti limitati dalla pelle. Per i mocassini soglia e massimo coincidono (200): la variabile x_2 è di fatto una binaria moltiplicata per 200.

Una soluzione ammissibile: il bound primale

Si scandiscono gli elementi in ordine di prezzo per grammo di pelle: $3/17$ per gli scarponi, $1/5$ per i mocassini, $13/70$ per le scarpe da passeggio. Si parte dai mocassini, se ne producono 200 e la pelle finisce esattamente. Nessun altro tipo raggiunge la propria soglia.

$$z(MILP) \geq LB = 24\,000.$$

Una soluzione duale: il bound duale

Con $\ell = \beta = 0$ e una sola risorsa valutata:

$$\text{pelle: } \frac{1}{5} \Rightarrow 24\,000, \quad \text{ore: } 60 \Rightarrow 420\,000, \quad \text{chiodi: } 8 \Rightarrow 320\,000.$$

Il bound migliore è quello della pelle:

$$z(MILP) \leq UB = 24\,000.$$

Il confronto finale

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 12 scarpe con soglia (max)	24 000	24 000	24 000	24 000	24 000

Tutti i numeri coincidono: il sandwich si chiude subito. La ragione è che una sola risorsa è stretta, la pelle, e il tipo che ne ricava di più per grammo la esaurisce da solo restando dentro la propria soglia e il proprio massimo. È un caso didatticamente utile proprio perché è facile: mostra che, quando il bound duale coincide con una soluzione ammissibile, il problema è finito.

Due variazioni cambiano il quadro. Raddoppiando le soglie, quella dei mocassini diventa 400 paia mentre la pelle ne consente al massimo 200: nessun tipo raggiunge più la propria soglia e la produzione si ferma del tutto, $z(MILP) = 0$. Portando la pelle a 200 000 grammi si producono 333 mocassini per 39 960 euro, e i big-M vanno ricalcolati: diventano (235, 333, 285). I dati del big-M non sono costanti del modello, ma funzioni dei dati dell'istanza.

EX 13 — Fondi acquistabili a lotti

Script: `python/ex13_fondi.py`

[Notebook in Colab](#)

EX 13 — due fondi comuni

Dopo una vincita, un investitore deve decidere come impiegare 100 milioni di lire. Un amico gli suggerisce due tipi di fondo comune. Le quote si acquistano solo a lotti indivisibili: un lotto del primo tipo costa 12 milioni, uno del secondo 20 milioni. Il rendimento annuo stimato è $1/6$ del capitale investito per il primo tipo e il 15% per il secondo. Inoltre il numero di lotti del secondo tipo non può superare il 50% del numero totale di lotti acquistati. Si vuole massimizzare il rendimento annuo atteso.

Il modello.

Il rendimento di un lotto è $12 \cdot 1/6 = 2$ milioni per il primo fondo e $20 \cdot 0,15 = 3$ per il secondo. Il vincolo di quota $x_2 \leq \frac{1}{2}(x_1 + x_2)$, moltiplicato per 2 e portato a sinistra, diventa $-x_1 + x_2 \leq 0$. Con x_j numero di lotti del fondo j :

EX 13 — il modello per esteso

$$\begin{aligned} \max \quad & 2x_1 + 3x_2 \\ \text{soggetto a} \quad & 12x_1 + 20x_2 \leq 100 \\ & -x_1 + x_2 \leq 0 \\ & x_1, \quad x_2 \in \mathbb{Z}_{\geq 0} \end{aligned}$$

Descrizione. Uno zaino intero, non binario: le variabili contano lotti, non scelte.

Una soluzione ammissibile: il bound primale

Si scandiscono gli elementi in ordine di rendimento per milione investito: $2/12 = 1/6$ per il primo fondo e $3/20$ per il secondo. Si parte dal primo, se ne comprano 8 lotti (96 milioni) e restano 4 milioni, che non bastano per un altro lotto di nessuno dei due tipi.

$$z(MILP) \geq LB = 16.$$

Una soluzione duale: il bound duale

Con $\alpha \geq 0$ sul budget e $\beta \geq 0$ sulla quota:

$$\min 100\alpha \quad \text{s.a.} \quad 12\alpha - \beta \geq 2, \quad 20\alpha + \beta \geq 3.$$

La bozza di partenza di questo esercizio proponeva $\bar{\alpha} = 5/32$, $\bar{\beta} = 1/8$. Verifichiamo il primo vincolo:

$$12 \cdot \frac{5}{32} - \frac{1}{8} = \frac{60}{32} - \frac{4}{32} = \frac{56}{32} = \frac{7}{4} < 2.$$

È violato di $1/4$: quella coppia *non* è una soluzione duale ammissibile, e il valore $100 \cdot 5/32$ non è un bound. Un valore duale si può leggere come bound solo dopo aver verificato *tutti* i vincoli, uno per uno.

La ricetta corretta è $\bar{\beta} = 0$ e

$$\bar{\alpha} = \max_j \frac{p_j}{c_j} = \max\left(\frac{2}{12}, \frac{3}{20}\right) = \frac{1}{6},$$

cioè il milione vale quanto rende nel fondo migliore. Entrambi i vincoli diventano $c_j \bar{\alpha} \geq p_j$ e sono soddisfatti, e

$$z(MILP) \leq UB = 100 \cdot \frac{1}{6} = \frac{50}{3} \approx 16,67.$$

Il confronto finale

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 13 fondi (max)	16	50/3	50/3	50/3	16

L'euristica trova l'ottimo, 8 lotti del primo fondo e nessuno del secondo, per 16 milioni di rendimento. La differenza $50/3 - 16 = 2/3$ è il prezzo dell'interrezza: il rilassamento comprenderebbe 25/3 lotti del primo fondo, che non si possono acquistare a pezzi.

Sui dati dell'istanza il vincolo di quota non morde, perché il primo fondo rende di più per milione investito e all'ottimo il secondo non si compra affatto. La quota diventa attiva appena il secondo fondo rende il 20%, cioè 4 milioni a lotto: allora senza quota l'ottimo sarebbe 20 (cinque lotti del secondo fondo) e con la quota scende a 18 (tre e tre). È l'unico modo per vedere che cosa fa davvero un vincolo: cambiare i dati finché non morde.

EX 14 — I turni del pronto soccorso

Script: `python/ex14_turni.py`

[Notebook in Colab](#)

EX 14 — turni settimanali

In un pronto soccorso è dato, per ogni giorno della settimana, il numero di addetti a tempo pieno necessari:

Giorno	lun	mar	mer	gio	ven	sab	dom
addetti	10	8	12	9	9	7	8

Un addetto a tempo pieno può essere sostituito da due addetti a mezzo servizio. Lo schema di turno è fisso: quattro giorni consecutivi a tempo pieno, poi un giorno a mezzo servizio, poi due giorni di riposo. Il costo settimanale di un addetto dipende dai giorni lavorati: 100 euro per ogni giorno feriale (da lunedì a venerdì), 110 per il sabato e 130 per la domenica; questi costi sono dimezzati nei giorni a mezzo servizio. Si vuole il piano settimanale di costo minimo.

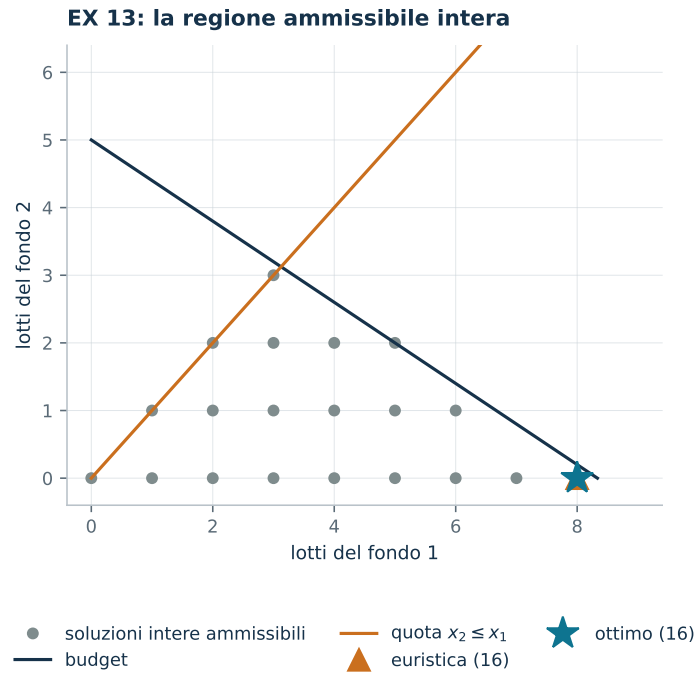


Figura 6.7: EX 13: la regione ammissibile intera, con la retta del budget, quella della quota, la soluzione euristica e l'ottimo (coincidenti).

Il modello.

Ci sono $n = 7$ schemi di turno, uno per giorno di inizio, e $m = 7$ giorni da coprire, con fabbisogno b_i . Con x_j il numero di addetti che seguono lo schema j (variabili intere), c_j il costo settimanale dello schema j e a_{ij} la quota del giorno i coperta dallo schema j (1 nei quattro giorni pieni, 1/2 nel giorno di mezzo servizio, 0 nei due di riposo):

EX 14

$$\begin{aligned} \min \quad & \sum_{j=1}^n c_j x_j \\ \text{soggetto a} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, & \forall i \in \{1, 2, \dots, m\}, \\ & x_j \in \mathbb{Z}_{\geq 0}, & \forall j \in \{1, 2, \dots, n\}. \end{aligned}$$

EX 14 sull'istanza — primale

$$\begin{aligned}
& \min 450x_1 + 455x_2 + 475x_3 + 490x_4 + 490x_5 + 490x_6 + 480x_7 \\
& \text{soggetto a} \quad x_1 + \frac{1}{2}x_4 + x_5 + x_6 + x_7 \geq 10 \\
& \quad x_1 + x_2 + \frac{1}{2}x_5 + x_6 + x_7 \geq 8 \\
& \quad x_1 + x_2 + x_3 + \frac{1}{2}x_6 + x_7 \geq 12 \\
& \quad x_1 + x_2 + x_3 + x_4 + \frac{1}{2}x_7 \geq 9 \\
& \quad \frac{1}{2}x_1 + x_2 + x_3 + x_4 + x_5 \geq 9 \\
& \quad \quad \frac{1}{2}x_2 + x_3 + x_4 + x_5 + x_6 \geq 7 \\
& \quad \quad \quad \frac{1}{2}x_3 + x_4 + x_5 + x_6 + x_7 \geq 8 \\
& \quad x_1, x_2, x_3, x_4, x_5, x_6, x_7 \in \mathbb{Z}_{\geq 0}
\end{aligned}$$

Descrizione. È un *set covering* con coefficienti 1 e 1/2 e variabili intere non binarie. I costi si calcolano dai dati:

Schema	giorni pieni	mezzo servizio	costo
1 (inizia lun)	lun, mar, mer, gio	ven	450
2 (inizia mar)	mar, mer, gio, ven	sab	455
3 (inizia mer)	mer, gio, ven, sab	dom	475
4 (inizia gio)	gio, ven, sab, dom	lun	490
5 (inizia ven)	ven, sab, dom, lun	mar	490
6 (inizia sab)	sab, dom, lun, mar	mer	490
7 (inizia dom)	dom, lun, mar, mer	gio	480

Per esempio lo schema 3 costa $100 + 100 + 100 + 110$ per i quattro giorni pieni (mer, gio, ven, sab) più $130/2 = 65$ per la domenica a mezzo servizio, in tutto 475.

Una soluzione ammissibile: il bound primale

Euristica costruttiva di copertura: finché resta fabbisogno scoperto si aggiunge una copia dello schema con il rapporto

$$\frac{\text{costo}}{\text{fabbisogno effettivamente coperto}}$$

più basso. Il rapporto si ricalcola a ogni passo, perché il fabbisogno residuo cambia. Sull'istanza l'euristica costruttiva aggiunge 18 turni: otto dello schema 1 (a 100 euro per unità di fabbisogno coperta), poi quattro del 3, due del 5, uno del 4 e tre del 7, con rapporti via via peggiori, fino a 480 per l'ultima unità.

$$z(\text{MILP}) \leq UB = 8410.$$

Una soluzione duale: il bound duale

Il duale è

$$\max \sum_{i=1}^7 b_i \pi_i \quad \text{s.a.} \quad \sum_{i=1}^7 a_{ij} \pi_i \leq c_j, \quad \forall j \in \{1, 2, \dots, 7\}, \quad \pi \geq 0.$$

Per esempio si può usare quella del *rapporto migliore*: si mette lo stesso prezzo t su tutti i giorni. Ogni schema copre $4 + 1/2 = 9/2$ giornate-uomo, quindi il vincolo duale diventa $\frac{9}{2}t \leq c_j$

per ogni schema, e il valore più grande ammissibile è

$$t = \min_j \frac{c_j}{9/2} = \min\left(100, \frac{910}{9}, \frac{950}{9}, \frac{980}{9}, \frac{980}{9}, \frac{980}{9}, \frac{320}{3}\right) = 100,$$

raggiunto dallo schema 1. Il fabbisogno totale è $10 + 8 + 12 + 9 + 9 + 7 + 8 = 63$, quindi

$$z(MILP) \geq LB = 100 \cdot 63 = 6300.$$

Il confronto finale

	<i>UB</i>	<i>LB</i>	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 14 turni	8 410	6 300	115970/17	115970/17	7 060

Il piano ottimo assume 5 addetti sullo schema 1, 4 sul 3, 1 sul 4, 2 sul 5 e 3 sul 7, per un costo di 7060. La copertura è satura mercoledì, sabato e domenica, e sovrabbondante negli altri giorni. Il gap dell'euristica è del 19,1%; la differenza $z(MILP) - z(LP) = 7060 - 115970/17 \approx 238$ è il prezzo dell'interezza, cioè del fatto che le persone si assumono una a una.

Il giorno a mezzo servizio conviene?

Confrontiamo il contratto dell'enunciato con uno alternativo: quattro giorni pieni e *tre* di riposo, senza mezzo servizio. Il prezzo minimo per giornata coperta è lo stesso nei due casi, 100 euro. Eppure l'ottimo del secondo contratto è 6710, contro i 7060 del primo: il mezzo servizio conviene di meno. La ragione è che la mezza giornata cade in un giorno prestabilito e spesso finisce dove la copertura c'è già; la flessibilità persa costa più di quanto valga la copertura in più. È un esempio in cui la conclusione «più copertura allo stesso prezzo unitario è sempre meglio» è falsa, e solo il modello lo mostra.

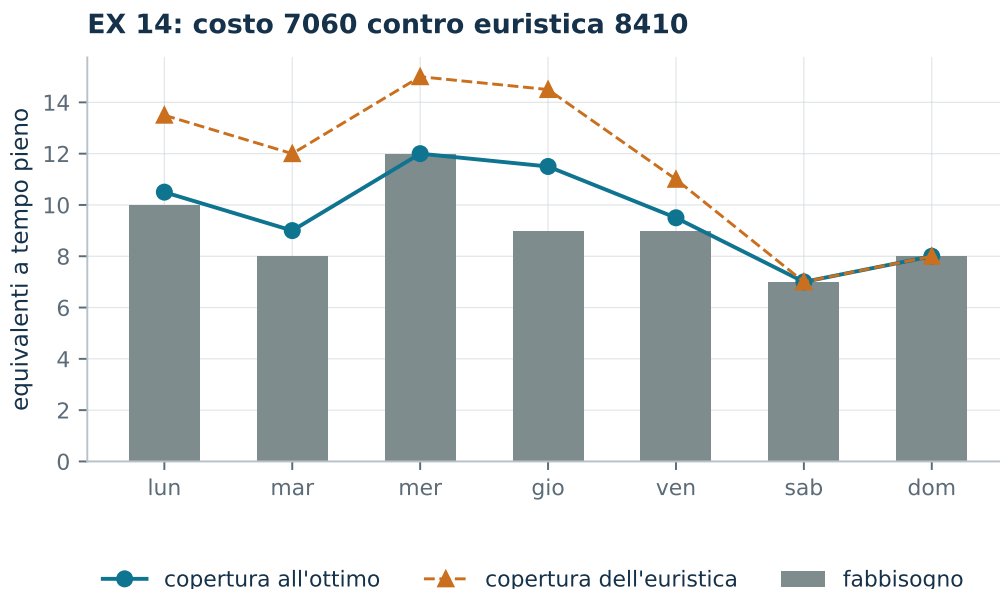


Figura 6.8: EX 14: fabbisogno giornaliero e copertura, nella soluzione euristica e in quella ottima.

EX 15 — L'orario della scuola di musica

Script: `python/ex15_orario.py`[Notebook in Colab](#)

EX 15 — orario settimanale

Una scuola di musica deve costruire l'orario settimanale. Ci sono quattro pomeriggi (lunedì, martedì, mercoledì e giovedì), ciascuno di tre ore. Vanno collocate 6 ore di chitarra, 3 di violino, 2 di pianoforte e 1 di arpa. Ogni giorno deve prevedere lezioni di almeno due strumenti diversi. Le lezioni di pianoforte e di arpa non possono stare in ore consecutive dello stesso giorno, per un conflitto fra i rispettivi docenti. Ogni docente ha delle preferenze da evitare, se possibile: il docente di chitarra preferisce non insegnare all'ora 1 di lunedì e martedì, quello di violino all'ora 2 di mercoledì e giovedì, quello di pianoforte all'ora 3 di ogni giorno, quello di arpa il martedì. Si vuole minimizzare il numero di ore che violano una preferenza.

Il modello.

Con D i giorni, T le ore, I gli strumenti, $x_{dti} = 1$ se all'ora t del giorno d si insegna lo strumento i , e $y_{di} = 1$ se lo strumento i compare nel giorno d :

EX 15

$$\begin{aligned}
 & \min \sum_{d \in D} \sum_{t \in T} \sum_{i \in I} c_{dti} x_{dti} \\
 & \text{soggetto a} \quad \sum_{d \in D} \sum_{t \in T} x_{dti} = h_i, & \forall i \in I, \\
 & \quad \sum_{i \in I} x_{dti} \leq 1, & \forall d \in D, \forall t \in T, \\
 & \quad \sum_{i \in I} y_{di} \geq 2, & \forall d \in D, \\
 & \quad x_{dti} - y_{di} \leq 0, & \forall d \in D, \forall t \in T, \forall i \in I, \\
 & \quad y_{di} - \sum_{t \in T} x_{dti} \leq 0, & \forall d \in D, \forall i \in I, \\
 & \quad x_{dt3} + x_{d,t+1,4} \leq 1, & \forall d \in D, \forall t \in \{1, 2\}, \\
 & \quad x_{dt4} + x_{d,t+1,3} \leq 1, & \forall d \in D, \forall t \in \{1, 2\}, \\
 & \quad x_{dti} \in \{0, 1\}, & \forall d \in D, \forall t \in T, \forall i \in I, \\
 & \quad y_{di} \in \{0, 1\}, & \forall d \in D, \forall i \in I.
 \end{aligned}$$

Descrizione. Le ore da collocare sono $6 + 3 + 2 + 1 = 12$ e le caselle sono $4 \cdot 3 = 12$: le due cifre coincidono, quindi ogni casella ospita esattamente una lezione.

La bozza di partenza di questo esercizio conteneva soltanto il legame $x_{dti} \leq y_{di}$, e non il suo inverso. Con quel solo verso il vincolo $\sum_i y_{di} \geq 2$ è *sempre* soddisfatto: basta porre $y_{di} = 1$ senza fare lezione. Risolvendo il modello così scritto il solver restituisce un orario in cui il giovedì compare un solo strumento — tre ore di chitarra — eppure i vincoli sono tutti verificati.

Il legame $x_{dti} \leq y_{di}$ dice «se c'è lezione allora l'indicatore è acceso», non il viceversa. Serve anche

$$y_{di} \leq \sum_{t \in T} x_{dti},$$

cioè la tecnica 3.10 (se e solo se). È lo stesso errore, e la stessa correzione, del problema 10.3: quando un vincolo *conta* degli indicatori, il legame deve valere nei due versi.

Una soluzione ammissibile.

Con il modello corretto si costruisce a mano un orario che non viola nessuna preferenza:

	ora 1	ora 2	ora 3
lunedì	pianoforte	chitarra	chitarra
martedì	violino	chitarra	chitarra
mercoledì	pianoforte	chitarra	chitarra
giovedì	violino	arpa	violino

Si verifica: 6 ore di chitarra, 3 di violino, 2 di pianoforte, 1 di arpa; almeno due strumenti al giorno; pianoforte e arpa mai in giorni comuni, quindi mai consecutivi; nessuna preferenza violata.

$$z(MILP) \leq UB = 0.$$

Il bound inferiore.

Tutti i costi c_{dti} valgono 0 oppure 1, quindi l'obiettivo è una somma di termini non negativi e $z(MILP) \geq 0$ senza bisogno di alcun duale. L'orario costruito a mano vale 0: è ottimo.

$$LB = z(LP) = z(LP^+) = z(MILP) = UB = 0.$$

Il modello serve dunque a *dimostrare* che un orario senza conflitti esiste, non a trovarne uno meno peggiore.

Che cosa succede se le preferenze si stringono.

Due varianti rendono il problema non banale.

- Se il docente di chitarra preferisce non insegnare né all'ora 1 né all'ora 2 di nessun giorno, restano 4 caselle senza penalità per la chitarra (le ore 3 dei quattro giorni), ma le ore da collocare sono 6: almeno $6 - 4 = 2$ lezioni violeranno la preferenza. È un bound inferiore che si legge dai soli dati, e il solver conferma $z(MILP) = 2$: il conteggio è esatto.
- Se si pretendono almeno *tre* strumenti diversi al giorno, la chitarra può occupare al più una casella al giorno, cioè 4 in tutto, mentre le ore di chitarra sono 6. Il modello è inammissibile, e lo si dimostra contando: Gurobi restituisce lo stato **INFEASIBLE**, come previsto.

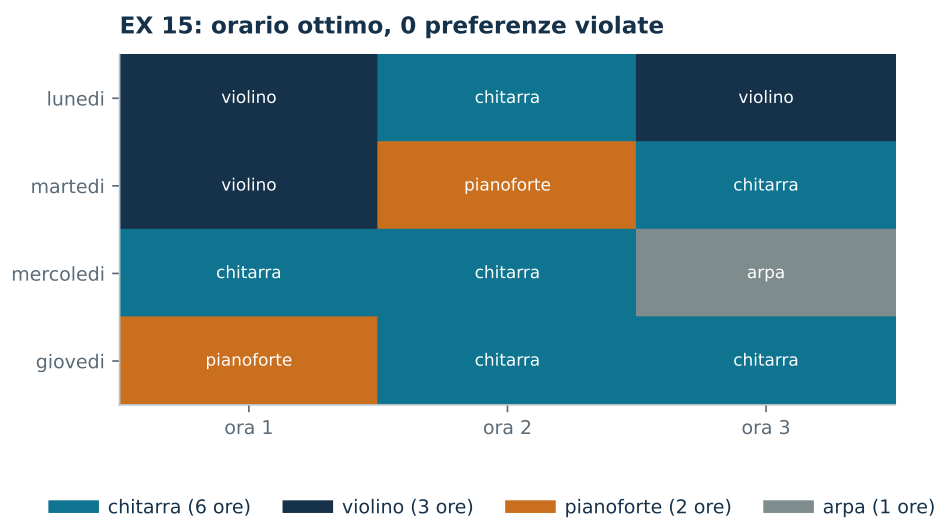


Figura 6.9: EX 15: l'orario ottimo trovato dal solver, un colore per strumento.

Assegnamento e scheduling

Classe: BIP / MILP **Script:** uno per problema, `python/fam07_1_assegnamento.py` ...

[Pagina online](#)

La famiglia

Sette problemi con lo stesso scheletro: dei *lavori* vanno assegnati a delle *macchine* con disponibilità limitata. Cambia, di problema in problema, che cosa si paga e che cosa si decide: il costo dell'assegnamento (problema 1), il costo fisso di ogni macchina accesa (2), il ricavo dei lavori che si scelgono di eseguire (3), il tempo di lavorazione quando i lavori sulla stessa macchina vanno in parallelo (4), il setup di una classe di lavori (5), un premio che si incassa *se e solo se* una classe è completa (6), il ritardo rispetto alle scadenze quando i lavori si susseguono su una sola macchina (7).

I legami fra variabili del capitolo 3 che qui si rivedono:

- **attivazione** (problemi 2, 3, 5): la variabile binaria «macchina usata» o «classe attivata» che comanda le assegnazioni, con il vincolo aggregato $\sum_j t_{jm} x_{jm} \leq a_m y_m$ o disaggregato $x_j \leq y_c$;
- **variabile di massimo** (problema 4): il tempo di lavorazione $y_m \geq t_{jm} x_{jm}$ per ogni lavoro, che all'ottimo vale esattamente il massimo;
- **se e solo se** (problema 6): un verso lo impone il vincolo, l'altro l'obiettivo;
- **big-M e disgiunzioni** (problema 7): « j prima di i o i prima di j », con M pari alla somma dei tempi.

Per ognuno: modello, dimostrazione dei legami, istanza, euristica costruttiva (un bound dal lato delle soluzioni ammissibili), duale del rilassamento LP con una soluzione duale costruita a mano (un bound dall'altro lato), soluzione del solver, considerazioni aggiuntive e due domande di modellazione con soluzione. Tutti i numeri vengono dagli script `fam07_1_assegnamento.py` ... `fam07_7_ritardo.py` della cartella `python/`, uno per problema, con il suo notebook.

Notazione della famiglia

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, indicizzati da $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	numero di macchine, indicizzate da $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	tempo di lavorazione (minuti) del lavoro j sulla macchina m ; t_j se non dipende dalla macchina
c_{jm}	$\in \mathbb{Q}_{>0}$	costo (euro) di eseguire il lavoro j sulla macchina m
c_m	$\in \mathbb{Q}_{>0}$	costo fisso (euro) se la macchina m viene usata
a_m	$\in \mathbb{Q}_{>0}$	disponibilità (minuti) della macchina m ; a se la macchina è una sola
p_m	$\in \mathbb{Z}_{\geq 1}$	numero massimo di lavori che la macchina m può eseguire
r_j	$\in \mathbb{Q}_{>0}$	ricavo (euro) se il lavoro j viene eseguito
d_j	$\in \mathbb{Q}_{>0}$	scadenza (minuti) del lavoro j
q	$\in \mathbb{Z}_{\geq 2}$	numero di classi di lavori, indicizzate da $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	lavori della classe c ; le classi partizionano i lavori
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	costo (euro) e tempo (minuti) di setup della classe c
v_c	$\in \mathbb{Q}_{>0}$	premio (euro) se tutti i lavori della classe c sono eseguiti
u	$\in \mathbb{Q}_{>0}$	riduzione (minuti) della disponibilità se si eseguono lavori di almeno due classi

7.1 Assegnamento a costo minimo con disponibilità

Script: `python/fam07_1_assegnamento.py`[Notebook in Colab](#)

Problema 7.1 — assegnamento a costo minimo

Un'azienda deve eseguire $n \in \mathbb{Z}_{\geq 1}$ lavori su $k \in \mathbb{Z}_{\geq 1}$ macchine. Per ogni lavoro $j \in \{1, 2, \dots, n\}$ e ogni macchina $m \in \{1, 2, \dots, k\}$, il valore $t_{jm} \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti e il valore $c_{jm} \in \mathbb{Q}_{>0}$ è il costo in euro di eseguire il lavoro j sulla macchina m . Per ogni macchina $m \in \{1, 2, \dots, k\}$, il valore $a_m \in \mathbb{Q}_{>0}$ è il tempo di lavorazione disponibile in minuti. Ogni macchina esegue un lavoro alla volta (niente lavorazioni in parallelo). L'azienda vuole assegnare tutti i lavori alle macchine a costo minimo.

Il problema a parole. Decidiamo su quale macchina eseguire ciascun lavoro. *L'obiettivo:* costo totale minimo. *I vincoli:* ogni lavoro va eseguito da esattamente una macchina; il tempo complessivo dei lavori assegnati a una macchina non supera la sua disponibilità. È il problema di *assegnamento generalizzato*: un assegnamento con un vincolo di zaino per macchina.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	numero di macchine, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	tempo di lavorazione del lavoro j sulla macchina m
c_{jm}	$\in \mathbb{Q}_{>0}$	costo di eseguire il lavoro j sulla macchina m
a_m	$\in \mathbb{Q}_{>0}$	disponibilità della macchina m

Variabili decisionali

Introduciamo le seguenti $n \cdot k$ variabili binarie:

$$x_{jm} = \begin{cases} 1 & \text{se il lavoro } j \text{ è eseguito dalla macchina } m, \\ 0 & \text{altrimenti,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Usando queste variabili, un modello BIP per il problema è il seguente:

Modello 7.1 — assegnamento a costo minimo

$$\min \sum_{j=1}^n \sum_{m=1}^k c_{jm} x_{jm} \quad (7.1a)$$

$$\text{soggetto a } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.1b)$$

$$\sum_{j=1}^n t_{jm} x_{jm} \leq a_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.1c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.1d)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.1a) minimizza il costo totale di lavorazione, somma dei costi degli assegnamenti scelti;
- i vincoli lineari (7.1b) assicurano che ogni lavoro sia assegnato a esattamente una macchina, così che tutti i lavori vengano eseguiti (n vincoli lineari);
- i vincoli lineari (7.1c) garantiscono che il tempo di lavorazione complessivo dei lavori assegnati a ciascuna macchina non superi la sua disponibilità (k vincoli lineari);
- i vincoli (7.1d) definiscono le variabili del modello.

Il modello ha una sola famiglia di variabili: non ci sono legami da dimostrare. I vincoli (7.1c) sono del tipo «capacità/risorsa»: due quantità della stessa natura (minuti richiesti e minuti disponibili) messe a confronto, nessuna implicazione logica da scrivere. I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

Ogni famiglia di vincoli è una `addConstrs` con il nome della sua etichetta; la matrice t_{jm} è una lista di liste, a_m una lista.

```
1 m = gp.Model("assegnamento"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 m.setObjective(gp.quicksum(c[j][mm] * x[j, mm] for j in range(n)
4                      for mm in range(k)), GRB.MINIMIZE)
```

```

5 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assegna")
6 m.addConstrs((gp.quicksum(t[j][mm] * x[j, mm] for j in range(n)) <= a[
    mm]
7                 for mm in range(k)), name="disponibilita")
8 m.optimize()

```

L'istanza

$n = 3$ lavori, $k = 3$ macchine:

t_{jm}	$m = 1$	$m = 2$	$m = 3$	c_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	2	1	3	$j = 1$	5	10	2
$j = 2$	3	4	2	$j = 2$	5	4	6
$j = 3$	4	5	3	$j = 3$	5	4	6

	$m = 1$	$m = 2$	$m = 3$
a_m	5	6	7

Il modello (7.1) per l'istanza:

Il modello dell'istanza

$$\begin{aligned}
 &\min 5x_{11} + 10x_{12} + 2x_{13} + 5x_{21} + 4x_{22} + 6x_{23} + 5x_{31} + 4x_{32} + 6x_{33} \\
 &\quad x_{11} + x_{12} + x_{13} = 1 \\
 &\quad \quad x_{21} + x_{22} + x_{23} = 1 \\
 &\quad \quad \quad x_{31} + x_{32} + x_{33} = 1 \\
 &\quad 2x_{11} + 3x_{21} + 4x_{31} \leq 5 \\
 &\quad \quad x_{12} + 4x_{22} + 5x_{32} \leq 6 \\
 &\quad \quad \quad 3x_{13} + 2x_{23} + 3x_{33} \leq 7 \\
 &\quad x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \in \{0, 1\}
 \end{aligned}$$

Euristica costruttiva: il bound primale

Tre euristiche ispirate al bin packing, tutte in tempo lineare nel numero di coppie lavoro-macchina. *Next-fit*: si carica una macchina alla volta e si passa alla successiva quando un lavoro non ci sta più. *First-fit*: ogni lavoro va sulla prima macchina con disponibilità residua sufficiente. *Best-fit*: fra le macchine con disponibilità sufficiente si sceglie quella di costo minimo. Riportiamo il best-fit, che qui è il più naturale perché l'obiettivo è un costo.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Esecuzione sull'istanza (output dello script):

- **Passo 1.** Lavoro 1: disponibilità residue $ra = (5, 6, 7)$; tutte le macchine bastano ($2 \leq 5$, $1 \leq 6$, $3 \leq 7$); costi 5, 10, 2: il minimo è la macchina 3, quindi $x[1][3] = 1$ e $ra[3] = 7 - 3 = 4$.
- **Passo 2.** Lavoro 2: $ra = (5, 6, 4)$; tutte bastano; costi 5, 4, 6: il minimo è la macchina 2, quindi $x[2][2] = 1$ e $ra[2] = 6 - 4 = 2$.

- **Passo 3.** Lavoro 3: $ra = (5, 2, 4)$; la macchina 2 non basta ($5 > 2$); fra le altre, costi 5 e 6: il minimo è la macchina 1, quindi $x[3][1] = 1$ e $ra[1] = 5 - 4 = 1$.

La soluzione ammissibile trovata è

$$\bar{x}_{13} = 1, \quad \bar{x}_{22} = 1, \quad \bar{x}_{31} = 1, \quad \text{tutte le altre } \bar{x}_{jm} = 0,$$

di valore $2 + 4 + 5 = 11$. Quindi $UB = 11$ è un upper bound sull'ottimo:

$$z(MILP) \leq 11.$$

Next-fit e first-fit trovano entrambe l'assegnamento $x_{11} = x_{21} = x_{32} = 1$, di valore 14: un bound peggiore.

Rilassamento LP e duale: il bound duale

Il rilassamento LP del modello (7.1) sostituisce i vincoli (7.1d) con

$$x_{jm} \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.2)$$

(Il vincolo $x_{jm} \leq 1$ è implicato dai vincoli (7.1b): aggiungerlo non cambierebbe nulla.) Asso- ciamo una variabile duale libera μ_j a ciascun vincolo (7.1b) (uguaglianza) e una variabile duale non positiva π_m a ciascun vincolo (7.1c) (verso $\leq$ in un problema di minimo). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 7.1

$$\max \sum_{j=1}^n \mu_j + \sum_{m=1}^k a_m \pi_m \quad (7.3a)$$

$$\text{soggetto a } \mu_j + t_{jm} \pi_m \leq c_{jm}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.3b)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.3c)$$

$$\pi_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.3d)$$

Descrizione. La variabile μ_j è il valore che si attribuisce al lavoro j , che va assegnato comun- que, e π_m il prezzo (non positivo) di una unità di capacità della macchina m . L'obiettivo (7.3a) valuta i lavori e la capacità disponibile. I vincoli (7.3b) sono le colonne delle x_{jm} : mettere il lavoro j sulla macchina m soddisfa il suo vincolo di assegnamento, e quindi vale μ_j , e consuma t_{jm} di capacità al prezzo π_m ; il saldo non può superare il costo c_{jm} di quell'abbinamento. Le μ_j sono libere perché i vincoli di assegnamento sono uguaglianze, le π_m non positive perché quelli di capacità hanno verso $\leq$ in un problema di minimo.

Per l'istanza:

Il modello dell'istanza

$$\begin{aligned}
\max \quad & \mu_1 + \mu_2 + \mu_3 + 5\pi_1 + 6\pi_2 + 7\pi_3 \\
& \mu_1 + 2\pi_1 \leq 5 \\
& \mu_1 + \pi_2 \leq 10 \\
& \mu_1 + 3\pi_3 \leq 2 \\
& \mu_2 + 3\pi_1 \leq 5 \\
& \mu_2 + 4\pi_2 \leq 4 \\
& \mu_2 + 2\pi_3 \leq 6 \\
& \mu_3 + 4\pi_1 \leq 5 \\
& \mu_3 + 5\pi_2 \leq 4 \\
& \mu_3 + 3\pi_3 \leq 6 \\
& \mu_1, \mu_2, \mu_3 \geq 0 \\
& \pi_1, \pi_2, \pi_3 \leq 0
\end{aligned}$$

Una soluzione duale a mano. Con $\bar{\pi}_m = 0$ per ogni macchina, i vincoli (7.3b) diventano $\mu_j \leq c_{jm}$ per ogni m : il valore più grande ammissibile è

$$\bar{\mu}_1 = \min\{5, 10, 2\} = 2, \quad \bar{\mu}_2 = \min\{5, 4, 6\} = 4, \quad \bar{\mu}_3 = \min\{5, 4, 6\} = 4,$$

con valore $2 + 4 + 4 + 0 = 10$. Per la dualità debole ogni soluzione duale ammissibile vale al più quanto il rilassamento LP, che a sua volta vale al più quanto il MILP:

$$10 \leq z(LP) \leq z(MILP) \leq 11.$$

La ricetta ha un significato: «ogni lavoro costa almeno il suo costo minimo» è un lower bound che chiunque scriverebbe; il duale lo formalizza e dice come migliorarlo, con $\pi_m < 0$ dove la disponibilità è stretta.

Quello che dice il solver. L'ottimo del rilassamento LP è $z(LP) = 53/5 = 10,6$ (uguale all'ottimo del duale: dualità forte), con duali $\tilde{\mu} = (2, 4, 8, 5)$ e $\tilde{\pi} = (0, -0,2, 0)$: la macchina 2 è la risorsa stretta. Il MILP ha ottimo $z(MILP) = 11$ con $\tilde{x}_{13} = \tilde{x}_{22} = \tilde{x}_{31} = 1$: l'euristica best-fit aveva trovato la soluzione ottima, ma solo il solver lo certifica, perché il bound duale si fermava a 10 (e il rilassamento a 10,6: nessun intero fra 10,6 e 11, dunque in realtà $\lceil 53/5 \rceil = 11$ chiude il gap — un argomento che vale perché i costi sono interi).

UB (best-fit)	LB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
11	10	53/5	11	0,0%

Considerazioni aggiuntive

- Per ogni lavoro j e macchina m , $x_{jm} \in \{0, 1\}$ implica $x_{jm} \leq 1$: le $n k$ disuguaglianze $x_{jm} \leq 1$ sono valide per il modello (7.1), ma sono implicate dai vincoli (7.1b) e non rafforzano il rilassamento (infatti $z(LP) = z(LP^+)$).
- Se un lavoro j non sta su una macchina m ($t_{jm} > a_m$), la variabile x_{jm} può essere fissata a zero prima di risolvere: il vincolo (7.1c) la forzerebbe comunque, ma il modello è più piccolo e il rilassamento non peggiora.

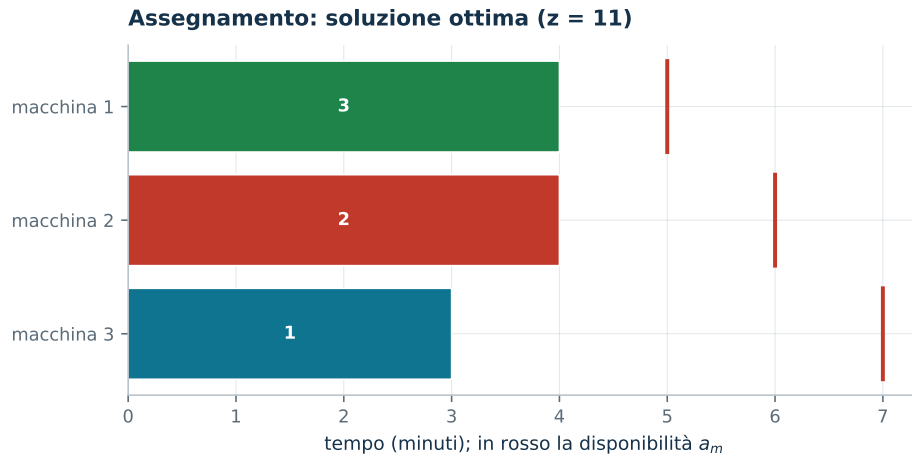


Figura 7.1: La soluzione ottima dell'istanza: i lavori assegnati a ciascuna macchina e, in rosso, la disponibilità a_m .

Domande di modellazione aggiuntive

Domanda di modellazione 7.1 — i lavori 1 e 3 sulla stessa macchina

I lavori 1 e 3 usano lo stesso utensile e devono essere eseguiti dalla stessa macchina. Come cambia il modello? Qual è il nuovo ottimo per l'istanza?

Domanda di modellazione 7.2 — costo fisso per macchina usata

Ogni macchina che esegue almeno un lavoro costa in più $g_m = 3$ euro di accensione. Modellare il costo fisso e trovare il nuovo ottimo. Quale legame del capitolo 3 entra in gioco?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	11	soluzione euristica (UB)
LB	10	bound certificato (LB)
$z(MILP)$	11	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 9.1\%.$$

7.2 Macchine con costo fisso di utilizzo

Script: `python/fam07_2_costofisso.py`

[Notebook in Colab](#)

Problema 7.2 — macchine con costo fisso

Un'azienda deve eseguire $n \in \mathbb{Z}_{\geq 1}$ lavori e dispone di $k \in \mathbb{Z}_{\geq 1}$ macchine. Per ogni lavoro $j \in \{1, 2, \dots, n\}$ e ogni macchina $m \in \{1, 2, \dots, k\}$, il valore $t_{jm} \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti del lavoro j sulla macchina m . Per ogni macchina $m \in \{1, 2, \dots, k\}$, il valore $a_m \in \mathbb{Q}_{>0}$ è la disponibilità in minuti e il valore $c_m \in \mathbb{Q}_{>0}$ è il costo in euro se la macchina viene usata. Ogni macchina esegue un lavoro alla volta. L'azienda vuole assegnare tutti i lavori alle macchine minimizzando il costo delle macchine usate.

Il problema a parole. Decidiamo quali macchine accendere e su quale macchina eseguire ciascun lavoro. *L'obiettivo:* costo totale delle macchine accese. *I vincoli:* ogni lavoro su esattamente una macchina; una macchina spenta non esegue lavori; una macchina accesa non supera la sua disponibilità. Rispetto al problema 7.1 il costo non sta più sugli assegnamenti ma sulle macchine: servono variabili di *attivazione*.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	numero di macchine, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	tempo di lavorazione del lavoro j sulla macchina m
a_m	$\in \mathbb{Q}_{>0}$	disponibilità della macchina m
c_m	$\in \mathbb{Q}_{>0}$	costo fisso se la macchina m è usata

Variabili decisionali

Introduciamo le seguenti $n k + k$ variabili binarie:

$$\begin{cases} x_{jm} = 1 \text{ se il lavoro } j \text{ è eseguito dalla macchina } m, 0 \text{ altrimenti,} \\ y_m = 1 \text{ se la macchina } m \text{ è usata, 0 altrimenti,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Usando queste variabili, un modello BIP per il problema è il seguente:

Modello 7.2 — macchine con costo fisso

$$\min \sum_{m=1}^k c_m y_m \quad (7.4a)$$

$$\text{soggetto a } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.4b)$$

$$-\sum_{j=1}^n t_{jm} x_{jm} + a_m y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.4c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.4d)$$

$$y_m \in \{0, 1\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.4e)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.4a) minimizza il costo totale delle macchine usate;
- i vincoli lineari (7.4b) assicurano che ogni lavoro sia assegnato a esattamente una macchina (n vincoli lineari);
- i vincoli lineari (7.4c) collegano le variabili di assegnamento con le variabili di utilizzo e impongono le restrizioni di capacità. In particolare, per ogni macchina, garantiscono che se almeno un lavoro le è assegnato allora la macchina è usata, che a una macchina non usata non è assegnato alcun lavoro e che, se la macchina è usata, il tempo complessivo dei lavori assegnati non supera la sua disponibilità (k vincoli lineari);
- i vincoli (7.4d) e (7.4e) definiscono le variabili del modello.

Legame fra le variabili

- Per ogni macchina m , se il tempo di lavorazione complessivo dei lavori assegnati alla macchina è positivo, allora la macchina deve essere usata. Questo legame logico si scrive come l'implicazione

$$\sum_{j=1}^n t_{jm} x_{jm} > 0 \implies y_m = 1.$$

Dalla contronominale otteniamo

$$y_m = 0 \implies \sum_{j=1}^n t_{jm} x_{jm} = 0 :$$

se la macchina m non è usata, il tempo complessivo dei lavori assegnati deve essere nullo — cioè, poiché $t_{jm} > 0$ per ogni j , nessun lavoro può esserle assegnato.

L'implicazione è imposta direttamente dai vincoli (7.4c) del modello (7.4). Infatti, per ogni macchina m , il vincolo (7.4c) dà $\sum_{j=1}^n t_{jm} x_{jm} \leq a_m y_m$; se $\sum_{j=1}^n t_{jm} x_{jm} > 0$, allora $a_m y_m \geq \sum_{j=1}^n t_{jm} x_{jm} > 0$, dunque $y_m > 0$ e, essendo y_m binaria, $y_m = 1$. Viceversa, se $y_m = 0$, il vincolo dà $\sum_{j=1}^n t_{jm} x_{jm} \leq a_m \cdot 0 = 0$ e, poiché $t_{jm} > 0$ e $x_{jm} \geq 0$ per ogni j , questo forza $x_{jm} = 0$ per ogni j , cioè $\sum_{j=1}^n t_{jm} x_{jm} = 0$.

- Viceversa, per ogni macchina m , se il tempo di lavorazione complessivo dei lavori assegnati alla macchina è nullo, allora la macchina non è usata:

$$\sum_{j=1}^n t_{jm} x_{jm} = 0 \implies y_m = 0,$$

e per contronominale

$$y_m = 1 \implies \sum_{j=1}^n t_{jm} x_{jm} > 0 :$$

se la macchina è usata, almeno un lavoro le è assegnato.

Questa implicazione *non* è imposta dai vincoli del modello (7.4) (la soluzione $y_m = 1$ senza lavori è ammissibile), ma segue dalla funzione obiettivo in ogni soluzione ottima.

Infatti, poiché $c_m > 0$, se $x_{jm} = 0$ per ogni j e $y_m = 1$, porre $y_m = 0$ mantiene tutti i vincoli soddisfatti (il vincolo (7.4c) diventa $0 \geq 0$) e riduce l'obiettivo della quantità strettamente positiva c_m : la soluzione di partenza non era ottima. Quindi, in ogni soluzione ottima, una macchina senza lavori non è usata.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 m = gp.Model("costo_fisso"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, vtype=GRB.BINARY, name="y") #
4 m.setObjective(gp.quicksum(c[mm] * y[mm] for mm in range(k)),
5                 GRB.MINIMIZE)
6 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assegna")
7 m.addConstrs((-gp.quicksum(t[j][mm] * x[j, mm] for j in range(n))
8               + a[mm] * y[mm] >= 0 for mm in range(k)),
9               name="link")
10 m.optimize()
```

L'istanza

$n = 3$ lavori, $k = 3$ macchine:

t_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	6	5	3
$j = 2$	5	10	2
$j = 3$	20	13	10

	$m = 1$	$m = 2$	$m = 3$
c_m	8	7	5
a_m	25	20	12

Il modello (7.4) per l'istanza — si minimizza, e le tre famiglie di vincoli sono nell'ordine assegnamento, attivazione e dominio:

Modello 7.2 sull'istanza

$$\begin{aligned}
 \min \quad & 8y_1 + 7y_2 + 5y_3 \\
 \text{vincoli} \quad & x_{11} + x_{12} + x_{13} = 1 \\
 & x_{21} + x_{22} + x_{23} = 1 \\
 & x_{31} + x_{32} + x_{33} = 1 \\
 & 25y_1 - 6x_{11} - 5x_{21} - 20x_{31} \geq 0 \\
 & 20y_2 - 5x_{12} - 10x_{22} - 13x_{32} \geq 0 \\
 & 12y_3 - 3x_{13} - 2x_{23} - 10x_{33} \geq 0 \\
 & x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \in \{0, 1\} \\
 & y_1, y_2, y_3 \in \{0, 1\}
 \end{aligned}$$

Euristica costruttiva: il bound primale

Le stesse tre famiglie del problema 7.1, con il criterio del best-fit che diventa il *tempo minimo* (non ci sono costi di assegnamento: conviene consumare poca disponibilità). Il best-fit trova $x_{13} = x_{23} = x_{32} = 1$:

- **Passo 1.** Lavoro 1: $ra = (25, 20, 12)$; tempi 6, 5, 3, tutti ammissibili; minimo sulla macchina 3: $x[1][3] = 1$, $ra[3] = 12 - 3 = 9$.
- **Passo 2.** Lavoro 2: $ra = (25, 20, 9)$; tempi 5, 10, 2; minimo sulla macchina 3: $x[2][3] = 1$, $ra[3] = 9 - 2 = 7$.
- **Passo 3.** Lavoro 3: $ra = (25, 20, 7)$; la macchina 3 non basta ($10 > 7$); fra 20 e 13 il minimo è la macchina 2: $x[3][2] = 1$, $ra[2] = 20 - 13 = 7$.

Le macchine usate sono la 2 e la 3: $\bar{y} = (0, 1, 1)$, valore $7 + 5 = 12$, quindi $z(MILP) \leq 12$. Next-fit e first-fit usano le macchine 1 e 2 (valore 15); anche le varianti «prima le macchine già aperte» danno 15. Il best-fit a tempo minimo è l'unica delle cinque a trovare 12.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Rilassamento LP e duale: il bound duale

Il rilassamento LP sostituisce (7.4d) e (7.4e) con $x_{jm} \geq 0$ e $y_m \geq 0$. Con una variabile duale libera μ_j per ogni vincolo (7.4b) e una non negativa π_m per ogni vincolo (7.4c) (verso $\geq$ in un minimo), il duale è:

Duale del rilassamento LP del modello 7.2

$$\max \sum_{j=1}^n \mu_j \quad (7.5a)$$

$$\text{soggetto a } \mu_j - t_{jm} \pi_m \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.5b)$$

$$a_m \pi_m \leq c_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.5c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.5d)$$

$$\pi_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.5e)$$

Descrizione. La variabile μ_j è il valore del lavoro j e π_m il prezzo di una unità di capacità della macchina m . L'obiettivo (7.5a) valuta i soli lavori: la capacità, di per sé, non si paga. I vincoli (7.5b) sono le colonne delle x_{jm} : assegnare il lavoro j alla macchina m incassa μ_j e consuma t_{jm} di capacità al prezzo π_m , e il saldo non può essere positivo perché le x_{jm} non compaiono nell'obiettivo primale. I vincoli (7.5c) sono le colonne delle y_m : tutta la capacità a_m che l'accensione della macchina m mette a disposizione, valutata a π_m , non può valere più del costo fisso c_m .

Per l'istanza:

Il modello dell'istanza

$$\begin{aligned}
& \max \mu_1 + \mu_2 + \mu_3 \\
& \mu_1 - 6\pi_1 \leq 0 \\
& \mu_1 - 5\pi_2 \leq 0 \\
& \mu_1 - 3\pi_3 \leq 0 \\
& \mu_2 - 5\pi_1 \leq 0 \\
& \mu_2 - 10\pi_2 \leq 0 \\
& \mu_2 - 2\pi_3 \leq 0 \\
& \mu_3 - 20\pi_1 \leq 0 \\
& \mu_3 - 13\pi_2 \leq 0 \\
& \mu_3 - 10\pi_3 \leq 0 \\
& 25\pi_1 \leq 8 \\
& 20\pi_2 \leq 7 \\
& 12\pi_3 \leq 5 \\
& \mu_1, \mu_2, \mu_3 \geq 0 \\
& \pi_1, \pi_2, \pi_3 \geq 0
\end{aligned}$$

Una soluzione duale a mano. I vincoli (7.5c) dicono quanto può valere al più ogni π_m : prendiamo il massimo, $\bar{\pi}_m = c_m/a_m$, cioè

$$\bar{\pi}_1 = \frac{8}{25}, \quad \bar{\pi}_2 = \frac{7}{20}, \quad \bar{\pi}_3 = \frac{5}{12}$$

(il costo per minuto di ogni macchina). I vincoli (7.5b) danno allora $\mu_j \leq t_{jm}\bar{\pi}_m$ per ogni m , e il massimo ammissibile è

$$\begin{aligned}
\bar{\mu}_1 &= \min\left\{6 \cdot \frac{8}{25}, 5 \cdot \frac{7}{20}, 3 \cdot \frac{5}{12}\right\} = \min\left\{\frac{48}{25}, \frac{7}{4}, \frac{5}{4}\right\} = \frac{5}{4}, \\
\bar{\mu}_2 &= \min\left\{5 \cdot \frac{8}{25}, 10 \cdot \frac{7}{20}, 2 \cdot \frac{5}{12}\right\} = \min\left\{\frac{8}{5}, \frac{7}{2}, \frac{5}{6}\right\} = \frac{5}{6}, \\
\bar{\mu}_3 &= \min\left\{20 \cdot \frac{8}{25}, 13 \cdot \frac{7}{20}, 10 \cdot \frac{5}{12}\right\} = \min\left\{\frac{32}{5}, \frac{91}{20}, \frac{25}{6}\right\} = \frac{25}{6}.
\end{aligned}$$

Valore: $\frac{5}{4} + \frac{5}{6} + \frac{25}{6} = \frac{25}{4}$. Quindi

$$\frac{25}{4} \leq z(MILP) \leq 12.$$

La lettura: ogni lavoro «paga» almeno il costo per minuto della macchina più economica per lui, moltiplicato per il suo tempo. È un bound debole: il costo fisso di una macchina si paga per intero appena la si usa, ma il rilassamento lo spalma sui minuti.

Quello che dice il solver. L'ottimo del rilassamento senza i bound è proprio $z(LP) = 25/4$: la soluzione costruita a mano è ottima per il duale. Il rilassamento con i bound con $y_m \leq 1$ e $x_{jm} \leq 1$ vale $z(LP^+) = 1273/200 = 6,365$; aggiungendo i vincoli disaggregati $x_{jm} \leq y_m$ (validi, si veda sotto) sale a $440/67 = 6,567$. L'ottimo intero è $z(MILP) = 12$, con le macchine 2 e 3 accese e $\tilde{x}_{12} = \tilde{x}_{23} = \tilde{x}_{33} = 1$: l'euristica aveva trovato l'ottimo, e il gap fra 6,25 e 12 misura quanto è lontano il rilassamento dall'intero in questa famiglia di modelli, dove i costi fissi «frazionati» ingannano l'LP.

LB (best-fit)	UB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
12	25/4	25/4	1273/200	12	0,0%

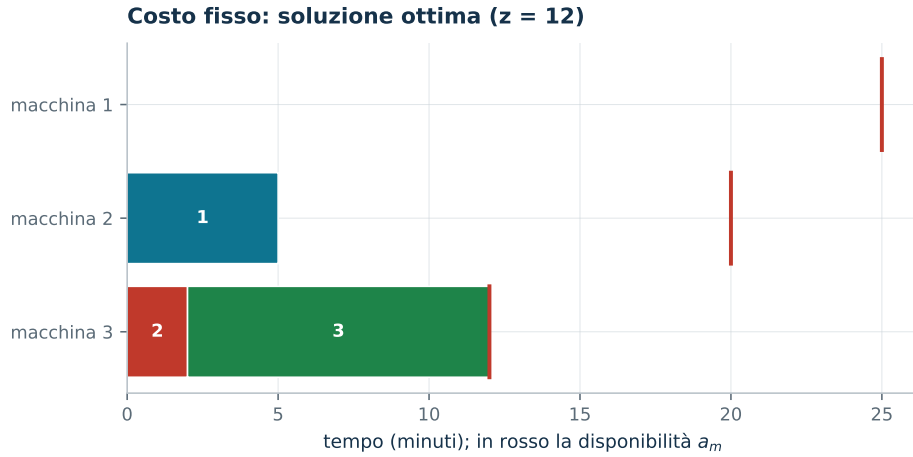


Figura 7.2: La soluzione ottima del problema 7.2: si accendono le macchine 2 e 3.

Considerazioni aggiuntive

- Le disuguaglianze $y_m \leq 1$ (k) e $x_{jm} \leq 1$ ($n k$) sono valide; le seconde sono implicate da (7.4b), le prime no: infatti rafforzano il rilassamento ($6,25 \rightarrow 6,365$).
- Per ogni macchina m , «se almeno un lavoro è assegnato a m allora m è usata» è l'implicazione booleana

$$(x_{1m} \text{ OR } x_{2m} \text{ OR } \dots \text{ OR } x_{nm}) \implies y_m,$$

la cui contronominale è

$$\text{NOT } y_m \implies (\text{NOT } x_{1m} \text{ AND } \dots \text{ AND } \text{NOT } x_{nm}) :$$

se la macchina non è usata, nessun lavoro le è assegnato. Come espressione booleana si scrive

$$\text{NOT } (x_{1m} \text{ OR } \dots \text{ OR } x_{nm}) \text{ OR } y_m,$$

che con De Morgan e la proprietà distributiva diventa la CNF

$$(\text{NOT } x_{1m} \text{ OR } y_m) \text{ AND } (\text{NOT } x_{2m} \text{ OR } y_m) \text{ AND } \dots \text{ AND } (\text{NOT } x_{nm} \text{ OR } y_m),$$

cioè i vincoli $1 - x_{jm} + y_m \geq 1$:

$$x_{jm} \leq y_m, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}$$

($n k$ disuguaglianze valide, la forma *disaggregata* del legame). Il modello le implica — lo abbiamo dimostrato sopra — ma il rilassamento no: aggiunte, portano $z(LP^+)$ a 440/67. È l'esempio concreto della sezione 3.1: stesso insieme di soluzioni intere, rilassamento più stretto.

- Il verso opposto, «se nessun lavoro è assegnato allora la macchina non è usata», è la clausola

$$\text{NOT } y_m \text{ OR } x_{1m} \text{ OR } \dots \text{ OR } x_{nm},$$

cioè

$$\sum_{j=1}^n x_{jm} \geq y_m, \quad \forall m \in \{1, 2, \dots, k\} :$$

non è una disuguaglianza valida (il modello ammette $y_m = 1$ senza lavori) ma si può aggiungere senza perdere l'ottimo, perché $c_m > 0$.

Domande di modellazione aggiuntive

Domanda di modellazione 7.3 — utilizzo minimo di una macchina accesa

Accendere una macchina per pochi minuti non ha senso: ogni macchina usata deve lavorare almeno $\ell = 8$ minuti. Modellare il vincolo e trovare il nuovo ottimo.

Domanda di modellazione 7.4 — legame fra due attivazioni

La macchina 3 condivide l'alimentazione con la macchina 1: se si usa la macchina 1 si deve usare anche la 3. Scrivere il vincolo e discutere che cosa impone e che cosa non impone.

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	12	soluzione euristica (UB)
LB	$\frac{25}{4}$	bound certificato (LB)
$z(MILP)$	12	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 47.9\%.$$

7.3 Selezione di lavori con ricavo e macchine a costo fisso

Script: `python/fam07_3_selezione.py`

[Notebook in Colab](#)

Problema 7.3 — selezione di lavori

Un'azienda può eseguire $n \in \mathbb{Z}_{\geq 1}$ lavori e dispone di $k \in \mathbb{Z}_{\geq 1}$ macchine. Per ogni lavoro $j \in \{1, 2, \dots, n\}$, il valore $t_j \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti (uguale su tutte le macchine) e il valore $r_j \in \mathbb{Q}_{>0}$ è il ricavo in euro se il lavoro viene eseguito. Per ogni macchina $m \in \{1, 2, \dots, k\}$, il valore $a_m \in \mathbb{Q}_{>0}$ è la disponibilità in minuti e il valore $c_m \in \mathbb{Q}_{>0}$ è il costo in euro se la macchina viene usata. Ogni macchina esegue un lavoro alla volta. L'azienda vuole scegliere quali lavori eseguire, e su quali macchine, per massimizzare il profitto: ricavi dei lavori eseguiti meno costi delle macchine usate.

Il problema a parole. Decidiamo quali lavori eseguire, su quali macchine, e quali macchine accendere. *L'obiettivo:* profitto massimo. *I vincoli:* ogni lavoro al più su una macchina; nessun lavoro su una macchina spenta; disponibilità rispettata. È il problema 7.2 in cui i lavori non sono più obbligatori e hanno un ricavo: un problema di *massimo*, e i ruoli dei bound si scambiano.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	numero di macchine, $m \in \{1, 2, \dots, k\}$
t_j	$\in \mathbb{Q}_{>0}$	tempo di lavorazione del lavoro j
r_j	$\in \mathbb{Q}_{>0}$	ricavo se il lavoro j è eseguito
a_m	$\in \mathbb{Q}_{>0}$	disponibilità della macchina m
c_m	$\in \mathbb{Q}_{>0}$	costo fisso se la macchina m è usata

Variabili decisionali

Introduciamo le seguenti $n k + k$ variabili binarie:

$$\begin{cases} x_{jm} = 1 \text{ se il lavoro } j \text{ è eseguito dalla macchina } m, 0 \text{ altrimenti,} \\ y_m = 1 \text{ se la macchina } m \text{ è usata, } 0 \text{ altrimenti,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Modello 7.3 — selezione di lavori

$$\max \sum_{j=1}^n \sum_{m=1}^k r_j x_{jm} - \sum_{m=1}^k c_m y_m \quad (7.6a)$$

$$\text{soggetto a } \sum_{m=1}^k x_{jm} \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.6b)$$

$$\sum_{j=1}^n t_j x_{jm} - a_m y_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.6c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.6d)$$

$$y_m \in \{0, 1\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.6e)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.6a) massimizza il profitto totale, differenza fra il ricavo dei lavori eseguiti e il costo delle macchine usate;
- i vincoli lineari (7.6b) assicurano che ogni lavoro sia assegnato ad al più una macchina, così che nessun lavoro venga eseguito due volte (n vincoli lineari);
- i vincoli lineari (7.6c) collegano le variabili di assegnamento con le variabili di utilizzo e impongono le restrizioni di capacità: se almeno un lavoro è assegnato a una macchina allora la macchina è usata, a una macchina non usata non è assegnato alcun lavoro, e il tempo complessivo dei lavori di una macchina usata non supera la disponibilità (k vincoli lineari);
- i vincoli (7.6d) e (7.6e) definiscono le variabili del modello.

Legame fra le variabili

Il legame è quello del problema 7.2, con t_j al posto di t_{jm} : riportiamo le due implicazioni, perché il verso «di ottimalità» cambia segno.

- L'implicazione

$$\sum_{j=1}^n t_j x_{jm} > 0 \implies y_m = 1,$$

$$\text{per contronominale: } y_m = 0 \implies \sum_{j=1}^n t_j x_{jm} = 0,$$

dice che una macchina non usata non ha lavori. È imposta dai vincoli (7.6c): se $\sum_{j=1}^n t_j x_{jm} > 0$ allora $a_m y_m \geq \sum_{j=1}^n t_j x_{jm} > 0$, quindi $y_m = 1$; viceversa, se $y_m = 0$ allora $\sum_{j=1}^n t_j x_{jm} \leq 0$ e, con $t_j > 0$ e $x_{jm} \geq 0$, tutte le x_{jm} sono nulle.

- L'implicazione opposta

$$\sum_{j=1}^n t_j x_{jm} = 0 \implies y_m = 0,$$

$$\text{per contronominale: } y_m = 1 \implies \sum_{j=1}^n t_j x_{jm} > 0,$$

dice che una macchina usata ha almeno un lavoro. Non è imposta dai vincoli, e segue dall'obiettivo in ogni ottimo: poiché $c_m > 0$, se $y_m = 1$ senza lavori, porre $y_m = 0$ resta ammissibile e *aumenta* il profitto di c_m — in un problema di massimo il verso del miglioramento si inverte, la struttura dell'argomento no.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 m = gp.Model("selezione"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(r[j] * x[j, mm] for j in range(n)
5                        for mm in range(k))
6                - gp.quicksum(c[mm] * y[mm] for mm in range(k)),
7                GRB.MAXIMIZE)
8 m.addConstrs((x.sum(j, "*") <= 1 for j in range(n)), name="al_piu_una")
9 m.addConstrs((gp.quicksum(t[j] * x[j, mm] for j in range(n))
10              - a[mm] * y[mm] <= 0 for mm in range(k)), name="link")
11 m.optimize()
```

L'istanza

$n = 3$ lavori, $k = 3$ macchine:

	$m = 1$	$m = 2$	$m = 3$
a_m	105	110	100
c_m	20	30	15

	$j = 1$	$j = 2$	$j = 3$
t_j	25	40	75
r_j	10	15	30

Il modello (7.6) per l'istanza — si massimizza, e le tre famiglie di vincoli sono nell'ordine selezione, attivazione e dominio:

Modello 7.3 sull'istanza

$$\begin{aligned}
 & \max 10x_{11} + 10x_{12} + 10x_{13} + 15x_{21} + 15x_{22} + 15x_{23} + 30x_{31} + 30x_{32} + 30x_{33} - 20y_1 - 30y_2 - 15y_3 \\
 & x_{11} + x_{12} + x_{13} \leq 1 \\
 & x_{21} + x_{22} + x_{23} \leq 1 \\
 & x_{31} + x_{32} + x_{33} \leq 1 \\
 & 25x_{11} + 40x_{21} + 75x_{31} - 105y_1 \leq 0 \\
 & 25x_{12} + 40x_{22} + 75x_{32} - 110y_2 \leq 0 \\
 & 25x_{13} + 40x_{23} + 75x_{33} - 100y_3 \leq 0 \\
 & x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \in \{0, 1\} \\
 & y_1, y_2, y_3 \in \{0, 1\}
 \end{aligned}$$

Euristica costruttiva: il bound primale

In un problema di massimo una soluzione ammissibile dà un *lower bound*. Le tre euristiche del problema 7.1 si adattano con una regola in più: un lavoro che non sta su nessuna macchina viene *saltato* (non è obbligatorio). Il best-fit sceglie la macchina *più piena* fra quelle che bastano (minima disponibilità residua), per concentrare i lavori su poche macchine:

- **Passo 1.** Lavoro 1 ($t_1 = 25$): $ra = (105, 110, 100)$, tutte bastano; la più piena è la macchina 3: $x[1][3] = 1$, $ra[3] = 75$.
- **Passo 2.** Lavoro 2 ($t_2 = 40$): $ra = (105, 110, 75)$; la più piena è ancora la 3: $x[2][3] = 1$, $ra[3] = 35$.
- **Passo 3.** Lavoro 3 ($t_3 = 75$): $ra = (105, 110, 35)$; la macchina 3 non basta; fra 1 e 2 la più piena è la 1: $x[3][1] = 1$, $ra[1] = 30$.

Soluzione: $\bar{x}_{13} = \bar{x}_{23} = \bar{x}_{31} = 1$, $\bar{y} = (1, 0, 1)$, profitto $10 + 15 + 30 - 20 - 15 = 20$, quindi $z(MILP) \geq 20$. Next-fit e first-fit riempiono prima la macchina 1 (lavori 1 e 2) e mettono il lavoro 3 sulla 2: profitto $55 - 50 = 5$.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Rilassamento LP e duale: il bound duale

Rilassando $x_{jm} \geq 0$ e $y_m \geq 0$, con variabili duali non negative μ_j per i vincoli (7.6b) e π_m per i vincoli (7.6c) (versi $\leq$ in un massimo), il duale è:

Duale del rilassamento LP del modello 7.3

$$\min \sum_{j=1}^n \mu_j \quad (7.7a)$$

$$\text{soggetto a } \mu_j + t_j \pi_m \geq r_j, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.7b)$$

$$-a_m \pi_m \geq -c_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.7c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.7d)$$

$$\pi_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.7e)$$

Descrizione. La variabile μ_j è il prezzo del diritto di eseguire il lavoro j e π_m quello di una unità di capacità della macchina m . L'obiettivo (7.7a) somma i prezzi dei diritti sui lavori. I vincoli (7.7b) sono le colonne delle x_{jm} : eseguire il lavoro j sulla macchina m consuma il diritto, al prezzo μ_j , e t_j di capacità, al prezzo π_m ; i due insieme devono coprire il ricavo r_j che quel lavoro porta. I vincoli (7.7c) sono le colonne delle y_m : la capacità a_m che l'accensione della macchina rende disponibile, valutata a π_m , non può costare più del costo fisso c_m che si paga per accenderla.

Per l'istanza:

Il modello dell'istanza

$$\begin{aligned} \min \quad & \mu_1 + \mu_2 + \mu_3 \\ & \mu_1 + 25\pi_1 \geq 10 \\ & \mu_1 + 25\pi_2 \geq 10 \\ & \mu_1 + 25\pi_3 \geq 10 \\ & \mu_2 + 40\pi_1 \geq 15 \\ & \mu_2 + 40\pi_2 \geq 15 \\ & \mu_2 + 40\pi_3 \geq 15 \\ & \mu_3 + 75\pi_1 \geq 30 \\ & \mu_3 + 75\pi_2 \geq 30 \\ & \mu_3 + 75\pi_3 \geq 30 \\ & -105\pi_1 \geq -20 \\ & -110\pi_2 \geq -30 \\ & -100\pi_3 \geq -15 \\ & \mu_1, \mu_2, \mu_3, \pi_1, \pi_2, \pi_3 \geq 0 \end{aligned}$$

Una soluzione duale a mano. I vincoli (7.7c) limitano ogni π_m dall'alto: $\bar{\pi}_m = c_m/a_m$, cioè $\bar{\pi}_1 = \frac{20}{105} = \frac{4}{21}$, $\bar{\pi}_2 = \frac{30}{110} = \frac{3}{11}$, $\bar{\pi}_3 = \frac{15}{100} = \frac{3}{20}$. Ogni μ_j deve poi soddisfare $\mu_j \geq r_j - t_j \bar{\pi}_m$ per ogni m e $\mu_j \geq 0$: il più piccolo valore ammissibile è il massimo di questi,

$$\begin{aligned} \bar{\mu}_1 &= \max\left\{10 - 25 \cdot \frac{4}{21}, 10 - 25 \cdot \frac{3}{11}, 10 - 25 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{110}{21}, \frac{35}{11}, \frac{25}{4}, 0\right\} = \frac{25}{4}, \\ \bar{\mu}_2 &= \max\left\{15 - 40 \cdot \frac{4}{21}, 15 - 40 \cdot \frac{3}{11}, 15 - 40 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{155}{21}, \frac{45}{11}, 9, 0\right\} = 9, \\ \bar{\mu}_3 &= \max\left\{30 - 75 \cdot \frac{4}{21}, 30 - 75 \cdot \frac{3}{11}, 30 - 75 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{110}{7}, \frac{105}{11}, \frac{75}{4}, 0\right\} = \frac{75}{4}. \end{aligned}$$

Valore: $\frac{25}{4} + 9 + \frac{75}{4} = 34$. Per la dualità debole

$$20 \leq z(MILP) \leq 34.$$

La lettura: μ_j è «quanto vale al più il lavoro j » una volta pagato, per ogni minuto, il costo fisso spalmato della macchina meno costosa al minuto (la 3, con $\frac{3}{20}$ euro al minuto).

Quello che dice il solver. L'ottimo del rilassamento senza i bound è $z(LP) = 34$: anche qui la soluzione a mano è ottima per il duale. Con i limiti $x_{jm} \leq 1$ e $y_m \leq 1$ il rilassamento con i bound scende a $z(LP^+) = 680/21 = 32,38$. L'ottimo intero è $z(MILP) = 25$: i lavori 1 e 3 sulla macchina 3 ($25 + 75 = 100 \leq 100$, esattamente la disponibilità), profitto $40 - 15$; il lavoro 2 non conviene, perché richiederebbe una seconda macchina ($c_1 = 20 > r_2 = 15$). L'euristica si era fermata a 20: aveva messo il lavoro 2 sulla macchina 3 lasciando fuori il 3, che poi ha aperto la macchina 1. Gap del 20%.

LB (best-fit)	UB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
20	34	34	680/21	25	20,0%

Considerazioni aggiuntive

- $y_m \leq 1$ (k) e $x_{jm} \leq 1$ (n k) sono disuguaglianze valide; le seconde sono implicate da (7.6b), le prime rafforzano il rilassamento ($34 \rightarrow 32,38$).
- I vincoli disaggregati $x_{jm} \leq y_m$, che vengono dall'implicazione

$$(x_{1m} \text{ OR } \dots \text{ OR } x_{nm}) \implies y_m$$

già vista nel problema 7.2, sono validi e implicati dal modello, ma rafforzano il rilassamento.

- Se $r_j < \min_m c_m$ e il lavoro j è l'unico su una macchina, eseguirlo non conviene mai: è quanto accade al lavoro 2 nell'istanza. Un ragionamento di dominanza che il solver fa da sé, ma che a mano aiuta a indovinare l'ottimo.

Domande di modellazione aggiuntive

Domanda di modellazione 7.5 — tutti i lavori obbligatori

L'azienda ha già firmato i contratti: tutti i lavori vanno eseguiti. Come cambia il modello e quanto costa l'obbligo sull'istanza?

Domanda di modellazione 7.6 — un lavoro condizionato a un altro

Il lavoro 3 richiede un semilavorato prodotto dal lavoro 2: si può eseguire il lavoro 3 solo se si esegue anche il lavoro 2 (su qualunque macchina). Scrivere il vincolo e trovare il nuovo ottimo.

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	20	soluzione euristica (LB)
UB	34	bound certificato (UB)
$z(MILP)$	25	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 20.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 56.0\%.$$

7.4 Lavori in parallelo: il tempo di lavorazione come massimo

Script: `python/fam07_4_parallel.py`

[Notebook in Colab](#)

Problema 7.4 — lavori in parallelo

Un'azienda deve eseguire $n \in \mathbb{Z}_{\geq 1}$ lavori con $k \in \mathbb{Z}_{\geq 1}$ macchine. Per ogni lavoro $j \in \{1, 2, \dots, n\}$ e ogni macchina $m \in \{1, 2, \dots, k\}$, il valore $t_{jm} \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti del lavoro j sulla macchina m . Ogni macchina $m \in \{1, 2, \dots, k\}$ può eseguire al più $p_m \in \mathbb{Z}_{\geq 1}$ lavori, e li esegue in parallelo: tutti i lavori assegnati alla stessa macchina partono insieme. L'azienda vuole assegnare i lavori minimizzando la somma dei tempi di lavorazione delle macchine, dove il tempo di lavorazione di una macchina è il tempo del lavoro più lungo fra quelli che le sono assegnati.

Il problema a parole. Decidiamo su quale macchina va ogni lavoro. *L'obiettivo:* somma dei tempi di lavorazione delle macchine, ciascuno pari al *massimo* dei tempi dei lavori assegnati. *I vincoli:* ogni lavoro a una macchina; al più p_m lavori sulla macchina m . La novità è l'obiettivo: un massimo non è lineare, ma si linearizza con una variabile continua e n vincoli « $\geq$ » per macchina.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	numero di macchine, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	tempo di lavorazione del lavoro j sulla macchina m
p_m	$\in \mathbb{Z}_{\geq 1}$	numero massimo di lavori (in parallelo) sulla macchina m

Variabili decisionali

Introduciamo le seguenti $n k$ variabili binarie e k variabili non negative:

$$\begin{cases} x_{jm} = 1 \text{ se il lavoro } j \text{ è eseguito dalla macchina } m, 0 \text{ altrimenti,} \\ y_m = \text{tempo di lavorazione della macchina } m, \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Modello 7.4 — lavori in parallelo

$$\min \sum_{m=1}^k y_m \quad (7.8a)$$

$$\text{soggetto a } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.8b)$$

$$\sum_{j=1}^n x_{jm} \leq p_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.8c)$$

$$-t_{jm} x_{jm} + y_m \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.8d)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.8e)$$

$$y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.8f)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.8a) minimizza la somma dei tempi di lavorazione delle macchine;
- i vincoli lineari (7.8b) assicurano che ogni lavoro sia assegnato a esattamente una macchina (n vincoli lineari);
- i vincoli lineari (7.8c) limitano il numero di lavori assegnati a ciascuna macchina (k vincoli lineari);
- i vincoli lineari (7.8d) collegano le variabili di assegnamento con le variabili di tempo di lavorazione. In particolare, per ogni lavoro e ogni macchina, garantiscono che se il lavoro è assegnato alla macchina allora il tempo di lavorazione della macchina è almeno il tempo richiesto da quel lavoro su quella macchina ($n k$ vincoli lineari);
- i vincoli (7.8e) e (7.8f) definiscono le variabili del modello.

Legame fra le variabili

È il pattern della *variabile definita come massimo* (sezione 3.5), in tre passi.

- Per ogni macchina m e ogni lavoro j , se il lavoro è assegnato alla macchina allora il tempo di lavorazione della macchina è almeno il tempo del lavoro:

$$x_{jm} = 1 \implies y_m \geq t_{jm},$$

$$\text{e per contronominale: } y_m < t_{jm} \implies x_{jm} = 0 :$$

se il tempo della macchina è strettamente minore di t_{jm} , il lavoro j non può esserle assegnato.

L'implicazione è imposta direttamente dai vincoli (7.8d), che danno $y_m \geq t_{jm} x_{jm}$: se $x_{jm} = 1$, $y_m \geq t_{jm}$. Viceversa, se $y_m < t_{jm}$, allora x_{jm} non può valere 1, perché il

vincolo forzerebbe $y_m \geq t_{jm}$; essendo binaria, $x_{jm} = 0$. Poiché vale per ogni j , la famiglia (7.8d) impone, per ogni macchina, $y_m \geq \max\{t_{jm}x_{jm} : j \in \{1, 2, \dots, n\}\}$.

- Viceversa, se nessun lavoro è assegnato alla macchina m , il suo tempo di lavorazione è zero:

$$\sum_{j=1}^n x_{jm} = 0 \implies y_m = 0,$$

$$\text{e per contronominale: } y_m > 0 \implies \sum_{j=1}^n x_{jm} > 0.$$

Non è imposta dai vincoli (con $x_{jm} = 0$ per ogni j i vincoli (7.8d) si riducono a $y_m \geq 0$), ma segue dall'obiettivo in ogni ottimo: y_m ha coefficiente $1 > 0$ nell'obiettivo; se nessun lavoro è sulla macchina e $y_m > 0$, porre $y_m = 0$ mantiene i vincoli ($0 \geq 0$) e riduce l'obiettivo della quantità strettamente positiva y_m .

- *Sintesi*: in ogni soluzione ottima

$$y_m = \max\{t_{jm}x_{jm} : j \in \{1, 2, \dots, n\}\},$$

cioè y_m è esattamente il tempo di lavorazione della macchina. Infatti il primo punto dà $y_m \geq$ massimo in ogni soluzione ammissibile; se in una soluzione fosse $y_m > \text{massimo}$, porre y_m uguale al massimo manterrebbe tutti i vincoli (7.8d) (per definizione di massimo) e ridurrebbe l'obiettivo della differenza, strettamente positiva: la soluzione non era ottima. Con nessun lavoro il massimo è 0, e si ritrova il secondo punto.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 m = gp.Model("parallelo"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, name="y")
4 m.setObjective(y.sum(), GRB.MINIMIZE)
5 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assegna")
6 m.addConstrs((x.sum("*", mm) <= p[mm] for mm in range(k)),
7               name="cardinalita")
8 m.addConstrs((-t[j][mm] * x[j, mm] + y[mm] >= 0
9               for j in range(n) for mm in range(k)), name="massimo")
10 m.optimize()
```

L'istanza

$n = 3$ lavori, $k = 3$ macchine, con i tempi del problema 7.2:

t_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	6	5	3
$j = 2$	5	10	2
$j = 3$	20	13	10

	$m = 1$	$m = 2$	$m = 3$
p_m	1	2	2

Il modello (7.8) per l'istanza ha 9 vincoli (7.8d), uno per coppia: $-6x_{11} + y_1 \geq 0$, $-5x_{12} + y_2 \geq 0$, $-3x_{13} + y_3 \geq 0$, $-5x_{21} + y_1 \geq 0$, $-10x_{22} + y_2 \geq 0$, $-2x_{23} + y_3 \geq 0$, $-20x_{31} + y_1 \geq 0$, $-13x_{32} + y_2 \geq 0$, $-10x_{33} + y_3 \geq 0$; i tre vincoli di assegnamento come nel problema 7.2; le cardinalità $x_{11} + x_{21} + x_{31} \leq 1$, $x_{12} + x_{22} + x_{32} \leq 2$, $x_{13} + x_{23} + x_{33} \leq 2$; obiettivo $\min y_1 + y_2 + y_3$.

Euristica costruttiva: il bound primale

Next-fit sulle cardinalità: si riempie la macchina 1 fino a p_1 lavori, poi la 2, e così via; il tempo di ogni macchina è il massimo dei tempi assegnati.

- **Passo 1.** Lavoro 1 sulla macchina 1 (un lavoro $\leq p_1 = 1$): $y[1] = \max(0, 6) = 6$.
- **Passo 2.** La macchina 1 è piena: lavoro 2 sulla macchina 2: $y[2] = \max(0, 10) = 10$.
- **Passo 3.** Lavoro 3 sulla macchina 2 (due lavori $\leq p_2 = 2$): $y[2] = \max(10, 13) = 13$.

Soluzione $\bar{x}_{11} = \bar{x}_{22} = \bar{x}_{32} = 1$, $\bar{y} = (6, 13, 0)$, valore 19: $z(MILP) \leq 19$.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Rilassamento LP e duale: il bound duale

Rilassando (7.8e) in $x_{jm} \geq 0$, con duali μ_j libere (7.8b), $\pi_m \leq 0$ (7.8c, verso $\leq$ in un minimo) e $\lambda_{jm} \geq 0$ (7.8d, verso $\geq$):

Duale del rilassamento LP del modello 7.4

$$\max \sum_{j=1}^n \mu_j + \sum_{m=1}^k p_m \pi_m \quad (7.9a)$$

$$\text{soggetto a } \mu_j + \pi_m - t_{jm} \lambda_{jm} \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.9b)$$

$$\sum_{j=1}^n \lambda_{jm} \leq 1, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.9c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.9d)$$

$$\pi_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.9e)$$

$$\lambda_{jm} \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.9f)$$

Descrizione. La variabile μ_j è il valore del lavoro j , π_m il prezzo (non positivo) di uno dei p_m posti della macchina m e λ_{jm} il prezzo del legame che spinge il massimo y_m sopra t_{jm} . L'obiettivo (7.9a) valuta i lavori e i posti disponibili. I vincoli (7.9b) sono le colonne delle x_{jm} : mettere il lavoro j sulla macchina m soddisfa l'assegnamento e occupa un posto, ma obbliga il massimo della macchina a salire fino a t_{jm} ; il saldo non può essere positivo, perché le x_{jm} non compaiono nell'obiettivo. I vincoli (7.9c) sono le colonne delle y_m : il massimo della macchina m costa 1 nell'obiettivo primale, e i prezzi dei legami che lo spingono verso l'alto non possono valere di più.

Il vincolo (7.9c) è il costo ridotto di y_m : il coefficiente 1 nell'obiettivo del primale limita la somma dei λ_{jm} . **Una soluzione duale a mano.** Ripartiamo quell'unità in parti uguali,

$\bar{\lambda}_{jm} = 1/n = 1/3$ per ogni coppia, e poniamo $\bar{\pi}_m = 0$; i vincoli (7.9b) diventano $\mu_j \leq t_{jm}/3$ per ogni m :

$$\bar{\mu}_1 = \min\{6, 5, 3\}/3 = 1, \quad \bar{\mu}_2 = \min\{5, 10, 2\}/3 = \frac{2}{3}, \quad \bar{\mu}_3 = \min\{20, 13, 10\}/3 = \frac{10}{3},$$

valore $1 + \frac{2}{3} + \frac{10}{3} = 5$. Quindi $5 \leq z(MILP) \leq 19$.

Quello che dice il solver. $z(LP) = 520/49 = 10,61$ (uguale all'ottimo del duale; il rilassamento con i bound coincide, perché $x_{jm} \leq 1$ è implicato). L'ottimo intero è $z(MILP) = 15$: il lavoro 1 sulla macchina 2, i lavori 2 e 3 sulla macchina 3, $\tilde{y} = (0, 5, 10)$. Il bound a mano (5) è lontano dal rilassamento (10,61): la ripartizione uniforme $\lambda_{jm} = 1/3$ è la prima che viene in mente, non la migliore — il duale ottimo distribuisce l'unità di ogni macchina in modo diverso.

LB (next-fit)	UB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
19	5	520/49	15	26,7%

Considerazioni aggiuntive

- $x_{jm} \leq 1$ ($n \cdot k$ disuguaglianze valide) è implicato da (7.8b).
- Il verso «nessun lavoro $\Rightarrow y_m = 0$ » si può imporre esplicitamente con un limite superiore: posto $\bar{t}_m = \max_j t_{jm}$, i vincoli

$$\bar{t}_m \sum_{j=1}^n x_{jm} - y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}$$

(k vincoli) forzano $y_m = 0$ se la macchina è vuota e lasciano a y_m spazio fino a $\bar{t}_m$ altrimenti. *Non* sono disuguaglianze valide (il modello ammette $y_m > 0$ a macchina vuota) ma sono *vincoli che preservano l'ottimo*: si possono aggiungere perché l'obiettivo minimizza $\sum_{m=1}^k y_m$. La distinzione fra «valida» e «preserva l'ottimo» è la stessa fra i due bullet del legame.

Domande di modellazione aggiuntive

Domanda di modellazione 7.7 — minimizzare il tempo della macchina più lenta

Le macchine lavorano tutte insieme e l'azienda vuole finire il prima possibile: minimizzare il *massimo* dei tempi di lavorazione delle macchine (il makespan), non la somma. Modellare e risolvere.

Domanda di modellazione 7.8 — costo fisso se la macchina lavora

Accendere una macchina costa $g_m = 4$ euro (una tantum) e un minuto di lavorazione costa 1 euro: minimizzare il costo totale. Quale legame serve e qual è il big-M più piccolo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	19	soluzione euristica (UB)
LB	5	bound certificato (LB)
$z(MILP)$	15	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 26.7\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 93.3\%.$$

7.5 Una macchina, classi di lavori con setup

Script: `python/fam07_5_classisetup.py`

[Notebook in Colab](#)

Problema 7.5 — classi con setup

Un'azienda ha $n \in \mathbb{Z}_{\geq 1}$ lavori eseguibili su una macchina con disponibilità $a \in \mathbb{Q}_{>0}$ minuti. Per ogni lavoro $j \in \{1, 2, \dots, n\}$, il valore $t_j \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti e il valore $r_j \in \mathbb{Q}_{>0}$ è il ricavo in euro se il lavoro viene eseguito. I lavori sono partizionati in $q \in \mathbb{Z}_{\geq 2}$ classi: per ogni classe $c \in \{1, 2, \dots, q\}$, l'insieme $\mathcal{J}_c \subseteq \{1, 2, \dots, n\}$ contiene i lavori della classe. Se la macchina esegue lavori di una classe c , si paga un costo di setup $f_c \in \mathbb{Q}_{\geq 0}$ euro e si consuma un tempo di setup $s_c \in \mathbb{Q}_{\geq 0}$ minuti per preparare la macchina. La macchina non esegue lavori in parallelo. L'azienda vuole scegliere i lavori da eseguire massimizzando il profitto.

Il problema a parole. Decidiamo quali lavori eseguire e, di conseguenza, quali classi attivare. *L'obiettivo:* ricavi meno costi di setup. *Il vincolo:* tempi dei lavori più tempi di setup delle classi attivate entro la disponibilità. Uno zaino con costi fissi per gruppo: il legame di attivazione, stavolta *disaggregato* fin dall'inizio.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	disponibilità della macchina
t_j, r_j	$\in \mathbb{Q}_{>0}$	tempo e ricavo del lavoro j
q	$\in \mathbb{Z}_{\geq 2}$	numero di classi, $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	lavori della classe c (partizione)
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	costo e tempo di setup della classe c

Variabili decisionali

Introduciamo le seguenti $n + q$ variabili binarie:

$$\begin{cases} x_j = 1 \text{ se il lavoro } j \text{ è eseguito, } 0 \text{ altrimenti,} & \forall j \in \{1, 2, \dots, n\}, \\ y_c = 1 \text{ se almeno un lavoro della classe } c \text{ è eseguito, } 0 \text{ altrimenti,} & \forall c \in \{1, 2, \dots, q\}. \end{cases}$$

Modello 7.5 — classi con setup

$$\max \sum_{j=1}^n r_j x_j - \sum_{c=1}^q f_c y_c \quad (7.10a)$$

$$\text{soggetto a } \sum_{j=1}^n t_j x_j + \sum_{c=1}^q s_c y_c \leq a, \quad (7.10b)$$

$$x_j - y_c \leq 0, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.10c)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.10d)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, q\}. \quad (7.10e)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.10a) massimizza il profitto, differenza fra il ricavo dei lavori eseguiti e il costo di setup delle classi attivate;
- il vincolo lineare (7.10b) assicura che il tempo dei lavori eseguiti, più i tempi di setup delle classi attivate, non superi la disponibilità della macchina (1 vincolo lineare);
- i vincoli lineari (7.10c) collegano le variabili di esecuzione con le variabili di attivazione delle classi: se un lavoro di una classe è eseguito, la classe è attivata (n vincoli lineari, uno per lavoro, perché le classi partizionano i lavori);
- i vincoli (7.10d) e (7.10e) definiscono le variabili del modello.

Legame fra le variabili

Entrambe le famiglie sono binarie: il vincolo di link è la linearizzazione diretta di un'implicazione booleana (sezione 2.3).

- Per ogni classe c , se almeno un lavoro della classe è eseguito, la classe è attivata:

$$\underbrace{(x_j \text{ OR } x_s \text{ OR } \dots)}_{j,s,\dots \in \mathcal{J}_c} \implies y_c,$$

e per contronominale (De Morgan)

$$\text{NOT } y_c \implies \underbrace{(\text{NOT } x_j \text{ AND NOT } x_s \text{ AND } \dots)}_{j,s,\dots \in \mathcal{J}_c} :$$

se la classe non è attivata, nessun suo lavoro è eseguito.

L'implicazione equivale all'espressione booleana

$$\text{NOT } (x_j \text{ OR } x_s \text{ OR } \dots) \text{ OR } y_c,$$

che con De Morgan e la proprietà distributiva diventa la CNF

$$(\text{NOT } x_j \text{ OR } y_c) \text{ AND } (\text{NOT } x_s \text{ OR } y_c) \text{ AND } \dots$$

Si traduce nei $|\mathcal{J}_c|$ vincoli lineari $1 - x_j + y_c \geq 1$, cioè $x_j - y_c \leq 0$ per ogni $j \in \mathcal{J}_c$: esattamente i vincoli (7.10c). Verifica nei due versi: se $x_j = 1$ per un $j \in \mathcal{J}_c$, il vincolo $x_j \leq y_c$ forza $y_c = 1$; se $y_c = 0$, i vincoli $x_j \leq y_c$ forzano $x_j = 0$ per ogni $j \in \mathcal{J}_c$.

- Viceversa, se nessun lavoro della classe è eseguito, la classe non è attivata:

$$\text{NOT}(x_j \text{ OR } x_s \text{ OR } \dots) \implies \text{NOT } y_c,$$

$$\text{cioè, per contronominale: } y_c \implies (x_j \text{ OR } x_s \text{ OR } \dots),$$

cioè una classe attivata ha almeno un lavoro eseguito.

Non è imposta dai vincoli, ma segue *senza perdita di ottimalità* dall'obiettivo e dal vincolo di capacità: se $x_j = 0$ per ogni $j \in \mathcal{J}_c$ e $y_c = 1$, porre $y_c = 0$ resta ammissibile, libera s_c minuti nel vincolo (7.10b) e *non diminuisce* l'obiettivo, perché $f_c \geq 0$. Poiché f_c può essere nullo, il miglioramento non è necessariamente stretto: la conclusione corretta è «esiste sempre un ottimo in cui le classi senza lavori non sono attivate», non «in ogni ottimo». Il segno del dato decide la forza della conclusione.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

Le classi sono liste di indici: $J = [[0, 1], [2, 3], [4, 5, 6]]$.

```

1 m = gp.Model("classi_setup"); m.Params.OutputFlag = 0
2 x = m.addVars(n, vtype=GRB.BINARY, name="x")
3 y = m.addVars(q, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(r[j] * x[j] for j in range(n))
5                 - gp.quicksum(f[c] * y[c] for c in range(q)), GRB.
6                   MAXIMIZE)
7 m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n))
8             + gp.quicksum(s[c] * y[c] for c in range(q)) <= a,
9             name="disponibilita")
10 m.addConstrs((x[j] - y[c] <= 0 for c in range(q) for j in J[c]),
11              name="link")

```

L'istanza

$n = 7$ lavori, $q = 3$ classi $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6, 7\}$, disponibilità $a = 50$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	$j = 7$
r_j	10	6	8	6	7	9	5
t_j	5	10	8	6	9	5	6

	$c = 1$	$c = 2$	$c = 3$
f_c	10	5	4
s_c	10	12	6

Il modello (7.10) per l'istanza:

Il modello dell'istanza

$$\begin{aligned}
& \max 10x_1 + 6x_2 + 8x_3 + 6x_4 + 7x_5 + 9x_6 + 5x_7 - 10y_1 - 5y_2 - 4y_3 \\
& 5x_1 + 10x_2 + 8x_3 + 6x_4 + 9x_5 + 5x_6 + 6x_7 + 10y_1 + 12y_2 + 6y_3 \leq 50 \\
& \begin{array}{rcccccccccc}
x_1 & & & & & & & & & & - & y_1 & & \leq 0 \\
& x_2 & & & & & & & & & - & y_1 & & \leq 0 \\
& & x_3 & & & & & & & & - & y_2 & & \leq 0 \\
& & & x_4 & & & & & & & - & y_2 & & \leq 0 \\
& & & & x_5 & & & & & & - & y_3 & \leq 0 \\
& & & & & x_6 & & & & & - & y_3 & \leq 0 \\
& & & & & & x_7 & & & & - & y_3 & \leq 0 \\
x_1, & x_2, & x_3, & x_4, & x_5, & x_6, & x_7, & y_1, & y_2, & y_3 \in \{0, 1\}
\end{array}
\end{aligned}$$

Euristica costruttiva: il bound primale

Classe per classe, lavoro per lavoro: il primo lavoro di una classe si esegue se ci stanno il suo tempo e il setup della classe; i successivi se ci sta il loro tempo.

- **Passo 1.** Classe 1 non attiva: $s_1 + t_1 = 10 + 5 = 15 \leq 50$; $y[1] = 1$, $x[1] = 1$, $ra = 35$.
- **Passo 2.** Classe 1 attiva: $t_2 = 10 \leq 35$; $x[2] = 1$, $ra = 25$.
- **Passo 3.** Classe 2 non attiva: $s_2 + t_3 = 12 + 8 = 20 \leq 25$; $y[2] = 1$, $x[3] = 1$, $ra = 5$.
- **Passo 4.** Classe 2 attiva: $t_4 = 6 > 5$; il lavoro 4 viene saltato.
- **Passi 5–7.** Classe 3 non attiva: $s_3 + t_j \in \{15, 11, 12\}$, tutti > 5 ; i lavori 5, 6, 7 vengono saltati.

Soluzione $\bar{x} = (1, 1, 1, 0, 0, 0, 0)$, $\bar{y} = (1, 1, 0)$, profitto $10 + 6 + 8 - 10 - 5 = 9$: $z(MILP) \geq 9$.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Rilassamento LP e duale: il bound duale

Rilassando $x_j \geq 0$ e $y_c \geq 0$, con duali $\pi \geq 0$ per il vincolo (7.10b) e $\lambda_j \geq 0$ per ciascun vincolo (7.10c):

Duale del rilassamento LP del modello 7.5

$$\min \quad a \pi \quad (7.11a)$$

$$\text{soggetto a} \quad t_j \pi + \lambda_j \geq r_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.11b)$$

$$s_c \pi - \sum_{j \in \mathcal{J}_c} \lambda_j \geq -f_c, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.11c)$$

$$\pi \geq 0, \quad (7.11d)$$

$$\lambda_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.11e)$$

Descrizione. La variabile π è il prezzo di una unità di tempo e λ_j il prezzo del legame fra il lavoro j e il setup della sua classe. L'obiettivo (7.11a) è il valore di tutto il tempo disponibile a . I vincoli (7.11b) sono le colonne delle x_j : il tempo t_j che il lavoro j consuma, valutato a π , più il prezzo del suo legame, deve coprire il ricavo r_j . I vincoli (7.11c) sono le colonne delle y_c : il setup della classe c consuma s_c di tempo e scioglie i legami di tutti i lavori della classe; il saldo non può scendere sotto $-f_c$, cioè sotto il costo di quel setup.

Per l'istanza: $\min 50\pi$ soggetto a $5\pi + \lambda_1 \geq 10$, $10\pi + \lambda_2 \geq 6$, $8\pi + \lambda_3 \geq 8$, $6\pi + \lambda_4 \geq 6$, $9\pi + \lambda_5 \geq 7$, $5\pi + \lambda_6 \geq 9$, $6\pi + \lambda_7 \geq 5$, $10\pi - \lambda_1 - \lambda_2 \geq -10$, $12\pi - \lambda_3 - \lambda_4 \geq -5$, $6\pi - \lambda_5 - \lambda_6 - \lambda_7 \geq -4$, $\pi, \lambda \geq 0$.

Una soluzione duale a mano. Con $\bar{\lambda}_j = 0$ i vincoli (7.11b) chiedono $\pi \geq r_j/t_j$ per ogni lavoro, e i vincoli (7.11c) sono soddisfatti per ogni $\pi \geq 0$:

$$\bar{\pi} = \max_j \frac{r_j}{t_j} = \max\left\{\frac{10}{5}, \frac{6}{10}, \frac{8}{8}, \frac{6}{6}, \frac{7}{9}, \frac{9}{5}, \frac{5}{6}\right\} = 2,$$

valore $50 \cdot 2 = 100$: $9 \leq z(MILP) \leq 100$. La lettura: π è il valore di un minuto di macchina; se ogni minuto valesse quanto il lavoro più redditizio per minuto, la macchina intera varrebbe 100 euro. Un bound grossolano, come spesso i bound «di zaino»: ignora setup e costi.

Quello che dice il solver. $z(LP) = 425/13 = 32,7$ (ottimo del duale, raggiunto con $\pi = \frac{17}{26}$ e alcuni $\lambda_j > 0$: il solver non si accontenta di $\lambda = 0$); con i limiti superiori, $z(LP^+) = 329/13 = 25,3$. L'ottimo intero è $z(MILP) = 21$: classi 2 e 3 attivate, lavori 3, 4, 5, 6 eseguiti ($8 + 6 + 9 + 5 + 12 + 6 = 46 \leq 50$), profitto $30 - 9$. L'euristica, che apre le classi nell'ordine dato e spende 10 + 10 minuti per la classe 1, resta a 9: un gap del 57%, il peggiore della famiglia — l'ordine di scansione conta, e una euristica costruttiva che ordina le classi per profitto potenziale farebbe meglio.

LB (euristica costruttiva)	UB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
9	100	425/13	329/13	21	57,1%

Considerazioni aggiuntive

- $y_c \leq 1$ (q) sono disuguaglianze valide che rafforzano il rilassamento; $x_j \leq 1$ (n) sono implicate da $y_c \leq 1$ e (7.10c).
- Il verso «nessun lavoro, quindi classe non attivata» è la clausola

$$\text{NOT } y_c \text{ OR } x_j \text{ OR } x_s \text{ OR } \dots,$$

cioè $\sum_{j \in \mathcal{J}_c} x_j \geq y_c$ (q vincoli): non valida, ma aggiungibile senza perdere l'ottimo (per $f_c \geq 0$ e $s_c \geq 0$).

- La forma aggregata del link, $\sum_{j \in \mathcal{J}_c} x_j \leq |\mathcal{J}_c| y_c$, ha lo stesso insieme di soluzioni intere e un rilassamento più debole: qui il modello nasce già nella forma disaggregata, quella giusta.

Domande di modellazione aggiuntive

Domanda di modellazione 7.9 — una sola classe

Cambiare classe richiede una pulizia della macchina che l'azienda vuole evitare del tutto: si può attivare al più una classe. Modellare e risolvere.

Domanda di modellazione 7.10 — una classe subordinata a un'altra

I lavori della classe 3 usano un utensile che va montato durante il setup della classe 1: la classe 3 si può attivare solo se si attiva anche la classe 1. Scrivere il vincolo e trovare il nuovo ottimo.

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	9	soluzione euristica (LB)
UB	100	bound certificato (UB)
$z(MILP)$	21	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 57.1\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 433.3\%.$$

7.6 Classi con premio di completamento e riduzione «se e solo se»

Script: `python/fam07_6_classipremio.py`

[Notebook in Colab](#)

Problema 7.6 — premio di completamento

Un'azienda ha $n \in \mathbb{Z}_{\geq 1}$ lavori eseguibili su una macchina con disponibilità $a \in \mathbb{Q}_{>0}$ minuti. Per ogni lavoro $j \in \{1, 2, \dots, n\}$, $t_j \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti e $r_j \in \mathbb{Q}_{>0}$ il ricavo in euro se eseguito. I lavori sono partizionati in $q \in \mathbb{Z}_{\geq 2}$ classi $\mathcal{J}_1, \dots, \mathcal{J}_q$. La disponibilità della macchina si riduce di $u \in \mathbb{Q}_{>0}$ minuti *se e solo se* la macchina esegue lavori di almeno due classi diverse. Per ogni classe $c \in \{1, 2, \dots, q\}$, si ottiene un ricavo extra di $v_c \in \mathbb{Q}_{>0}$ euro *se e solo se* tutti i lavori della classe sono eseguiti. La macchina non esegue lavori in parallelo. L'azienda vuole scegliere i lavori da eseguire massimizzando il ricavo totale.

Il problema a parole. Decidiamo quali lavori eseguire. *L'obiettivo*: ricavi dei lavori più i premi delle classi complete. *Il vincolo*: tempi dei lavori, più la riduzione u se le classi coinvolte sono almeno due, entro la disponibilità. Due «se e solo se»: uno premia (il completamento), uno penalizza (la mescolanza). Vedremo che di ciascuno basta imporre un verso: l'altro lo impone l'obiettivo.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	disponibilità della macchina
t_j, r_j	$\in \mathbb{Q}_{>0}$	tempo e ricavo del lavoro j
q	$\in \mathbb{Z}_{\geq 2}$	numero di classi, $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	lavori della classe c (partizione)
v_c	$\in \mathbb{Q}_{>0}$	premio se tutti i lavori della classe c sono eseguiti
u	$\in \mathbb{Q}_{>0}$	riduzione della disponibilità se si eseguono lavori di almeno due classi

Variabili decisionali

Introduciamo le seguenti $n + q + 1$ variabili binarie:

$$\begin{cases} x_j = 1 \text{ se il lavoro } j \text{ è eseguito, } 0 \text{ altrimenti,} & \forall j \in \{1, 2, \dots, n\}, \\ y_c = 1 \text{ se tutti i lavori della classe } c \text{ sono eseguiti, } 0 \text{ altrimenti,} & \forall c \in \{1, 2, \dots, q\}, \\ z = 1 \text{ se si eseguono lavori di almeno due classi diverse, } 0 \text{ altrimenti.} \end{cases}$$

Modello 7.6 — premio di completamento

$$\max \sum_{j=1}^n r_j x_j + \sum_{c=1}^q v_c y_c \quad (7.12a)$$

$$\text{soggetto a } x_j - y_c \geq 0, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.12b)$$

$$x_j + x_i - z \leq 1, \quad \forall c, g \in \{1, 2, \dots, q\}, c < g, \forall j \in \mathcal{J}_c, \forall i \in \mathcal{J}_g, \quad (7.12c)$$

$$\sum_{j=1}^n t_j x_j + u z \leq a, \quad (7.12d)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.12e)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.12f)$$

$$z \in \{0, 1\}. \quad (7.12g)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.12a) massimizza il ricavo totale, somma dei ricavi dei lavori eseguiti e dei premi delle classi complete;
- i vincoli lineari (7.12b) collegano le variabili di esecuzione con le variabili di completamento: se una classe è dichiarata completa, tutti i suoi lavori sono eseguiti (n vincoli lineari);

- i vincoli lineari (7.12c) collegano le variabili di esecuzione con la variabile di mescolanza: se si eseguono due lavori di classi diverse, allora $z = 1$ ($\sum_{c < g} |\mathcal{J}_c| |\mathcal{J}_g|$ vincoli lineari, uno per coppia di lavori di classi diverse);
- il vincolo lineare (7.12d) assicura che il tempo dei lavori eseguiti, più la riduzione u quando si mescolano classi, non superi la disponibilità (1 vincolo lineare);
- i vincoli (7.12e)–(7.12g) definiscono le variabili del modello.

Legame fra le variabili

Quattro implicazioni: due per y_c , due per z . In ogni coppia una è imposta dal vincolo, l'altra dall'ottimo — ed è così che si modella un «se e solo se» (sezione 3.10).

- Per ogni classe c , se non tutti i lavori della classe sono eseguiti, la classe non è completa:

$$\text{NOT}(x_j \text{ AND } x_i \text{ AND } \dots) \implies \text{NOT } y_c,$$

$$\text{cioè, per contronominale: } y_c \implies (x_j \text{ AND } x_i \text{ AND } \dots),$$

con $j, i, \dots \in \mathcal{J}_c$. L'espressione booleana diventa, per distributività, la CNF

$$\begin{aligned} & (x_j \text{ AND } x_i \text{ AND } \dots) \text{ OR } \text{NOT } y_c \\ & \equiv (x_j \text{ OR } \text{NOT } y_c) \text{ AND } (x_i \text{ OR } \text{NOT } y_c) \text{ AND } \dots, \end{aligned}$$

cioè $x_j + (1 - y_c) \geq 1$: sono i vincoli (7.12b), $x_j - y_c \geq 0$ per ogni $j \in \mathcal{J}_c$. Verifica: se $y_c = 1$, ogni vincolo $x_j \geq y_c$ forza $x_j = 1$; se un $x_j = 0$ con $j \in \mathcal{J}_c$, il vincolo $x_j \geq y_c$ forza $y_c = 0$.

- Viceversa, se tutti i lavori della classe sono eseguiti, la classe è completa:

$$(x_j \text{ AND } x_i \text{ AND } \dots) \implies y_c,$$

$$\text{contronominale: } \text{NOT } y_c \implies (\text{NOT } x_j \text{ OR } \text{NOT } x_i \text{ OR } \dots).$$

Non è imposta dai vincoli, e segue dall'obiettivo in ogni ottimo: poiché $v_c > 0$, se tutti gli x_j della classe valgono 1 e $y_c = 0$, porre $y_c = 1$ resta ammissibile (i vincoli (7.12b) chiedono $x_j \geq 1$, vero) e aumenta l'obiettivo di v_c . Quindi in ogni ottimo il premio si incassa appena la classe è completa.

- Per ogni coppia di lavori j, i di classi diverse, se entrambi sono eseguiti allora $z = 1$:

$$x_j \text{ AND } x_i \implies z,$$

$$\text{contronominale: } \text{NOT } z \implies \text{NOT}(x_j \text{ AND } x_i),$$

cioè: se $z = 0$, due lavori di classi diverse non sono entrambi eseguiti. L'espressione booleana, con De Morgan, è

$$\text{NOT}(x_j \text{ AND } x_i) \text{ OR } z \equiv \text{NOT } x_j \text{ OR } \text{NOT } x_i \text{ OR } z,$$

cioè $(1 - x_j) + (1 - x_i) + z \geq 1$: sono i vincoli (7.12c), $x_j + x_i - z \leq 1$. Verifica: se $x_j = x_i = 1$ il vincolo forza $z = 1$; se $z = 0$ diventa $x_j + x_i \leq 1$.

- Viceversa, se i lavori eseguiti stanno tutti in una classe, $z = 0$; per contronominale, $z = 1$ implica che per ogni classe c almeno un lavoro eseguito sta fuori da $\mathcal{J}_c$. Non imposta dai vincoli, segue dal vincolo di capacità *senza perdita di ottimalità*: se i lavori eseguiti stanno in una sola classe e $z = 1$, porre $z = 0$ resta ammissibile (i vincoli (7.12c) diventano $x_j + x_i \leq 1$, veri perché nessuna coppia mista è eseguita), libera $u > 0$ minuti nel vincolo (7.12d) e non cambia l'obiettivo, dove z non compare. Quindi esiste sempre un ottimo in cui $z = 1$ solo se si mescolano classi.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 coppie = [(j, i) for c in range(q) for g in range(c + 1, q)
2             for j in J[c] for i in J[g]]
3 m = gp.Model("classi_premio"); m.Params.OutputFlag = 0
4 x = m.addVars(n, vtype=GRB.BINARY, name="x")
5 y = m.addVars(q, vtype=GRB.BINARY, name="y")
6 z = m.addVar(vtype=GRB.BINARY, name="z")
7 m.setObjective(gp.quicksum(r[j] * x[j] for j in range(n))
8               + gp.quicksum(v[c] * y[c] for c in range(q)), GRB.
9               MAXIMIZE)
10 m.addConstrs((x[j] - y[c] >= 0 for c in range(q) for j in J[c]),
11              name="tutti")
12 m.addConstrs((x[j] + x[i] - z <= 1 for (j, i) in coppie), name="miste")
13 m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n)) + u * z <= a,
14              name="disponibilita")
15 m.optimize()

```

L'istanza

$n = 6$ lavori, $q = 3$ classi $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6\}$, $a = 50$, $u = 10$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$		$c = 1$	$c = 2$	$c = 3$
r_j	10	5	20	12	10	22	v_c	5	4	10
t_j	5	15	25	15	10	38				

Il modello per l'istanza ha obiettivo $\max 10x_1 + 5x_2 + 20x_3 + 12x_4 + 10x_5 + 22x_6 + 5y_1 + 4y_2 + 10y_3$, sei vincoli (7.12b) ($x_1 \geq y_1$, $x_2 \geq y_1$, $x_3 \geq y_2$, $x_4 \geq y_2$, $x_5 \geq y_3$, $x_6 \geq y_3$), dodici vincoli (7.12c) ($x_1 + x_3 - z \leq 1$, $x_1 + x_4 - z \leq 1$, $x_2 + x_3 - z \leq 1$, $x_2 + x_4 - z \leq 1$, $x_1 + x_5 - z \leq 1$, $x_1 + x_6 - z \leq 1$, $x_2 + x_5 - z \leq 1$, $x_2 + x_6 - z \leq 1$, $x_3 + x_5 - z \leq 1$, $x_3 + x_6 - z \leq 1$, $x_4 + x_5 - z \leq 1$, $x_4 + x_6 - z \leq 1$) e il vincolo di capacità $5x_1 + 15x_2 + 25x_3 + 15x_4 + 10x_5 + 38x_6 + 10z \leq 50$.

Euristica costruttiva: il bound primale

Classe per classe; dalla seconda classe in poi, il primo lavoro eseguito paga anche la riduzione u ; a fine classe, se tutti i suoi lavori sono eseguiti, si incassa il premio.

- **Passi 1–2.** Classe 1: $t_1 = 5 \leq 50$, $x[1] = 1$, $ra = 45$; $t_2 = 15 \leq 45$, $x[2] = 1$, $ra = 30$. Classe completa: $y[1] = 1$.

- **Passo 3.** Classe 2, riduzione non ancora applicata: $t_3 + u = 35 > 30$, il lavoro 3 viene saltato.
- **Passo 4.** $t_4 + u = 25 \leq 30$: $x[4] = 1$, $z = 1$, $ra = 5$. Classe 2 incompleta.
- **Passi 5–6.** Classe 3: $t_5 = 10 > 5$ e $t_6 = 38 > 5$, saltati.

Soluzione $\bar{x} = (1, 1, 0, 1, 0, 0)$, $\bar{y} = (1, 0, 0)$, $\bar{z} = 1$, ricavo $10 + 5 + 12 + 5 = 32$: $z(MILP) \geq 32$. È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Rilassamento LP e duale: il bound duale

Rilassando tutte le binarie a ≥ 0 , con duali $\pi_j \leq 0$ per i vincoli (7.12b) (verso $\geq$ in un massimo), $\lambda_{ji} \geq 0$ per i vincoli (7.12c) e $\mu \geq 0$ per il vincolo (7.12d):

Duale del rilassamento LP del modello 7.6

$$\min \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + a \mu \quad (7.13a)$$

$$\text{soggetto a } \pi_j + \sum_{i \notin \mathcal{J}_c} \lambda_{ji} + t_j \mu \geq r_j, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.13b)$$

$$- \sum_{j \in \mathcal{J}_c} \pi_j \geq v_c, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.13c)$$

$$- \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + u \mu \geq 0, \quad (7.13d)$$

$$\pi_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.13e)$$

$$\lambda_{ji} \geq 0, \quad \forall c, g \in \{1, 2, \dots, q\}, c < g, \forall j \in \mathcal{J}_c, \forall i \in \mathcal{J}_g, \quad (7.13f)$$

$$\mu \geq 0. \quad (7.13g)$$

Descrizione. La variabile π_j è il prezzo (non positivo) del legame «se la classe è premiata, allora il lavoro j va fatto», λ_{ji} il prezzo del vincolo sulla coppia mista (j, i) e μ il prezzo di una unità di tempo. L'obiettivo (7.13a) somma i prezzi di tutte le coppie miste e il valore del tempo disponibile a . I vincoli (7.13b) sono le colonne delle x_j : fare il lavoro j scioglie il legame con la sua classe, tocca tutti i vincoli delle coppie miste che lo contengono e consuma t_j di tempo; il totale deve coprire il ricavo r_j . I vincoli (7.13c) sono le colonne delle y_c : il premio v_c della classe c va coperto dai legami dei lavori che la compongono. Il vincolo (7.13d) è la colonna di z : la variabile che segnala la presenza di due classi diverse scioglie tutti i vincoli delle coppie miste e consuma u di tempo.

(In (7.13b) λ_{ji} indica la variabile della coppia $\{j, i\}$, qualunque sia l'ordine dei due indici.)

Una soluzione duale a mano. I vincoli (7.13c) chiedono che i π_j di ogni classe «coprano» il premio: basta caricarlo su un solo lavoro per classe, $\bar{\pi}_1 = -5$, $\bar{\pi}_3 = -4$, $\bar{\pi}_5 = -10$, gli altri 0. Con $\bar{\lambda} = 0$ il vincolo (7.13d) è soddisfatto per ogni $\mu \geq 0$ e i vincoli (7.13b) chiedono $\mu \geq (r_j - \bar{\pi}_j)/t_j$:

$$\bar{\mu} = \max\left\{\frac{10+5}{5}, \frac{5}{15}, \frac{20+4}{25}, \frac{12}{15}, \frac{10+10}{10}, \frac{22}{38}\right\} = \max\left\{3, \frac{1}{3}, \frac{24}{25}, \frac{4}{5}, 2, \frac{11}{19}\right\} = 3,$$

valore $a\bar{\mu} = 150$: $32 \leq z(MILP) \leq 150$. Il bound è dominato dal lavoro 1, breve e col premio caricato addosso: spalmare il premio sui due lavori della classe ($\pi_1 = \pi_2 = -2,5$) lo abbasserebbe — un buon esercizio.

Quello che dice il solver. $z(LP) = z(LP^+) = 5280/113 = 46,7$. L'ottimo intero è $z(MILP) = 42$: la sola classe 3 completa, lavori 5 e 6 ($10 + 38 = 48 \leq 50$, $z = 0$), ricavo $10 + 22 + 10$. L'euristica, partita dalla classe 1, si ferma a 32: gap del 24%.

LB (euristica costruttiva)	UB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
32	150	5280/113	42	23,8%

Considerazioni aggiuntive

- Il verso di ottimalità per y_c si può imporre esplicitamente con la clausola NOT x_j OR NOT x_i OR ... OR y_c , cioè

$$\sum_{j \in \mathcal{J}_c} x_j - y_c \leq |\mathcal{J}_c| - 1, \quad \forall c \in \{1, 2, \dots, q\}$$

(q vincoli che preservano l'ottimo, non validi: il modello ammette la classe completa con $y_c = 0$).

- Il verso di ottimalità per z : per ogni classe c , $z \leq \sum_{j \notin \mathcal{J}_c} x_j$ (q vincoli), da $z \implies (x_j$ OR x_i OR ...) sui lavori fuori da $\mathcal{J}_c$. Anche questi preservano l'ottimo senza essere validi.
- I vincoli (7.12c) sono tanti quante le coppie miste. Una forma aggregata con variabili di classe, $w_c \geq x_j$ per $j \in \mathcal{J}_c$ e $\sum_{c=1}^q w_c - 1 \leq (q-1)z$, usa q variabili in più e $n+1$ vincoli al posto delle coppie: stesso insieme di soluzioni intere, rilassamento diverso. Confrontarli è la domanda 7.6.2.

Domande di modellazione aggiuntive

Domanda di modellazione 7.11 — almeno un lavoro per classe

Un accordo sindacale impone di eseguire almeno un lavoro di ogni classe. Modellare, dire che cosa succede a z e trovare il nuovo ottimo.

Domanda di modellazione 7.12 — penalità per classe iniziata e non finita

Iniziare una classe senza completarla costa $w = 3$ euro di riorganizzazione. Modellare la penalità: quale nuova variabile serve, e in che verso va dimostrato il suo legame?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	32	soluzione euristica (LB)
UB	150	bound certificato (UB)
$z(MILP)$	42	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 23.8\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 281.0\%.$$

7.7 Ritardo totale su una macchina: sequenziamento con big-M

Script: `python/fam07_7_ritardo.py`

[Notebook in Colab](#)

Problema 7.7 — ritardo totale

Un'azienda deve eseguire $n \in \mathbb{Z}_{\geq 1}$ lavori su una sola macchina. Per ogni lavoro $j \in \{1, 2, \dots, n\}$, $t_j \in \mathbb{Q}_{>0}$ è il tempo di lavorazione in minuti e $d_j \in \mathbb{Q}_{>0}$ la scadenza in minuti. Il ritardo del lavoro j è $\tau_j = \max\{0, \kappa_j - d_j\}$, dove κ_j è l'istante di completamento. La macchina esegue un lavoro alla volta. L'azienda vuole una sequenza dei lavori che minimizzi il ritardo totale.

Il problema a parole. Decidiamo l'ordine dei lavori. L'obiettivo: somma dei ritardi. I vincoli: per ogni coppia, uno dei due precede l'altro; chi viene dopo finisce almeno t minuti dopo il completamento di chi viene prima. L'ordine è una *disgiunzione* (« j prima di i o i prima di j »): si linearizza con una binaria e un big-M.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di lavori, $j \in \{1, 2, \dots, n\}$
t_j	$\in \mathbb{Q}_{>0}$	tempo di lavorazione del lavoro j
d_j	$\in \mathbb{Q}_{>0}$	scadenza del lavoro j
M	$\in \mathbb{Q}_{>0}$	costante big-M, $M = \sum_{j=1}^n t_j$

Variabili decisionali

Introduciamo $n(n-1)$ variabili binarie di precedenza e $2n$ variabili non negative:

$$\begin{cases} s_{ji} = 1 \text{ se il lavoro } j \text{ precede il lavoro } i, \text{ 0 altrimenti,} & \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \\ \kappa_j = \text{istante di completamento del lavoro } j, & \forall j \in \{1, 2, \dots, n\}, \\ \tau_j = \text{ritardo del lavoro } j, & \forall j \in \{1, 2, \dots, n\}. \end{cases}$$

Modello 7.7 — ritardo totale

$$\min \sum_{j=1}^n \tau_j \quad (7.14a)$$

$$\text{soggetto a } s_{ji} + s_{ij} = 1, \quad \forall j, i \in \{1, 2, \dots, n\}, j < i, \quad (7.14b)$$

$$-M s_{ji} - \kappa_j + \kappa_i \geq t_i - M, \quad \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \quad (7.14c)$$

$$-\kappa_j + \tau_j \geq -d_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14d)$$

$$\kappa_j \geq t_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14e)$$

$$s_{ji} \in \{0, 1\}, \quad \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \quad (7.14f)$$

$$\kappa_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14g)$$

$$\tau_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.14h)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (7.14a) minimizza il ritardo totale;
- i vincoli lineari (7.14b) collegano le variabili di precedenza di ogni coppia: o j precede i o i precede j ($n(n-1)/2$ vincoli lineari);
- i vincoli lineari (7.14c) collegano le variabili di precedenza con quelle di completamento tramite il big-M: se j precede i , il lavoro i non può completarsi prima che j sia completato e i sia stato lavorato ($n(n-1)$ vincoli lineari). Un valore valido è $M = \sum_{j=1}^n t_j$: esiste sempre una sequenza ottima senza tempi morti, quindi nessun completamento deve superare il tempo totale;
- i vincoli lineari (7.14d), con i vincoli (7.14h), collegano i completamenti con i ritardi e definiscono correttamente il ritardo di ogni lavoro (n vincoli lineari);
- i vincoli lineari (7.14e) impongono che ogni lavoro si completi non prima del suo tempo di lavorazione (n vincoli lineari);
- i vincoli (7.14f)–(7.14h) definiscono le variabili del modello.

Legame fra le variabili

- Per ogni coppia j, i : se j precede i , il completamento di i è almeno il completamento di j più il tempo di i :

$$s_{ji} = 1 \implies \kappa_i \geq \kappa_j + t_i,$$

$$\text{e per contronominale: } \kappa_i < \kappa_j + t_i \implies s_{ji} = 0.$$

Si impone con il big-M: $\kappa_i \geq \kappa_j + t_i - M(1 - s_{ji})$, cioè i vincoli (7.14c). Se $s_{ji} = 1$ il vincolo diventa $\kappa_i \geq \kappa_j + t_i$; se $s_{ji} = 0$ diventa $\kappa_i \geq \kappa_j + t_i - M$, sempre vero per i valori ammissibili di κ quando M è abbastanza grande: con $M = \sum_{j=1}^n t_j$ e completamenti entro M , il membro destro è $\leq t_i \leq \kappa_i$. Questo è il pattern di sezione 3.8: il big-M «spegne» il vincolo.

- Per ogni lavoro j , la variabile di ritardo soddisfa il limite quantitativo

$$\tau_j \geq \max\{0, \kappa_j - d_j\},$$

imposto direttamente dai vincoli (7.14d) ($\tau_j \geq \kappa_j - d_j$) e (7.14h) ($\tau_j \geq 0$), qualunque siano κ_j e d_j : non c'è un'implicazione da dimostrare, il legame è la disuguaglianza. C'è invece un'implicazione di ottimalità: se il lavoro finisce in tempo, il ritardo è nullo:

$$\kappa_j \leq d_j \implies \tau_j = 0,$$

$$\text{contronominale: } \tau_j > 0 \implies \kappa_j > d_j.$$

I vincoli non la impongono (con $\kappa_j \leq d_j$ chiedono solo $\tau_j \geq 0$), ma segue dall'obiettivo in ogni ottimo: se $\kappa_j \leq d_j$ e $\tau_j > 0$, porre $\tau_j = 0$ resta ammissibile ($0 \geq \kappa_j - d_j$ perché $\kappa_j - d_j \leq 0$) e riduce l'obiettivo di $\tau_j > 0$.

- *Sintesi*: in ogni ottimo $\tau_j = \max\{0, \kappa_j - d_j\}$, esattamente il ritardo. Il limite inferiore vale in ogni soluzione ammissibile; se τ_j lo superasse, abbassarla al massimo manterrebbe entrambi i vincoli e ridurrebbe l'obiettivo della differenza, positiva. Nessun vincolo in più è necessario.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 M = sum(t)
2 m = gp.Model("ritardo"); m.Params.OutputFlag = 0
3 s = m.addVars([(j, i) for j in range(n) for i in range(n) if j != i],
4               vtype=GRB.BINARY, name="s")
5 kappa = m.addVars(n, name="kappa"); tau = m.addVars(n, name="tau")
6 m.setObjective(tau.sum(), GRB.MINIMIZE)
7 m.addConstrs((s[j, i] + s[i, j] == 1 for j in range(n)
8               for i in range(j + 1, n)), name="ordine")
9 m.addConstrs((-M * s[j, i] - kappa[j] + kappa[i] >= t[i] - M
10               for j in range(n) for i in range(n) if j != i),
11               name="precedenza")
12 m.addConstrs((-kappa[j] + tau[j] >= -d[j] for j in range(n)),
13               name="ritardo")
14 m.addConstrs((kappa[j] >= t[j] for j in range(n)), name="inizio")
15 m.optimize()

```

L'istanza

$n = 3$ lavori, $M = 5 + 4 + 6 = 15$:

	$j = 1$	$j = 2$	$j = 3$
t_j	5	4	6
d_j	3	4	10

Il modello (7.14) per l'istanza:

Modello 7.7 sull'istanza

$$\begin{aligned}
\min \quad & \tau_1 + \tau_2 + \tau_3 \\
& s_{12} + s_{21} = 1 \\
& s_{13} + s_{31} = 1 \\
& s_{23} + s_{32} = 1 \\
& -15s_{12} - \kappa_1 + \kappa_2 \geq -11 \\
& -15s_{21} + \kappa_1 - \kappa_2 \geq -10 \\
& -15s_{13} - \kappa_1 + \kappa_3 \geq -9 \\
& -15s_{31} + \kappa_1 - \kappa_3 \geq -10 \\
& -15s_{23} - \kappa_2 + \kappa_3 \geq -9 \\
& -15s_{32} + \kappa_2 - \kappa_3 \geq -11 \\
& \tau_1 - \kappa_1 \geq -3 \\
& \kappa_1 \geq 5 \\
& \tau_2 - \kappa_2 \geq -4 \\
& \kappa_2 \geq 4 \\
& \tau_3 - \kappa_3 \geq -10 \\
& \kappa_3 \geq 6 \\
& s_{12}, s_{21}, s_{13}, s_{31}, s_{23}, s_{32} \in \{0, 1\} \\
& \kappa_1, \kappa_2, \kappa_3 \geq 0 \\
& \tau_1, \tau_2, \tau_3 \geq 0
\end{aligned}$$

Euristica costruttiva: il bound primale

La più semplice: eseguire i lavori nell'ordine dato $1 \rightarrow 2 \rightarrow 3$ (FIFO); una regola migliore ordina per scadenza crescente (EDD, *earliest due date*), che qui coincide.

- **Passo 1.** Lavoro 1: $\kappa_1 = 5$, $\tau_1 = \max\{0, 5 - 3\} = 2$.
- **Passo 2.** Lavoro 2: $\kappa_2 = 5 + 4 = 9$, $\tau_2 = \max\{0, 9 - 4\} = 5$.
- **Passo 3.** Lavoro 3: $\kappa_3 = 9 + 6 = 15$, $\tau_3 = \max\{0, 15 - 10\} = 5$.

Soluzione $\bar{s}_{12} = \bar{s}_{13} = \bar{s}_{23} = 1$ (le altre 0), $\bar{\kappa} = (5, 9, 15)$, $\bar{\tau} = (2, 5, 5)$, valore 12: $z(MILP) \leq 12$. È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Rilassamento LP e duale: il bound duale

Rilassando (7.14f) in $s_{ji} \geq 0$, con duali α_{ji} libere (7.14b), $\beta_{ji} \geq 0$ (7.14c), $\gamma_j \geq 0$ (7.14d) e $\delta_j \geq 0$ (7.14e):

Duale del rilassamento LP del modello 7.7

$$\max \sum_{j < i} \alpha_{ji} + \sum_{j \neq i} (t_i - M) \beta_{ji} \quad (7.15a)$$

$$- \sum_{j=1}^n d_j \gamma_j + \sum_{j=1}^n t_j \delta_j$$

$$\text{soggetto a } \alpha_{ji} - M \beta_{ji} \leq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15b)$$

$$\alpha_{ji} - M \beta_{ij} \leq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15c)$$

$$- \sum_{i \neq j} \beta_{ji} + \sum_{i \neq j} \beta_{ij} - \gamma_j + \delta_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15d)$$

$$\gamma_j \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15e)$$

$$\alpha_{ji} \geq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15f)$$

$$\beta_{ji} \geq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j \neq i, \quad (7.15g)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15h)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.15i)$$

Descrizione. La variabile α_{ji} è il prezzo della scelta d'ordine fra i lavori j e i , β_{ji} il prezzo della precedenza scritta con il big-M, γ_j il prezzo del vincolo che definisce il ritardo del lavoro j e δ_j quello del suo tempo minimo di completamento. L'obiettivo (7.15a) raccoglie questi prezzi con i termini noti dei rispettivi vincoli. I vincoli (7.15b) e (7.15c) sono le colonne delle s_{ji} e delle s_{ij} : la scelta d'ordine entra nell'uguaglianza con coefficiente 1 e nella precedenza con $-M$, e il saldo non può essere positivo perché quelle variabili non compaiono nell'obiettivo. I vincoli (7.15d) sono le colonne delle κ_j : il tempo di completamento entra nelle precedenze con segni opposti, nel ritardo con -1 e nel tempo minimo con $+1$. I vincoli (7.15e) sono le colonne delle τ_j : il ritardo costa 1 nell'obiettivo primale, quindi il suo prezzo non può superare 1.

Per l'istanza l'obiettivo è $\max \alpha_{12} + \alpha_{13} + \alpha_{23} - 11\beta_{12} - 9\beta_{13} - 10\beta_{21} - 9\beta_{23} - 10\beta_{31} - 11\beta_{32} - 3\gamma_1 - 4\gamma_2 - 10\gamma_3 + 5\delta_1 + 4\delta_2 + 6\delta_3$.

Una soluzione duale a mano. Ogni β_{ji} ha coefficiente $t_i - M < 0$: conviene tenerle a zero, e allora (7.15b) e (7.15c) impongono $\alpha = 0$. Restano γ e δ , con $\delta_j \leq \gamma_j \leq 1$ dai vincoli (7.15d) e (7.15e): ogni lavoro contribuisce al più $t_j \delta_j - d_j \gamma_j \leq t_j - d_j$, positivo solo se il lavoro è in ritardo anche eseguito per primo. Solo il lavoro 1 lo è ($t_1 = 5 > d_1 = 3$): $\bar{\gamma}_1 = \bar{\delta}_1 = 1$, tutto il resto 0, valore $-3 + 5 = 2$. Quindi $2 \leq z(MILP) \leq 12$. La lettura: il lavoro 1 sarà in ritardo di almeno 2 minuti comunque lo si collochi.

Quello che dice il solver. $z(LP) = 2$: la soluzione a mano è ottima per il duale, e il rilassamento LP di questo modello è *debolissimo*, come tutti i modelli con big-M — il rilassamento mette $s_{ji} = 1/2$ e rilascia le precedenze. L'ottimo intero è $z(MILP) = 11$, sequenza $2 \rightarrow 1 \rightarrow 3$: $\tilde{\kappa} = (9, 4, 15)$, $\tilde{\tau} = (6, 0, 5)$. L'ordine dato costava 12; scambiare i primi due lavori risparmia 1.

LB (ordine dato)	UB (duale a mano)	$z(LP)$	$z(MILP)$	gap euristica
12	2	2	11	9,1%

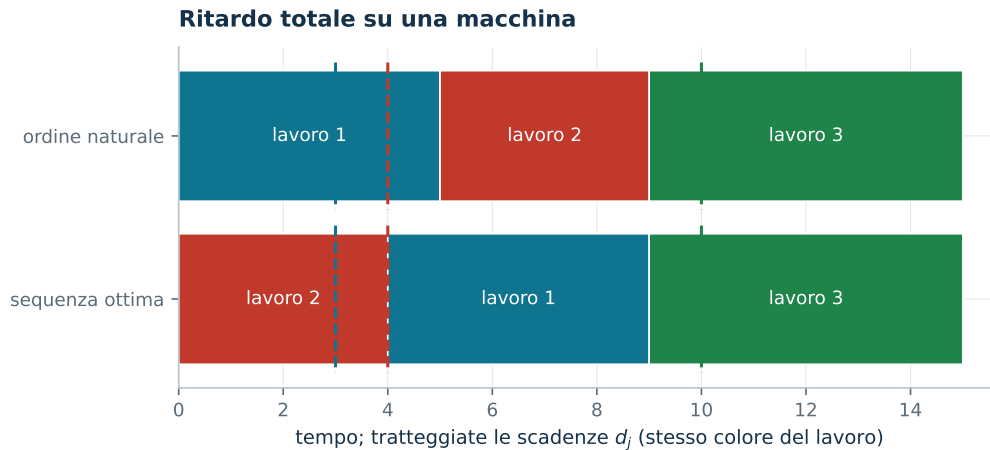


Figura 7.3: Le due sequenze sull'istanza: l'ordine dato (12) e la sequenza ottima $2 \rightarrow 1 \rightarrow 3$ (11); tratteggiate le scadenze.

Considerazioni aggiuntive

- $s_{ji} \leq 1$ ($n(n-1)$) sono valide e implicate da (7.14b).
- *Transitività*: per tre lavori distinti j, i, k , se j precede i e i precede k allora j precede k :

$$s_{ji} \text{ AND } s_{ik} \implies s_{jk},$$

$$\text{con De Morgan: NOT } s_{ji} \text{ OR NOT } s_{ik} \text{ OR } s_{jk},$$

cioè

$$s_{ji} + s_{ik} - s_{jk} \leq 1$$

$(n(n-1)(n-2)/6)$ vincoli per terna non ordinata, con le sue permutazioni). Sono disuguaglianze valide: in ogni sequenza reale la transitività vale, quindi i vincoli (7.14c) le implicano (un ciclo $j \rightarrow i \rightarrow k \rightarrow j$ darebbe $\kappa_j \geq \kappa_j + t_i + t_k + t_j$, impossibile). Nel rilassamento tagliano le soluzioni a $1/2$: sono il primo passo per rafforzare un modello big-M.

- La scelta di M : $\sum_{j=1}^n t_j$ è il più piccolo valore che funziona in generale; M più grandi lasciano lo stesso insieme intero e un rilassamento più debole (sezione 3.8).

Domande di modellazione aggiuntive

Domanda di modellazione 7.13 — date di rilascio

Il lavoro 2 non può iniziare prima dell'istante $\rho_2 = 2$ (il materiale arriva in ritardo); gli altri sono disponibili da subito. Modellare e trovare il nuovo ottimo.

Domanda di modellazione 7.14 — minimizzare il ritardo massimo

L'azienda vuole minimizzare il ritardo del lavoro più in ritardo, non la somma. Modellare e risolvere; che cosa cambia nel legame di τ_j ?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	12	soluzione euristica (UB)
LB	2	bound certificato (LB)
$z(MILP)$	11	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.1\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 90.9\%.$$

Localizzazione e copertura

Classe: BIP / MILP

Script: uno per problema, `python/fam08_1_capacitata.py` ...

[Pagina online](#)

La famiglia

Quattro problemi in cui si decide *dove* aprire una struttura — una sede, un hub — e come questo vincola le variabili che dipendono da quella decisione: dove spedire la merce (problema 1), quali clienti servire (problema 2), quali clienti coprire (problema 3), a quale hub connettere ciascun terminale (problema 4). Cambia, di problema in problema, la natura del legame fra la variabile di apertura e le variabili che ne dipendono e il modo in cui il budget sulle aperture entra nel modello.

I legami fra variabili del capitolo 3 che qui si rivedono:

- **attivazione aggregata** (problema 1): un'unica famiglia di vincoli fa da legame e da vincolo di capacità insieme, $\sum_c y_{lc} \leq u_l x_l$;
- **attivazione disaggregata** (problema 2): $x_l \geq y_{lc}$ per ogni sede e cliente, dedotta dalla CNF di un'implicazione booleana come nel problema 7.5, qui con un budget k sul numero di sedi anziché sul tempo;
- **se e solo se** (problema 3): un verso imposto da due famiglie di vincoli (soglia di segnale e interferenza), l'altro che segue dall'obiettivo;
- **attivazione aggregata e variabile di massimo** (problema 4): la stessa attivazione del problema 1 insieme al legame di massimo $z_j \geq c_{ij} x_{ij}$ già visto nel problema 7.7.

Per ognuno: modello, dimostrazione dei legami, istanza, euristica costruttiva (un bound dal lato delle soluzioni ammissibili), duale del rilassamento LP con una soluzione duale costruita a mano (un bound dall'altro lato), soluzione del solver, considerazioni aggiuntive e due domande di modellazione con soluzione. Tutti i numeri vengono dai quattro script `fam08_1_capacitata.py` ... `fam08_4_hub.py` della cartella `python/`, uno per problema, con il suo notebook.

Notazione della famiglia

Simbolo	Tipo	Significato
m	$\in \mathbb{Z}_{\geq 1}$	numero di sedi/hub candidati, indicizzati da $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	numero di clienti/terminali, indicizzati da $c \in \{1, 2, \dots, n\}$
t_{lc}	$\in \mathbb{Q}_{>0}$	costo di trasporto/conconnessione dalla sede/hub l al cliente/terminale c
i_l, f_l	$\in \mathbb{Q}_{\geq 0}$	costo di apertura/attivazione della sede/hub l
u_l	$\in \mathbb{Q}_{>0}$	capacità della sede l
d_c	$\in \mathbb{Q}_{>0}$	domanda del cliente c
k	$\in \mathbb{Z}_{\geq 1}$	numero massimo di sedi aperte / capacità di ciascun hub
s_{lc}	$\in \mathbb{Q}_{\geq 0}$	intensità del segnale dalla sede l al cliente c
p_c	$\in \mathbb{Q}_{>0}$	profitto se il cliente c è coperto
t, b	$\in \mathbb{Q}_{>0}$	soglia di segnale e limite di interferenza

8.1 Localizzazione capacitata

Script: `python/fam08_1_capacitata.py`

[Notebook in Colab](#)

Problema 8.1 — localizzazione capacitata

Un'azienda deve servire $n \in \mathbb{Z}_{\geq 1}$ clienti con il proprio prodotto e ha individuato $m \in \mathbb{Z}_{\geq 1}$ sedi candidate dove installare una struttura. Per ogni cliente $c \in \{1, 2, \dots, n\}$, il valore $d_c \in \mathbb{Q}_{>0}$ è la domanda, in litri di prodotto, di quel cliente. Per ogni sede $l \in \{1, 2, \dots, m\}$ e ogni cliente c , il valore $t_{lc} \in \mathbb{Q}_{>0}$ è il costo, in euro, di trasportare un litro di prodotto dalla sede l al cliente c . Per ogni sede l , il valore $u_l \in \mathbb{Q}_{>0}$ è la capacità, in litri, della struttura installata in quella sede, e $i_l \in \mathbb{Q}_{>0}$ è il costo di installazione. L'azienda vuole decidere dove installare le strutture e come servire i clienti, a costo totale minimo.

Il problema a parole. Decidiamo dove installare le strutture e quanto prodotto trasportare da ciascuna sede a ciascun cliente. *L'obiettivo:* costo totale (installazione più trasporto) minimo. *I vincoli:* da una sede non installata non parte nulla, e una sede installata non supera la propria capacità; la domanda di ogni cliente va soddisfatta esattamente. È il problema di *localizzazione capacitata* (capacitated facility location).

Dati (input del modello)

Simbolo	Tipo	Significato
m	$\in \mathbb{Z}_{\geq 1}$	numero di sedi candidate, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	numero di clienti, $c \in \{1, 2, \dots, n\}$
t_{lc}	$\in \mathbb{Q}_{>0}$	costo di trasporto (per litro) dalla sede l al cliente c
u_l	$\in \mathbb{Q}_{>0}$	capacità (litri) della sede l
i_l	$\in \mathbb{Q}_{>0}$	costo di installazione della sede l
d_c	$\in \mathbb{Q}_{>0}$	domanda (litri) del cliente c

Variabili decisionali

Introduciamo le seguenti m variabili binarie:

$$x_l = \begin{cases} 1 & \text{se si installa una struttura nella sede } l, \\ 0 & \text{altrimenti,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

e le seguenti $m n$ variabili continue non negative:

y_{lc} = litri di prodotto trasportati dalla sede l al cliente c

$$\forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}.$$

Usando queste variabili, un modello MILP per il problema è il seguente:

Modello 8.1 — localizzazione capacitata

$$\min \sum_{l=1}^m i_l x_l + \sum_{l=1}^m \sum_{c=1}^n t_{lc} y_{lc} \quad (8.1a)$$

$$\text{soggetto a } u_l x_l - \sum_{c=1}^n y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.1b)$$

$$\sum_{l=1}^m y_{lc} = d_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.1c)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.1d)$$

$$y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.1e)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (8.1a) minimizza il costo totale, somma dei costi di installazione delle sedi aperte e dei costi di trasporto;
- i vincoli lineari (8.1b) legano le variabili di trasporto a quelle di installazione e impongono il vincolo di capacità: assicurano che se si trasporta una quantità positiva da una sede, quella sede sia installata, e che da una sede non installata non parta nulla; inoltre, se una sede è installata, la quantità totale trasportata non supera la sua capacità u_l (m vincoli lineari);
- i vincoli lineari (8.1c) assicurano che la domanda di ogni cliente sia soddisfatta esattamente (n vincoli lineari);
- i vincoli (8.1d) e (8.1e) definiscono le variabili del modello.

Il legame fra le variabili. Per ogni sede l , se una quantità positiva di prodotto parte dalla sede l , allora la sede deve essere installata:

$$\sum_{c=1}^n y_{lc} > 0 \implies x_l = 1.$$

Dalla contronominale segue anche

$$x_l = 0 \implies \sum_{c=1}^n y_{lc} = 0 :$$

da una sede non installata non può partire nulla. Questa implicazione — insieme alla sua contronominale — è imposta direttamente dai vincoli (8.1b): se $\sum_{c=1}^n y_{lc} > 0$, il vincolo forza $u_l x_l \geq \sum_{c=1}^n y_{lc} > 0$ e, poiché $u_l > 0$, si ha $x_l > 0$, cioè $x_l = 1$ (essendo binaria). Viceversa, se $x_l = 0$, il vincolo diventa $\sum_{c=1}^n y_{lc} \leq 0$ e, poiché ogni $y_{lc} \geq 0$, si ha $y_{lc} = 0$ per ogni c . (Quando $x_l = 1$, il vincolo si riduce a $\sum_{c=1}^n y_{lc} \leq u_l$: la capacità.)

Il verso opposto — se non parte nulla da una sede, quella sede non è installata:

$$x_l = 1 \implies \sum_{c=1}^n y_{lc} > 0$$

— non è imposto esplicitamente dai vincoli, ma segue dalla funzione obiettivo in ogni soluzione ottima: poiché $i_l > 0$, se $\sum_{c=1}^n y_{lc} = 0$ e $x_l = 1$, porre $x_l = 0$ resta ammissibile e riduce il costo di i_l . Quindi, in ogni soluzione ottima, nessuna sede installata resta inutilizzata. Una sola famiglia di vincoli, dunque, fa sia da legame di attivazione sia da vincolo di capacità.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```
1 mod = gp.Model("localizzazione_capacitata")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, n, name="y")
4 mod.setObjective(gp.quicksum(i[l] * x[l] for l in range(m))
5                  + gp.quicksum(t[l][c] * y[l, c] for l in range(m) for
6                  c in range(n)), GRB.MINIMIZE)
7 mod.addConstrs((u[l] * x[l] - gp.quicksum(y[l, c] for c in range(n)) >=
8                0
9                for l in range(m)), name="capacita")
10 mod.addConstrs((gp.quicksum(y[l, c] for l in range(m)) == d[c] for c in
11                range(n)), name="domanda")
```

L'istanza

$m = 2$ sedi, $n = 3$ clienti:

t_{lc}	$c = 1$	$c = 2$	$c = 3$
$l = 1$	4	5	6
$l = 2$	6	4	3

	$l = 1$	$l = 2$
u_l	50	50
i_l	60	90

	$c = 1$	$c = 2$	$c = 3$
d_c	8	25	27

Euristica costruttiva: il bound primale

Si scandiscono le sedi in ordine naturale; per ciascuna, si scandiscono i clienti e si spedisce a ognuno il minimo fra la capacità residua della sede e la domanda residua del cliente, senza superare né l'una né l'altra; una sede che ha spedito qualcosa si apre.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Esecuzione sull'istanza (output dello script): la sede 1 spedisce 8 al cliente 1 (domanda esaurita, capacità residua 42), poi 25 al cliente 2 (domanda esaurita, capacità residua 17), poi 17 al cliente 3 (capacità della sede 1 esaurita, ne restano 10 da soddisfare); la sede 2 spedisce i restanti 10 al cliente 3. La soluzione ammissibile trovata è

$$\bar{x}_1 = \bar{x}_2 = 1, \quad \bar{y}_{11} = 8, \bar{y}_{12} = 25, \bar{y}_{13} = 17, \bar{y}_{23} = 10, \quad \text{tutte le altre } \bar{y}_{lc} = 0,$$

di valore

$$\underbrace{60 + 90}_{\text{installazione}} + \underbrace{4 \cdot 8 + 5 \cdot 25 + 6 \cdot 17 + 3 \cdot 10}_{\text{trasporto} = 32 + 125 + 102 + 30 = 289} = 150 + 289 = 439.$$

Quindi

$$z(MILP) \leq UB = 439.$$

Rilassamento LP e duale: il bound duale

Associamo una variabile duale non negativa μ_l a ciascun vincolo (8.1b) (verso $\geq$) e una variabile duale libera π_c a ciascun vincolo (8.1c) (uguaglianza). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 8.1

$$\max \sum_{c=1}^n d_c \pi_c \quad (8.2a)$$

$$\text{soggetto a } u_l \mu_l \leq i_l, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.2b)$$

$$-\mu_l + \pi_c \leq t_{lc}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \quad (8.2c)$$

$$\mu_l \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.2d)$$

$$\pi_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}. \quad (8.2e)$$

Descrizione. La variabile μ_l è il prezzo di una unità di capacità della sede l e π_c il valore di una unità di domanda del cliente c . L'obiettivo (8.2a) valuta a quei prezzi tutta la domanda da soddisfare. I vincoli (8.2b) sono le colonne delle x_l : la capacità u_l che l'apertura della sede l mette a disposizione, valutata a μ_l , non può valere più del costo di installazione i_l . I vincoli (8.2c) sono le colonne delle y_{lc} : portare una unità dalla sede l al cliente c vale π_c e consuma una unità di capacità al prezzo μ_l ; il saldo non può superare il costo di trasporto t_{lc} .

Una soluzione duale a mano. Poniamo $\bar{\mu}_l = i_l/u_l$: il vincolo (8.2b) è soddisfatto con uguaglianza, il costo di installazione «spalmato» sulla capacità. Sull'istanza,

$$\bar{\mu}_1 = 60/50 = 6/5, \quad \bar{\mu}_2 = 90/50 = 9/5.$$

Il vincolo (8.2c) diventa $\pi_c \leq t_{lc} + \bar{\mu}_l$ per ogni sede l : il valore più grande ammissibile è $\bar{\pi}_c = \min_l(t_{lc} + \bar{\mu}_l)$:

$$\bar{\pi}_1 = \min(4 + 6/5, 6 + 9/5) = 26/5,$$

$$\bar{\pi}_2 = \min(5 + 6/5, 4 + 9/5) = 29/5,$$

$$\bar{\pi}_3 = \min(6 + 6/5, 3 + 9/5) = 24/5.$$

Il valore di questa soluzione duale ammissibile è

$$8 \cdot \frac{26}{5} + 25 \cdot \frac{29}{5} + 27 \cdot \frac{24}{5} = \frac{208+725+648}{5} = \frac{1581}{5}.$$

Per la dualità debole, $LB = 1581/5 \leq z(LP) \leq z(MILP) \leq UB = 439$. La ricetta ha un significato gestionale: «spalmare» il costo fisso di ciascuna sede sulla sua capacità e poi servire ogni cliente al costo marginale minimo è un lower bound che chiunque scriverebbe; il duale lo formalizza.

Quello che dice il solver. L'ottimo del rilassamento LP è esattamente $z(LP) = 1581/5$: la soluzione duale a mano costruita sopra è già ottima (dualità forte, nessun margine di miglioramento). Rafforzando il rilassamento con $x_l \leq 1$ si ottiene $z(LP^+) = 317$: un salto notevole, segno che il vincolo $x_l \leq 1$ — implicito nel MILP ma non nel rilassamento senza i bound — è qui rilevante. Il MILP ha ottimo $z(MILP) = 365$, con $\tilde{x}_1 = \tilde{x}_2 = 1$, $\tilde{y}_{11} = 8$, $\tilde{y}_{12} = 2$, $\tilde{y}_{22} = 23$, $\tilde{y}_{23} = 27$: entrambe le sedi aperte, ma la sede 1 serve solo parte del cliente 2 (il resto arriva dalla sede 2, più conveniente per quel cliente). Il gap dell'euristica rispetto all'ottimo è del 20,3%.

UB (euristica)	LB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
439	1581/5	1581/5	317	365	20,3%

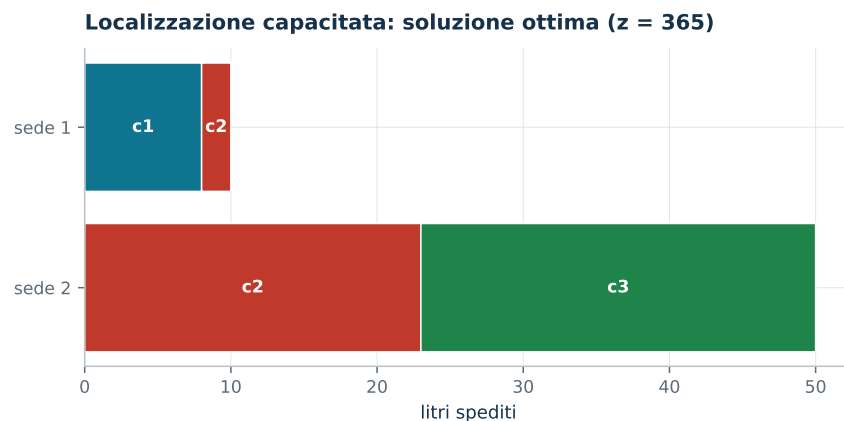


Figura 8.1: Soluzione ottima: per ciascuna sede aperta, i litri spediti a ciascun cliente.

Considerazioni aggiuntive

- Per ogni sede l e cliente c , se $u_l < d_c$ nessuna sede da sola può soddisfare l'intera domanda del cliente c : non è un caso di questa istanza, ma va verificato prima di assumere che una sola sede basti.
- I vincoli $y_{lc} \leq d_c x_l$ (per ogni l, c) sono disuguaglianze valide implicate dai vincoli (8.1b) e (8.1c) insieme: se $x_l = 0$ forzano $y_{lc} = 0$, e se $x_l = 1$ non sono mai più strette di $y_{lc} \leq d_c$ (che segue comunque dal soddisfacimento della domanda).

Domande di modellazione aggiuntive

Domanda di modellazione 8.1 — lotto minimo per ogni sede aperta

Ogni sede aperta deve spedire almeno 5 litri complessivi (non ha senso sostenere il costo di installazione per una quantità trascurabile). Come cambia il modello? Qual è il nuovo ottimo per l'istanza?

Domanda di modellazione 8.2 — apertura condizionata

La sede 2 può essere installata solo se lo è anche la sede 1 (ad esempio, un vincolo logistico di supervisione). Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	439	soluzione euristica (UB)
LB	$\frac{1581}{5}$	bound certificato (LB)
$z(MILP)$	365	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 20.3\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 33.6\%.$$

8.2 p-mediana: al più k sedi

Script: `python/fam08_2_pmediana.py`

[Notebook in Colab](#)

Problema 8.2 — p-mediana

Un'azienda deve scegliere al più $k \in \mathbb{Z}_{\geq 1}$ sedi, fra $m \in \mathbb{Z}_{\geq 1}$ candidate, dove aprire un punto di servizio, e assegnare ciascuno degli $n \in \mathbb{Z}_{\geq 1}$ clienti alla sede aperta più conveniente. Per ogni sede $l \in \{1, 2, \dots, m\}$ e ogni cliente $c \in \{1, 2, \dots, n\}$, il valore $d_{lc} \in \mathbb{Q}_{>0}$ è la distanza (o il costo di servizio) fra la sede l e il cliente c . Ogni cliente va assegnato a esattamente una sede aperta. L'azienda vuole decidere quali sedi aprire e come assegnare i clienti, in modo da minimizzare la somma delle distanze fra ciascun cliente e la sede a cui è assegnato.

Il problema a parole. Decidiamo quali sedi aprire (al più k) e a quale sede aperta assegnare ciascun cliente. *L'obiettivo:* somma delle distanze cliente-sede minima. *I vincoli:* ogni cliente assegnato a esattamente una sede; un cliente può essere assegnato solo a una sede aperta; al più k sedi aperte. È il classico problema della *p-mediana*.

Dati (input del modello)

Simbolo	Tipo	Significato
m	$\in \mathbb{Z}_{\geq 1}$	numero di sedi candidate, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	numero di clienti, $c \in \{1, 2, \dots, n\}$
d_{lc}	$\in \mathbb{Q}_{>0}$	distanza fra la sede l e il cliente c
k	$\in \mathbb{Z}_{\geq 1}$	numero massimo di sedi aperte

Variabili decisionali

Introduciamo le seguenti m variabili binarie:

$$x_l = \begin{cases} 1 & \text{se la sede } l \text{ è aperta,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

e le seguenti $m n$ variabili binarie:

$$y_{lc} = \begin{cases} 1 & \text{se il cliente } c \text{ è servito dalla sede } l, \\ 0 & \text{altrimenti,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}.$$

Usando queste variabili, un modello BIP per il problema è il seguente:

Modello 8.2 — p-mediana

$$\min \sum_{l=1}^m \sum_{c=1}^n d_{lc} y_{lc} \quad (8.3a)$$

$$\text{soggetto a } \sum_{l=1}^m y_{lc} = 1, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.3b)$$

$$\sum_{l=1}^m x_l \leq k, \quad (8.3c)$$

$$x_l - y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \quad (8.3d)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.3e)$$

$$y_{lc} \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.3f)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (8.3a) minimizza la somma delle distanze fra ciascun cliente e la sede a cui è assegnato;
- i vincoli lineari (8.3b) assicurano che ogni cliente sia assegnato a esattamente una sede (n vincoli lineari);
- il vincolo lineare (8.3c) limita a k il numero di sedi aperte (un vincolo lineare);
- i vincoli lineari (8.3d) legano le variabili di assegnamento a quelle di apertura, in forma disaggregata ($m n$ vincoli lineari);

- i vincoli (8.3e) e (8.3f) definiscono le variabili del modello.

Il legame fra le variabili. Per ogni sede l e cliente c , se il cliente c è servito dalla sede l , allora la sede l deve essere aperta:

$$y_{lc} = 1 \implies x_l = 1.$$

In forma booleana l'implicazione equivale, per la CNF, a

$$y_{lc} \implies x_l \quad \equiv \quad \text{NOT } y_{lc} \text{ OR } x_l,$$

cioè $x_l \geq y_{lc}$: esattamente il vincolo (8.3d), imposto direttamente (non serve l'obiettivo). Dalla contronominale,

$$x_l = 0 \implies y_{lc} = 0 :$$

da una sede chiusa non può partire nessun assegnamento — di nuovo imposto direttamente dallo stesso vincolo. A differenza del problema 8.1, qui non c'è un costo di apertura che scoraggi sedi aperte ma inutilizzate: il vincolo (8.3c) impone solo un tetto al numero di sedi aperte, quindi il verso « $x_l = 1 \implies$ almeno un cliente servito da l » non è né imposto né garantito dall'ottimalità, ed è disaggregato — una riga per ogni coppia (l, c) , come nel problema 7.5 — perché il legame è binario contro binario per ciascuna coppia, non un totale contro un'attivazione.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 mod = gp.Model("p_mediana")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, n, vtype=GRB.BINARY, name="y")
4 mod.setObjective(gp.quicksum(dist[l][c] * y[l, c] for l in range(m) for
   c in range(n)), GRB.MINIMIZE)
5 mod.addConstrs((y.sum("*", c) == 1 for c in range(n)), name="assegna")
6 mod.addConstr(x.sum() <= k, name="numero_sedi")
7 mod.addConstrs((x[l] - y[l, c] >= 0 for l in range(m) for c in range(n)
   ), name="link")

```

L'istanza

$m = 3$ sedi, $n = 3$ clienti, $k = 2$:

d_{lc}	$c = 1$	$c = 2$	$c = 3$
$l = 1$	5	6	10
$l = 2$	3	12	9
$l = 3$	10	9	4

Euristica costruttiva: il bound primale

Si aprono le prime k sedi nell'ordine naturale; poi ogni cliente va servito dalla sede aperta più vicina.

Esecuzione sull'istanza (output dello script): si aprono le sedi 1 e 2 ($x_1 = x_2 = 1$). Cliente 1: la sede aperta più vicina è la 2 (distanza 3). Cliente 2: la sede aperta più vicina è la 1 (distanza 6). Cliente 3: la sede aperta più vicina è la 2 (distanza 9). La soluzione ammissibile trovata è

$$\bar{y}_{21} = 1, \quad \bar{y}_{12} = 1, \quad \bar{y}_{23} = 1, \quad \text{tutte le altre } \bar{y}_{lc} = 0,$$

di valore $3 + 6 + 9 = 18$. Quindi

$$z(MILP) \leq UB = 18.$$

Rilassamento LP e duale: il bound duale

Associamo una variabile duale libera μ_c a ciascun vincolo (8.3b) (uguaglianza), una variabile duale non positiva ϱ al vincolo (8.3c) (verso $\leq$ in un problema di minimo) e una variabile duale non negativa π_{lc} a ciascun vincolo (8.3d) (verso $\geq$). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 8.2

$$\max \sum_{c=1}^n \mu_c + k \varrho \quad (8.4a)$$

$$\text{soggetto a } \varrho + \sum_{c=1}^n \pi_{lc} \leq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.4b)$$

$$\mu_c - \pi_{lc} \leq d_{lc}, \quad \forall l \in \{1, 2, \dots, m\}, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.4c)$$

$$\mu_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.4d)$$

$$\varrho \leq 0, \quad (8.4e)$$

$$\pi_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (8.4f)$$

Descrizione. La variabile μ_c è il valore dell'assegnamento del cliente c , ϱ il prezzo (non positivo) del budget di k sedi e π_{lc} il prezzo del legame «per servire c da l bisogna aprire l ». L'obiettivo (8.4a) valuta gli assegnamenti e il budget. I vincoli (8.4b) sono le colonne delle x_l : aprire la sede l consuma una unità di budget e scioglie i legami di tutti i clienti; il saldo non può essere positivo, perché aprire una sede non costa nulla nell'obiettivo di questo modello. I vincoli (8.4c) sono le colonne delle y_{lc} : servire il cliente c dalla sede l vale μ_c e consuma il legame π_{lc} ; il saldo non può superare la distanza d_{lc} .

Una soluzione duale a mano. Con $\bar{\varrho} = 0$ e $\bar{\pi}_{lc} = 0$ per ogni coppia, il vincolo (8.4b) è soddisfatto ($0 \leq 0$) e il vincolo (8.4c) diventa $\mu_c \leq d_{lc}$ per ogni sede l : il valore più grande ammissibile è $\bar{\mu}_c = \min_l d_{lc}$, cioè la distanza dalla sede più vicina in assoluto (aperta o no):

$$\bar{\mu}_1 = \min(5, 3, 10) = 3, \quad \bar{\mu}_2 = \min(6, 12, 9) = 6, \quad \bar{\mu}_3 = \min(10, 9, 4) = 4,$$

di valore $3 + 6 + 4 + 2 \cdot 0 = 13$. Per la dualità debole, $LB = 13 \leq z(LP) \leq z(MILP) \leq UB = 18$. La ricetta ha un significato: «nessun cliente può costare meno della sua sede più vicina in

assoluto» è un lower bound banale che ignora il vincolo di budget k ; il duale lo rappresenta esattamente con $\varrho = 0$.

Quello che dice il solver. L'ottimo del rilassamento LP è $z(LP) = 15$, uguale al rilassamento con i bound $z(LP^+) = 15$: il vincolo $y_{lc} \leq 1$ non è mai rilevante qui. Il MILP ha ottimo $z(MILP) = 15$, con $\tilde{x}_1 = \tilde{x}_3 = 1$ (si aprono le sedi 1 e 3, non 1 e 2 come nell'euristica) e $\tilde{y}_{11} = \tilde{y}_{12} = \tilde{y}_{33} = 1$: il rilassamento LP è già intero su questa istanza, e l'euristica — che apre sempre le prime k sedi senza guardare i costi — perde $18/15 - 1 = 20,0\%$.

UB (euristica)	LB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
18	13	15	15	15	20,0%

p-mediana: soluzione ottima ($z = 15$); quadrato = sede aperta

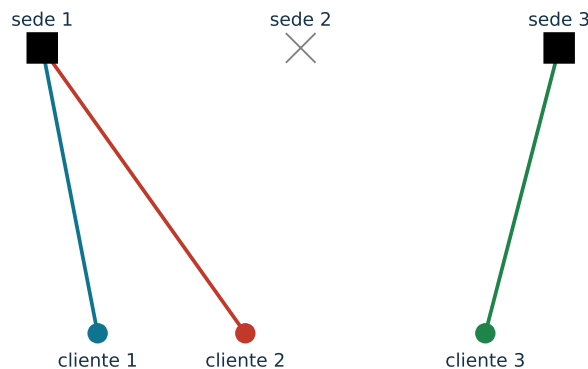


Figura 8.2: Soluzione ottima: quadrato = sede aperta, croce = sede chiusa, linee = assegnamento cliente–sede.

Considerazioni aggiuntive

- Il vincolo (8.3c) è «al più k », non «esattamente k »: aprire meno di k sedi non è mai vantaggioso quando le distanze sono positive (aprire una sede in più non può che avvicinare qualche cliente), ma il modello non lo impone esplicitamente. La domanda 2a qui sotto verifica che, su questa istanza, l'ottimo non cambia imponendo l'uguaglianza.
- I vincoli (8.3d), sommati su c per una sede l fissata, danno $nx_l \geq \sum_{c=1}^n y_{lc}$: una disuguaglianza valida aggregata, più debole di quella disaggregata (che è quella usata nel modello).

Domande di modellazione aggiuntive

Domanda di modellazione 8.3 — esattamente k sedi aperte

Per motivi organizzativi, si devono aprire esattamente k sedi (non al più). Come cambia il modello? Qual è il nuovo ottimo per l'istanza?

Domanda di modellazione 8.4 — copertura di prossimità per un cliente

Il cliente 1 deve essere servito entro distanza 4 (un vincolo di livello di servizio). Come si modella? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	18	soluzione euristica (UB)
LB	13	bound certificato (LB)
$z(MILP)$	15	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 20.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 33.3\%.$$

8.3 Copertura del segnale con interferenza

Script: `python/fam08_3_copertura.py`

[Notebook in Colab](#)

Problema 8.3 — copertura con interferenza

Un operatore deve scegliere al più $k \in \mathbb{Z}_{\geq 1}$ sedi, fra $m \in \mathbb{Z}_{\geq 1}$ candidate, dove installare un'antenna, per servire $n \in \mathbb{Z}_{\geq 1}$ clienti. Per ogni sede $l \in \{1, 2, \dots, m\}$ e ogni cliente $c \in \{1, 2, \dots, n\}$, il valore $s_{lc} \in \mathbb{Q}_{\geq 0}$ è l'intensità del segnale ricevuto dal cliente c se la sede l è installata. Un cliente c è *coperto* se e solo se l'intensità totale di segnale ricevuta da tutte le sedi installate è almeno pari alla soglia $t \in \mathbb{Q}_{>0}$ e al più una sede installata genera per quel cliente un segnale maggiore o uguale al limite di interferenza $b \in \mathbb{Q}_{>0}$. Per ogni cliente c , il valore $p_c \in \mathbb{Q}_{>0}$ è il profitto se il cliente è coperto. L'operatore vuole decidere quali sedi installare per massimizzare il profitto totale.

Il problema a parole. Decidiamo quali sedi installare (al più k). L'obiettivo: profitto totale massimo, somma dei profitti dei clienti coperti. I vincoli: un cliente è coperto se e solo se riceve segnale sufficiente e non troppa interferenza; il numero di sedi installate non supera k .

Dati (input del modello)

Simbolo	Tipo	Significato
m	$\in \mathbb{Z}_{\geq 1}$	numero di sedi candidate, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	numero di clienti, $c \in \{1, 2, \dots, n\}$
s_{lc}	$\in \mathbb{Q}_{\geq 0}$	segnale ricevuto dal cliente c se la sede l è installata
p_c	$\in \mathbb{Q}_{> 0}$	profitto se il cliente c è coperto
t	$\in \mathbb{Q}_{> 0}$	soglia di segnale totale
b	$\in \mathbb{Q}_{> 0}$	limite di interferenza
k	$\in \mathbb{Z}_{\geq 1}$	numero massimo di sedi installate

Per ogni cliente c definiamo $\mathcal{L}_c = \{l : s_{lc} \geq b\}$, le sedi «forti» per quel cliente.

Variabili decisionali

Introduciamo le seguenti m variabili binarie:

$$x_l = \begin{cases} 1 & \text{se si installa la sede } l, \\ 0 & \text{altrimenti,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

e le seguenti n variabili binarie:

$$y_c = \begin{cases} 1 & \text{se il cliente } c \text{ è coperto,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall c \in \{1, 2, \dots, n\}.$$

Usando queste variabili, un modello BIP per il problema è il seguente:

Modello 8.3 — copertura con interferenza

$$\max \sum_{c=1}^n p_c y_c \quad (8.5a)$$

$$\text{soggetto a} \quad - \sum_{l=1}^m s_{lc} x_l + t y_c \leq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.5b)$$

$$\sum_{l \in \mathcal{L}_c} x_l + (m-1) y_c \leq m, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.5c)$$

$$\sum_{l=1}^m x_l \leq k, \quad (8.5d)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.5e)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (8.5f)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (8.5a) massimizza il profitto totale, somma dei profitti dei clienti coperti;

- i vincoli lineari (8.5b) legano le variabili di copertura al segnale ricevuto: impongono che se un cliente è coperto, il segnale totale ricevuto sia almeno t (n vincoli lineari);
- i vincoli lineari (8.5c) legano le variabili di copertura all'interferenza: impongono che se un cliente è coperto, al più una sede in $\mathcal{L}_c$ sia installata (n vincoli lineari);
- il vincolo lineare (8.5d) limita a k il numero di sedi installate (un vincolo lineare);
- i vincoli (8.5e) e (8.5f) definiscono le variabili del modello.

Il legame fra le variabili: un «se e solo se». Per ogni cliente c , il cliente è coperto se e solo se le sedi installate soddisfano insieme la condizione di soglia e quella di interferenza:

$$y_c = 1 \iff \left(\sum_{l=1}^m s_{lc} x_l \geq t \right) \text{ AND } \left(\sum_{l \in \mathcal{L}_c} x_l \leq 1 \right).$$

Un verso, imposto direttamente dai vincoli. Se $y_c = 1$, il vincolo (8.5b) forza $\sum_{l=1}^m s_{lc} x_l \geq t y_c = t$: la soglia è rispettata. Il vincolo (8.5c) forza $\sum_{l \in \mathcal{L}_c} x_l \leq m - (m - 1) y_c = 1$: al più una sede forte è installata. Quindi

$$y_c = 1 \implies \text{entrambe le condizioni.}$$

L'altro verso, che segue dall'ottimalità. Se entrambe le condizioni sono soddisfatte da una data apertura delle sedi, i vincoli (8.5b) e (8.5c) permettono comunque sia $y_c = 0$ sia $y_c = 1$: non c'è niente, nei vincoli, che forzi $y_c = 1$. Ma poiché $p_c > 0$ e la variabile y_c compare solo in questi due vincoli (dove appare con coefficiente positivo sul lato che la limita superiormente), aumentare y_c da 0 a 1 resta ammissibile — per ipotesi le condizioni sono soddisfatte — e aumenta l'obiettivo di p_c . Quindi, in ogni soluzione ottima, $y_c = 1$ ogni volta che le due condizioni sono soddisfatte. Lo stesso schema «un verso dai vincoli, l'altro dall'ottimalità in un problema di massimo» del problema 7.6.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 mod = gp.Model("copertura_interferenza")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(n, vtype=GRB.BINARY, name="y")
4 mod.setObjective(gp.quicksum(p[c] * y[c] for c in range(n)), GRB.
    MAXIMIZE)
5 mod.addConstrs((-gp.quicksum(s[l][c] * x[l] for l in range(m)) + t * y[
    c] <= 0
6                 for c in range(n)), name="soglia")
7 mod.addConstrs((gp.quicksum(x[l] for l in L[c]) + (m - 1) * y[c] <= m
8                 for c in range(n)), name="interferenza")
9 mod.addConstr(x.sum() <= k, name="budget")

```

L'istanza

$m = 3$ sedi, $n = 5$ clienti, $t = 5$, $b = 4$, $k = 2$:

s_{lc}	$c = 1$	$c = 2$	$c = 3$	$c = 4$	$c = 5$
$l = 1$	6	0	5	3	1
$l = 2$	4	5	2	0	0
$l = 3$	0	7	5	4	2

	$c = 1$	$c = 2$	$c = 3$	$c = 4$	$c = 5$
p_c	10	20	5	15	25

Con $b = 4$: $\mathcal{L}_1 = \{1, 2\}$, $\mathcal{L}_2 = \{3\}$, $\mathcal{L}_3 = \{1, 3\}$, $\mathcal{L}_4 = \{3\}$, $\mathcal{L}_5 = \emptyset$.

Euristica costruttiva: il bound primale

Si aprono le prime k sedi nell'ordine naturale; un cliente è coperto se il segnale totale ricevuto dalle sedi aperte raggiunge la soglia t e al più una sede aperta in $\mathcal{L}_c$ è aperta.

Esecuzione sull'istanza (output dello script): si aprono le sedi 1 e 2. Cliente 1: segnale totale $6 + 4 = 10 \geq 5$, ma sedi forti aperte in $\mathcal{L}_1 = \{1, 2\}$ sono entrambe ($2 > 1$): **non coperto**. Cliente 2: segnale totale $0 + 5 = 5 \geq 5$, sedi forti in $\mathcal{L}_2 = \{3\}$ aperte: nessuna ($0 \leq 1$): **coperto**. Cliente 3: segnale totale $5 + 2 = 7 \geq 5$, sedi forti in $\mathcal{L}_3 = \{1, 3\}$ aperte: solo la sede 1 ($1 \leq 1$): **coperto**. Cliente 4: segnale totale $3 + 0 = 3 < 5$: **non coperto**. Cliente 5: segnale totale $1 + 0 = 1 < 5$: **non coperto**. La soluzione ammissibile trovata copre i clienti 2 e 3, di valore $20 + 5 = 25$. Quindi

$$z(MILP) \geq LB = 25.$$

Rilassamento LP e duale: il bound duale

Associamo una variabile duale non negativa π_c a ciascun vincolo (8.5b) (verso $\leq$ in un problema di massimo), una variabile duale non negativa λ_c a ciascun vincolo (8.5c) e una variabile duale non negativa μ al vincolo (8.5d). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 8.3

$$\min \quad m \sum_{c=1}^n \lambda_c + k \mu \quad (8.6a)$$

$$\text{soggetto a} \quad - \sum_{c=1}^n s_{lc} \pi_c + \sum_{c: l \in \mathcal{L}_c} \lambda_c + \mu \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.6b)$$

$$t \pi_c + (m - 1) \lambda_c \geq p_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6c)$$

$$\pi_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6d)$$

$$\lambda_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6e)$$

$$\mu \geq 0. \quad (8.6f)$$

Descrizione. La variabile π_c è il prezzo del vincolo di soglia del cliente c , λ_c quello del suo vincolo di interferenza e μ il prezzo di una unità di budget. L'obiettivo (8.6a) somma i termini noti di quei vincoli valutati a quei prezzi. I vincoli (8.6b) sono le colonne delle x_l : installare l'antenna l porta segnale ai clienti che copre, entra nei vincoli di interferenza dei clienti la cui lista la contiene e consuma una unità di budget; il saldo deve restare non negativo, perché le

x_l non compaiono nell'obiettivo. I vincoli (8.6c) sono le colonne delle y_c : dichiarare servito il cliente c richiede t unità di segnale al prezzo π_c e $m - 1$ unità di interferenza al prezzo λ_c , e le due insieme devono coprire il premio p_c .

Una soluzione duale a mano. Con $\bar{\pi}_c = 0$ e $\bar{\mu} = 0$, il vincolo (8.6b) è soddisfatto ($\sum \lambda_c \geq 0$), e il vincolo (8.6c) diventa $(m - 1)\bar{\lambda}_c \geq p_c$, cioè $\bar{\lambda}_c = p_c/(m - 1) = p_c/2$:

$$\bar{\lambda}_1 = 5, \quad \bar{\lambda}_2 = 10, \quad \bar{\lambda}_3 = 5/2, \quad \bar{\lambda}_4 = 15/2, \quad \bar{\lambda}_5 = 25/2.$$

Il valore di questa soluzione duale ammissibile è $m \sum_{c=1}^n \bar{\lambda}_c + k \cdot 0 = 3 \cdot (5 + 10 + 5/2 + 15/2 + 25/2) = 3 \cdot 75/2 = 225/2$. Per la dualità debole, $LB = 25 \leq z(MILP) \leq z(LP) \leq UB = 225/2$ (problema di massimo: l'euristica dà il lower bound, il duale l'upper bound).

Quello che dice il solver. L'ottimo del rilassamento LP è $z(LP) = 41925/646 \approx 64,9$, mentre rafforzando con $x_l \leq 1$ si ottiene $z(LP^+) = 125/2 = 62,5$: un miglioramento netto. Il MILP ha ottimo $z(MILP) = 45$, con $\tilde{x}_1 = \tilde{x}_3 = 1$ e $\tilde{y}_1 = \tilde{y}_2 = \tilde{y}_4 = 1$ (i clienti 1, 2 e 4 coperti, non il 3 né il 5): una configurazione diversa da quella trovata dall'euristica, che copriva i clienti 2 e 3. Il gap dell'euristica è del 44,4%: su questo problema di massimo, aprire semplicemente le prime k sedi è una scelta molto meno efficace che nei problemi di minimo costo della famiglia.

LB (euristica)	UB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
25	225/2	41925/646	125/2	45	44,4%

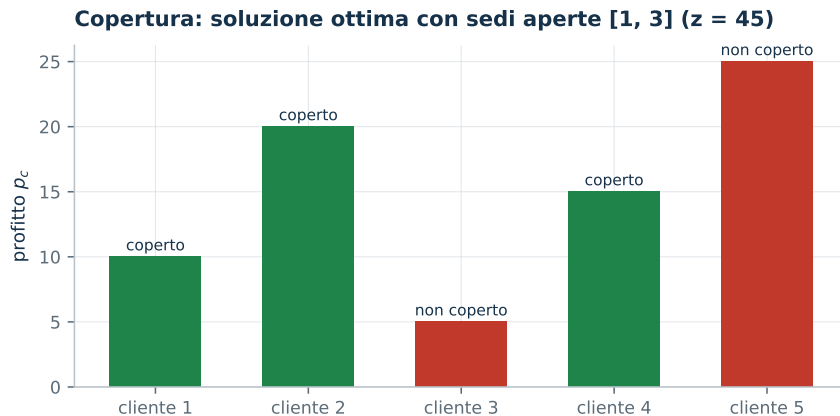


Figura 8.3: Soluzione ottima: profitto di ciascun cliente, verde se coperto, rosso se non coperto.

Considerazioni aggiuntive

- Il cliente 5 non può mai essere coperto in questa istanza: $\mathcal{L}_5 = \emptyset$, ma il segnale massimo ricevibile è $s_{15} + s_{25} + s_{35} = 1 + 0 + 2 = 3 < t = 5$ anche aprendo tutte le sedi. La variabile y_5 potrebbe essere fissata a 0 a priori.
- I vincoli (8.5c), per un cliente c con $|\mathcal{L}_c| \leq 1$, sono ridondanti (mai stretti): è il caso dei clienti 2, 4 e 5 in questa istanza.

Domande di modellazione aggiuntive

Domanda di modellazione 8.5 — copertura minima garantita

Per contratto, almeno 3 clienti devono essere coperti. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 8.6 — installazione condizionata

Per motivi di manutenzione, la sede 1 può essere installata solo se lo è anche la sede 3. Come si modella? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	25	soluzione euristica (LB)
UB	$\frac{225}{2}$	bound certificato (UB)
$z(MILP)$	45	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 44.4\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 194.4\%.$$

8.4 Localizzazione di hub con costo massimo

Script: `python/fam08_4_hub.py`

[Notebook in Colab](#)

Problema 8.4 — hub con costo di connessione massimo

In una rete di computer ci sono $n \in \mathbb{Z}_{\geq 1}$ terminali, ciascuno da connettere a esattamente un hub. Sono disponibili $m \in \mathbb{Z}_{\geq 1}$ hub, e ogni hub può servire al più $k \in \mathbb{Z}_{\geq 1}$ terminali. Un hub va attivato prima di poter servire dei terminali: per ogni hub $j \in \{1, 2, \dots, m\}$, il valore $f_j \in \mathbb{Q}_{\geq 0}$ è il costo di attivazione. Per ogni terminale $i \in \{1, 2, \dots, n\}$ e ogni hub j , il valore $c_{ij} \in \mathbb{Q}_{\geq 0}$ è il costo di connettere il terminale i all'hub j . L'obiettivo è decidere come assegnare i terminali agli hub per minimizzare la somma dei costi di attivazione degli hub usati e del costo di connessione massimo di ciascun hub, nel rispetto dei vincoli descritti.

Il problema a parole. Decidiamo quali hub attivare e a quale hub connettere ciascun terminale. *L'obiettivo:* costi di attivazione più, per ciascun hub, il costo di connessione più alto fra i terminali che gli sono assegnati (non la somma: il collo di bottiglia). *I vincoli:* ogni terminale connesso a esattamente un hub; un hub non attivato non può servire nessuno, e uno attivato serve al più k terminali.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di terminali, $i \in \{1, 2, \dots, n\}$
m	$\in \mathbb{Z}_{\geq 1}$	numero di hub, $j \in \{1, 2, \dots, m\}$
k	$\in \mathbb{Z}_{\geq 1}$	capacità (terminali) di ciascun hub
c_{ij}	$\in \mathbb{Q}_{\geq 0}$	costo di connettere il terminale i all'hub j
f_j	$\in \mathbb{Q}_{\geq 0}$	costo di attivazione dell'hub j

Variabili decisionali

Introduciamo le seguenti $n m$ variabili binarie:

$$x_{ij} = \begin{cases} 1 & \text{se il terminale } i \text{ è connesso all'hub } j, \\ 0 & \text{altrimenti,} \end{cases} \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\},$$

le seguenti m variabili binarie:

$$y_j = \begin{cases} 1 & \text{se l'hub } j \text{ è attivato,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, \dots, m\},$$

e le seguenti m variabili continue non negative:

$$z_j = \text{costo di connessione massimo fra i terminali assegnati all'hub } j, \quad \forall j \in \{1, 2, \dots, m\}.$$

Usando queste variabili, un modello MILP per il problema è il seguente:

Modello 8.4 — hub con costo massimo

$$\min \sum_{j=1}^m f_j y_j + \sum_{j=1}^m z_j \quad (8.7a)$$

$$\text{soggetto a } \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\}, \quad (8.7b)$$

$$-\sum_{i=1}^n x_{ij} + k y_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.7c)$$

$$-c_{ij} x_{ij} + z_j \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.7d)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.7e)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.7f)$$

$$z_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}. \quad (8.7g)$$

Descrizione della funzione obiettivo e dei vincoli

- la funzione obiettivo lineare (8.7a) minimizza il costo totale, somma dei costi di attivazione degli hub usati e del costo di connessione massimo di ciascun hub;

- i vincoli lineari (8.7b) assicurano che ogni terminale sia connesso a esattamente un hub (n vincoli lineari);
- i vincoli lineari (8.7c) legano le variabili di assegnamento a quelle di attivazione e impongono la capacità: se un hub non è attivato non può servire nessuno, e se è attivato serve al più k terminali (m vincoli lineari);
- i vincoli lineari (8.7d) legano le variabili di assegnamento alla variabile di massimo: se il terminale i è connesso all'hub j , allora z_j non può essere inferiore a c_{ij} ($n m$ vincoli lineari);
- i vincoli (8.7e), (8.7f) e (8.7g) definiscono le variabili del modello.

Primo legame: attivazione (aggregata). Per ogni hub j , se almeno un terminale è connesso a j , allora j deve essere attivato:

$$\sum_{i=1}^n x_{ij} \geq 1 \implies y_j = 1,$$

$$\text{per contronominale: } y_j = 0 \implies \sum_{i=1}^n x_{ij} = 0.$$

Questa implicazione — insieme alla sua contronominale — è imposta direttamente, in forma aggregata, dal vincolo (8.7c): se $y_j = 0$ il vincolo diventa $\sum_{i=1}^n x_{ij} \leq 0$, cioè $x_{ij} = 0$ per ogni i ; se $y_j = 1$ diventa $\sum_{i=1}^n x_{ij} \leq k$, la capacità. Il verso opposto — un hub attivato serve almeno un terminale:

$$y_j = 1 \implies \sum_{i=1}^n x_{ij} \geq 1$$

— non è imposto dai vincoli: $y_j = 1$ con tutte le $x_{ij} = 0$ soddisfa (8.7c). Segue dall'ottimalità, con una forza che dipende dal segno di f_j . Se $f_j > 0$: data una soluzione ottima con $\sum_{i=1}^n x_{ij} = 0$ e $y_j = 1$, porre $y_j = 0$ resta ammissibile e riduce il costo di $f_j > 0$, contro l'ottimalità; quindi **in ogni ottimo** nessun hub attivato resta vuoto. Se invece $f_j = 0$ — che l'enunciato ammette, avendo dichiarato $f_j \in \mathbb{Q}_{\geq 0}$ — lo scambio non peggiora ma nemmeno migliora, e la conclusione corretta è la più debole: **esiste un ottimo** in cui gli hub vuoti sono spenti. Sull'istanza $f = (5, 6, 7)$ è strettamente positivo e vale la versione forte; ma la dimostrazione va scritta con l'ipotesi che usa, non con quella che fa comodo. Esattamente lo schema di attivazione già visto nel problema 7.2, qui in forma aggregata.

Secondo legame: variabile di massimo. Per ogni terminale i e hub j , se il terminale i è connesso all'hub j , allora $z_j \geq c_{ij}$:

$$x_{ij} = 1 \implies z_j \geq c_{ij},$$

imposto direttamente dal vincolo (8.7d) (equivalente a $z_j \geq c_{ij}x_{ij}$). Combinando su tutti i terminali connessi a j , $z_j \geq \max_{i: x_{ij}=1} c_{ij}$ (con la convenzione che il massimo sull'insieme vuoto è 0, garantita da $z_j \geq 0$). Poiché l'obiettivo minimizza $\sum_{j=1}^m z_j$ e z_j non compare in nessun altro vincolo, in ogni soluzione ottima z_j assume il valore più piccolo ammissibile: se $z_j > \max_{i: x_{ij}=1} c_{ij}$, diminuirlo fino a quel massimo resta ammissibile e riduce l'obiettivo. Quindi,

in ogni ottimo, $z_j = \max_{i: x_{ij}=1} c_{ij}$ esattamente: lo stesso schema della variabile di massimo del problema 7.7.

I due bound che costruiamo qui sotto — quello primale, dall’euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 mod = gp.Model("hub_max")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, vtype=GRB.BINARY, name="y")
4 z = mod.addVars(m, name="z")
5 mod.setObjective(gp.quicksum(f[j] * y[j] for j in range(m)) + z.sum(),
6                      GRB.MINIMIZE)
7 mod.addConstrs((gp.quicksum(x[i, j] for j in range(m)) == 1 for i in
8                      range(n)), name="assegnamento")
9 mod.addConstrs((-gp.quicksum(x[i, j] for i in range(n)) + k * y[j] >= 0
10                      for j in range(m)), name="attivazione")
11 mod.addConstrs((-c[i][j] * x[i, j] + z[j] >= 0 for i in range(n) for j
12                      in range(m)), name="massimo")

```

L’istanza

$n = 3$ terminali, $m = 3$ hub, $k = 2$:

c_{ij}	$j = 1$	$j = 2$	$j = 3$
$i = 1$	5	10	2
$i = 2$	5	4	6
$i = 3$	5	4	6

	$j = 1$	$j = 2$	$j = 3$
f_j	5	6	7

Euristica costruttiva: il bound primale

Un’euristica *next-fit*, ispirata al bin packing: si riempie un hub alla volta fino a k terminali, poi si passa al successivo; se tutti gli hub sono stati usati prima di aver assegnato tutti i terminali, l’algoritmo fallisce. È la stessa euristica generica dei problemi di scheduling della famiglia precedente (gli hub sono le «macchine» con capacità k , i terminali i «lavori» con tempo unitario indipendente dalla macchina): lo script la richiama da `euristiche.py` invece di riscriverla.

Esecuzione sull’istanza (output dello script): il terminale 1 va sull’hub 1 (disponibilità residua $2 \rightarrow 1$); il terminale 2 va sull’hub 1 (disponibilità residua $1 \rightarrow 0$); il terminale 3 non ci sta più sull’hub 1 (pieno), passa sull’hub 2. Si attivano gli hub 1 e 2: il costo di connessione massimo dell’hub 1 è $\max(c_{11}, c_{21}) = \max(5, 5) = 5$, quello dell’hub 2 è $c_{32} = 4$. La soluzione ammissibile trovata è

$$\bar{x}_{11} = \bar{x}_{21} = \bar{x}_{32} = 1, \quad \bar{y}_1 = \bar{y}_2 = 1, \quad \bar{z}_1 = 5, \bar{z}_2 = 4,$$

di valore $5 + 6 + 5 + 4 = 20$. Quindi

$$z(MILP) \leq UB = 20.$$

Rilassamento LP e duale: il bound duale

Associamo una variabile duale libera α_i a ciascun vincolo (8.7b) (uguaglianza), una variabile duale non negativa β_j a ciascun vincolo (8.7c) (verso $\geq$) e una variabile duale non negativa γ_{ij} a ciascun vincolo (8.7d) (verso $\geq$). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 8.4

$$\max \sum_{i=1}^n \alpha_i \quad (8.8a)$$

$$\text{soggetto a } \alpha_i - \beta_j - c_{ij} \gamma_{ij} \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.8b)$$

$$k \beta_j \leq f_j, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8c)$$

$$\sum_{i=1}^n \gamma_{ij} \leq 1, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8d)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \quad (8.8e)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8f)$$

$$\gamma_{ij} \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}. \quad (8.8g)$$

Descrizione. La variabile α_i è il valore del terminale i , β_j il prezzo di uno dei k posti dell'hub j e γ_{ij} il prezzo del legame che spinge il costo massimo dell'hub j sopra c_{ij} . L'obiettivo (8.8a) valuta i terminali, che vanno assegnati tutti. I vincoli (8.8b) sono le colonne delle x_{ij} : assegnare il terminale i all'hub j soddisfa il suo vincolo di assegnamento, occupa un posto e obbliga il costo massimo dell'hub a salire fino a c_{ij} ; il saldo non può essere positivo. I vincoli (8.8c) sono le colonne delle y_j : i k posti che l'apertura dell'hub j mette a disposizione non possono valere più del costo fisso f_j . I vincoli (8.8d) sono le colonne delle z_j : il costo massimo dell'hub j pesa 1 nell'obiettivo primale, quindi i prezzi dei legami che lo spingono verso l'alto sommano al più a uno.

Una soluzione duale a mano. Poniamo $\bar{\gamma}_{ij} = 0$ per ogni coppia (il vincolo (8.8d) è automaticamente soddisfatto) e $\bar{\beta}_j = f_j/k$, il valore più grande ammissibile per il vincolo (8.8c): sull'istanza $\bar{\beta} = (5/2, 3, 7/2)$. Il vincolo (8.8b) diventa allora $\alpha_i \leq \bar{\beta}_j$ per *ogni* hub j (non solo per il più conveniente): il valore più grande ammissibile è $\bar{\alpha}_i = \min_j \bar{\beta}_j = 5/2$ per ogni terminale i . Il valore di questa soluzione duale ammissibile è $3 \cdot 5/2 = 15/2$. Per la dualità debole, $LB = 15/2 \leq z(LP) \leq z(MILP) \leq UB = 20$.

Fissare $\bar{\gamma}_{ij} = 0$ solo per gli hub «non convenienti» e sperare che il vincolo (8.8b) imponga α_i soltanto rispetto all'hub più economico è un errore: il vincolo vale per *ogni* j , quindi α_i è limitato dal *minimo* su tutti gli hub, non da uno solo. Una soluzione duale che ignori questo — ad esempio ponendo $\bar{\gamma}_{i1} = 1/n$ solo per l'hub più economico e lasciando $\bar{\beta}_j = 0$ altrove — non è ammissibile: violerebbe il vincolo (8.8b) per gli altri hub.

Quello che dice il solver. L'ottimo del rilassamento LP è $z(LP) = 25/2$, che il rafforzamento con $x_{ij} \leq 1$ migliora a $z(LP^+) = 1015/78 \approx 13,0$. Il MILP ha ottimo $z(MILP) = 19$,

con $\tilde{x}_{13} = \tilde{x}_{21} = \tilde{x}_{31} = 1$, $\tilde{y}_1 = \tilde{y}_3 = 1$, $\tilde{z}_1 = 5$, $\tilde{z}_3 = 2$: si attivano gli hub 1 e 3 (non 1 e 2 come nell'euristica next-fit), con il terminale 1 da solo sull'hub 3 (il più economico per lui, $c_{13} = 2$) e i terminali 2 e 3 insieme sull'hub 1. Il gap dell'euristica è del 5,3%: qui il next-fit, che non guarda i costi, è comunque abbastanza vicino all'ottimo.

UB (euristica)	LB (duale a mano)	$z(LP)$	$z(LP^+)$	$z(MILP)$	gap euristica
20	15/2	25/2	1015/78	19	5,3%

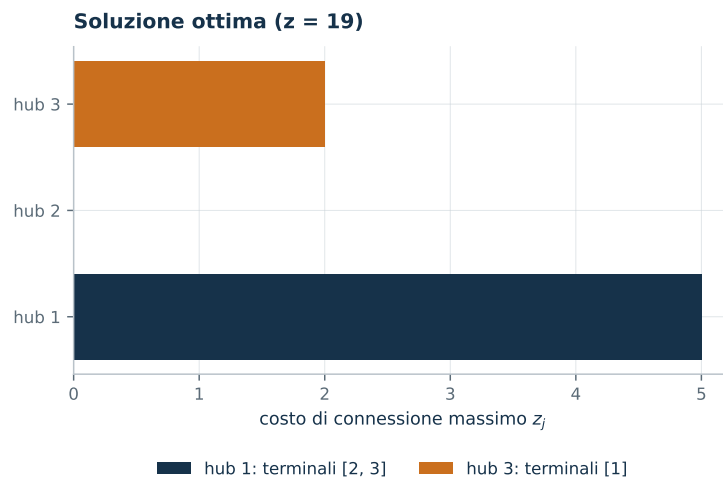


Figura 8.4: Soluzione ottima: costo di connessione massimo di ciascun hub attivato, con i terminali assegnati.

Considerazioni aggiuntive

- I vincoli disaggregati $x_{ij} \leq y_j$ (per ogni i, j) sono disuguaglianze valide implicate dal vincolo (8.7c) aggregato: danno la stessa logica di attivazione in forma diversa, e la domanda 4a qui sotto verifica che non cambiano l'ottimo su questa istanza.
- Ponendo $M_j = \max_i c_{ij}$, i vincoli $z_j \leq M_j y_j$ non sono disuguaglianze valide per il modello (che ammette $z_j > 0$ anche con $y_j = 0$), ma sono *vincoli che preservano l'ottimalità*: in ogni ottimo $z_j = 0$ quando $y_j = 0$, perché l'obiettivo minimizza z_j .

Domande di modellazione aggiuntive

Domanda di modellazione 8.7 — link di attivazione disaggregato

Si *aggiungano* al modello (8.7) i link disaggregati $x_{ij} \leq y_j$. Cambia l'ottimo per l'istanza? Cambia il rilassamento? E che cosa succede se, invece di aggiungerli, si *sostituisce* con essi il vincolo aggregato (8.7c)?

Domanda di modellazione 8.8 — connessione vietata

Il terminale 1 non può essere connesso all'hub 2 (un vincolo di sicurezza). Come si modella? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	20	soluzione euristica (UB)
LB	$\frac{15}{2}$	bound certificato (LB)
$z(MILP)$	19	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 5.3\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 65.8\%.$$

Pianificazione della produzione

Classe: MILP

Script: uno per problema, `python/fam09_1_lotti.py` ...

[Pagina online](#)

La famiglia

Tre problemi in cui si decide *quanto* produrre, non soltanto *se* fare qualcosa. Le variabili di quantità sono continue o intere, e sopra di esse si appoggiano delle binarie che accendono, limitano o premiano: il lancio della produzione con il suo costo fisso (problema 1), l'organico con le assunzioni (2), il lotto minimo per tipo con un premio per la varietà (3).

Il tratto comune è il *bilancio*: una quantità che entra, una che esce e una che resta. Nel problema 1 e nel 2 è il bilancio delle scorte, $x_t + s_{t-1} - s_t = d_t$; nel problema 2 c'è in più il bilancio dell'organico, $y_t - y_{t-1} - z_t = 0$. Sono vincoli di uguaglianza, e i loro duali sono variabili libere: è la prima famiglia del corso in cui questo succede in modo sistematico.

I legami fra variabili del capitolo 3 che qui si rivedono:

- **costo fisso** (problema 1): $x_t \leq M_t y_t$, con M_t letto dai dati (la domanda residua) e non scelto a caso;
- **conteggi interi** (problema 2): l'organico y_t e le assunzioni z_t sono numeri di persone, e le ore disponibili sono $r y_t$;
- **lotto minimo** (problema 3): la variabile semicontinua $q_j y_j \leq x_j \leq M_j y_j$, che vale zero oppure almeno q_j ;
- **contare i tipi e se e solo se** (problema 3): il premio per la varietà si incassa solo se il conteggio dei tipi attivi arriva a due, e il verso mancante lo impone l'ottimalità perché il premio è positivo.

Per ognuno: modello, dimostrazione dei legami, istanza, euristica costruttiva, duale del rilassamento LP con una soluzione duale costruita a mano, soluzione del solver, considerazioni aggiuntive e due domande di modellazione con soluzione. Tutti i numeri vengono dagli script `fam09_1_lotti.py`, `fam09_2_manodopera.py` e `fam09_3_veicoli.py` della cartella `python/`, uno per problema, con il suo notebook.

Notazione della famiglia

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di periodi, indicizzati da $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Q}_{\geq 0}$	domanda del periodo t
p_t	$\in \mathbb{Q}_{> 0}$	costo unitario di produzione nel periodo t
q_t	$\in \mathbb{Q}_{\geq 0}$	costo fisso di lancio della produzione nel periodo t
h_t	$\in \mathbb{Q}_{\geq 0}$	costo di magazzino per una unità a fine periodo t
r_0, r_n	$\in \mathbb{Q}_{\geq 0}$	scorta iniziale e scorta finale richiesta
M_t	$\in \mathbb{Q}_{> 0}$	massima produzione utile nel periodo t : $\sum_{\tau \geq t} d_\tau + r_n$
m_0	$\in \mathbb{Z}_{\geq 0}$	operai in servizio all'inizio dell'orizzonte
w	$\in \mathbb{Q}_{> 0}$	salario di un operaio per periodo
u	$\in \mathbb{Q}_{\geq 0}$	costo di assunzione di un operaio
r	$\in \mathbb{Q}_{> 0}$	ore lavorate da un operaio in un periodo
g	$\in \mathbb{Q}_{> 0}$	ore di lavoro necessarie per una unità di prodotto
s	$\in \mathbb{Z}_{\geq 1}$	numero di tipi di prodotto, indicizzati da j
k	$\in \mathbb{Z}_{\geq 1}$	numero di risorse, indicizzate da i
a_{ij}	$\in \mathbb{Q}_{\geq 0}$	quantità di risorsa i per una unità del tipo j
b_i	$\in \mathbb{Q}_{> 0}$	disponibilità della risorsa i
$\bar{p}_j$	$\in \mathbb{Q}_{> 0}$	profitto unitario del tipo j
$\bar{q}_j$	$\in \mathbb{Z}_{\geq 1}$	lotto minimo del tipo j , se lo si produce
$\bar{r}$	$\in \mathbb{Q}_{> 0}$	premio se si producono almeno due tipi diversi

9.1 Produzione e lotti con costo fisso di lancio

Script: `python/fam09_1_lotti.py`[Notebook in Colab](#)**Problema 9.1 — lotti con costo fisso**

Un'azienda deve pianificare la produzione di un unico prodotto su $n \in \mathbb{Z}_{\geq 1}$ periodi. Per ogni periodo $t \in \{1, 2, \dots, n\}$, il valore $d_t \in \mathbb{Q}_{\geq 0}$ è la domanda da soddisfare, $p_t \in \mathbb{Q}_{> 0}$ il costo di produrre una unità e $q_t \in \mathbb{Q}_{\geq 0}$ il costo fisso da sostenere se in quel periodo la produzione viene lanciata. Per ogni periodo $t \in \{1, 2, \dots, n-1\}$, il valore $h_t \in \mathbb{Q}_{\geq 0}$ è il costo di tenere in magazzino una unità dalla fine del periodo t all'inizio del successivo. La scorta iniziale è $r_0 \in \mathbb{Q}_{\geq 0}$ e quella richiesta alla fine dell'orizzonte è $r_n \in \mathbb{Q}_{\geq 0}$. L'azienda vuole decidere quanto produrre in ogni periodo in modo da soddisfare tutta la domanda al costo totale minimo.

Il problema a parole. Decidiamo quanto produrre in ogni periodo, e di conseguenza quanto resta in magazzino. *L'obiettivo:* costo totale (produzione, lanci e magazzino) minimo. *I vincoli:* la domanda di ogni periodo va soddisfatta esattamente, con quello che si produce e con quello che si è tenuto da parte; e non si può produrre senza pagare il costo fisso di lancio. È il problema di *lot sizing* con setup.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di periodi, $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Q}_{\geq 0}$	domanda del periodo t
p_t	$\in \mathbb{Q}_{> 0}$	costo unitario di produzione nel periodo t
q_t	$\in \mathbb{Q}_{\geq 0}$	costo fisso di lancio nel periodo t
h_t	$\in \mathbb{Q}_{\geq 0}$	costo di magazzino a fine periodo t , $t \leq n - 1$
r_0, r_n	$\in \mathbb{Q}_{\geq 0}$	scorta iniziale e finale richiesta

Variabili decisionali

Introduciamo le seguenti n variabili continue non negative:

$$x_t = \text{unità prodotte nel periodo } t, \quad \forall t \in \{1, 2, \dots, n\},$$

le seguenti $n - 1$ variabili continue non negative:

$$s_t = \text{unità in magazzino a fine periodo } t, \quad \forall t \in \{1, 2, \dots, n - 1\},$$

e le seguenti n variabili binarie:

$$y_t = \begin{cases} 1 & \text{se nel periodo } t \text{ la produzione viene lanciata,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall t \in \{1, 2, \dots, n\}.$$

Usando queste variabili, un modello MILP per il problema è il seguente:

Modello 9.1 — lotti con costo fisso

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^n q_t y_t + \sum_{t=1}^{n-1} h_t s_t \quad (9.1a)$$

$$\text{soggetto a } x_1 - s_1 = d_1 - r_0, \quad (9.1b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, \dots, n - 1\}, \quad (9.1c)$$

$$x_n + s_{n-1} = d_n + r_n, \quad (9.1d)$$

$$-x_t + M_t y_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.1e)$$

$$x_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.1f)$$

$$s_t \geq 0, \quad \forall t \in \{1, 2, \dots, n - 1\}, \quad (9.1g)$$

$$y_t \in \{0, 1\}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.1h)$$

Descrizione. La funzione obiettivo (9.1a) somma il costo di produzione, quello dei lanci e quello del magazzino. I vincoli (9.1b)–(9.1d) sono il bilancio delle scorte, uno per periodo (n vincoli di uguaglianza): quello che si aveva più quello che si produce meno quello che si vende è quello che resta. I vincoli (9.1e) legano quantità e lancio (n vincoli lineari).

Il legame fra le variabili. Il vincolo (9.1e) si legge $x_t \leq M_t y_t$. Se $y_t = 0$ allora $x_t \leq 0$ e, con $x_t \geq 0$, si ottiene $x_t = 0$: senza lancio non si produce. Se $y_t = 1$ il vincolo diventa $x_t \leq M_t$ e non è restrittivo, purché M_t sia scelto abbastanza grande. Il verso opposto — se $x_t = 0$ allora $y_t = 0$ — non è imposto da alcun vincolo, ma segue dall'ottimalità: se $x_t = 0$ e $y_t = 1$, porre $y_t = 0$ resta ammissibile e fa risparmiare $q_t \geq 0$.

La tavola di verità del legame $x_t \leq M_t y_t$

Il legame va verificato su tutte le combinazioni dei valori di y_t , tenendo conto che $x_t \geq 0$:

y_t	che cosa impone il vincolo	x_t ammesse	lettura
0	$x_t \leq 0$	solo $x_t = 0$	senza lancio non si produce
1	$x_t \leq M_t$	$0 \leq x_t \leq M_t$	col lancio si produce quanto si vuole

Resta scoperta la combinazione $x_t = 0$ con $y_t = 1$: è ammissibile per il modello, ma non compare in alcuna soluzione ottima, perché

$$y_t = 1 \text{ e } x_t = 0 \implies \text{porre } y_t = 0 \text{ resta ammissibile e risparmia } q_t \geq 0.$$

Un M_t valido deve essere almeno pari alla massima quantità che conviene produrre nel periodo t . In una soluzione ottima non si produce mai più della domanda che resta da coprire, scorta finale inclusa:

$$M_t = \sum_{\tau=t}^n d_{\tau} + r_n.$$

Ogni valore più grande è ancora valido ma indebolisce il rilassamento LP, e ogni valore più piccolo può tagliare soluzioni ottime. Sull'istanza, $M = (110, 90, 80, 50, 10)$: il big-M dell'ultimo periodo vale 10, non 110.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 m = gp.Model("lotti")
2 x = m.addVars(n, name="x")
3 s = m.addVars(n - 1, name="s")
4 y = m.addVars(n, vtype=GRB.BINARY, name="y")
5 m.setObjective(gp.quicksum(p[t] * x[t] for t in range(n))
6                 + gp.quicksum(q[t] * y[t] for t in range(n))
7                 + gp.quicksum(h[t] * s[t] for t in range(n - 1)), GRB.
MINIMIZE)
8 m.addConstr(x[0] - s[0] == d[0] - r0, name="bilancio[0]")
9 m.addConstrs((x[t] + s[t - 1] - s[t] == d[t] for t in range(1, n - 1)),
name="bilancio")
10 m.addConstr(x[n - 1] + s[n - 2] == d[n - 1] + rn, name=f"bilancio[{n -
1}]")
11 m.addConstrs((-x[t] + M[t] * y[t] >= 0 for t in range(n)), name="lancio
")

```

L'istanza

$n = 5$ periodi (giorni), $r_0 = r_n = 0$:

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
d_t	20	10	30	40	10
p_t	2	3	2	3	2
q_t	50	50	50	50	50
M_t	110	90	80	50	10

	$t = 1$	$t = 2$	$t = 3$	$t = 4$
h_t	1	1	1	1

La domanda totale è 110 unità.

Scritto per esteso, con i coefficienti dell'istanza, il modello (9.1) diventa il seguente. Le cinque variabili x_t sono la produzione, le quattro s_t le scorte, le cinque y_t i lanci.

Modello 9.1 sull'istanza

$$\begin{aligned}
 &\min 2x_1 + 3x_2 + 2x_3 + 3x_4 + 2x_5 + s_1 + s_2 + s_3 + s_4 + 50y_1 + 50y_2 + 50y_3 + 50y_4 + 50y_5 \\
 &\quad x_1 \qquad \qquad \qquad - s_1 \qquad \qquad \qquad = 20 \\
 &\quad \quad x_2 \qquad \qquad \qquad + s_1 - s_2 \qquad \qquad \qquad = 10 \\
 &\quad \quad \quad x_3 \qquad \qquad \qquad + s_2 - s_3 \qquad \qquad \qquad = 30 \\
 &\quad \quad \quad \quad x_4 \qquad \qquad \qquad + s_3 - s_4 \qquad \qquad \qquad = 40 \\
 &\quad \quad \quad \quad \quad x_5 \qquad \qquad \qquad + s_4 \qquad \qquad \qquad = 10 \\
 &\quad -x_1 \qquad \qquad \qquad \qquad \qquad \qquad + 110y_1 \qquad \qquad \geq 0 \\
 &\quad \quad -x_2 \qquad \qquad \qquad \qquad \qquad \qquad + 90y_2 \qquad \qquad \geq 0 \\
 &\quad \quad \quad -x_3 \qquad \qquad \qquad \qquad \qquad \qquad + 80y_3 \qquad \qquad \geq 0 \\
 &\quad \quad \quad \quad -x_4 \qquad \qquad \qquad \qquad \qquad \qquad + 50y_4 \qquad \qquad \geq 0 \\
 &\quad \quad \quad \quad \quad -x_5 \qquad \qquad \qquad \qquad \qquad \qquad + 10y_5 \geq 0 \\
 &\quad x_1, \quad x_2, \quad x_3, \quad x_4, \quad x_5, \quad s_1, \quad s_2, \quad s_3, \quad s_4 \qquad \qquad \geq 0 \\
 &\quad \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad y_1, \quad y_2, \quad y_3, \quad y_4, \quad y_5 \in \{0, 1\}
 \end{aligned}$$

Il duale del rilassamento LP ha una variabile libera μ_t per ciascuno dei cinque bilanci e una variabile non negativa π_t per ciascuno dei cinque vincoli di lancio. Le quattordici colonne del primale diventano quattordici vincoli duali: cinque per le x_t , quattro per le s_t , cinque per le y_t .

Duale sull'istanza

$$\begin{array}{rcl}
\max & 20\mu_1 + 10\mu_2 + 30\mu_3 + 40\mu_4 + 10\mu_5 & \\
& \mu_1 & - \pi_1 \leq 2 \\
& & \mu_2 & - \pi_2 \leq 3 \\
& & & \mu_3 & - \pi_3 \leq 2 \\
& & & & \mu_4 & - \pi_4 \leq 3 \\
& & & & & \mu_5 & - \pi_5 \leq 2 \\
& -\mu_1 + \mu_2 & & & & & \leq 1 \\
& & 110\pi_1 & & & & \leq 50 \\
& -\mu_2 + \mu_3 & & & & & \leq 1 \\
& & 90\pi_2 & & & & \leq 50 \\
& -\mu_3 + \mu_4 & & & & & \leq 1 \\
& & 80\pi_3 & & & & \leq 50 \\
& -\mu_4 + \mu_5 & & & & & \leq 1 \\
& & 50\pi_4 & & & & \leq 50 \\
& & 10\pi_5 & & & & \leq 50 \\
& \mu_1, \mu_2, \mu_3, \mu_4, \mu_5 & & & & & \geq 0 \\
& & \pi_1, \pi_2, \pi_3, \pi_4, \pi_5 & & & & \geq 0
\end{array}$$

Due cose da notare in questo duale:

- i vincoli sulle colonne delle scorte, $-\mu_t + \mu_{t+1} \leq 1$, dicono che il prezzo di una unità non può crescere di più del costo di magazzino da un periodo al successivo;
- il vincolo $10\pi_5 \leq 50$ è molto più lasco degli altri, perché il big-M dell'ultimo periodo vale 10 e non 110: un big-M piccolo nel primale lascia più spazio alla corrispondente variabile duale, ed è un'altra faccia dello stesso fatto per cui i big-M piccoli danno rilassamenti più stretti.

Euristiche costruttive: il bound primale

Due regole classiche, entrambe ammissibili per costruzione.

(a) **Lot-for-lot.** Si produce ogni giorno esattamente la domanda del giorno: nessuna scorta, ma un lancio ogni giorno. Il costo è

$$\sum_{t=1}^5 p_t d_t + \sum_{t=1}^5 q_t = 270 + 250 = 520.$$

(b) **Least unit cost.** Si parte dal primo periodo scoperto e si sceglie di coprire con un solo lancio i k periodi che minimizzano il costo medio per unità, cioè

$$\frac{q + \sum_{i=1}^{k-1} h \cdot (\text{unità tenute } i \text{ periodi})}{\sum_{i=0}^{k-1} d_{t+i}},$$

poi si ricomincia dal primo periodo ancora scoperto.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Il tempo di esecuzione è $O(n^2)$. Sull'istanza:

- periodo 1: si copre fino al periodo 2, quantità 30, costo medio 2 per unità;
- periodo 3: si copre fino al periodo 4, quantità 70, costo medio $\approx 1,286$;
- periodo 5: si copre il solo periodo 5, quantità 10, costo medio 5.

Si lancia nei giorni 1, 3, 5; il costo totale è 420. Tenendo la migliore delle due euristiche,

$$z(MILP) \leq UB = 420.$$

Rilassamento LP e duale: il bound duale

Associamo una variabile duale libera μ_t a ciascun vincolo di bilancio (9.1b)–(9.1d) (uguaglianze) e una variabile duale non negativa π_t a ciascun vincolo (9.1e) (verso $\geq$). Con $b_1 = d_1 - r_0$, $b_t = d_t$ per $2 \leq t \leq n-1$ e $b_n = d_n + r_n$, il duale del rilassamento LP è:

Duale del rilassamento LP del modello 9.1

$$\max \sum_{t=1}^n b_t \mu_t \quad (9.2a)$$

$$\text{soggetto a } \mu_t - \pi_t \leq p_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2b)$$

$$M_t \pi_t \leq q_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2c)$$

$$-\mu_t + \mu_{t+1} \leq h_t, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.2d)$$

$$\mu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2e)$$

$$\pi_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.2f)$$

Descrizione. La variabile μ_t è il valore che si attribuisce a una unità disponibile nel periodo t , e π_t il prezzo di una unità di capacità di lancio. L'obiettivo (9.2a) valuta a quei prezzi la domanda di ciascun periodo. I vincoli (9.2b), uno per periodo, sono le colonne delle x_t : produrre una unità nel periodo t vale μ_t e consuma capacità di lancio al prezzo π_t , e il saldo non può superare il costo unitario p_t . I vincoli (9.2c) sono le colonne delle y_t : tutta la capacità M_t che un lancio mette a disposizione, valutata a π_t , non può valere più del costo fisso q_t . I vincoli (9.2d) sono le colonne delle s_t : portare una unità dal periodo t al $t+1$ fa guadagnare $\mu_{t+1} - \mu_t$, che non può superare il costo di magazzino h_t . Le μ_t sono libere perché i bilanci sono uguaglianze; le π_t sono non negative perché i vincoli di lancio hanno verso $\geq$ in un problema di minimo.

Una soluzione duale a mano. Poniamo $\bar{\pi}_t = 0$ per ogni t : i vincoli (9.2c) sono soddisfatti perché $q_t \geq 0$, e i lanci diventano gratuiti. Restano $\mu_t \leq p_t$ e $\mu_{t+1} \leq \mu_t + h_t$: il valore più grande ammissibile si costruisce in avanti,

$$\bar{\mu}_1 = p_1, \quad \bar{\mu}_t = \min(\bar{\mu}_{t-1} + h_{t-1}, p_t) \quad \text{per } t \geq 2.$$

La lettura è diretta: $\bar{\mu}_t$ è il costo unitario più basso per avere una unità disponibile nel periodo t , o producendola allora, oppure producendola prima e tenendola in magazzino. Sull'istanza

$h = 1$ è basso e i costi di produzione sono 2, 3, 2, 3, 2, quindi

$$\bar{\mu} = (2, 3, 2, 3, 2),$$

e il valore duale è

$$LB = \sum_{t=1}^5 d_t \bar{\mu}_t = 2 \cdot 20 + 3 \cdot 10 + 2 \cdot 30 + 3 \cdot 40 + 2 \cdot 10 = 270.$$

È il costo di produzione se i lanci fossero gratuiti: un bound valido, e volutamente ottimista. Il rilassamento LP, che i lanci li paga in frazione ($\pi_t = q_t/M_t$ è ammissibile), arriva più in alto:

$$z(LP) = z(LP^+) = \frac{3890}{11} \approx 353,64.$$

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci.

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
lancio y_t	1	0	1	0	0
produzione x_t	30	0	80	0	0
scorta s_t	10	0	50	10	—

Il costo è $z(MILP) = 390$, e la catena dei bound è

$$270 = LB \leq z(LP) = \frac{3890}{11} \leq z(MILP) = 390 \leq UB = 420.$$

Il gap dell'euristica è $(420 - 390)/390 = 7,7\%$; il gap certificato, prima di risolvere il MILP, è $(420 - 270)/390 = 38,5\%$.

Considerazioni aggiuntive

- Il modello è un caso particolare del problema risolto in tempo $O(n^2)$ dalla programmazione dinamica di Wagner–Whitin. La least unit cost *non* è quell'algoritmo: è una regola miope che guarda un lancio per volta, e in generale non trova l'ottimo. Sull'istanza infatti si ferma a 420 contro 390.
- La disuguaglianza valida $x_t \leq M_t$, ottenuta dal vincolo (9.1e) con $y_t \leq 1$, non aggiunge nulla: M_t è già la massima produzione utile. Infatti $z(LP) = z(LP^+)$.
- Una formulazione alternativa, molto più stretta, usa le variabili $x_{t\tau}$ = «unità prodotte in t e vendute in τ » (formulazione *facility location* del lot sizing). Il suo rilassamento LP è intero, ma ha $O(n^2)$ variabili: è il tipico scambio fra dimensione del modello e qualità del rilassamento.

Domande di modellazione aggiuntive

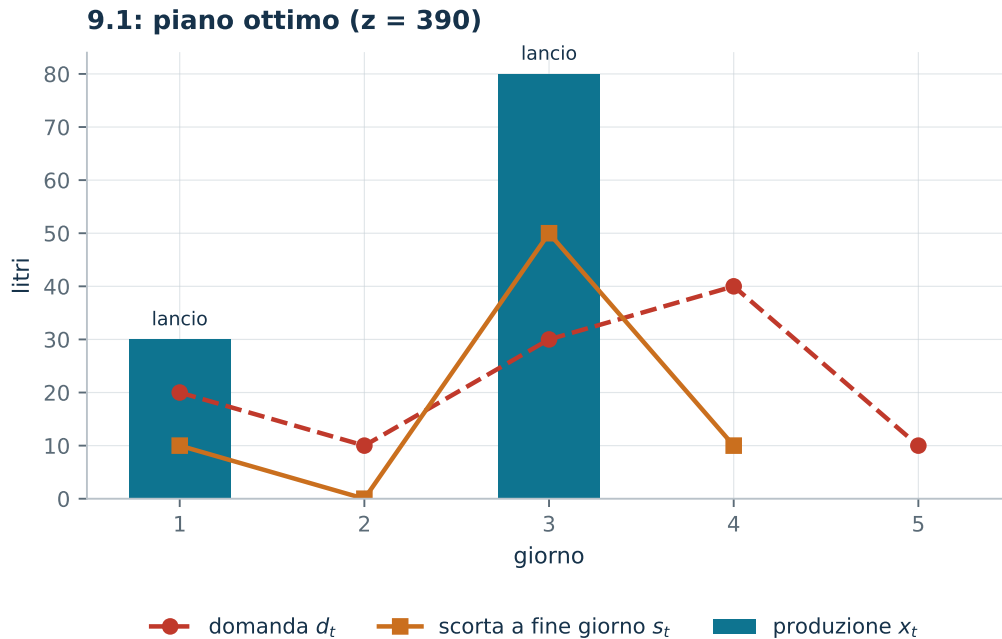


Figura 9.1: Piano ottimo: produzione, domanda e scorta a fine giorno. I due lanci coprono, il primo, i giorni 1–2 e, il secondo, i giorni 3–5.

Domanda di modellazione 9.1 — capacità giornaliera

L'impianto non può produrre più di 35 unità al giorno. Come cambia il modello? Qual è il nuovo ottimo per l'istanza?

Domanda di modellazione 9.2 — lotto minimo

Se in un giorno si produce, si devono produrre almeno 25 unità (sotto quella soglia l'attrezzaggio non è giustificato). Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	420	soluzione euristica (UB)
LB	270	bound certificato (LB)
$z(MILP)$	390	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 7.7\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 38.5\%.$$

9.2 Produzione e manodopera: due formulazioni equivalenti

Script: `python/fam09_2_manodopera.py`

Notebook in Colab

Problema 9.2 — produzione e manodopera

Un calzaturificio deve pianificare la produzione su $n \in \mathbb{Z}_{\geq 1}$ mesi. Per ogni mese $t \in \{1, 2, \dots, n\}$, il valore $d_t \in \mathbb{Z}_{\geq 0}$ è la domanda in paia e $p_t \in \mathbb{Q}_{>0}$ il costo delle materie prime per un paio. Per ogni mese $t \in \{1, 2, \dots, n-1\}$, il valore $h_t \in \mathbb{Q}_{\geq 0}$ è il costo di tenere un paio in magazzino a fine mese. All'inizio dell'orizzonte l'azienda ha $m_0 \in \mathbb{Z}_{\geq 0}$ operai; ogni operaio lavora $r \in \mathbb{Q}_{>0}$ ore al mese e costa $w \in \mathbb{Q}_{>0}$ euro al mese, e produrre un paio richiede $g \in \mathbb{Q}_{>0}$ ore di lavoro. All'inizio di ogni mese si possono assumere operai, al costo di $u \in \mathbb{Q}_{\geq 0}$ euro ciascuno; non si licenzia nessuno. L'azienda vuole decidere quanto produrre e quanti operai avere in ogni mese, al costo totale minimo.

Il problema a parole. Decidiamo quanto produrre, quanto tenere in magazzino e quanti operai assumere. *L'obiettivo:* costo totale (materie prime, magazzino, salari e assunzioni) minimo. *I vincoli:* la domanda va soddisfatta esattamente; la produzione di un mese non può richiedere più ore di quelle che gli operai in servizio riescono a fare.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di mesi, $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Z}_{\geq 0}$	domanda (paia) del mese t
p_t	$\in \mathbb{Q}_{>0}$	costo delle materie prime per un paio
h_t	$\in \mathbb{Q}_{\geq 0}$	costo di magazzino a fine mese t
m_0	$\in \mathbb{Z}_{\geq 0}$	operai in servizio all'inizio
w	$\in \mathbb{Q}_{>0}$	salario mensile di un operaio
u	$\in \mathbb{Q}_{\geq 0}$	costo di assunzione di un operaio
r	$\in \mathbb{Q}_{>0}$	ore lavorate da un operaio in un mese
g	$\in \mathbb{Q}_{>0}$	ore di lavoro per un paio

Due formulazioni

La stessa decisione si può scrivere in due modi, e vale la pena metterli uno accanto all'altro perché è il primo caso del corso in cui due modelli apparentemente diversi descrivono lo stesso insieme di piani.

Formulazione A: le assunzioni. Le variabili di personale sono

$$z_t = \text{operai assunti all'inizio del mese } t, \quad \forall t \in \{1, 2, \dots, n\},$$

interi e non negative. Un operaio assunto al mese t resta in servizio fino alla fine dell'orizzonte, quindi costa u una volta più w per ciascuno dei $n - t + 1$ mesi restanti. Il salario degli m_0 operai iniziali, $m_0 w n$, è un termine costante che non dipende dalle decisioni: lo si lascia fuori dal modello e lo si somma al valore finale.

Formulazione B: l'organico. Le variabili di personale sono

$$y_t = \text{operai in servizio nel mese } t, \quad \forall t \in \{1, 2, \dots, n\},$$

interi e non negative, con $y_t \geq y_{t-1}$ perché non si licenzia. Il costo è $w y_t$ ogni mese, più u per ogni assunzione, cioè $u(y_n - m_0)$ in totale, perché le assunzioni sono gli incrementi dell'organico e la somma telescopica lascia solo gli estremi.

Modello 9.2A — con le assunzioni

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^{n-1} h_t s_t + \sum_{t=1}^n (u + w(n-t+1)) z_t \quad (9.3a)$$

$$\text{soggetto a } x_1 - s_1 = d_1, \quad (9.3b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, \dots, n-1\}, \quad (9.3c)$$

$$x_n + s_{n-1} = d_n, \quad (9.3d)$$

$$-g x_t + r \sum_{j=1}^t z_j \geq -r m_0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.3e)$$

$$x_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.3f)$$

$$s_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.3g)$$

$$z_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.3h)$$

Modello 9.2B — con l'organico

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^{n-1} h_t s_t + \sum_{t=1}^n w y_t + u (y_n - m_0) \quad (9.4a)$$

$$\text{soggetto a } x_1 - s_1 = d_1, \quad (9.4b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, \dots, n-1\}, \quad (9.4c)$$

$$x_n + s_{n-1} = d_n, \quad (9.4d)$$

$$-g x_t + r y_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.4e)$$

$$y_1 \geq m_0, \quad (9.4f)$$

$$-y_{t-1} + y_t \geq 0, \quad \forall t \in \{2, \dots, n\}, \quad (9.4g)$$

$$x_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.4h)$$

$$s_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.4i)$$

$$y_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.4j)$$

Descrizione. Le due formulazioni condividono i vincoli di bilancio delle scorte, (9.3b)–(9.3d) e (9.4b)–(9.4d): uno per mese, dicono che quanto si produce più quanto si ha in magazzino copre esattamente la domanda. I vincoli (9.3e) e (9.4e), uno per mese, sono le ore disponibili: la produzione del mese non può richiedere più ore di quante ne facciano gli operai in servizio. Nella formulazione *B* ci sono in più il vincolo (9.4f), che parte dagli m_0 operai iniziali, e i vincoli (9.4g), uno per mese a partire dal secondo, che vietano i licenziamenti. Nella *A* quegli stessi fatti sono nascosti nel dominio delle z_t , che sono non negative.

Le due formulazioni descrivono lo stesso problema

La corrispondenza è

$$y_t = m_0 + \sum_{j=1}^t z_j, \quad \text{cioè} \quad z_t = y_t - y_{t-1} \quad (\text{con } y_0 = m_0).$$

Ammissibilità. Con questa sostituzione il vincolo (9.3e) diventa esattamente (9.4e). Le condizioni $z_t \geq 0$ diventano $y_t \geq y_{t-1}$, cioè (9.4g), e $y_1 \geq m_0$ cioè (9.4f). I vincoli di bilancio non contengono variabili di personale e restano identici. Quindi la corrispondenza è una biiezione fra i piani ammissibili dei due modelli.

Costo. Basta calcolare il costo del personale nei due modi. In *B* è

$$\sum_{t=1}^n w y_t + u (y_n - m_0) = \sum_{t=1}^n w \left(m_0 + \sum_{j \leq t} z_j \right) + u \sum_{t=1}^n z_t.$$

Il primo termine vale $m_0 w n + w \sum_{t=1}^n (n - t + 1) z_t$, perché z_j compare in tutti i mesi da j in poi, cioè $n - j + 1$ volte. Sommando il secondo si ottiene

$$m_0 w n + \sum_{t=1}^n (u + w (n - t + 1)) z_t,$$

cioè la costante m_0 *wn* più esattamente il terzo addendo di (9.3a). Le due funzioni obiettivo differiscono dunque per la sola costante, e i due modelli hanno gli stessi ottimi.

Perché tenerle entrambe

La formulazione *A* ha meno vincoli (niente monotonia) ma coefficienti di costo che dipendono dal periodo; la *B* ha coefficienti uniformi e si estende meglio se si aggiungono i licenziamenti (basta una seconda famiglia di variabili $\ell_t \geq 0$ e il bilancio $y_t = y_{t-1} + z_t - \ell_t$). Sull'istanza anche i *rilassamenti* coincidono: $z(LP) = 15\,960$ per entrambe. Due formulazioni equivalenti sull'intero non lo sono sempre sul rilassamento; qui lo sono, e la verifica va fatta, non data per scontata.

Il modello in gurobipy

```

1 m = gp.Model("manodopera_B")
2 x = m.addVars(n, vtype=GRB.INTEGER, name="x")
3 s = m.addVars(n - 1, vtype=GRB.INTEGER, name="s")
4 y = m.addVars(n, vtype=GRB.INTEGER, name="y")
5 m.setObjective(gp.quicksum(p[t] * x[t] for t in range(n))
6                 + gp.quicksum(h[t] * s[t] for t in range(n - 1))
7                 + gp.quicksum(w * y[t] for t in range(n)) + u * (y[n -
8                     1] - m0), GRB.MINIMIZE)
9 m.addConstr(x[0] - s[0] == d[0], name="bilancio[0]")
10 m.addConstrs((x[t] + s[t - 1] - s[t] == d[t] for t in range(1, n - 1)),
11              name="bilancio")
12 m.addConstr(x[n - 1] + s[n - 2] == d[n - 1], name=f"bilancio[{n - 1}]")
13 m.addConstrs((-g * x[t] + r * y[t] >= 0 for t in range(n)), name="ore")
14 m.addConstr(y[0] >= m0, name="organico_iniziale")
15 m.addConstrs((-y[t - 1] + y[t] >= 0 for t in range(1, n)), name="
    mai_licenziamenti")

```

L'istanza

$n = 3$ mesi, $m_0 = 2$ operai, $w = 1500$, $u = 100$, $r = 160$ ore, $g = 4$ ore per paio, $h_t = 3$:

	$t = 1$	$t = 2$	$t = 3$
d_t	60	100	140
p_t	15	15	15

Con due operai la capacità iniziale è $2 \cdot 160/4 = 80$ paia al mese: basta per il primo mese, non per gli altri due.

Scritta per esteso sui dati dell'istanza, la formulazione *B* è la seguente. Le tre variabili x_t sono le paia prodotte, le due s_t le scorte, le tre y_t l'organico.

Modello 9.2B sull'istanza

$$\begin{array}{rcl}
\min & 15x_1 + 15x_2 + 15x_3 + 3s_1 + 3s_2 + 1500y_1 + 1500y_2 + 1600y_3 - 200 & \\
& x_1 & - s_1 = 60 \\
& & x_2 + s_1 - s_2 = 100 \\
& & x_3 + s_2 = 140 \\
& -4x_1 & + 160y_1 \geq 0 \\
& & y_1 \geq 2 \\
& -4x_2 & + 160y_2 \geq 0 \\
& & -y_1 + y_2 \geq 0 \\
& & -4x_3 + 160y_3 \geq 0 \\
& & -y_2 + y_3 \geq 0 \\
& x_1, & x_2, & x_3 \in \mathbb{Z}_{\geq 0} \\
& & s_1, & s_2 \in \mathbb{Z}_{\geq 0} \\
& & y_1, & y_2, & y_3 \in \mathbb{Z}_{\geq 0}
\end{array}$$

Il duale del rilassamento LP della formulazione A , sempre sui dati dell'istanza, ha tre variabili libere μ_t (una per bilancio) e tre variabili non negative ν_t (una per vincolo di ore).

Duale della formulazione 9.2A sull'istanza

$$\begin{array}{rcl}
\max & 60\mu_1 + 100\mu_2 + 140\mu_3 - 320\nu_1 - 320\nu_2 - 320\nu_3 & \\
& \mu_1 & - 4\nu_1 \leq 15 \\
& & 160\nu_1 + 160\nu_2 + 160\nu_3 \leq 4600 \\
& & \mu_2 - 4\nu_2 \leq 15 \\
& & 160\nu_2 + 160\nu_3 \leq 3100 \\
& & \mu_3 - 4\nu_3 \leq 15 \\
& & 160\nu_3 \leq 1600 \\
& -\mu_1 + \mu_2 & \leq 3 \\
& -\mu_2 + \mu_3 & \leq 3 \\
& & \mu_t \geq 0 \\
& & \nu_t \geq 0
\end{array}$$

I tre membri destri della colonna di destra sono i costi delle assunzioni nei tre mesi: $100 + 1500 \cdot 3 = 4600$ per un operaio assunto al primo mese, $100 + 1500 \cdot 2 = 3100$ al secondo, $100 + 1500 = 1600$ al terzo. Il coefficiente $320 = 160 \cdot 2$ dell'obiettivo duale sono le ore già disponibili con i due operai iniziali.

Euristica costruttiva: il bound primale

Produzione «just in time»: ogni mese si produce esattamente la domanda, e si assume il numero minimo di operai che serve.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza: mese 1, $\lceil 4 \cdot 60 / 160 \rceil = 2$ operai, nessuna assunzione; mese 2, $\lceil 4 \cdot 100 / 160 \rceil = 3$, una assunzione; mese 3, $\lceil 4 \cdot 140 / 160 \rceil = 4$, un'altra assunzione. Il costo, incluso il termine costante

$m_0 w n = 9000$, è

$$z(MILP) \leq UB = 18\,200.$$

Rilassamento LP e duale: il bound duale

Sulla formulazione A, associamo una variabile duale libera μ_t a ciascun bilancio e una non negativa ν_t a ciascun vincolo (9.3e). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 9.2A

$$\max \sum_{t=1}^n d_t \mu_t - r m_0 \sum_{t=1}^n \nu_t \quad (9.5a)$$

$$\text{soggetto a } \mu_t - g \nu_t \leq p_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.5b)$$

$$-\mu_t + \mu_{t+1} \leq h_t, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.5c)$$

$$r \sum_{t=j}^n \nu_t \leq u + w(n-j+1), \quad \forall j \in \{1, 2, \dots, n\}, \quad (9.5d)$$

$$\mu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.5e)$$

$$\nu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.5f)$$

Descrizione. La variabile μ_t è il valore di un paio disponibile nel mese t e ν_t il prezzo di un'ora di lavoro nel mese t . L'obiettivo (9.5a) valuta la domanda a quei prezzi e sottrae $r m_0 \sum_t \nu_t$, cioè le ore che i m_0 operai iniziali offrono gratuitamente. I vincoli (9.5b) sono le colonne delle x_t : produrre un paio nel mese t vale μ_t e consuma g ore al prezzo ν_t , e il saldo non può superare il costo p_t delle materie prime. I vincoli (9.5c) sono le colonne delle s_t : tenere un paio in magazzino fa guadagnare $\mu_{t+1} - \mu_t$, che non può superare h_t . I vincoli (9.5d) sono le colonne delle z_j : un operaio assunto all'inizio del mese j mette a disposizione r ore in ciascuno dei mesi da j in poi, e il loro valore complessivo non può superare quanto quell'assunzione costa, cioè $u + w(n-j+1)$.

Una soluzione duale a mano. Poniamo $\bar{\nu}_t = 0$: le ore di lavoro non si valutano, i vincoli (9.5d) sono soddisfatti perché il membro destro è positivo, e restano $\mu_t \leq p_t$ e $\mu_{t+1} \leq \mu_t + h_t$. Come nel problema 9.1, il valore più grande ammissibile è

$$\bar{\mu}_1 = p_1, \quad \bar{\mu}_t = \min(\bar{\mu}_{t-1} + h_{t-1}, p_t).$$

Sull'istanza $\bar{\mu} = (15, 15, 15)$ e

$$\sum_{t=1}^n d_t \bar{\mu}_t = 15 \cdot 300 = 4500,$$

a cui va sommato il termine costante 9000:

$$LB = 4500 + 9000 = 13\,500.$$

È il costo delle sole materie prime più il salario degli operai già in servizio: la manodopera aggiuntiva è regalata. Il rilassamento LP, che la paga in frazione di persona, arriva a $z(LP) = z(LP^+) = 15\,960$.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci.

	$t = 1$	$t = 2$	$t = 3$
produzione x_t	60	120	120
organico y_t	2	3	3
assunzioni z_t	0	1	0
scorta s_t	0	20	—

Il costo è $z(MILP) = 16\,660$, con la catena

$$13\,500 = LB \leq z(LP) = 15\,960 \leq z(MILP) = 16\,660 \leq UB = 18\,200.$$

Il gap dell'euristica è 9,2%. Il piano ottimo assume *un* operaio invece di due e anticipa venti paia dal terzo mese al secondo: pagare 3 euro di magazzino per venti paia costa 60 euro contro i 1600 di un'assunzione al terzo mese.

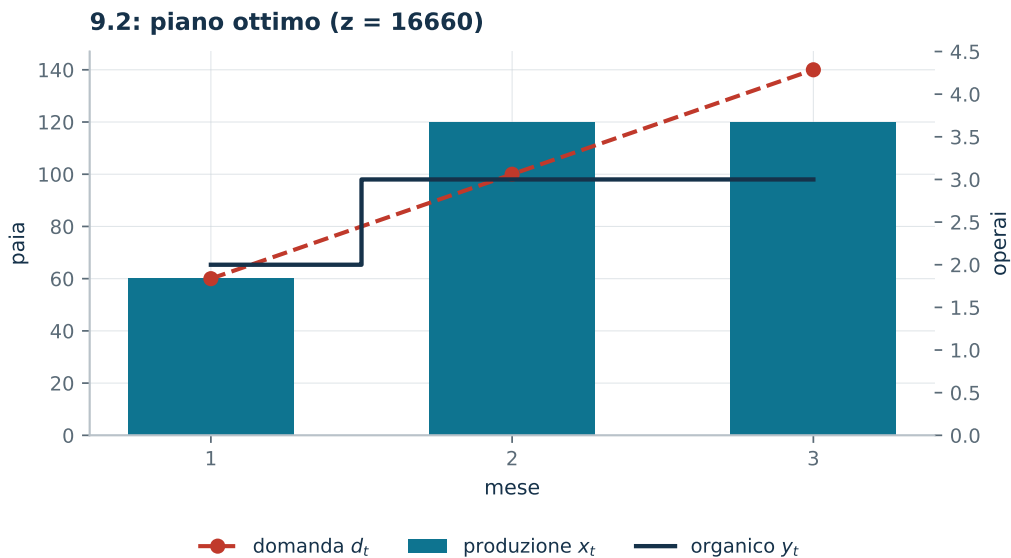


Figura 9.2: Piano ottimo: produzione, domanda e organico. L'organico sale una volta sola, e la produzione anticipata copre il picco finale.

Considerazioni aggiuntive

- Il vincolo di monotonia (9.4g) è ciò che rende il problema non banale: se si potesse licenziare a costo zero, la formulazione B si spezzerebbe in n problemi indipendenti, uno per mese.
- Le variabili x_t e s_t sono dichiarate intere perché le paia di scarpe non si spezzano. In questo modello si potrebbero lasciare continue senza cambiare l'ottimo (i dati sono interi e la matrice dei bilanci è totalmente unimodulare), ma la dichiarazione è comunque quella corretta dal punto di vista del modello.
- Il termine costante $m_0 w n$ va ricordato in ogni confronto: dimenticarlo fa apparire la formulazione A molto più economica della B .

Domande di modellazione aggiuntive

Domanda di modellazione 9.3 — assunzioni molto costose

Il costo di assunzione sale da 100 a 3000 euro (selezione e formazione). Come cambia il piano ottimo?

Domanda di modellazione 9.4 — straordinari

Ogni operaio può fare fino a 40 ore di straordinario al mese, pagate 25 euro l'ora. Come cambia il modello? Conviene usarli?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	18200	soluzione euristica (UB)
LB	13500	bound certificato (LB)
$z(MILP)$	16660	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.2\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 28.2\%.$$

9.3 Veicoli: lotto minimo e premio per la varietà

Script: `python/fam09_3_veicoli.py`

[Notebook in Colab](#)

Problema 9.3 — veicoli con lotto minimo e premio

Un'azienda produce $s \in \mathbb{Z}_{\geq 1}$ tipi di veicolo usando $k \in \mathbb{Z}_{\geq 1}$ risorse. Per ogni risorsa $i \in \{1, 2, \dots, k\}$ e ogni tipo $j \in \{1, \dots, s\}$, il valore $a_{ij} \in \mathbb{Q}_{\geq 0}$ è la quantità di risorsa i necessaria per una unità del tipo j , e $b_i \in \mathbb{Q}_{>0}$ è la disponibilità della risorsa i . Per ogni tipo j , il valore $\bar{p}_j \in \mathbb{Q}_{>0}$ è il profitto di una unità e $\bar{q}_j \in \mathbb{Z}_{\geq 1}$ la quantità minima da produrre se si decide di produrre quel tipo. Se la produzione comprende almeno due tipi diversi, l'azienda incassa un premio $\bar{r} \in \mathbb{Q}_{>0}$ (un contributo per la diversificazione). L'azienda vuole massimizzare il profitto totale.

Il problema a parole. Decidiamo quante unità produrre di ciascun tipo. *L'obiettivo:* profitto massimo, premio compreso. *I vincoli:* le risorse non si superano; e di un tipo si producono zero unità oppure almeno $\bar{q}_j$.

Dati (input del modello)

Simbolo	Tipo	Significato
s	$\in \mathbb{Z}_{\geq 1}$	numero di tipi, $j \in \{1, 2, \dots, s\}$
k	$\in \mathbb{Z}_{\geq 1}$	numero di risorse, $i \in \{1, 2, \dots, k\}$
a_{ij}	$\in \mathbb{Q}_{>0}$	risorsa i per una unità del tipo j
b_i	$\in \mathbb{Q}_{>0}$	disponibilità della risorsa i
$\bar{p}_j$	$\in \mathbb{Q}_{>0}$	profitto unitario del tipo j
$\bar{q}_j$	$\in \mathbb{Z}_{\geq 1}$	lotto minimo del tipo j
$\bar{r}$	$\in \mathbb{Q}_{>0}$	premio per la varietà
M_j	$\in \mathbb{Z}_{\geq 1}$	$\min_i \lfloor b_i / a_{ij} \rfloor$: massimo producibile del solo tipo j

Variabili decisionali

Introduciamo s variabili intere non negative, s binarie e una binaria:

$$x_j = \text{unità prodotte del tipo } j, \quad \forall j \in \{1, 2, \dots, s\},$$

$$y_j = \begin{cases} 1 & \text{se il tipo } j \text{ viene prodotto,} \\ 0 & \text{altrimenti,} \end{cases} \quad z = \begin{cases} 1 & \text{se si incassa il premio,} \\ 0 & \text{altrimenti.} \end{cases}$$

Modello 9.3 — veicoli con lotto minimo e premio

$$\max \sum_{j=1}^s \bar{p}_j x_j + \bar{r} z \quad (9.6a)$$

$$\text{soggetto a } \sum_{j=1}^s a_{ij} x_j \leq b_i, \quad \forall i \in \{1, 2, \dots, k\}, \quad (9.6b)$$

$$x_j - \bar{q}_j y_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6c)$$

$$x_j - M_j y_j \leq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6d)$$

$$-\sum_{j=1}^s y_j + 2z \leq 0, \quad (9.6e)$$

$$x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6f)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6g)$$

$$z \in \{0, 1\}. \quad (9.6h)$$

Descrizione. L'obiettivo (9.6a) somma i profitti dei veicoli prodotti e il premio per la varietà. I vincoli (9.6b), uno per risorsa, sono le disponibilità. I vincoli (9.6c) e (9.6d), uno per tipo ciascuno, rendono x_j semicontinua: o zero, o almeno il lotto minimo $\bar{q}_j$ e al più M_j . Il vincolo (9.6e), uno solo, dice che il premio si incassa solo se i tipi attivi sono almeno due.

Il legame fra le variabili: la semicontinuità. I due vincoli (9.6c) e (9.6d) insieme dicono

$$\bar{q}_j y_j \leq x_j \leq M_j y_j.$$

Se $y_j = 0$ entrambi danno $x_j = 0$. Se $y_j = 1$ si ottiene $\bar{q}_j \leq x_j \leq M_j$. La variabile x_j vale dunque zero oppure sta fra il lotto minimo e il massimo: è una variabile *semicontinua*. Nessuno dei due vincoli da solo basta: senza (9.6d) un tipo potrebbe essere prodotto con $y_j = 0$; senza (9.6c) il lotto minimo sarebbe vuoto.

Il premio si incassa solo con almeno due tipi

Il vincolo (9.6e) si legge $2z \leq \sum_{j=1}^s y_j$. Se $z = 1$ allora $\sum_{j=1}^s y_j \geq 2$: almeno due tipi sono attivati, e per la semicontinuità almeno due tipi sono effettivamente prodotti (in quantità $\geq \bar{q}_j > 0$). Il verso opposto — se ci sono due tipi attivi allora $z = 1$ — non è imposto dal vincolo, ma segue dall'ottimalità: se $\sum_{j=1}^s y_j \geq 2$ e $z = 0$, porre $z = 1$ resta ammissibile e aumenta l'obiettivo di $\bar{r} > 0$. Quindi in ogni soluzione ottima z è un indicatore fedele. La domanda 3b qui sotto mostra che con $\bar{r} = 0$ questo argomento cade, e z smette di essere fedele.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```
1 mm = gp.Model("veicoli")
2 x = mm.addVars(n, vtype=GRB.INTEGER, name="x")
3 y = mm.addVars(n, vtype=GRB.BINARY, name="y")
4 z = mm.addVar(vtype=GRB.BINARY, name="z")
5 mm.setObjective(gp.quicksum(p[j] * x[j] for j in range(n)) + r * z, GRB
    .MAXIMIZE)
6 mm.addConstrs((gp.quicksum(a[i][j] * x[j] for j in range(n)) <= b[i]
    for i in range(m)), name="risorsa")
7 mm.addConstrs((x[j] - q[j] * y[j] >= 0 for j in range(n)), name="
    lotto_minimo")
8 mm.addConstrs((x[j] - M[j] * y[j] <= 0 for j in range(n)), name="attiva
    ")
9 mm.addConstr(-gp.quicksum(y[j] for j in range(n)) + 2 * z <= 0, name="
    premio")
```

L'istanza

$s = 3$ tipi, $k = 2$ risorse (acciaio in tonnellate e ore di manodopera), $\bar{q}_j = 10$ per ogni tipo, $\bar{r} = 500$:

a_{ij}	$j = 1$	$j = 2$	$j = 3$		$i = 1$	$i = 2$
$i = 1$ (acciaio)	2	3	5	b_i	100	1200
$i = 2$ (ore)	30	25	40			

	$j = 1$	$j = 2$	$j = 3$
$\bar{p}_j$	200	250	300
M_j	40	33	20

I big-M sono calcolati dai dati: $M_1 = \min(\lfloor 100/2 \rfloor, \lfloor 1200/30 \rfloor) = 40$, e analogamente $M_2 = 33$, $M_3 = 20$.

Euristica costruttiva: il bound primale

Il problema è di massimo, quindi l'euristica dà il bound *primale*, che sta sotto l'ottimo. Si ordinano i tipi per profitto rapportato al consumo della risorsa più stretta, si attivano i due migliori al lotto minimo (così il premio è garantito) e poi si riempie con il più redditizio fra quelli attivati.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza si attivano i tipi 2 e 1 al lotto minimo, poi si riempie: la produzione è (11, 26, 0), l'acciaio si esaurisce e restano 220 ore. Il profitto è 8700 più il premio di 500:

$$z(MILP) \geq LB = 9200.$$

Rilassamento LP e duale: il bound duale

Il primale è di massimo con vincoli $\leq$ e $\geq$: seguendo la tabella di conversione del capitolo 4, associamo $\pi_i \geq 0$ ai vincoli (9.6b), $\lambda_j \leq 0$ a (9.6c), $\beta_j \geq 0$ a (9.6d) e $\gamma \geq 0$ a (9.6e). Scrivendo $\lambda_j = -\ell_j$ con $\ell_j \geq 0$, il duale è:

Duale del rilassamento LP del modello 9.3

$$\min \sum_{i=1}^k b_i \pi_i \quad (9.7a)$$

$$\text{soggetto a } \sum_{i=1}^k a_{ij} \pi_i - \ell_j + \beta_j \geq \bar{p}_j, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7b)$$

$$\bar{q}_j \ell_j - M_j \beta_j - \gamma \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7c)$$

$$2\gamma \geq \bar{r}, \quad (9.7d)$$

$$\pi_i \geq 0, \quad \forall i \in \{1, 2, \dots, k\}, \quad (9.7e)$$

$$\ell_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7f)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7g)$$

$$\gamma \geq 0. \quad (9.7h)$$

Descrizione. La variabile π_i è il prezzo di una unità della risorsa i ; ℓ_j e β_j sono i prezzi dei due vincoli di semicontinuità del tipo j , e γ quello del premio. L'obiettivo (9.7a) valuta a quei prezzi tutte le risorse disponibili. I vincoli (9.7b) sono le colonne delle x_j : le risorse che una unità del tipo j consuma, valutate ai prezzi π_i e corrette dai due vincoli di semicontinuità, devono coprire il profitto $\bar{p}_j$ di quella unità. I vincoli (9.7c) sono le colonne delle y_j : accendere il tipo j obbliga a produrne almeno $\bar{q}_j$ e ne concede al più M_j , e il saldo dei due prezzi deve coprire il premio γ . Il vincolo (9.7d) è la colonna di z : il premio $\bar{r}$ si incassa solo con due tipi attivi, e infatti va coperto da 2γ .

Una soluzione duale a mano. Per esempio si può procedere in tre passi.

1. Il vincolo (9.7d) obbliga $\gamma \geq \bar{r}/2$: si prende il minimo, $\bar{\gamma} = \bar{r}/2 = 250$.

2. Si pone $\bar{\beta}_j = 0$ e si ricava dal vincolo (9.7c) il più piccolo ℓ_j ammissibile,

$$\bar{\ell}_j = \frac{\bar{\gamma}}{\bar{q}_j} = \frac{250}{10} = 25 \quad \text{per ogni } j.$$

La lettura è che ogni tipo attivato «porta con sé» la sua quota di premio.

3. Resta da soddisfare (9.7b), che diventa $\sum_{i=1}^k a_{ij} \pi_i \geq \bar{p}_j + \bar{\ell}_j$. Si valuta *una sola* risorsa, al prezzo che copre tutti i tipi:

$$\bar{\pi}_i = \max_j \frac{\bar{p}_j + \bar{\ell}_j}{a_{ij}}, \quad \text{bound} = b_i \bar{\pi}_i,$$

e si tiene la risorsa che dà il bound più piccolo.

Sull'istanza:

$$\begin{aligned} \bar{\pi}_1 &= \max\left(\frac{225}{2}, \frac{275}{3}, \frac{325}{5}\right) = \frac{225}{2}, & b_1 \bar{\pi}_1 &= 100 \cdot \frac{225}{2} = 11\,250, \\ \bar{\pi}_2 &= \max\left(\frac{225}{30}, \frac{275}{25}, \frac{325}{40}\right) = 11, & b_2 \bar{\pi}_2 &= 1200 \cdot 11 = 13\,200. \end{aligned}$$

Il bound migliore è quello dell'acciaio:

$$z(MILP) \leq UB = 11\,250.$$

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. La produzione ottima è (26, 16, 0): si attivano i tipi 1 e 2, si incassa il premio, si consumano tutte e 100 le tonnellate di acciaio e 1180 ore sulle 1200 disponibili. Il valore è

$$z(MILP) = 200 \cdot 26 + 250 \cdot 16 + 500 = 9700,$$

e la catena dei bound è

$$9200 = LB \leq z(MILP) = 9700 \leq z(LP^+) = 9750 \leq z(LP) = \frac{20625}{2} \leq UB = 11\,250.$$

Il gap dell'euristica è 5,2%.

Qui il rafforzato batte il duale a mano

È l'unico problema del capitolo in cui $z(LP^+) = 9750$ è *migliore* del bound duale costruito a mano (11 250), e di pochissimo peggiore dell'ottimo (9700). La ragione è che il rilassamento senza i bound lascia y_j e z crescere sopra 1, e con esse il premio: $z(LP) = 20625/2 \approx 10\,312$. Aggiungere $y_j \leq 1$ e $z \leq 1$ toglie proprio quella libertà. Quando il modello contiene indicatori premiati, il rafforzato non è un dettaglio.

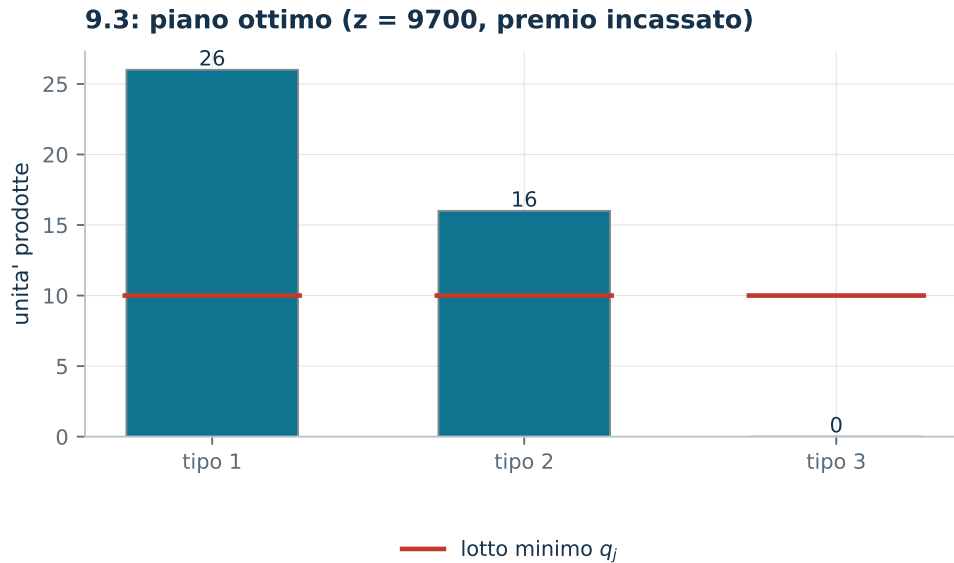


Figura 9.3: Piano ottimo: unità prodotte per tipo, con il lotto minimo $\bar{q}_j$ segnato in rosso. Il tipo 3 resta a zero.

Considerazioni aggiuntive

- Il big-M M_j è il massimo producibile del *solo* tipo j , non del piano complessivo: è già un valore molto più stretto di una costante arbitraria, e infatti $M_3 = 20$ mentre l'acciaio da solo permetterebbe 20 unità e le ore 30.
- Se per qualche tipo si avesse $\bar{q}_j > M_j$, quel tipo sarebbe impossibile da produrre e si potrebbe eliminare dal modello con un controllo sui dati. È un pre-processing che vale la pena fare sempre.
- Il premio è modellato con *una* binaria e *un* vincolo. Con una soglia diversa (almeno f tipi) basta sostituire il coefficiente 2 con f ; con più premi a scaglioni servirebbe una variabile per scaglione e una catena di vincoli, come nelle funzioni a tratti della sezione 3.14.

Domande di modellazione aggiuntive

Domanda di modellazione 9.5 — premio più esigente

Il premio si incassa solo se si producono almeno *tre* tipi diversi. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 9.6 — premio nullo

Il contributo per la diversificazione viene abolito, cioè $\bar{r} = 0$. Che cosa succede alla variabile z ?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	9200	soluzione euristica (LB)
UB	11250	bound certificato (UB)
$z(MILP)$	9700	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 5.2\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 21.1\%.$$

Modelli misti

Classe: BIP / MILP

Script: uno per problema, `python/fam10_1_premi.py` ...

[Pagina online](#)

Perché un capitolo di modelli misti

Le tre famiglie dei capitoli precedenti hanno una struttura riconoscibile: si assegna, si localizza, si pianifica. I nove problemi di questo capitolo non ce l'hanno, e non gliene va imposta una: ciascuno mette insieme pezzi diversi, e il lavoro di modellazione consiste proprio nel riconoscere quali.

- **Selezione con modalità alternative** (problemi 10.1 e 10.3): si sceglie un sottoinsieme, ma ogni oggetto ha più di un modo di essere scelto, e i modi si escludono a vicenda.
- **Conteggi con lotto minimo** (problema 10.2): le variabili non sono binarie ma quantità intere, e una quantità può stare a zero oppure sopra una soglia.
- **Copertura con contenitori** (problemi 10.4 e 10.5): un fabbisogno va coperto acquistando confezioni di composizione fissa, e la quantità in eccesso si paga o si spreca.
- **Divisione e bilanciamento** (problemi 10.6–10.9): un insieme va diviso fra più contenitori e la qualità si misura su quanto i contenitori si somigliano.

Quattro di questi problemi hanno in comune un tratto che nelle tre famiglie non si era presentato: il **rilassamento LP è debole**, e in due casi vale esattamente zero. La ragione è sempre la stessa: una soluzione frazionaria può spezzare a metà ogni oggetto e metterne una metà in ciascun contenitore, pareggiando tutto. Il bound duale utile va allora cercato altrove:

- nella **parità**, quando un conteggio non può che essere pari;
- nel **numero di contenitori** necessari, letto dalle capacità;
- nella **dominanza di una classe** di oggetti sulle altre.

Sono argomenti *combinatori*: nascono dall'inezienza, e il duale del rilassamento non può vederli.

Per ogni problema, come nelle famiglie: descrizione, variabili, modello simbolico, modello dell'istanza, una soluzione ammissibile con il suo bound primale, il rilassamento LP con il suo duale, una soluzione duale con il suo bound duale, la soluzione ottima e il confronto finale.

10.1 Premi acquistabili con due modalità

Script: `python/fam10_1_premi.py`

[Notebook in Colab](#)

Problema 10.1 — premi con due modalità

Un programma fedeltà mette a disposizione $s \in \mathbb{Z}_{\geq 1}$ premi e un cliente dispone di $p \in \mathbb{Q}_{>0}$ punti. Ogni premio $i \in \{1, 2, \dots, s\}$ si può ottenere in due modi alternativi: con i soli punti, spendendone $a_i \in \mathbb{Q}_{>0}$; oppure spendendone soltanto $b_i \in \mathbb{Q}_{>0}$ (con $b_i < a_i$) e aggiungendo un contributo in denaro di $c_i \in \mathbb{Q}_{\geq 0}$ euro. Ogni premio si può ottenere al più una volta, e in una sola delle due modalità. A ogni premio è associato un valore di preferenza $d_i \in \mathbb{Q}_{>0}$, e il cliente vuole raggiungere una preferenza complessiva di almeno $\ell \in \mathbb{Q}_{>0}$. Il cliente vuole minimizzare il denaro speso.

Il problema a parole. Decidiamo quali premi prendere e, per ciascuno, con quale delle due modalità. *L'obiettivo:* contributo in denaro minimo. *I vincoli:* i punti non bastano per tutto; la preferenza complessiva deve arrivare alla soglia; e uno stesso premio non si prende due volte.

Dati (input del modello)

Simbolo	Tipo	Significato
s	$\in \mathbb{Z}_{\geq 1}$	numero di premi, $i \in \{1, 2, \dots, s\}$
a_i	$\in \mathbb{Q}_{>0}$	punti richiesti dalla modalità a soli punti
b_i	$\in \mathbb{Q}_{>0}$	punti richiesti dalla modalità con contributo, $b_i < a_i$
c_i	$\in \mathbb{Q}_{\geq 0}$	contributo in euro della seconda modalità
d_i	$\in \mathbb{Q}_{>0}$	valore di preferenza del premio i
p	$\in \mathbb{Q}_{>0}$	punti disponibili
ℓ	$\in \mathbb{Q}_{>0}$	preferenza minima richiesta

Variabili decisionali

Servono due famiglie di binarie, una per modalità:

$$x_i = \begin{cases} 1 & \text{se il premio } i \text{ si prende con i soli punti,} \\ 0 & \text{altrimenti,} \end{cases}$$

$$y_i = \begin{cases} 1 & \text{se il premio } i \text{ si prende con punti e contributo,} \\ 0 & \text{altrimenti,} \end{cases}$$

per ogni $i \in \{1, 2, \dots, s\}$: in tutto $2s$ variabili binarie.

Modello 10.1 — premi con due modalità

$$\min \sum_{i=1}^s c_i y_i \quad (10.1a)$$

$$\text{soggetto a } x_i + y_i \leq 1, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.1b)$$

$$\sum_{i=1}^s (a_i x_i + b_i y_i) \leq p, \quad (10.1c)$$

$$\sum_{i=1}^s d_i (x_i + y_i) \geq \ell, \quad (10.1d)$$

$$x_i, y_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}. \quad (10.1e)$$

Descrizione. L'obiettivo (10.1a) conta solo il denaro: la modalità a soli punti non compare, perché non costa euro. I vincoli (10.1b) sono la mutua esclusione (s vincoli lineari); il vincolo (10.1c) è il budget in punti (uno solo); il vincolo (10.1d) è la soglia di preferenza (uno solo).

Che cosa dice e che cosa non dice la mutua esclusione

Il vincolo $x_i + y_i \leq 1$ ammette *tre* configurazioni: $(0, 0)$, $(1, 0)$ e $(0, 1)$. Vieta soltanto $(1, 1)$. In particolare:

- non dice «il premio i va preso»: la configurazione $(0, 0)$ è legittima e significa che quel premio si lascia perdere;
- non dice «se non lo prendo a punti allora lo prendo con contributo»: la conversa

$$x_i = 0 \implies y_i = 1$$

è falsa, e il controesempio è proprio $(0, 0)$;

- se si volesse imporre che ogni premio venga preso, il vincolo andrebbe scritto con l'uguaglianza, $x_i + y_i = 1$: è un set *partitioning* invece di un set *packing*, e il problema cambia (vedi la domanda 10.3 più avanti).

La quantità $x_i + y_i$ è dunque l'indicatore «il premio i è stato preso, in un modo o nell'altro», ed è esattamente questa somma che compare nel vincolo di preferenza.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```
1 m = gp.Model("premi")
2 x = m.addVars(s, vtype=GRB.BINARY, name="x")
3 y = m.addVars(s, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(c[i] * y[i] for i in range(s)), GRB.MINIMIZE)
5 m.addConstrs((x[i] + y[i] <= 1 for i in range(s)), name="una_modalita")
```

```

6 m.addConstr(gp.quicksum(a[i] * x[i] + b[i] * y[i] for i in range(s)) <=
    p, name="punti")
7 m.addConstr(gp.quicksum(d[i] * (x[i] + y[i]) for i in range(s)) >= ell,
    name="preferenza")

```

L'istanza

$s = 5$ premi, $p = 20$ punti, $\ell = 16$:

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
a_i	8	6	10	5	7
b_i	4	3	6	2	4
c_i	10	8	15	5	9
d_i	5	4	7	3	6

Euristica costruttiva: il bound primale

Si scorrono i premi per preferenza decrescente. Ciascuno si prende con i soli punti se bastano, altrimenti con il contributo se bastano i punti ridotti, altrimenti si salta; ci si ferma appena la preferenza richiesta è raggiunta.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza l'ordine per preferenza è 3, 5, 1, 2, 4.

- Premio 3 (preferenza 7): bastano i soli punti ($10 \leq 20$), si prende; restano 10 punti, preferenza 7.
- Premio 5 (preferenza 6): bastano i soli punti ($7 \leq 10$), si prende; restano 3 punti, preferenza 13.
- Premio 1 (preferenza 5): i punti non bastano per la modalità a ($8 > 3$), né per la b ($4 > 3$): si salta.
- Premio 2 (preferenza 4): i punti non bastano per la modalità a ($6 > 3$), ma bastano per la b ($3 \leq 3$): si prende con il contributo di 8 euro; la preferenza arriva a $17 \geq 16$ e ci si ferma.

Il contributo totale è

$$z(MILP) \leq UB = 8.$$

Rilassamento LP e duale: il bound duale

Associamo $\sigma_i \geq 0$ ai vincoli (10.1b), $\pi \geq 0$ al vincolo dei punti (10.1c) e $\rho \geq 0$ alla soglia di preferenza (10.1d). Il primale è di minimo, quindi i vincoli $\leq$ danno duali di segno negativo: scriviamo $-\sigma_i$ e $-\pi$ con $\sigma_i, \pi \geq 0$. Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 10.1

$$\max \quad -\sum_{i=1}^s \sigma_i - p\pi + \ell\rho \quad (10.2a)$$

$$\text{soggetto a} \quad -\sigma_i - a_i\pi + d_i\rho \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2b)$$

$$-\sigma_i - b_i\pi + d_i\rho \leq c_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2c)$$

$$\sigma_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2d)$$

$$\pi \geq 0, \quad (10.2e)$$

$$\rho \geq 0. \quad (10.2f)$$

Descrizione. La variabile π è il prezzo di un punto, ρ il valore di una unità di preferenza e σ_i il prezzo della mutua esclusione del premio i . L'obiettivo (10.2a) incassa la soglia ℓ valutata a ρ e paga il budget di punti p valutato a π e i moltiplicatori σ_i . I vincoli (10.2b) sono le colonne delle x_i : prendere il premio i con i soli punti frutta $d_i\rho$ di preferenza, consuma $a_i\pi$ di punti e σ_i di mutua esclusione, e il saldo non può superare il costo in denaro di quella modalità, che è zero. I vincoli (10.2c) dicono la stessa cosa per la seconda modalità, che consuma solo b_i punti: lì il saldo può arrivare fino a c_i , il contributo che quella modalità fa pagare.

Una soluzione duale a mano. Qui la ricetta ha *due* parametri liberi, non uno: il prezzo π di un punto e il prezzo ρ di una unità di preferenza. Fissati questi due, i moltiplicatori della mutua esclusione si ricavano dal vincolo (10.2b) prendendo il più piccolo valore ammissibile,

$$\bar{\sigma}_i = \max(0, d_i\rho - a_i\pi),$$

e restano da verificare soltanto i vincoli (10.2c) sulla seconda modalità. La funzione

$$V(\pi, \rho) = -\sum_{i=1}^s \bar{\sigma}_i(\pi, \rho) - p\pi + \ell\rho$$

è concava e lineare a tratti: si può esplorare su una griglia. Sull'istanza il massimo si trova in

$$\bar{\pi} = 2, \quad \bar{\rho} = 3, \quad \bar{\sigma} = (0, 0, 1, 0, 4),$$

con valore

$$LB = -(0 + 0 + 1 + 0 + 4) - 20 \cdot 2 + 16 \cdot 3 = -5 - 40 + 48 = 3.$$

Questa soluzione è *ottima* per il rilassamento senza i bound: infatti $z(LP) = z(LP^+) = 3$.

Qui $LB = 3$ e $z(MILP) = 5$: il gap fra il bound duale e l'ottimo intero è del 40%, e il gap certificato fra euristica e duale è del 100%. Non c'è nulla di sbagliato: il rilassamento LP può prendere «mezzo premio» a metà preferenza, e questa libertà vale molto. È il caso più estremo del corso, e serve a ricordare che un bound valido non è automaticamente un bound utile.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
soli punti x_i	0	0	1	0	1
con contributo y_i	0	0	0	1	0

Si prendono i premi 3 e 5 con i soli punti ($10 + 7 = 17$) e il premio 4 con il contributo (2 punti e 5 euro): i punti usati sono 19 su 20, la preferenza è $7 + 6 + 3 = 16$, esattamente la soglia, e il contributo è

$$z(MILP) = 5.$$

La catena dei bound è

$$3 = LB = z(LP) \leq z(MILP) = 5 \leq UB = 8,$$

con un gap dell'euristica del 60%. L'euristica sbaglia perché guarda solo la preferenza: prende il premio 5 a soli punti (giusto) ma poi si ritrova con 3 punti e deve comprare il premio 2 a 8 euro, mentre l'ottimo tiene da parte i punti per il premio 4, che costa solo 5 euro di contributo.

10.1: le modalita' scelte (contributo totale 5 EUR)

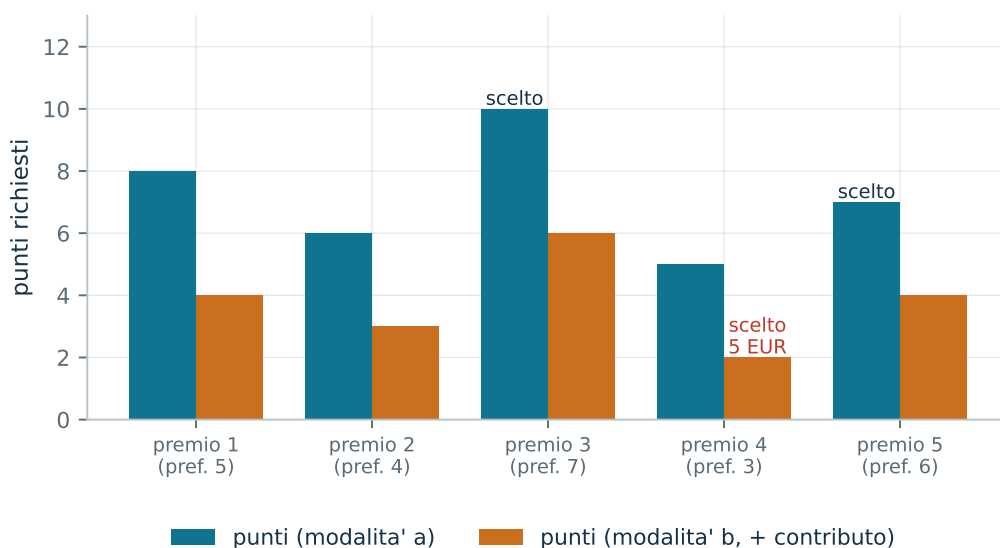


Figura 10.1: Per ogni premio, i punti richiesti dalle due modalità; le etichette segnalano quale modalità è scelta all'ottimo e il contributo pagato.

Considerazioni aggiuntive

- Le disuguaglianze valide $x_i \leq 1$ e $y_i \leq 1$ sono implicate dai vincoli (10.1b): infatti $z(LP) = z(LP^+)$.
- Se per qualche premio fosse $c_i = 0$, la seconda modalità dominerebbe la prima (meno punti, stesso costo) e la variabile x_i potrebbe essere eliminata. È un controllo sui dati che vale la pena fare.

- Il modello si estende a k modalità senza fatica: bastano k famiglie di binarie e il vincolo

$$\sum_{m=1}^k x_i^{(m)} \leq 1.$$

La struttura resta un set packing per righe.

Domande di modellazione aggiuntive

Domanda di modellazione 10.1 — premi alternativi

I premi 3 e 5 provengono dallo stesso fornitore e sono alternativi: se ne può prendere al più uno, in qualunque modalità. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 10.2 — almeno quattro premi

Oltre alla soglia di preferenza, il cliente vuole almeno quattro premi diversi. Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	8	soluzione euristica (UB)
LB	3	bound certificato (LB)
$z(MILP)$	5	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 60.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 100.0\%.$$

10.2 Asta combinatoria

Script: `python/fam10_2_asta.py`

[Notebook in Colab](#)

Problema 10.2 — asta combinatoria

Un banditore ha un insieme $S = \{1, 2, \dots, n\}$ di $n \in \mathbb{Z}_{\geq 1}$ oggetti da vendere e ha ricevuto $r \in \mathbb{Z}_{\geq 1}$ offerte. Per ogni offerta $j \in \{1, 2, \dots, r\}$, l'insieme $B_j \subseteq S$ è il sottoinsieme di oggetti richiesti e $p_j \in \mathbb{Q}_{>0}$ il profitto in euro se l'offerta viene accettata. Ogni oggetto può essere venduto al più una volta, e un'offerta può essere accettata solo se tutti i suoi oggetti sono disponibili. Il banditore vuole scegliere un insieme di offerte di profitto totale massimo.

Il problema a parole. Decidiamo quali offerte accettare. *L'obiettivo:* profitto massimo. *Il vincolo:* due offerte accettate non possono chiedere lo stesso oggetto. Un'offerta è «tutto o niente»: non se ne accetta una parte.

Dati (input del modello)

Simbolo	Tipo	Significato
n	$\in \mathbb{Z}_{\geq 1}$	numero di oggetti, $i \in S = \{1, 2, \dots, n\}$
r	$\in \mathbb{Z}_{\geq 1}$	numero di offerte, $j \in \{1, 2, \dots, r\}$
B_j	$\subseteq S$	oggetti richiesti dall'offerta j
p_j	$\in \mathbb{Q}_{>0}$	profitto dell'offerta j

Variabili decisionali

Una sola famiglia di r variabili binarie:

$$x_j = \begin{cases} 1 & \text{se l'offerta } j \text{ viene accettata,} \\ 0 & \text{altrimenti,} \end{cases} \quad \forall j \in \{1, 2, \dots, r\}.$$

Modello 10.2 — asta combinatoria

$$\max \sum_{j=1}^r p_j x_j \quad (10.3a)$$

$$\text{soggetto a } \sum_{j: i \in B_j} x_j \leq 1, \quad \forall i \in S, \quad (10.3b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, r\}. \quad (10.3c)$$

Descrizione. È un *set packing* puro: un vincolo per oggetto (n vincoli lineari) e una variabile per offerta. Il vincolo (10.3b) dice che l'oggetto i viene venduto al più una volta.

Per righe, non per coppie

Un modo alternativo, e peggiore, di scrivere lo stesso vincolo è elencare tutte le *coppie* di offerte in conflitto: $x_j + x_{j'} \leq 1$ per ogni $j \neq j'$ con $B_j \cap B_{j'} \neq \emptyset$. Sull'istanza sarebbero nove vincoli invece di quattro, e su istanze reali il numero di coppie cresce come r^2 mentre i vincoli per oggetto restano n . La forma per righe è anche più stretta: se tre offerte chiedono tutte l'oggetto i , il vincolo di riga dice $x_1 + x_2 + x_3 \leq 1$, mentre le tre disuguaglianze a coppie ammettono $x_1 = x_2 = x_3 = 1/2$. È lo stesso confronto fra forma aggregata e disaggregata già visto nella sezione 3.1, ma qui il verso della convenienza è opposto.

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```
1 m = gp.Model("asta")
2 x = m.addVars(r, vtype=GRB.BINARY, name="x")
```

```

3 m.setObjective(gp.quicksum(p[j] * x[j] for j in range(r)), GRB.MAXIMIZE
  )
4 m.addConstrs((gp.quicksum(x[j] for j in range(r) if i in B[j]) <= 1
5               for i in range(n)), name="oggetto")

```

L'istanza

$n = 4$ oggetti, $r = 6$ offerte:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
B_j	{1}	{2}	{3, 4}	{1, 3}	{2, 4}	{1, 3, 4}
p_j	6	3	12	12	10	16

Il modello dell'istanza è

$$\max 6x_1 + 3x_2 + 12x_3 + 12x_4 + 10x_5 + 16x_6$$

soggetto a

$$\begin{aligned}
 x_1 + x_4 + x_6 &\leq 1 && \text{(oggetto 1)} \\
 x_2 + x_5 &\leq 1 && \text{(oggetto 2)} \\
 x_3 + x_4 + x_6 &\leq 1 && \text{(oggetto 3)} \\
 x_3 + x_5 + x_6 &\leq 1 && \text{(oggetto 4)}
 \end{aligned}$$

con $x_j \in \{0, 1\}$.

Euristica costruttiva: il bound primale

Il problema è di massimo, quindi l'euristica dà il bound *primale*, che sta sotto l'ottimo. Si ordinano le offerte per profitto *per oggetto* decrescente e si accettano quelle i cui oggetti sono ancora liberi.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Il tempo di esecuzione è $O(r \log r + rn)$. Sull'istanza i rapporti sono 6, 3, 6, 6, 5 e $16/3 \approx 5,33$, quindi l'ordine è 1, 3, 4, 6, 5, 2:

- offerta 1 {1}: accettata (profitto 6);
- offerta 3 {3, 4}: accettata (profitto 12);
- offerta 4 {1, 3}: scartata, gli oggetti 1 e 3 sono già venduti;
- offerta 6 {1, 3, 4}: scartata;
- offerta 5 {2, 4}: scartata, l'oggetto 4 è già venduto;
- offerta 2 {2}: accettata (profitto 3).

Si accettano le offerte 1, 2, 3:

$$z(MILP) \geq LB = 21.$$

Rilassamento LP e duale: il bound duale

Associamo una variabile duale non negativa λ_i a ciascun vincolo (10.3b). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 10.2

$$\min \sum_{i \in S} \lambda_i \quad (10.4a)$$

$$\text{soggetto a } \sum_{i \in B_j} \lambda_i \geq p_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.4b)$$

$$\lambda_i \geq 0, \quad \forall i \in S. \quad (10.4c)$$

Descrizione. La variabile λ_i è il prezzo che il banditore attribuisce all'oggetto i . L'obiettivo (10.4a) è il valore complessivo dei lotti a quei prezzi. I vincoli (10.4b) sono le colonne delle x_j , uno per offerta: gli oggetti che l'offerta j richiede, valutati ai prezzi λ_i , devono valere almeno quanto l'offerta paga, cioè p_j . Nessuna offerta, insomma, deve risultare un affare.

Una soluzione duale a mano. Per esempio si può procedere così: si spalma il profitto di ogni offerta sui suoi oggetti e si prende, per ogni oggetto, il massimo fra le offerte che lo chiedono,

$$\bar{\lambda}_i = \max_{j: i \in B_j} \frac{p_j}{|B_j|}.$$

L'ammissibilità è immediata: per ogni offerta j ,

$$\sum_{i \in B_j} \bar{\lambda}_i \geq \sum_{i \in B_j} \frac{p_j}{|B_j|} = |B_j| \cdot \frac{p_j}{|B_j|} = p_j.$$

Sull'istanza:

$$\begin{aligned} \bar{\lambda}_1 &= \max\left(\frac{6}{1}, \frac{12}{2}, \frac{16}{3}\right) = 6, & \bar{\lambda}_2 &= \max\left(\frac{3}{1}, \frac{10}{2}\right) = 5, \\ \bar{\lambda}_3 &= \max\left(\frac{12}{2}, \frac{12}{2}, \frac{16}{3}\right) = 6, & \bar{\lambda}_4 &= \max\left(\frac{12}{2}, \frac{10}{2}, \frac{16}{3}\right) = 6, \end{aligned}$$

e

$$z(MILP) \leq UB = 6 + 5 + 6 + 6 = 23.$$

La ricetta «ovvia» $\bar{\lambda}_i = \max_{j: i \in B_j} p_j$ (senza dividere per $|B_j|$) è anch'essa ammissibile, ma dà $16 + 10 + 16 + 16 = 58$: più del doppio. Dividere per la cardinalità dell'insieme richiesto è ciò che rende il bound utile. Vale la pena provare più ricette e tenere la migliore: sono tutte valide, non tutte informative.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. All'ottimo si accettano le offerte 4 ($\{1, 3\}$, profitto 12) e 5 ($\{2, 4\}$, profitto 10): tutti e quattro gli oggetti vengono venduti e

$$z(MILP) = 22.$$

La catena dei bound è

$$21 = LB \leq z(MILP) = 22 = z(LP^+) = z(LP) \leq UB = 23,$$

con un gap dell'euristica del 4,5%. Il banditore vende tutto, ma non perché un vincolo lo imponga: i vincoli sono $\leq$, non $=$. Con altre offerte la soluzione ottima potrebbe lasciare oggetti sullo scaffale.

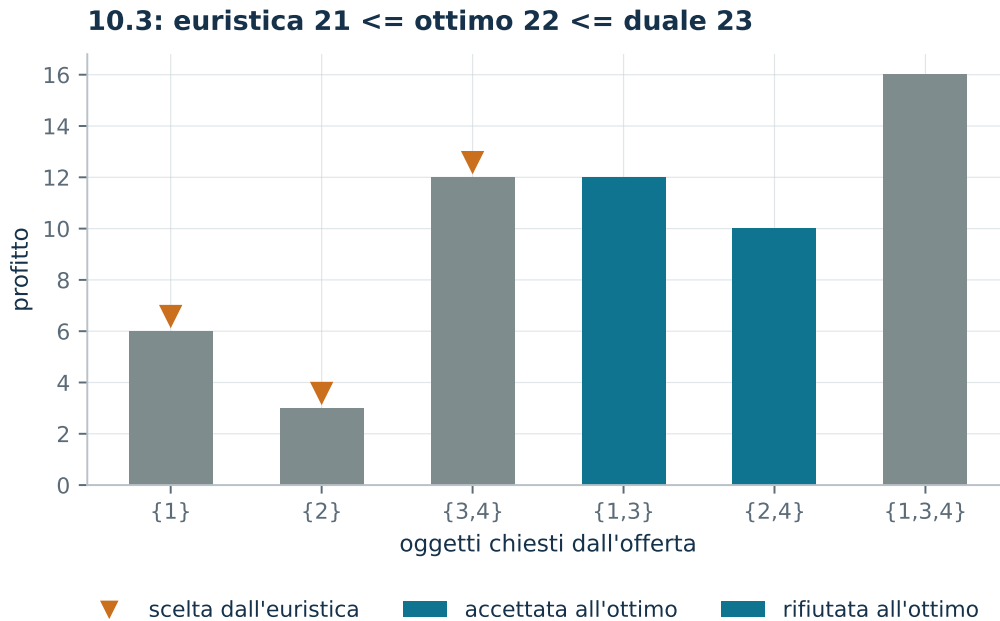


Figura 10.2: Le sei offerte con il loro profitto: in teal quelle accettate all'ottimo, in grigio quelle rifiutate; il triangolo segna quelle scelte dall'euristica.

Considerazioni aggiuntive

- Le disuguaglianze valide $x_j \leq 1$ sono implicate dai vincoli (10.3b) ogni volta che $B_j \neq \emptyset$: infatti $z(LP) = z(LP^+) = 22$.
- Su questa istanza si ha anche $z(LP) = z(MILP)$: il rilassamento cade in un vertice intero. È un caso fortunato, non una proprietà del set packing. Il controesempio minimo è il *triangolo*: tre oggetti e tre offerte che ne chiedono due ciascuna, tutte di profitto 1. Lì $z(LP) = 3/2$ (con $x = 1/2$ su tutte e tre) contro $z(MILP) = 1$.
- Il set packing è NP-difficile in generale, ma diventa facile quando la matrice di incidenza è *perfetta* o *bilanciata*. Il triangolo è proprio il più piccolo grafo non perfetto in questo senso, ed è il motivo per cui le *clique inequalities* sono i tagli classici per questa famiglia.

Domande di modellazione aggiuntive

Domanda di modellazione 10.3 — offerte dello stesso partecipante

Le offerte 4 e 5 provengono dallo stesso partecipante, che può vincerne al più una. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 10.4 — consegne limitate

In questa tornata il banditore può consegnare al più due oggetti in totale. Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	21	soluzione euristica (LB)
UB	23	bound certificato (UB)
$z(MILP)$	22	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 4.5\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 9.1\%.$$

10.3 Dieta con conteggio dei cibi e lotto minimo

Script: `python/fam10_3_dieta.py`

[Notebook in Colab](#)

Problema 10.3 — dieta con varietà

Un nutrizionista deve comporre una dieta mensile scegliendo fra $s \in \mathbb{Z}_{\geq 1}$ cibi e controllando $r \in \mathbb{Z}_{\geq 1}$ nutrienti. Per ogni cibo $i \in \{1, 2, \dots, s\}$, il valore $w_i \in \mathbb{Q}_{>0}$ è il costo di un chilo; per ogni cibo i e ogni nutriente $j \in \{1, 2, \dots, r\}$, il valore $g_{ij} \in \mathbb{Q}_{\geq 0}$ è la quantità di nutriente j contenuta in un chilo di cibo i . Per ogni nutriente j , l'assunzione mensile deve stare fra $a_j \in \mathbb{Q}_{>0}$ e $b_j \in \mathbb{Q}_{>0}$. Se un cibo entra nella dieta se ne devono consumare almeno $c_i \in \mathbb{Q}_{>0}$ chili e al più $d_i \in \mathbb{Q}_{>0}$. La dieta deve comprendere almeno $t \in \mathbb{Z}_{\geq 1}$ cibi diversi. Si vuole la dieta di costo minimo.

Il problema a parole. Decidiamo quali cibi usare e in che quantità. *L'obiettivo:* costo minimo. *I vincoli:* ogni nutriente entro la sua finestra; ogni cibo scelto in quantità fra il lotto minimo e il tetto; almeno t cibi diversi.

Dati (input del modello)

Simbolo	Tipo	Significato
s	$\in \mathbb{Z}_{\geq 1}$	numero di cibi, $i \in \{1, 2, \dots, s\}$
r	$\in \mathbb{Z}_{\geq 1}$	numero di nutrienti, $j \in \{1, 2, \dots, r\}$
w_i	$\in \mathbb{Q}_{>0}$	costo di un chilo di cibo i
g_{ij}	$\in \mathbb{Q}_{\geq 0}$	grammi di nutriente j per chilo di cibo i
a_j, b_j	$\in \mathbb{Q}_{>0}$	minimo e massimo mensile del nutriente j
c_i, d_i	$\in \mathbb{Q}_{>0}$	quantità minima e massima del cibo i , se scelto
t	$\in \mathbb{Z}_{\geq 1}$	numero minimo di cibi diversi

Variabili decisionali

s variabili continue non negative e s binarie:

$$x_i = \text{chili di cibo } i \text{ consumati nel mese,} \quad y_i = \begin{cases} 1 & \text{se il cibo } i \text{ entra nella dieta,} \\ 0 & \text{altrimenti.} \end{cases}$$

Modello 10.3 — dieta con varietà

$$\min \sum_{i=1}^s w_i x_i \quad (10.5a)$$

$$\text{soggetto a } \sum_{i=1}^s g_{ij} x_i \geq a_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.5b)$$

$$\sum_{i=1}^s g_{ij} x_i \leq b_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.5c)$$

$$x_i - c_i y_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5d)$$

$$x_i - d_i y_i \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5e)$$

$$\sum_{i=1}^s y_i \geq t, \quad (10.5f)$$

$$x_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5g)$$

$$y_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}. \quad (10.5h)$$

Descrizione. L'obiettivo (10.5a) è la spesa complessiva. I vincoli (10.5b), uno per nutriente, impongono il fabbisogno minimo; i vincoli (10.5c), sempre uno per nutriente, impongono il tetto. I vincoli (10.5d) e (10.5e), uno per cibo ciascuno, rendono x_i una variabile semicontinua: o vale zero, o sta fra il lotto minimo c_i e il massimo d_i . Il vincolo (10.5f), uno solo, chiede che i cibi effettivamente comprati siano almeno t .

Perché il lotto minimo serve al conteggio

Il vincolo di varietà (10.5f) conta gli indicatori y_i , non le quantità. Se ci fosse soltanto il vincolo di attivazione (10.5e), l'indicatore potrebbe valere 1 con $x_i = 0$: si «accenderebbero» cibi senza consumarne un grammo, e il vincolo di varietà sarebbe soddisfatto da indicatori vuoti. È il legame a senso unico della sezione 3.10, e qui produce un modello che non descrive il problema.

Il vincolo di lotto minimo (10.5d) chiude il cerchio: con $y_i = 1$ si ha $x_i \geq c_i > 0$, quindi ogni indicatore acceso corrisponde a un cibo davvero consumato. Sull'istanza, ponendo $c_i = 0$ l'ottimo scende da $48/5$ a $46/5$ e la soluzione «accende» quattro cibi di cui due con quantità nulla: il conteggio non dice più niente.

Il modello in gurobipy

```

1 m = gp.Model("dieta")
2 x = m.addVars(s, name="x")
3 y = m.addVars(s, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(w[i] * x[i] for i in range(s)), GRB.MINIMIZE
5 )
6 m.addConstrs((gp.quicksum(g[i][j] * x[i] for i in range(s)) >= a[j] for
7 j in range(r)),
8 name="minimo")

```

```

7 m.addConstrs((gp.quicksum(g[i][j] * x[i] for i in range(s)) <= b[j] for
  j in range(r)),
8               name="massimo")
9 m.addConstrs((x[i] - c[i] * y[i] >= 0 for i in range(s)), name="
  lotto_minimo")
10 m.addConstrs((x[i] - d[i] * y[i] <= 0 for i in range(s)), name="attiva"
  )
11 m.addConstr(gp.quicksum(y[i] for i in range(s)) >= t, name="varietà")

```

L'istanza

$s = 4$ cibi, $r = 2$ nutrienti, $c_i = 1$, $d_i = 8$, $t = 3$:

	latte	riso	pane	patate		ferro	calcio
w_i (euro/kg)	2	3	1	4	a_j	60	40
ferro (g/kg)	10	20	5	25	b_j	200	150
calcio (g/kg)	5	10	15	5			

Euristica costruttiva: il bound primale

Si accendono al lotto minimo i t cibi più economici, poi si copre il fabbisogno residuo di ciascun nutriente aggiungendo il cibo già acceso con il costo per grammo più basso.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza i tre cibi più economici sono pane (1), latte (2) e riso (3): si accendono al lotto minimo, cioè 1 kg ciascuno. Con queste quantità si hanno $5 + 10 + 20 = 35$ grammi di ferro (ne servono 60) e $15 + 5 + 10 = 30$ di calcio (ne servono 40).

- **Ferro:** mancano 25 grammi. Fra i cibi accesi il costo per grammo di ferro più basso è quello del riso, $3/20 = 0,15$; ne servono $25/20 = 1,25$ kg in più, e il riso arriva a 2,25 kg.
- **Calcio:** dopo l'aggiunta si hanno $30 + 12,5 = 42,5$ grammi, già oltre il minimo di 40: non serve altro.

La dieta finale è latte 1 kg, riso 2,25 kg, pane 1 kg, per un costo di

$$z(MILP) \leq UB = \frac{39}{4} = 9,75.$$

Rilassamento LP e duale: il bound duale

Associamo $\alpha_j \geq 0$ ai minimi (10.5b), $\beta_j \geq 0$ ai massimi (10.5c), $\lambda_i \geq 0$ ai lotti minimi (10.5d), $\mu_i \geq 0$ ai tetti (10.5e) e $\tau \geq 0$ alla varietà (10.5f). Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 10.3

$$\max \sum_{j=1}^r a_j \alpha_j - \sum_{j=1}^r b_j \beta_j + t \tau \quad (10.6a)$$

$$\text{soggetto a } \sum_{j=1}^r g_{ij} (\alpha_j - \beta_j) + \lambda_i - \mu_i \leq w_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6b)$$

$$-c_i \lambda_i + d_i \mu_i + \tau \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6c)$$

$$\alpha_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.6d)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.6e)$$

$$\lambda_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6f)$$

$$\mu_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6g)$$

$$\tau \geq 0. \quad (10.6h)$$

Descrizione. La variabile α_j è il prezzo di una unità del nutriente j quando serve a raggiungere il fabbisogno minimo, β_j quello che si paga per non sforare il tetto, λ_i e μ_i i prezzi dei due vincoli di semicontinuità del cibo i e τ il prezzo della varietà. L'obiettivo (10.6a) incassa i fabbisogni valutati ad α , paga i tetti valutati a β e incassa la soglia t valutata a τ . I vincoli (10.6b) sono le colonne delle x_i : il contenuto nutrizionale di una unità del cibo i , valutato ai prezzi $\alpha_j - \beta_j$ e corretto dai due vincoli di semicontinuità, non può superare il prezzo w_i di quel cibo. I vincoli (10.6c) sono le colonne delle y_i : accendere il cibo i concede c_i unità di lotto minimo al prezzo λ_i , ne obbliga al più d_i al prezzo μ_i , e deve coprire il prezzo τ della varietà.

Una soluzione duale a mano. Poniamo $\beta = \mu = \tau = 0$: i massimi, i tetti e la varietà non si valutano. I vincoli (10.6c) diventano $-c_i \lambda_i \leq 0$, soddisfatti con $\lambda = 0$. Resta da soddisfare $\sum_{j=1}^r g_{ij} \alpha_j \leq w_i$ per ogni cibo. Per esempio si può valutare *un solo* nutriente, al prezzo per grammo più basso fra i cibi:

$$\bar{\alpha}_j = \min_{i: g_{ij} > 0} \frac{w_i}{g_{ij}}, \quad \text{bound} = a_j \bar{\alpha}_j,$$

e si tiene il nutriente che dà il bound più alto. Sull'istanza:

$$\text{ferro: } \bar{\alpha}_1 = \min\left(\frac{2}{10}, \frac{3}{20}, \frac{1}{5}, \frac{4}{25}\right) = \frac{3}{20}, \quad 60 \cdot \frac{3}{20} = 9,$$

$$\text{calcio: } \bar{\alpha}_2 = \min\left(\frac{2}{5}, \frac{3}{10}, \frac{1}{15}, \frac{4}{5}\right) = \frac{1}{15}, \quad 40 \cdot \frac{1}{15} = \frac{8}{3}.$$

Il migliore è il ferro:

$$z(MILP) \geq LB = 9.$$

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. La dieta ottima è riso 1,4 kg, pane 1,4 kg, patate 1 kg: tre cibi diversi, come richiesto, con 60 grammi di ferro (il minimo esatto) e 40 di calcio (di nuovo il minimo esatto). Il costo è

$$z(MILP) = \frac{48}{5} = 9,6,$$

e la catena dei bound è

$$9 = LB \leq z(LP) = \frac{46}{5} \leq z(LP^+) = \frac{48}{5} = z(MILP) \leq UB = \frac{39}{4}.$$

Il gap dell'euristica è appena l'1,6%: è il sandwich più stretto del capitolo. Si noti che qui il rilassamento con i bound coincide con l'ottimo intero, mentre quello senza i bound sta sotto: aggiungere $y_i \leq 1$ serve, perché i tetti $d_i = 8$ sono molto più larghi delle quantità in gioco e senza $y_i \leq 1$ il rilassamento userebbe indicatori frazionari maggiori di uno.

10.2: dieta euristica (39/4 EUR) e ottima (48/5 EUR)

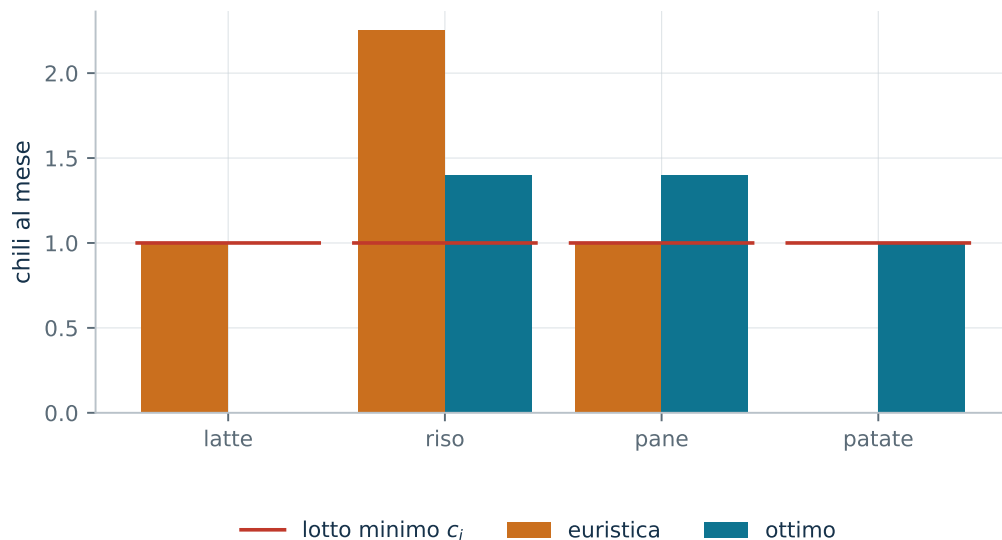


Figura 10.3: Quantità per cibo nella dieta euristica e in quella ottima; la barra rossa segna il lotto minimo c_i .

Considerazioni aggiuntive

- I vincoli di massimo (10.5c) non sono attivi all'ottimo ($60 \leq 200$ e $40 \leq 150$): in questa istanza si potrebbero togliere senza cambiare nulla. Restano nel modello perché il problema li richiede, e perché su altre istanze morderebbero.
- La forma classica del problema della dieta (Stigler, 1945) non ha né lotti minimi né conteggi: è un LP puro. Sono proprio le due aggiunte intere a renderlo un MILP, ed è per questo che il problema sta in questo capitolo.
- Il vincolo di varietà si potrebbe scrivere anche in forma di *copertura*: almeno un cibo da ciascun gruppo alimentare, cioè $\sum_{i \in G_k} y_i \geq 1$ per ogni gruppo k . È una formulazione più informativa dal punto di vista nutrizionale, e non più difficile.

Domande di modellazione aggiuntive

Domanda di modellazione 10.5 — lotto minimo più alto

Il lotto minimo sale da 1 a 2 chili per ogni cibo scelto. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 10.6 — più varietà

Si vogliono almeno quattro cibi diversi invece di tre. Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	$\frac{39}{4}$	soluzione euristica (UB)
LB	9	bound certificato (LB)
$z(MILP)$	$\frac{48}{5}$	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 1.6\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 7.8\%.$$

10.4 Alberi di Natale e scatole di luci

Script: `python/fam10_4_luci.py`

[Notebook in Colab](#)

Problema 10.4 — alberi e luci

L'amministrazione comunale vuole decorare $q \in \mathbb{Z}_{\geq 1}$ alberi per le festività. Ogni albero può essere decorato secondo una delle $n \in \mathbb{Z}_{\geq 1}$ configurazioni possibili, ciascuna caratterizzata da luci di $m \in \mathbb{Z}_{\geq 1}$ colori diversi. Per ogni configurazione $c \in \{1, 2, \dots, n\}$ e ogni colore $l \in \{1, 2, \dots, m\}$, il valore $u_{cl} \in \mathbb{Z}_{\geq 0}$ è il numero di luci di colore l richieste dalla configurazione c , e $i_c \in \mathbb{Q}_{>0}$ è il costo di installazione di un albero decorato secondo quella configurazione. Tutte le luci vanno acquistate sul mercato, dove si vendono in scatole di $k \in \mathbb{Z}_{\geq 1}$ tipi: per ogni tipo $b \in \{1, 2, \dots, k\}$ e ogni colore l , il valore $v_{bl} \in \mathbb{Z}_{\geq 0}$ è il numero di luci di colore l contenute in una scatola di tipo b , e $p_b \in \mathbb{Q}_{\geq 0}$ è il suo costo. Per garantire una varietà visiva gradevole si devono usare almeno $f \in \mathbb{Z}_{\geq 1}$ configurazioni diverse. Si vuole decorare gli alberi al costo totale minimo.

Il problema a parole. Decidiamo quanti alberi decorare con ciascuna configurazione e quante scatole comprare di ciascun tipo. *L'obiettivo:* costo totale (installazione più scatole) minimo. *I vincoli:* tutti gli alberi vanno decorati; le luci comprate devono bastare a quelle richieste, colore per colore; almeno f configurazioni diverse.

Variabili decisionali

n variabili intere, k variabili intere e n binarie:

x_c = alberi decorati con la configurazione c , y_b = scatole comprate di tipo b ,

$$z_c = \begin{cases} 1 & \text{se la configurazione } c \text{ viene usata,} \\ 0 & \text{altrimenti.} \end{cases}$$

Modello 10.4 — alberi e luci

$$\min \sum_{c=1}^n i_c x_c + \sum_{b=1}^k p_b y_b \quad (10.7a)$$

$$\text{soggetto a } \sum_{c=1}^n x_c = q, \quad (10.7b)$$

$$\sum_{b=1}^k v_{bl} y_b - \sum_{c=1}^n u_{cl} x_c \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (10.7c)$$

$$\sum_{c=1}^n z_c \geq f, \quad (10.7d)$$

$$x_c - z_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.7e)$$

$$x_c \in \mathbb{Z}_{\geq 0}, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.7f)$$

$$y_b \in \mathbb{Z}_{\geq 0}, \quad \forall b \in \{1, 2, \dots, k\}, \quad (10.7g)$$

$$z_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (10.7h)$$

Descrizione. L'obiettivo (10.7a) somma il costo delle installazioni e quello delle scatole comprate. Il vincolo (10.7b), uno solo, dice che gli alberi decorati sono esattamente q . I vincoli (10.7c), uno per colore, dicono che le luci comprate bastano a coprire quelle usate. Il vincolo (10.7d), uno solo, impone almeno f configurazioni diverse. I vincoli (10.7e), uno per configurazione, legano l'indicatore z_c all'uso effettivo della configurazione.

Il legame fra i due piani. Il vincolo (10.7c) è la disponibilità: per ogni colore, le luci comprate devono essere almeno quelle richieste. Non c'è alcun big-M, perché entrambi i membri sono somme di quantità reali. Il conteggio delle scatole nasce da qui: se le luci di colore l richieste sono u_l e una scatola ne contiene v_{bl} , comprando solo scatole di tipo b servirebbero $\lceil u_l/v_{bl} \rceil$ scatole. Il modello non scrive il tetto: lo ottiene imponendo y_b intera.

Il legame «configurazione usata»

Il vincolo (10.7e) si legge $z_c \leq x_c$. Se $z_c = 1$ allora $x_c \geq 1$: la configurazione è davvero usata su almeno un albero. Il verso opposto non è imposto, ma non serve: l'obiettivo di minimo non ha alcun interesse ad alzare z_c , e il vincolo (10.7d) lo costringe a farlo esattamente f volte. Si noti che qui non serve un big-M perché x_c e z_c sono confrontabili direttamente: x_c è un conteggio, non una quantità continua.

Un errore frequente è scrivere $x_c \leq M z_c$ pensando all'attivazione: sarebbe il legame nell'altro verso, e imporrebbe che una configurazione non usata abbia $x_c = 0$ — vero ma inutile, perché è già garantito da $\sum_{c=1}^n x_c = q$ e dal fatto che $x_c \geq 0$.

Il modello in gurobipy

```

1 m = gp.Model("luci")
2 x = m.addVars(nc, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(nb, vtype=GRB.INTEGER, name="y")
4 z = m.addVars(nc, vtype=GRB.BINARY, name="z")
5 m.setObjective(gp.quicksum(i[c] * x[c] for c in range(nc))
6                 + gp.quicksum(p[b] * y[b] for b in range(nb)), GRB.
9                 MINIMIZE)
7 m.addConstr(x.sum() == q, name="alberi")
8 m.addConstrs((gp.quicksum(v[b][l] * y[b] for b in range(nb))
9                 - gp.quicksum(u[c][l] * x[c] for c in range(nc)) >= 0
10                for l in range(nl)), name="luci")
11 m.addConstr(z.sum() >= f, name="varietà")
12 m.addConstrs((x[c] - z[c] >= 0 for c in range(nc)), name="usata")

```

L'istanza

$q = 20$ alberi, $n = 3$ configurazioni, $m = 2$ colori, $k = 2$ tipi di scatola, $f = 2$:

u_{cl}	colore 1	colore 2	i_c	v_{bl}	colore 1	colore 2	p_b
$c = 1$	4	2	7	$b = 1$	10	2	100
$c = 2$	2	3	6	$b = 2$	15	4	200
$c = 3$	2	2	8				

Il prezzo di una luce dipende dal colore e dal tipo di scatola: colore 1 a 10 nella scatola 1 e a 40/3 nella 2; colore 2 a 50 in entrambe. Il colore 2 è molto più caro, ed è questo a comandare la soluzione.

Euristica costruttiva: il bound primale

Due fasi. Prima le configurazioni: $q - f + 1$ alberi con quella di installazione più economica e un albero per ciascuna delle altre $f - 1$, così la varietà è soddisfatta al minimo costo di installazione. Poi le scatole: finché manca qualche luce si compra la scatola con il prezzo per luce mancante più basso.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza la configurazione più economica da installare è la 2 (costo 6), la seconda la 1 (costo 7): si decorano 19 alberi con la 2 e uno con la 1. Servono allora 42 luci del colore 1 e 59 del colore 2, e l'euristica costruttiva compra 30 scatole di tipo 1. Il costo totale è

$$z(MILP) \leq UB = 3121.$$

L'euristica sceglie la configurazione più economica da installare, la 2, che però è la più avida di luci del colore costoso: il conto lo pagano le scatole. È il tipico errore di un'euristica costruttiva che guarda una sola voce di costo.

Rilassamento LP e duale: il bound duale

Associamo α libera al vincolo (10.7b) (uguaglianza), $\beta_l \geq 0$ ai vincoli (10.7c), $\gamma \geq 0$ a (10.7d) e $\delta_c \geq 0$ a (10.7e). Il duale è:

Duale del rilassamento LP del modello 10.4

$$\max \quad q\alpha + f\gamma \quad (10.8a)$$

$$\text{soggetto a} \quad \alpha - \sum_{l=1}^m u_{cl} \beta_l + \delta_c \leq i_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.8b)$$

$$\sum_{l=1}^m v_{bl} \beta_l \leq p_b, \quad \forall b \in \{1, 2, \dots, k\}, \quad (10.8c)$$

$$\gamma - \delta_c \leq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.8d)$$

$$\alpha \geq 0, \quad (10.8e)$$

$$\beta_l \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (10.8f)$$

$$\gamma \geq 0, \quad (10.8g)$$

$$\delta_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}. \quad (10.8h)$$

Descrizione. La variabile α è il valore di un albero decorato, β_l il prezzo di una luce di colore l , γ il prezzo della varietà e δ_c quello del legame fra la configurazione c e il suo indicatore. L'obiettivo (10.8a) valuta i q alberi al prezzo α e la soglia f di varietà al prezzo γ . I vincoli (10.8b) sono le colonne delle x_c : usare la configurazione c su un albero vale α , consuma u_{cl} luci di ciascun colore e libera δ_c , e il saldo non può superare il costo i_c dell'installazione. I vincoli (10.8c) sono le colonne delle y_b : una scatola di tipo b mette a disposizione v_{bl} luci di ciascun colore, e il loro valore complessivo non può superare il prezzo p_b della scatola. I vincoli (10.8d) sono le colonne delle z_c : il prezzo della varietà non può superare quello del legame che la rende esigibile.

Una soluzione duale a mano. Poniamo $\gamma = 0$ e $\delta_c = 0$: la varietà non si valuta e il vincolo (10.8d) è soddisfatto. Restano i vincoli (10.8c) sulle scatole, che limitano β , e i (10.8b) sulle configurazioni, che limitano α .

Si valuta *un solo* colore, al prezzo per luce più alto che nessuna scatola riesce a battere:

$$\bar{\beta}_l = \min_{b: v_{bl} > 0} \frac{p_b}{v_{bl}},$$

e poi si prende il più grande α ammissibile,

$$\bar{\alpha} = \min_c (i_c + u_{cl} \bar{\beta}_l),$$

cioè: ogni albero costa almeno l'installazione della configurazione più conveniente, luci comprese. Sull'istanza il colore migliore è il 2, dove entrambi i tipi di scatola danno lo stesso prezzo per luce, $\bar{\beta}_2 = 50$:

$$i_1 + 2 \cdot 50 = 107, \quad i_2 + 3 \cdot 50 = 156, \quad i_3 + 2 \cdot 50 = 108,$$

dunque $\bar{\alpha} = 107$ e

$$z(MILP) \geq LB = 20 \cdot 107 = 2140.$$

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. Si decorano 19 alberi con la configurazione 1 e uno con la 3, e si comprano 20 scatole di tipo 1:

$$z(MILP) = 19 \cdot 7 + 8 + 20 \cdot 100 = 2141.$$

Le luci servono 78 del colore 1 (se ne comprano 200: ne avanzano molte) e 40 del colore 2 (se ne comprano esattamente 40). La catena dei bound è

$$2140 = LB = z(LP) \leq z(LP^+) = 2141 = z(MILP) \leq UB = 3121.$$

Il bound duale sbaglia di *una* unità su 2141, e il rilassamento rafforzato è esatto. Il gap dell'euristica è del 45,8%.

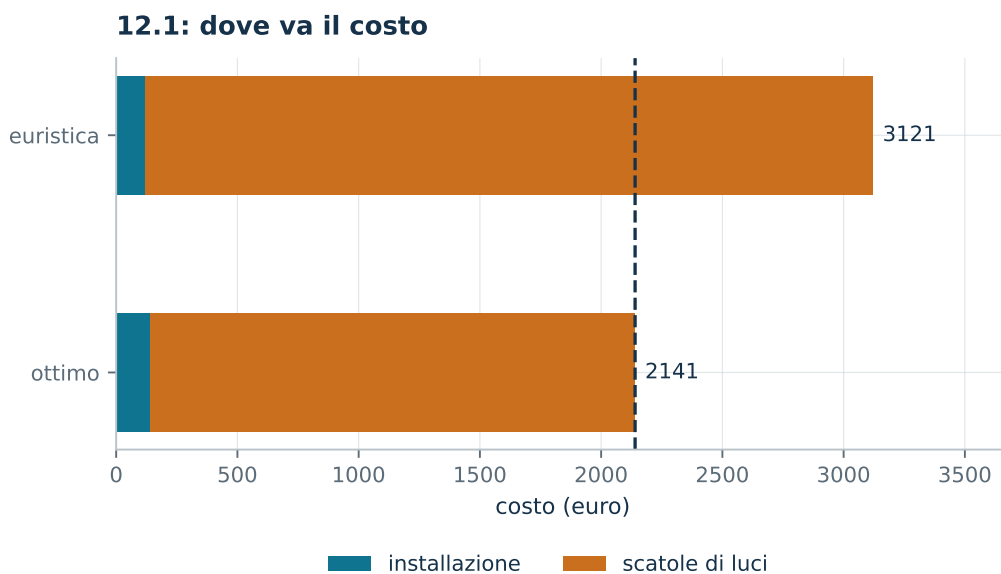


Figura 10.4: Dove va il costo nelle due soluzioni: installazione e scatole, con il bound duale tratteggiato.

Considerazioni aggiuntive

- Il colore 1 avanza in abbondanza: le scatole si comprano per il colore 2, e il colore 1 viene «in omaggio». È il motivo per cui il bound duale che valuta il solo colore 2 è quasi esatto.
- Il vincolo (10.7b) è un'uguaglianza: tutti gli alberi vanno decorati. Se fosse $\leq q$ il problema diventerebbe banale ($x = 0$ e costo zero); se fosse $\geq q$ non cambierebbe nulla, perché decorare più alberi costa di più.
- La struttura «due piani legati da una disponibilità» si ritrova identica nei problemi di *cutting stock*: sotto i pezzi da tagliare, sopra le barre da comprare.

Domande di modellazione aggiuntive

Domanda di modellazione 10.7 — tutte le configurazioni

Si vuole che compaiano tutte e tre le configurazioni. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 10.8 — lotto minimo per configurazione

Ogni configurazione usata deve decorare almeno tre alberi (sotto quella soglia non vale la pena attrezzare la squadra). Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	3121	soluzione euristica (UB)
LB	2140	bound certificato (LB)
$z(MILP)$	2141	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 45.8\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 45.8\%.$$

10.5 Spedizioni in scatole

Script: `python/fam10_5_spedizioni.py`

[Notebook in Colab](#)

Problema 10.5 — spedizioni in scatole

Un'azienda deve organizzare le spedizioni dai suoi $n \in \mathbb{Z}_{\geq 1}$ stabilimenti ai suoi $m \in \mathbb{Z}_{\geq 1}$ clienti. L'azienda produce $k \in \mathbb{Z}_{\geq 1}$ tipi di prodotto. I prodotti si spediscono in scatole: ogni scatola viaggia direttamente da uno stabilimento a un cliente e ha capacità massima $w \in \mathbb{Z}_{\geq 1}$, espressa come numero massimo di unità di prodotto trasportabili. Per ogni prodotto $p \in \{1, 2, \dots, k\}$ e ogni cliente $c \in \{1, 2, \dots, m\}$, il valore $d_{pc} \in \mathbb{Z}_{\geq 0}$ è il numero di unità ordinate. Per ogni prodotto p e ogni stabilimento $s \in \{1, 2, \dots, n\}$, il valore $a_{ps} \in \mathbb{Z}_{\geq 0}$ è il numero di unità disponibili. L'azienda vuole un piano di spedizione che minimizzi il numero totale di scatole usate.

Il problema a parole. Decidiamo quante unità di ciascun prodotto spedire da ciascuno stabilimento a ciascun cliente, e quante scatole servono su ciascuna tratta. *L'obiettivo*: numero minimo di scatole. *I vincoli*: ogni ordine soddisfatto esattamente, nessuno stabilimento oltre le proprie scorte, e su ogni tratta abbastanza scatole per le unità spedite.

Per brevità si scrivono $P = \{1, 2, \dots, k\}$ l'insieme dei prodotti, $S = \{1, 2, \dots, n\}$ quello degli stabilimenti e $C = \{1, 2, \dots, m\}$ quello dei clienti.

Variabili decisionali

$k n m$ variabili intere non negative per il flusso e $n m$ per le scatole:

x_{psc} = unità del prodotto p spedite da s a c , y_{sc} = scatole spedite da s a c .

Modello 10.5 — spedizioni in scatole

$$\min \sum_{s=1}^n \sum_{c=1}^m y_{sc} \quad (10.9a)$$

$$\text{soggetto a } \sum_{s=1}^n x_{psc} = d_{pc}, \quad \forall p \in P, \forall c \in C, \quad (10.9b)$$

$$\sum_{c=1}^m x_{psc} \leq a_{ps}, \quad \forall p \in P, \forall s \in S, \quad (10.9c)$$

$$-\sum_{p=1}^k x_{psc} + w y_{sc} \geq 0, \quad \forall s \in S, \forall c \in C, \quad (10.9d)$$

$$x_{psc} \in \mathbb{Z}_{\geq 0}, \quad \forall p \in P, \forall s \in S, \forall c \in C, \quad (10.9e)$$

$$y_{sc} \in \mathbb{Z}_{\geq 0}, \quad \forall s \in S, \forall c \in C. \quad (10.9f)$$

Descrizione. L'obiettivo (10.9a) conta le scatole usate su tutte le tratte. I vincoli (10.9b), uno per coppia prodotto–cliente, dicono che ogni ordine è soddisfatto esattamente. I vincoli (10.9c), uno per coppia prodotto–stabilimento, non lasciano spedire più di quanto lo stabilimento abbia. I vincoli (10.9d), uno per tratta, dicono che le scatole spedite da s a c bastano a contenere le unità caricate.

Il legame è un arrotondamento per eccesso

Il vincolo (10.9d) si legge

$$\sum_{p=1}^k x_{psc} \leq w y_{sc} \quad \Longleftrightarrow \quad y_{sc} \geq \frac{\sum_{p=1}^k x_{psc}}{w},$$

e poiché y_{sc} è intera e non negativa questo equivale a

$$y_{sc} \geq \left\lceil \frac{\sum_{p=1}^k x_{psc}}{w} \right\rceil.$$

Il tetto non compare mai nel modello: lo produce l'interezza. Lo stesso vincolo impone anche l'implicazione

$$\sum_{p=1}^k x_{psc} > 0 \implies y_{sc} \geq 1,$$

cioè: se da uno stabilimento parte anche una sola unità verso un cliente, allora almeno una scatola deve viaggiare su quella tratta. La contronominale è più leggibile: se $y_{sc} = 0$ allora $\sum_{p=1}^k x_{psc} = 0$.

Il verso opposto — se non si spedisce niente allora nessuna scatola — non è imposto dal vincolo ma segue dall'ottimalità, perché ogni scatola costa 1 e l'obiettivo è di minimo.

Il modello in gurobipy

```

1 m = gp.Model("spedizioni")
2 x = m.addVars(nk, nn, nm, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(nn, nm, vtype=GRB.INTEGER, name="y")
4 m.setObjective(y.sum(), GRB.MINIMIZE)
5 m.addConstrs((x.sum(p, "*", c) == d[p][c] for p in range(nk) for c in
    range(nm)),
6               name="domanda")
7 m.addConstrs((x.sum(p, s, "*") <= a[p][s] for p in range(nk) for s in
    range(nn)),
8               name="disponibilita")
9 m.addConstrs((w * y[s, c] - x.sum("s", s, c) >= 0 for s in range(nn)
    for c in range(nm)),
10              name="capacita")

```

L'istanza

$n = 2$ stabilimenti, $m = 2$ clienti, $k = 2$ prodotti, $w = 10$:

d_{pc}	$c = 1$	$c = 2$	a_{ps}	$s = 1$	$s = 2$
$p = 1$	5	0	$p = 1$	8	6
$p = 2$	2	4	$p = 2$	5	7

Le unità da spedire sono $5 + 0 + 2 + 4 = 11$ in tutto.

Euristica costruttiva: il bound primale

Cliente per cliente: si cerca di servirlo da un solo stabilimento, quello che ha tutto quello che serve; se nessuno basta si spezza l'ordine fra più stabilimenti. Alla fine si conta $\lceil \cdot / w \rceil$ scatole per tratta.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza il cliente 1 chiede 5 unità del prodotto 1 e 2 del 2: lo stabilimento 1 ha 8 e 5, quindi basta da solo. Il cliente 2 chiede 4 unità del prodotto 2: dopo la prima spedizione allo stabilimento 1 ne restano 3, allo stabilimento 2 ce ne sono 7, quindi si spedisce da lì. Le tratte sono due, con 7 e 4 unità: una scatola ciascuna.

$$z(MILP) \leq UB = 2.$$

Rilassamento LP e duale: il bound duale

Associamo α_{pc} libera ai vincoli di domanda (10.9b), $\beta_{ps} \leq 0$ a quelli di disponibilità (10.9c) e $\gamma_{sc} \geq 0$ al legame (10.9d). Il duale è:

Duale del rilassamento LP del modello 10.5

$$\max \sum_{p=1}^k \sum_{c=1}^m d_{pc} \alpha_{pc} + \sum_{p=1}^k \sum_{s=1}^n a_{ps} \beta_{ps} \quad (10.10a)$$

$$\text{soggetto a } \alpha_{pc} + \beta_{ps} - \gamma_{sc} \leq 0, \quad \forall p \in P, \forall s \in S, \forall c \in C, \quad (10.10b)$$

$$w \gamma_{sc} \leq 1, \quad \forall s \in S, \forall c \in C, \quad (10.10c)$$

$$\alpha_{pc} \geq 0, \quad \forall p \in P, \forall c \in C, \quad (10.10d)$$

$$\beta_{ps} \leq 0, \quad \forall p \in P, \forall s \in S, \quad (10.10e)$$

$$\gamma_{sc} \geq 0, \quad \forall s \in S, \forall c \in C. \quad (10.10f)$$

Descrizione. La variabile α_{pc} è il valore di una unità del prodotto p consegnata al cliente c , β_{ps} il prezzo (non positivo) della disponibilità del prodotto p nello stabilimento s e γ_{sc} il prezzo di una unità di spazio sulla tratta da s a c . L'obiettivo (10.10a) valuta a quei prezzi gli ordini e le disponibilità. I vincoli (10.10b) sono le colonne delle x_{psc} : spedire una unità del prodotto p da s a c vale $\alpha_{pc} + \beta_{ps}$ e occupa una unità di spazio al prezzo γ_{sc} ; il saldo non può essere positivo, perché quelle variabili non compaiono nell'obiettivo. I vincoli (10.10c) sono le colonne delle y_{sc} : una scatola offre w unità di spazio, e il loro valore non può superare 1, cioè quanto costa la scatola.

Una soluzione duale a mano. Il vincolo (10.10c) limita $\gamma_{sc} \leq 1/w$: prendiamo il massimo, $\bar{\gamma}_{sc} = 1/w$. Con $\bar{\beta} = 0$ il vincolo (10.10b) diventa $\alpha_{pc} \leq \bar{\gamma}_{sc} = 1/w$: prendiamo $\bar{\alpha}_{pc} = 1/w$. Il valore è

$$LB = \frac{1}{w} \sum_{p=1}^k \sum_{c=1}^m d_{pc} = \frac{11}{10}.$$

La lettura è che ogni unità ordinata occupa $1/w$ di scatola: è il bound «volumetrico», e coincide con $z(LP)$.

Un bound intero più forte

Il bound volumetrico ignora che una scatola non si divide fra due clienti. Ogni cliente c con almeno un'unità ordinata riceve almeno

$$\left\lceil \frac{\sum_{p=1}^k d_{pc}}{w} \right\rceil$$

scatole, e queste scatole sono diverse da quelle degli altri clienti. Sommando su tutti i clienti:

Cliente	unità ordinate	scatole minime
1	7	1
2	4	1
totale	11	2

$$z(MILP) \geq LB = 2,$$

quasi il doppio del bound volumetrico $11/10$.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. Lo stabilimento 2 serve entrambi i clienti: al cliente 1 con una scatola contenente 5 unità del prodotto 1 e 2 del 2, al cliente 2 con una scatola contenente 4 unità del prodotto 2.

$$z(MILP) = 2, \quad 2 = LB \leq z(MILP) = 2 \leq UB = 2, \quad z(LP) = z(LP^+) = \frac{11}{10}.$$

Il gap dell'euristica è nullo, e il bound intero certifica l'ottimalità. La differenza $2 - 11/10 = 9/10$ è tutta dovuta all'interezza: il 45% del valore ottimo.

12.2: piano ottimo con 2 scatole

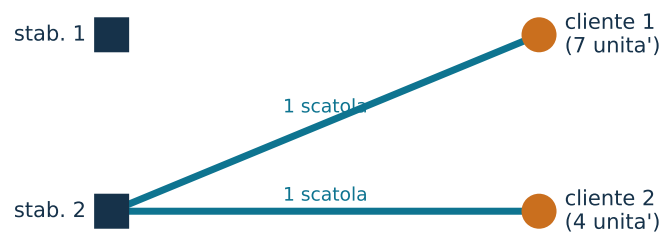


Figura 10.5: Il piano ottimo: quali stabilimenti servono quali clienti e con quante scatole.

Considerazioni aggiuntive

- Il modello è un flusso multiprodotto con costi solo sui contenitori. Se i costi fossero sul flusso (per unità trasportata) e non ci fossero scatole, il problema sarebbe un trasporto puro, risolvibile in tempo polinomiale e con rilassamento intero.
- Le variabili x_{psc} potrebbero essere continue senza cambiare l'ottimo, perché i dati sono interi e la matrice del trasporto è totalmente unimodulare *a y fissato*. Restano intere perché il problema parla di unità di prodotto.
- Il vincolo (10.9d) usa la stessa capacità w su ogni tratta. Con scatole di dimensione diversa servirebbero più famiglie $y_{sc}^{(t)}$, una per tipo, come nel problema 10.4.

Domande di modellazione aggiuntive

Domanda di modellazione 10.9 — scatole più piccole

Le scatole contengono 4 unità invece di 10. Qual è il nuovo ottimo?

Domanda di modellazione 10.10 — prodotti separati

Prodotti diversi non possono viaggiare nella stessa scatola. Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	2	soluzione euristica (UB)
LB	2	bound certificato (LB)
$z(MILP)$	2	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 0.0\%.$$

10.6 Campi estivi

Script: `python/fam10_6_campi.py`

[Notebook in Colab](#)

Problema 10.6 — campi estivi

Un'azienda gestisce $r \in \mathbb{Z}_{\geq 1}$ campi estivi per ospitare bambini durante le vacanze. Per ogni campo $j \in \{1, 2, \dots, r\}$, il valore $d_j \in \mathbb{Z}_{\geq 1}$ è il numero massimo di bambini ospitabili. L'azienda ha ricevuto richieste da bambini di $s \in \mathbb{Z}_{\geq 1}$ nazionalità diverse: per ogni nazionalità $i \in \{1, 2, \dots, s\}$ ci sono $f_i \in \mathbb{Z}_{\geq 0}$ bambine e $g_i \in \mathbb{Z}_{\geq 0}$ bambini. In ogni campo il numero di bambine deve essere maggiore o uguale a quello dei bambini, e il numero di bambini della nazionalità $c \in \{1, 2, \dots, s\}$ deve essere maggiore o uguale a quello di ogni altra nazionalità. L'azienda vuole massimizzare il numero totale di bambini accettati.

Il problema a parole. Decidiamo quanti bambini di ciascuna nazionalità e di ciascun sesso mandare in ciascun campo. *L'obiettivo:* massimo numero di bambini accettati. *I vincoli:* non si superano né le disponibilità né le capacità; e in ogni campo valgono le due regole di composizione.

Dati (input del modello)

Simbolo	Tipo	Significato
s	$\in \mathbb{Z}_{\geq 1}$	numero di nazionalità, $i \in \{1, 2, \dots, s\}$
r	$\in \mathbb{Z}_{\geq 1}$	numero di campi, $j \in \{1, 2, \dots, r\}$
f_i	$\in \mathbb{Z}_{\geq 0}$	bambine disponibili della nazionalità i
g_i	$\in \mathbb{Z}_{\geq 0}$	bambini disponibili della nazionalità i
d_j	$\in \mathbb{Z}_{\geq 1}$	capacità del campo j
c	$\in \{1, 2, \dots, s\}$	nazionalità che deve essere maggioritaria

Variabili decisionali

Le variabili non sono binarie ma *conteggi*: $2rs$ variabili intere non negative,

x_{ij} = bambine della nazionalità i nel campo j ,

y_{ij} = bambini della nazionalità i nel campo j .

Modello 10.6 — campi estivi

$$\max \sum_{i=1}^s \sum_{j=1}^r (x_{ij} + y_{ij}) \quad (10.11a)$$

$$\text{soggetto a } \sum_{j=1}^r x_{ij} \leq f_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.11b)$$

$$\sum_{j=1}^r y_{ij} \leq g_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.11c)$$

$$\sum_{i=1}^s (x_{ij} + y_{ij}) \leq d_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11d)$$

$$\sum_{i=1}^s (x_{ij} - y_{ij}) \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11e)$$

$$x_{cj} + y_{cj} - \sum_{i \neq c} (x_{ij} + y_{ij}) \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11f)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.11g)$$

$$y_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}. \quad (10.11h)$$

Descrizione. I vincoli (10.11b) e (10.11c) limitano le disponibilità ($2s$ vincoli); i vincoli (10.11d) le capacità (r vincoli); i (10.11e) impongono «bambine $\geq$ bambini» in ogni campo (r vincoli); i (10.11f) impongono che la nazionalità c non sia in minoranza (r vincoli).

Il vincolo di maggioranza con più di due nazionalità

Con $s = 2$ nazionalità il vincolo «la nazionalità c non è meno di ogni altra» si scrive una volta sola: c'è una sola «altra» nazionalità. Con $s > 2$ il testo del problema chiede

$$x_{cj} + y_{cj} \geq x_{ij} + y_{ij} \quad \forall i \in \{1, 2, \dots, s\}, i \neq c, \forall j \in \{1, 2, \dots, r\},$$

cioè $(s - 1)r$ disuguaglianze. La forma (10.11f) scritta qui è *più forte*: impone che la nazionalità c non sia meno di *tutte le altre messe insieme*, cioè che occupi almeno metà dei posti di ogni campo. Le due letture coincidono per $s = 2$ e divergono per $s > 2$; la scelta va fatta esplicitamente, leggendo l'enunciato, non per comodità di scrittura. Qui teniamo la forma aggregata perché sull'istanza $s = 2$ e perché dà subito un argomento combinatorio (vedi più avanti).

I due bound che costruiamo qui sotto — quello primale, dall'euristica, e quello duale, dal rilassamento — sono ciò su cui lavora il solver.

Il modello in gurobipy

```

1 m = gp.Model("campi")
2 x = m.addVars(s, r, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(s, r, vtype=GRB.INTEGER, name="y")
4 m.setObjective(gp.quicksum(x[i, j] + y[i, j] for i in range(s) for j in
    range(r)),
5               GRB.MAXIMIZE)
6 m.addConstrs((x.sum(i, "*") <= f[i] for i in range(s)), name="bambine")
7 m.addConstrs((y.sum(i, "*") <= g[i] for i in range(s)), name="bambini")
8 m.addConstrs((gp.quicksum(x[i, j] + y[i, j] for i in range(s)) <= d[j]
9               for j in range(r)), name="capacita")
10 m.addConstrs((gp.quicksum(x[i, j] - y[i, j] for i in range(s)) >= 0
11               for j in range(r)), name="parita")
12 m.addConstrs((x[c, j] + y[c, j] - gp.quicksum(x[i, j] + y[i, j]
13               for i in range(s) if i != c) >= 0 for j in range(r)),
               name="maggioranza")

```

L'istanza

$s = 2$ nazionalità, $r = 2$ campi, $c = 1$:

	$i = 1$	$i = 2$		$j = 1$	$j = 2$
f_i	8	10	d_j	15	8
g_i	4	12			

In tutto ci sono 34 bambini disponibili e 23 posti.

Euristica costruttiva: il bound primale

Il problema è di massimo. Si riempie un campo per volta, prendendo prima la nazionalità maggioritaria (bambine e poi bambini) e poi le altre, e fermandosi appena uno dei tre vincoli si romperebbe.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza:

- **Campo 1** (capacità 15): si prendono tutte e 8 le bambine e tutti e 4 i bambini della nazionalità 1, poi 3 bambine della nazionalità 2. Il campo è pieno: 15 posti, 12 della nazionalità 1 contro 3 della 2 (maggioranza rispettata), 11 bambine contro 4 bambini (parità rispettata).
- **Campo 2** (capacità 8): la nazionalità 1 è esaurita, quindi qualunque bambino della nazionalità 2 violerebbe la maggioranza. Il campo resta vuoto.

$$z(MILP) \geq LB = 15.$$

Rilassamento LP e duale: il bound duale

Associamo $\alpha_i, \beta_i, \gamma_j \geq 0$ ai tre gruppi di vincoli $\leq$ e $\delta_j, \varepsilon_j \geq 0$ ai due gruppi di vincoli di composizione. Il duale del rilassamento LP è:

Duale del rilassamento LP del modello 10.6

$$\min \sum_{i=1}^s f_i \alpha_i + \sum_{i=1}^s g_i \beta_i + \sum_{j=1}^r d_j \gamma_j \quad (10.12a)$$

$$\text{soggetto a } \alpha_i + \gamma_j - \delta_j + \sigma_i \varepsilon_j \geq 1, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.12b)$$

$$\beta_i + \gamma_j + \delta_j + \sigma_i \varepsilon_j \geq 1, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.12c)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.12d)$$

$$\beta_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.12e)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12f)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12g)$$

$$\varepsilon_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12h)$$

Descrizione. Qui $\sigma_i = -1$ per $i = c$ e $\sigma_i = +1$ altrimenti. La variabile α_i è il prezzo di un posto per le bambine della nazionalità i e β_i quello per i bambini; γ_j è il prezzo di un posto nel campo j , δ_j il prezzo del vincolo di parità e ε_j quello del vincolo di maggioranza. L'obiettivo (10.12a) valuta a quei prezzi le disponibilità di ciascuna nazionalità e le capacità dei campi. I vincoli (10.12b) sono le colonne delle x_{ij} : accettare una bambina della nazionalità i nel campo j consuma un posto della sua nazionalità e uno del campo, alza di una unità la parità e sposta di σ_i la maggioranza; il valore complessivo deve coprire l'unità che quella bambina porta all'obiettivo primale. I vincoli (10.12c) dicono la stessa cosa per i bambini, con il segno della parità rovesciato.

Una soluzione duale a mano. La ricetta più semplice valuta la sola capacità: $\alpha = \beta = \delta = \varepsilon = 0$ e $\gamma_j = 1$ per ogni campo. Tutti i vincoli duali diventano $\gamma_j \geq 1$ e sono soddisfatti, e

$$UB = \sum_{j=1}^r d_j = 15 + 8 = 23.$$

La lettura è ovvia: ogni bambino accettato occupa un posto, quindi non se ne possono accettare più di quanti sono i posti. Ed è anche *ottima*: sul rilassamento senza i bound $z(LP) = 23$.

Altri due argomenti combinatori

Il bound 23 non è l'unico che si può leggere dai dati. Il vincolo (10.11f) dice che in ogni campo la nazionalità c occupa almeno metà dei posti; poiché di quella nazionalità ci sono in tutto $f_c + g_c = 12$ bambini, gli accettati sono al più $2 \cdot 12 = 24$. Analogamente il vincolo (10.11e) dice che in ogni campo le bambine sono almeno la metà, e di bambine ce ne sono 18: gli accettati sono al più $2 \cdot 18 = 36$.

Argomento	bound superiore
capacità dei campi	23
nazionalità maggioritaria	24
bambine disponibili	36

Su questa istanza vince la capacità, ma non è una regola: la variante 11a più avanti allarga il campo 1 e fa passare il comando alla nazionalità maggioritaria.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci.

	campo 1		campo 2	
	bambine	bambini	bambine	bambini
nazionalità 1	8	0	0	4
nazionalità 2	1	6	4	0
totale	15 su 15		8 su 8	

Entrambi i campi sono pieni: $z(MILP) = 23$, e la catena dei bound è

$$15 = LB \leq z(MILP) = 23 = z(LP) = UB.$$

Il gap dell'euristica è del 34,8%, e il bound duale chiude il problema: tutto il divario stava dal lato della soluzione, non del certificato. L'errore dell'euristica è chiaro: esaurisce la nazionalità maggioritaria nel primo campo, e nel secondo non resta nessuno che possa fare da maggioranza.

11.1: euristica 15 contro ottimo 23

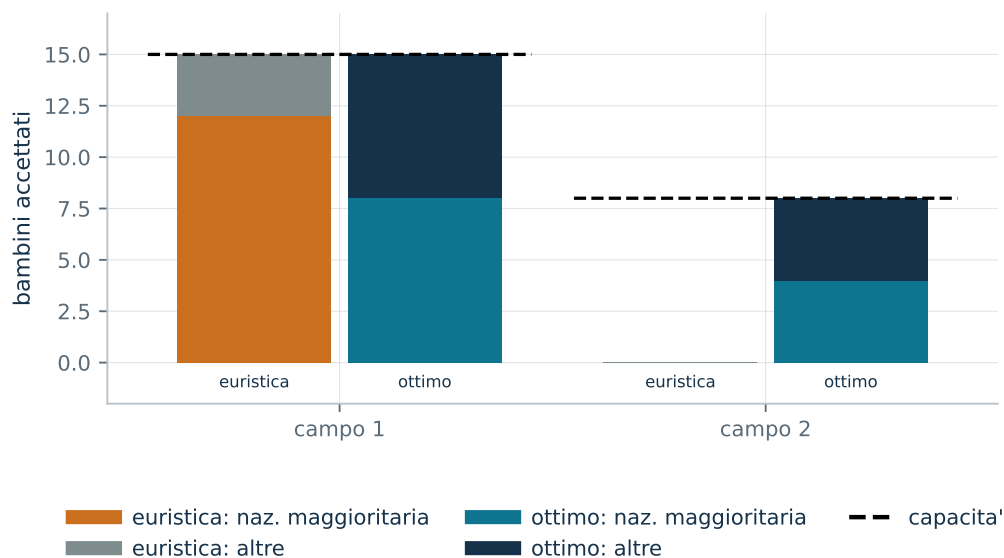


Figura 10.6: Bambini accettati per campo, divisi fra nazionalità maggioritaria e altre, nella soluzione euristica e in quella ottima; la linea tratteggiata è la capacità.

Considerazioni aggiuntive

- Le variabili sono intere ma non binarie: è la prima famiglia del corso in cui i conteggi hanno valori grandi, e il rilassamento resta comunque stretto perché i vincoli sono tutti di somma.
- Il vincolo di parità e quello di maggioranza sono *indipendenti*: si possono avere campi con molte bambine e poca nazionalità c , e viceversa. Sull'istanza è proprio il secondo a mordere.

- Se una nazionalità avesse zero bambini disponibili, il vincolo di maggioranza la escluderebbe automaticamente da ogni campo in cui compare qualcun altro: un caso limite che vale la pena controllare sui dati.

Domande di modellazione aggiuntive

Domanda di modellazione 10.11 — un campo più grande

Il campo 1 viene ampliato e arriva a 20 posti. Qual è il nuovo ottimo?

Domanda di modellazione 10.12 — una nazionalità non divisibile

Per motivi organizzativi i bambini della nazionalità 1 devono stare tutti nello stesso campo. Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
LB	15	soluzione euristica (LB)
UB	23	bound certificato (UB)
$z(MILP)$	23	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 34.8\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 34.8\%.$$

10.7 Suddivisione antitrust

Script: `python/fam10_7_antitrust.py`

[Notebook in Colab](#)

Problema 10.7 — suddivisione antitrust

Un'azienda ha $s \in \mathbb{Z}_{\geq 1}$ filiali e vende $r \in \mathbb{Z}_{\geq 1}$ prodotti. Per ogni filiale $i \in \{1, 2, \dots, s\}$ e ogni prodotto $j \in \{1, 2, \dots, r\}$, il valore $v_{ij} \in \mathbb{Q}_{\geq 0}$ è il fatturato in milioni di euro generato da quella filiale con quel prodotto. Per effetto di una nuova norma antitrust l'azienda deve dividersi in due società più piccole; ogni filiale è indivisibile e va assegnata a esattamente una delle due. L'azienda vuole partizionare le filiali minimizzando la massima differenza, fra tutti i prodotti, dei fatturati delle due nuove società.

Il problema a parole. Decidiamo a quale delle due società va ciascuna filiale. *L'obiettivo:* rendere le due società il più simili possibile, misurando la somiglianza sul prodotto in cui vanno peggio. *Il vincolo:* ogni filiale a una sola società.

Variabili decisionali

Basta *una* famiglia di s binarie:

$$x_i = \begin{cases} 1 & \text{se la filiale } i \text{ va alla societ\`a } A, \\ 0 & \text{se va alla societ\`a } B, \end{cases} \quad \forall i \in \{1, 2, \dots, s\},$$

più una variabile libera z per il min-max.

Una famiglia o due?

L'enunciato classico introduce *due* famiglie, x_i e y_i , con il vincolo $x_i + y_i = 1$. Le due scritture sono equivalenti, perché $y_i = 1 - x_i$: sostituendo si ottiene esattamente il modello qui sotto, con s variabili e s vincoli in meno. Conviene tenere la forma aggregata finché le società sono *due*; con tre o più società la forma disaggregata $\sum_{k=1}^2 x_{ik} = 1$ è l'unica possibile, ed è quella che si estende. Riconoscere quando due modelli sono la stessa cosa è parte del mestiere: qui la verifica è una sostituzione, in altri casi (problema 9.2) richiede una dimostrazione.

Con $T_j = \sum_{i=1}^s v_{ij}$ il fatturato totale sul prodotto j , il fatturato di A è $\sum_{i=1}^s v_{ij} x_i$ e quello di B è $T_j - \sum_{i=1}^s v_{ij} x_i$: la loro differenza è $2 \sum_{i=1}^s v_{ij} x_i - T_j$.

Modello 10.7 — suddivisione antitrust

$$\min \quad z \quad (10.13a)$$

$$\text{soggetto a} \quad z - 2 \sum_{i=1}^s v_{ij} x_i + T_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.13b)$$

$$z + 2 \sum_{i=1}^s v_{ij} x_i - T_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.13c)$$

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.13d)$$

$$z \geq 0. \quad (10.13e)$$

Descrizione. L'obiettivo (10.13a) è lo squilibrio peggiore. I vincoli (10.13b) e (10.13c), uno per prodotto ciascuno, dicono che z non è più piccolo dello squilibrio del prodotto j né del suo opposto: insieme impongono $z \geq |2 \sum_i v_{ij} x_i - T_j|$, e il minimo li rende un'uguaglianza sul prodotto peggiore.

Valore assoluto e min-max in due righe

Le due disuguaglianze (10.13b) e (10.13c) dicono

$$-z \leq 2 \sum_{i=1}^s v_{ij} x_i - T_j \leq z,$$

cioè $z \geq |2 \sum_{i=1}^s v_{ij} x_i - T_j|$ per ogni prodotto j . Poiché z compare da sola nell'obiettivo di minimo, all'ottimo vale esattamente il massimo dei valori assoluti: è la tecnica 3.6 (min-max) applicata alla tecnica 3.7 (valore assoluto). Il verso « z non è più grande del necessario» non

è imposto da alcun vincolo, ma dall'ottimalità: se z fosse più grande, si potrebbe abbassarlo restando ammissibili.

Il modello in gurobipy

```

1 m = gp.Model("antitrust")
2 x = m.addVars(s, vtype=GRB.BINARY, name="x")
3 z = m.addVar(lb=-GRB.INFINITY, name="z")
4 m.setObjective(z, GRB.MINIMIZE)
5 for j in range(r):
6     tot = sum(v[i][j] for i in range(s))
7     m.addConstr(z - 2 * gp.quicksum(v[i][j] * x[i] for i in range(s)) +
8                 tot >= 0)
9     m.addConstr(z + 2 * gp.quicksum(v[i][j] * x[i] for i in range(s)) -
10                 tot >= 0)

```

L'istanza

$s = 4$ filiali, $r = 3$ prodotti:

v_{ij}	$j = 1$	$j = 2$	$j = 3$	totale filiale
$i = 1$	3	3	2	8
$i = 2$	6	8	5	19
$i = 3$	3	4	4	11
$i = 4$	2	7	9	18
T_j	14	22	20	

Euristica costruttiva: il bound primale

Si assegnano le filiali in ordine di fatturato complessivo decrescente, ciascuna alla società che al momento fattura meno in totale. È l'analogo della LPT per lo scheduling.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

Sull'istanza i totali sono 8, 19, 11, 18, quindi l'ordine è 2, 4, 3, 1: la filiale 2 ad A ($A = 19$), la 4 a B ($B = 18$), la 3 a B ($B = 29$), la 1 ad A ($A = 27$). Si ottiene $A = \{1, 2\}$, $B = \{3, 4\}$, con differenze 4, 0 e 6 sui tre prodotti:

$$z(MILP) \leq UB = 6.$$

Rilassamento LP e duale: il bound vale zero

Associamo $\lambda_j \geq 0$ ai vincoli (10.13b) e $\mu_j \geq 0$ a quelli (10.13c). La colonna di z (variabile libera) dà un vincolo di uguaglianza:

Duale del rilassamento LP del modello 10.7

$$\max \sum_{j=1}^r T_j (\mu_j - \lambda_j) \quad (10.14a)$$

$$\text{soggetto a } \sum_{j=1}^r (\lambda_j + \mu_j) = 1, \quad (10.14b)$$

$$2 \sum_{j=1}^r v_{ij} (\mu_j - \lambda_j) \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.14c)$$

$$\lambda_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.14d)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}. \quad (10.14e)$$

Descrizione. Le variabili λ_j e μ_j sono i prezzi dei due vincoli che stringono lo squilibrio del prodotto j , uno da sopra e uno da sotto. L'obiettivo (10.14a) valuta a quei prezzi il totale T_j di ciascun prodotto. Il vincolo (10.14b) è la colonna di z : la variabile compare in tutti i vincoli con coefficiente 1 e nell'obiettivo primale con costo 1, quindi i prezzi dei $2r$ vincoli devono sommare esattamente a uno — il duale distribuisce un'unica unità di peso fra i prodotti. I vincoli (10.14c) sono le colonne delle x_i : spostare la filiale i da una società all'altra cambia di $2v_{ij}$ lo squilibrio del prodotto j , e la somma pesata di questi spostamenti non può essere positiva, perché le x_i non compaiono nell'obiettivo.

Una soluzione duale a mano Prendiamo $\bar{\lambda}_1 = \bar{\mu}_1 = 1/2$ e tutto il resto zero: il vincolo (10.14b) è soddisfatto, $\mu_j - \lambda_j = 0$ per ogni j e quindi anche i vincoli (10.14c). Il valore è $LB = 0$.

Nessuna soluzione duale ammissibile vale più di zero. Infatti, posto $\theta_j = \mu_j - \lambda_j$, l'obiettivo è $\sum_{j=1}^r T_j \theta_j$ e i vincoli (10.14c) impongono $\sum_{j=1}^r v_{ij} \theta_j \leq 0$ per ogni filiale; sommando su tutte le filiali si ottiene $\sum_{j=1}^r T_j \theta_j \leq 0$. Il duale vale dunque al più 0, e per dualità forte $z(LP) = 0$.

Il certificato primale è ancora più semplice: la soluzione frazionaria $x_i = 1/2$ per ogni filiale con $z = 0$ è ammissibile per il rilassamento e pareggia esattamente ogni prodotto. Mezza filiale a ciascuna società: legittimo per l'LP, privo di senso per il problema.

Un bound combinatorio, prodotto per prodotto

Se il rilassamento non dice niente, bisogna cercare altrove. Per ogni prodotto j si può calcolare, guardando *quel solo prodotto*, il minimo squilibrio ottenibile:

$$g_j = \min_{S \subseteq \{1, 2, \dots, s\}} \left| 2 \sum_{i \in S} v_{ij} - T_j \right|.$$

È il classico problema della partizione su una sola colonna, e con s piccolo si risolve per enumerazione (2^s sottoinsiemi). Ogni partizione ammissibile del problema completo è in particolare una partizione per il prodotto j , quindi il suo squilibrio sul prodotto j è almeno g_j ; poiché l'obiettivo è il massimo su tutti i prodotti,

$$z(MILP) \geq \max_j g_j.$$

Sull'istanza:

Prodotto	totale T_j	g_j
1	14	2
2	22	0
3	20	2

(sul prodotto 1 i valori sono 3, 6, 3, 2: la somma 7 non è raggiungibile da alcun sottoinsieme, e il meglio è 6 contro 8, cioè $g_1 = 2$). Quindi

$$z(MILP) \geq LB = 2,$$

un bound che il rilassamento LP non vede, perché nasce dall'interezza e non dai vincoli.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci. La partizione ottima è $A = \{2, 3\}$ e $B = \{1, 4\}$, con differenze 4, 2 e 2 sui tre prodotti:

$$z(MILP) = 4,$$

e la catena dei bound è

$$2 = LB \leq z(MILP) = 4 \leq UB = 6, \quad z(LP) = z(LP^+) = 0.$$

Il gap dell'euristica è del 50%. Si noti che LB non è il valore del duale (0) ma il bound combinatorio: nella tabella riassuntiva del capitolo la colonna si chiama «bound certificato» proprio per questo.

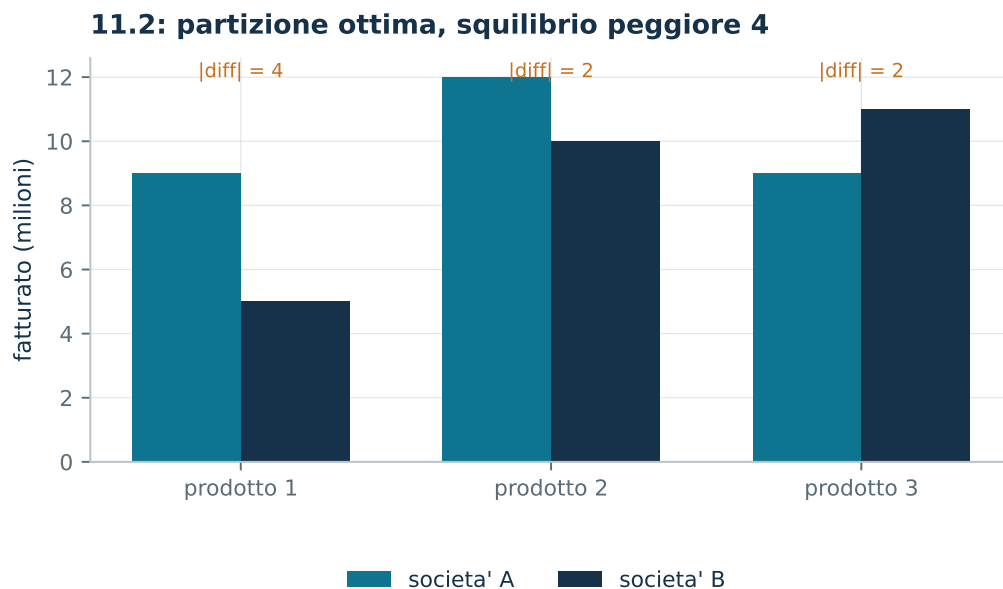


Figura 10.7: Fatturato delle due società ottime, prodotto per prodotto, con la differenza in evidenza.

Considerazioni aggiuntive

- Il problema con un solo prodotto è il *number partitioning*, uno dei ventuno problemi NP-completi di Karp. Con più prodotti è la sua versione vettoriale, e resta NP-difficile.
- Il bound combinatorio si può rafforzare: invece del massimo dei g_j si può cercare, per ogni coppia di prodotti, il minimo del massimo dei due squilibri. Il costo cresce ma il bound migliora, ed è la stessa idea del *surrogate relaxation*.
- L'obiettivo min-max non è l'unico possibile: minimizzare la somma delle differenze è un'altra scelta legittima e dà un modello con r variabili ausiliarie invece di una (vedi la domanda 10.9).

Domande di modellazione aggiuntive

Domanda di modellazione 10.13 — due filiali inseparabili

Le filiali 1 e 2 condividono la sede e devono restare nella stessa società. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 10.14 — min-sum invece di min-max

Si vuole minimizzare la *somma* delle differenze su tutti i prodotti invece della differenza peggiore. Come cambia il modello? La partizione ottima è la stessa?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	6	soluzione euristica (UB)
LB	2	bound certificato (LB)
$z(MILP)$	4	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 50.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 100.0\%.$$

10.8 Distribuzione dei brani sui CD

Script: `python/fam10_8_cd.py`

[Notebook in Colab](#)

Problema 10.8 — brani sui CD

Una casa discografica pubblica una raccolta di $n \in \mathbb{Z}_{\geq 1}$ brani. Per ogni brano $i \in \{1, 2, \dots, n\}$, il valore $d_i \in \mathbb{Q}_{\geq 1}$ è la durata in minuti. La raccolta va distribuita su $m \in \mathbb{Z}_{\geq 1}$

compact disc; ogni brano va assegnato a esattamente un CD, e ogni CD $j \in \{1, 2, \dots, m\}$ deve contenere almeno $w_j \in \mathbb{Z}_{\geq 1}$ brani. La casa discografica vuole minimizzare la differenza fra la durata complessiva del CD più lungo e quella del più corto.

Il problema a parole. Decidiamo su quale CD finisce ogni brano. *L'obiettivo:* CD il più possibile omogenei per durata. *I vincoli:* ogni brano su un CD solo, e nessun CD troppo scarso.

Variabili decisionali

nm variabili binarie e due variabili continue non negative:

$$x_{ij} = \begin{cases} 1 & \text{se il brano } i \text{ va sul CD } j, \\ 0 & \text{altrimenti,} \end{cases}$$

y = durata del CD più lungo, z = durata del CD più corto.

Modello 10.8 — brani sui CD

$$\min \quad y - z \quad (10.15a)$$

$$\text{soggetto a} \quad \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\}, \quad (10.15b)$$

$$\sum_{i=1}^n x_{ij} \geq w_j, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15c)$$

$$-\sum_{i=1}^n d_i x_{ij} + y \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15d)$$

$$\sum_{i=1}^n d_i x_{ij} - z \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15e)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15f)$$

$$y, z \geq 0. \quad (10.15g)$$

Descrizione. L'obiettivo (10.15a) è la differenza fra la durata più lunga e la più corta. I vincoli (10.15b), uno per brano, dicono che ogni brano finisce su esattamente un CD. I vincoli (10.15c), uno per CD, impongono il numero minimo di brani. I vincoli (10.15d), uno per CD, spingono y sopra ogni durata, e i vincoli (10.15e), sempre uno per CD, spingono z sotto ogni durata: nel minimo la prima diventa il massimo delle durate e la seconda il minimo.

Due variabili ausiliarie, due versi diversi

I vincoli (10.15d) dicono $y \geq \sum_{i=1}^n d_i x_{ij}$ per ogni CD: y è *almeno* il massimo. I vincoli (10.15e) dicono $z \leq \sum_{i=1}^n d_i x_{ij}$ per ogni CD: z è *al più* il minimo. L'obiettivo $\min(y - z)$ spinge y verso il basso e z verso l'alto, quindi all'ottimo y vale esattamente la durata massima e z esattamente la minima. È la tecnica 3.5 usata due volte, in due versi opposti: si noti che serve un obiettivo che spinga nella direzione giusta *entrambe* le variabili, altrimenti

una delle due resterebbe indeterminata.

Il modello in gurobipy

```

1 mod = gp.Model("cd")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVar(name="y")
4 z = mod.addVar(name="z")
5 mod.setObjective(y - z, GRB.MINIMIZE)
6 mod.addConstrs((x.sum(i, "*") == 1 for i in range(n)), name="brano")
7 mod.addConstrs((x.sum("*, j) >= w[j] for j in range(m)), name="minimo"
8 )
9 mod.addConstrs((y - gp.quicksum(d[i] * x[i, j] for i in range(n)) >= 0
10 for j in range(m)), name="massimo")
11 mod.addConstrs((gp.quicksum(d[i] * x[i, j] for i in range(n)) - z >= 0
for j in range(m)), name="minimo_durata")

```

L'istanza

$n = 6$ brani, $m = 2$ CD, $w_j = 1$:

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$
d_i	5	6	7	3	4	10

La durata totale è 35 minuti.

Due euristiche a confronto: il bound primale

La regola di inserimento è sempre la stessa — ogni brano sul CD attualmente più corto — ma l'ordine in cui si scorrono i brani cambia molto il risultato.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

(a) Ordine naturale. $5 \rightarrow \text{CD1}$, $6 \rightarrow \text{CD2}$, $7 \rightarrow \text{CD1}$, $3 \rightarrow \text{CD2}$, $4 \rightarrow \text{CD2}$, $10 \rightarrow \text{CD1}$. Durate finali 22 e 13: differenza 9.

(b) Ordine per durata decrescente (LPT). $10 \rightarrow \text{CD1}$, $7 \rightarrow \text{CD2}$, $6 \rightarrow \text{CD2}$, $5 \rightarrow \text{CD1}$, $4 \rightarrow \text{CD2}$, $3 \rightarrow \text{CD1}$. Durate finali 18 e 17: differenza 1.

La stessa regola, con due ordini diversi, dà 9 e 1. Si tiene la migliore:

$$z(\text{MILP}) \leq UB = 1.$$

Rilassamento LP e duale: di nuovo zero

Associamo α_i libera ai vincoli (10.15b) (uguaglianze), $\beta_j \geq 0$ a (10.15c), $\gamma_j \geq 0$ a (10.15d) e $\delta_j \geq 0$ a (10.15e). Le colonne di y e di z danno $\sum_{j=1}^m \gamma_j = 1$ e $\sum_{j=1}^m \delta_j = 1$.

Duale del rilassamento LP del modello 10.8

$$\max \sum_{i=1}^n \alpha_i + \sum_{j=1}^m w_j \beta_j \quad (10.16a)$$

$$\text{soggetto a } \sum_{j=1}^m \gamma_j = 1, \quad (10.16b)$$

$$\sum_{j=1}^m \delta_j = 1, \quad (10.16c)$$

$$\alpha_i + \beta_j - d_i \gamma_j + d_i \delta_j \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (10.16d)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \quad (10.16e)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.16f)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.16g)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}. \quad (10.16h)$$

Descrizione. La variabile α_i è il valore del brano i , β_j il prezzo del numero minimo di brani sul CD j , mentre γ_j e δ_j sono i pesi con cui il CD j entra rispettivamente nella durata massima e in quella minima. L'obiettivo (10.16a) valuta i brani e le soglie w_j . I vincoli (10.16b) e (10.16c) sono le colonne di y e di z : ciascuna delle due variabili compare in un vincolo per CD e nell'obiettivo primale con coefficiente ± 1 , quindi i rispettivi pesi devono sommare a uno. I vincoli (10.16d) sono le colonne delle x_{ij} : mettere il brano i sul CD j soddisfa il suo vincolo di assegnamento (α_i), contribuisce al minimo di brani (β_j) e sposta di d_i le due durate; il saldo non può essere positivo, perché le x_{ij} non compaiono nell'obiettivo.

Una soluzione duale a mano. Con $\bar{\gamma}_j = \bar{\delta}_j = 1/m$ e $\alpha = \beta = 0$ i due vincoli di uguaglianza sono soddisfatti e i vincoli (10.16d) diventano $-d_i/m + d_i/m = 0 \leq 0$. Il valore è $LB = 0$.

Che il rilassamento valga zero si vede meglio dal lato primale: mettendo $x_{ij} = 1/m$ per ogni brano e ogni CD, ogni CD «dura» $35/2 = 17,5$ minuti e la differenza è nulla. È ammissibile per il rilassamento e privo di senso per il problema, perché un brano non si spezza. Il rilassamento LP dei modelli di bilanciamento è quasi sempre così: pareggia tutto e vale zero.

Un argomento di parità che chiude il problema

Le durate sono numeri interi e i CD sono due: le loro durate q_1 e q_2 sono interi che sommano a $q = 35$, che è *dispari*. Due interi che sommano a un numero dispari non possono essere uguali, e la loro differenza $|q_1 - q_2|$ ha la stessa parità di q , cioè è dispari. Una differenza dispari e non negativa vale almeno 1:

$$z(MILP) \geq LB = 1.$$

L'euristica LPT raggiunge esattamente 1: i due bound coincidono, e la soluzione euristica è ottima. Lo si sa *prima* di chiamare il solver, ed è il caso più netto del corso in cui un argomento combinatorio di due righe fa il lavoro che il rilassamento LP non riesce a fare.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci.

	brani	durata
CD 1	1, 2, 3	18
CD 2	4, 5, 6	17

$$1 = LB \leq z(MILP) = 1 \leq UB = 1, \quad z(LP) = z(LP^+) = 0.$$

Il gap dell'euristica è nullo. Si noti che la soluzione trovata dal solver non è quella dell'euristica LPT (che metteva i brani 6, 1, 4 su un CD e 3, 2, 5 sull'altro), ma ha lo stesso valore: il problema ha più ottimi.

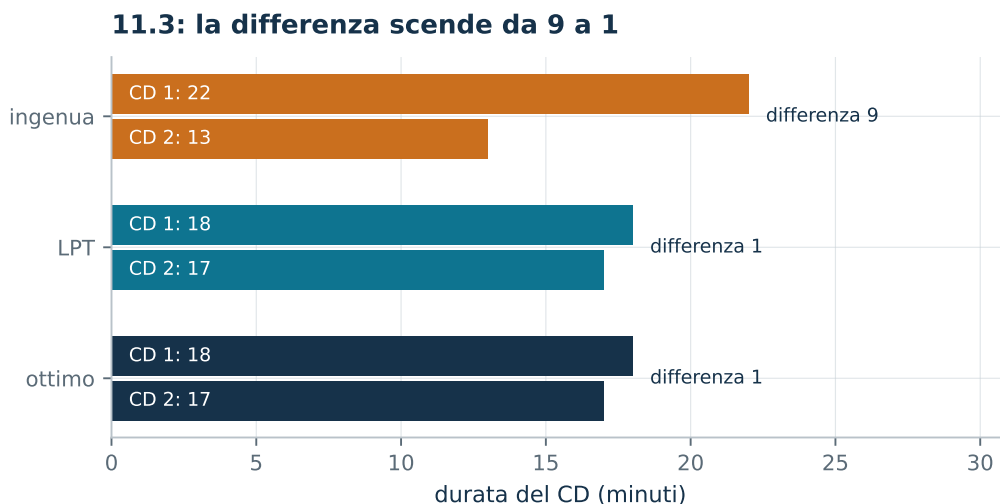


Figura 10.8: Durata dei due CD nelle tre soluzioni: euristica ingenua, LPT e ottimo.

Considerazioni aggiuntive

- L'argomento di parità vale così com'è per $m = 2$. Con $m = 3$ va rifatto: 35 non è divisibile per 3, quindi le tre durate non possono essere tutte uguali, ma il minimo scarto non è più immediato. Sull'istanza con tre CD l'ottimo vale 2 (durate 13, 11, 11).
- Il vincolo $\sum_{i=1}^n x_{ij} \geq w_j$ con $w_j = 1$ non morde mai all'ottimo: una soluzione che lasciasse un CD vuoto avrebbe $z = 0$ e quindi differenza pari alla durata totale. Serve però nel modello, perché senza di esso il rilassamento avrebbe soluzioni degeneri.
- Minimizzare $y - z$ non è equivalente a minimizzare y (il makespan): la seconda è la formulazione classica dello scheduling su macchine parallele, e ha ottimi diversi. È la stessa differenza fra «bilanciare» e «finire presto».

Domande di modellazione aggiuntive

Domanda di modellazione 10.15 — un supporto più piccolo

Il CD 1 è un supporto ridotto e non può superare i 15 minuti. Come cambia il modello? Qual è il nuovo ottimo?

Domanda di modellazione 10.16 — tre CD

La raccolta si distribuisce su tre CD invece che su due. Come cambia il modello? Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	1	soluzione euristica (UB)
LB	1	bound certificato (LB)
$z(MILP)$	1	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 0.0\%.$$

10.9 Libri sugli scaffali

Script: `python/fam10_9_scaffali.py`

[Notebook in Colab](#)

Problema 10.9 — libri sugli scaffali

Una biblioteca deve sistemare $n \in \mathbb{Z}_{\geq 1}$ libri su degli scaffali. Ogni libro $b \in \{1, 2, \dots, n\}$ ha larghezza $w_b \in \mathbb{Z}_{\geq 1}$ e altezza $h_b \in \mathbb{Z}_{\geq 1}$. Sono disponibili $m \in \mathbb{Z}_{\geq 1}$ scaffali, ciascuno di larghezza massima $c \in \mathbb{Q}_{>0}$. Ogni libro va assegnato a esattamente uno scaffale, e la larghezza complessiva dei libri su uno scaffale non può superare c . L'altezza di uno scaffale è quella del libro più alto che vi si trova. La biblioteca vuole minimizzare la somma delle altezze degli scaffali.

Il problema a parole. Decidiamo su quale scaffale va ogni libro. L'obiettivo: somma delle altezze minima (è il legno che si risparmia). I vincoli: ogni libro su uno scaffale solo, e nessuno scaffale più largo di c .

Variabili decisionali

n variabili binarie e m variabili continue non negative:

$$x_{bs} = \begin{cases} 1 & \text{se il libro } b \text{ va sullo scaffale } s, \\ 0 & \text{altrimenti,} \end{cases} \quad y_s = \text{altezza dello scaffale } s.$$

Modello 10.9 — libri sugli scaffali

$$\min \sum_{s=1}^m y_s \quad (10.17a)$$

$$\text{soggetto a } \sum_{s=1}^m x_{bs} = 1, \quad \forall b \in \{1, 2, \dots, n\}, \quad (10.17b)$$

$$\sum_{b=1}^n w_b x_{bs} \leq c, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.17c)$$

$$-h_b x_{bs} + y_s \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.17d)$$

$$x_{bs} \in \{0, 1\}, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.17e)$$

$$y_s \geq 0, \quad \forall s \in \{1, 2, \dots, m\}. \quad (10.17f)$$

Descrizione. L'obiettivo (10.17a) è l'altezza complessiva degli scaffali. I vincoli (10.17b), uno per libro, dicono che ogni libro sta su esattamente uno scaffale. I vincoli (10.17c), uno per scaffale, sono la larghezza disponibile. I vincoli (10.17d), uno per coppia libro-scaffale, spingono l'altezza y_s sopra quella di ogni libro sistemato su quello scaffale.

Il legame fra le variabili. I vincoli (10.17d) dicono $y_s \geq h_b x_{bs}$: se il libro b è sullo scaffale s allora $y_s \geq h_b$, altrimenti il vincolo è vuoto ($y_s \geq 0$). Poiché y_s compare nell'obiettivo di minimo con coefficiente positivo, all'ottimo vale esattamente il massimo delle altezze dei libri presenti, e vale 0 se lo scaffale è vuoto. È la tecnica 3.5, la variabile di massimo, nella sua forma disaggregata: un vincolo per libro invece di uno solo con un big-M.

Uno scaffale vuoto non costa

Con $y_s \geq 0$ e nessun altro vincolo, uno scaffale vuoto ha $y_s = 0$ e non contribuisce all'obiettivo. Ne segue che dare alla biblioteca più scaffali non può mai peggiorare l'ottimo, ma non è detto che lo migliori: spezzare i libri su più scaffali fa pagare più altezze. Sull'istanza, con tre scaffali invece di due, l'ottimo resta 15 (vedi la domanda 10.12).

Il modello in gurobipy

```
1 mod = gp.Model("scaffali")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, name="y")
4 mod.setObjective(y.sum(), GRB.MINIMIZE)
5 mod.addConstrs((x.sum(b, "*") == 1 for b in range(n)), name="libro")
6 mod.addConstrs((gp.quicksum(w[b] * x[b, s] for b in range(n)) <= c
```

```

7         for s in range(m)), name="larghezza")
8 mod.addConstrs((y[s] - h[b] * x[b, s] >= 0 for b in range(n) for s in
    range(m)),
9             name="altezza")

```

L'istanza

$n = 4$ libri, $m = 2$ scaffali, $c = 10$:

	$b = 1$	$b = 2$	$b = 3$	$b = 4$
w_b	3	5	4	6
h_b	8	5	7	4

La larghezza totale dei libri è 18, la capacità complessiva $2 \cdot 10 = 20$.

Euristica costruttiva: due ordini, due esiti

La regola è first-fit: ogni libro sul primo scaffale in cui entra. Come nel problema 10.8, l'ordine cambia tutto — ma qui, in un ordine, l'euristica *fallisce*.

È un'euristica costruttiva della famiglia vista nel capitolo 5: si costruisce una sola soluzione, un elemento per volta, senza mai tornare indietro. Qui sotto la sua esecuzione sull'istanza.

(a) Ordine per altezza decrescente (1, 3, 2, 4). Libro 1 (largo 3) sullo scaffale 1; libro 3 (largo 4) sullo scaffale 1, che arriva a 7; libro 2 (largo 5) sullo scaffale 2. Resta il libro 4, largo 6: sullo scaffale 1 ci sono 3 di residuo, sul 2 ce ne sono 5. **L'euristica fallisce.**

(b) Ordine per larghezza decrescente (4, 2, 3, 1). Libro 4 (largo 6) sullo scaffale 1; libro 2 (largo 5) sullo scaffale 2; libro 3 (largo 4) sullo scaffale 1, che arriva a 10; libro 1 (largo 3) sullo scaffale 2, che arriva a 8. Altezze: $\max(4, 7) = 7$ e $\max(5, 8) = 8$, somma 15.

$$z(MILP) \leq UB = 15.$$

L'ordine per altezza decrescente è quello suggerito dall'*obiettivo*, ma il vincolo che può rendere infattibile un inserimento è la *larghezza*: è su quella che va ordinato. È la stessa logica del first-fit decreasing per il bin packing, dove si ordina per dimensione e non per valore. Un'euristica costruttiva che può fallire non è di per sé sbagliata — basta prevedere il caso e cambiare ordine — ma va detto esplicitamente, perché un'euristica «che a volte non restituisce niente» non fornisce alcun bound.

Rilassamento LP e duale: il bound duale

Associamo α_b libera ai vincoli (10.17b) (uguaglianze), $\beta_s \leq 0$ a (10.17c) (verso $\leq$ in un minimo) e $\gamma_{bs} \geq 0$ a (10.17d). Il duale è:

Duale del rilassamento LP del modello 10.9

$$\max \sum_{b=1}^n \alpha_b + c \sum_{s=1}^m \beta_s \quad (10.18a)$$

$$\text{soggetto a } \sum_{b=1}^n \gamma_{bs} \leq 1, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.18b)$$

$$\alpha_b + w_b \beta_s - h_b \gamma_{bs} \leq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.18c)$$

$$\alpha_b \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \quad (10.18d)$$

$$\beta_s \leq 0, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.18e)$$

$$\gamma_{bs} \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}. \quad (10.18f)$$

Descrizione. La variabile α_b è il valore del libro b , β_s il prezzo (non positivo) della larghezza dello scaffale s e γ_{bs} il prezzo del legame «l'altezza dello scaffale s copre il libro b ». L'obiettivo (10.18a) valuta i libri e la larghezza disponibile su ciascuno scaffale. I vincoli (10.18b) sono le colonne delle y_s : l'altezza dello scaffale s costa 1 nell'obiettivo primale, e i prezzi dei legami che la spingono verso l'alto non possono valere di più. I vincoli (10.18c) sono le colonne delle x_{bs} : mettere il libro b sullo scaffale s soddisfa il suo vincolo di assegnamento (α_b), occupa w_b di larghezza e costringe l'altezza a salire fino a h_b ; il saldo non può essere positivo, perché le x_{bs} non compaiono nell'obiettivo.

Una soluzione duale a mano. Poniamo $\beta = 0$ (la larghezza non si valuta) e concentriamo tutto il «peso» γ sul libro più alto, il 1 con $h_1 = 8$: $\bar{\gamma}_{1s} = 1$ per ogni s e $\bar{\gamma}_{bs} = 0$ per $b \neq 1$. I vincoli (10.18b) diventano $1 \leq 1$. I vincoli (10.18c) diventano $\alpha_1 \leq 8$ per $b = 1$ e $\alpha_b \leq 0$ per $b \neq 1$: prendiamo $\bar{\alpha}_1 = 8$ e $\bar{\alpha}_b = 0$. Il valore è

$$LB = 8.$$

È l'osservazione ovvia: lo scaffale che ospita il libro più alto è alto almeno quanto lui.

Un bound combinatorio più forte

La larghezza totale dei libri è 18 e ogni scaffale ne regge 10: servono almeno

$$\left\lceil \frac{18}{10} \right\rceil = 2$$

scaffali non vuoti. Uno di essi ospita il libro più alto e misura almeno 8; l'altro contiene almeno un libro, quindi misura almeno $\min_{b \neq 1} h_b = 4$. Sommando,

$$z(MILP) \geq LB = 8 + 4 = 12,$$

meglio del bound duale 8. Anche qui il salto viene dall'interrezza: il rilassamento può mettere metà libro su ciascuno scaffale e pagare metà altezza due volte, cioè in totale ancora 8.

Soluzione ottima

L'ottimo si ottiene risolvendo il modello: i due bound appena costruiti, il primale e il duale, sono ciò che aiuta il solver ad arrivarci.

	libri	larghezza	altezza
scaffale 1	1, 2	8 su 10	8
scaffale 2	3, 4	10 su 10	7

$$z(MILP) = 15, \quad 12 = LB \leq z(MILP) = 15 \leq UB = 15, \quad z(LP) = z(LP^+) = 8.$$

Il gap dell'euristica è nullo: la first-fit per larghezza decrescente trova l'ottimo. Il gap certificato, prima di risolvere il MILP, è $(15 - 12)/15 = 20\%$.

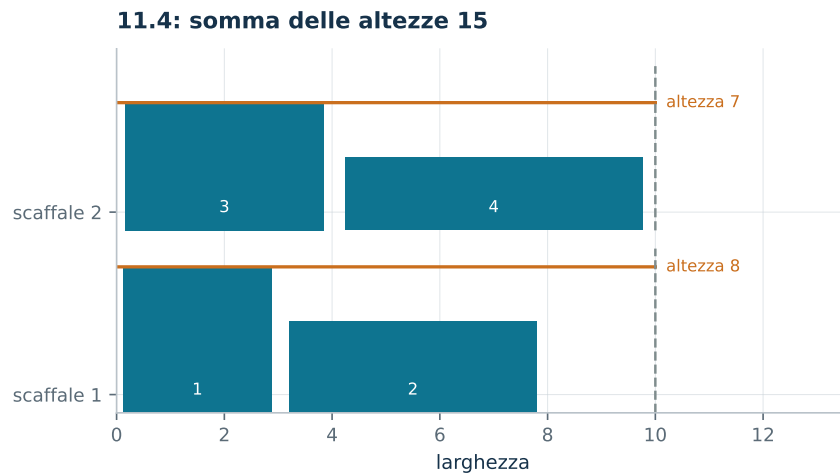


Figura 10.9: Disposizione ottima: i libri su ciascuno scaffale, con l'altezza dello scaffale segnata in arancio e la larghezza massima tratteggiata.

Considerazioni aggiuntive

- Il problema è una variante del bin packing in cui il costo di un contenitore non è fisso ma dipende dal contenuto. È noto come *bin packing with item fragmentation* oppure, nella letteratura sui magazzini, come *shelf space allocation*.
- Il modello ha una simmetria fastidiosa: scambiando i due scaffali si ottiene la stessa soluzione con valore uguale. Su istanze più grandi conviene romperla, per esempio imponendo $y_1 \geq y_2 \geq \dots \geq y_m$.
- La forma disaggregata (10.17d) usa nm vincoli. La forma aggregata con big-M, $y_s \geq h_b - M(1 - x_{bs})$, ne userebbe altrettanti ed è più debole: qui la disaggregata non costa nulla ed è preferibile.

Domande di modellazione aggiuntive

Domanda di modellazione 10.17 — un terzo scaffale

La biblioteca compra un terzo scaffale, largo come gli altri. Qual è il nuovo ottimo?

Domanda di modellazione 10.18 — scaffali più larghi

Gli scaffali sono larghi 12 invece di 10. Qual è il nuovo ottimo?

Il confronto finale

I tre numeri che ogni problema del corso produce, messi uno accanto all'altro:

	valore	che cos'è
UB	15	soluzione euristica (UB)
LB	12	bound certificato (LB)
$z(MILP)$	15	ottimo del MILP

- Gap dell'euristica rispetto all'ottimo:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Gap certificato, cioè quello che si conosce *prima* di risolvere il MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 20.0\%.$$

10.10 Due problemi da modellare

Gli otto problemi precedenti arrivano al lettore già sezionati: variabili, modello, istanza, bound. I due che chiudono il capitolo no. Sono dati come arrivano davvero — un testo, dei dati, una domanda — e il lavoro di riconoscere la struttura resta da fare.

I passi sono sempre gli stessi:

1. si elencano le **decisioni**, una per una, e si dà a ciascuna una variabile con il suo dominio;
2. si traduce ogni frase dell'enunciato in **un vincolo**, e si conta quanti vincoli sono in funzione dei dati;
3. si controllano i **legami** fra variabili con la tavola di verità;
4. si costruisce a mano una **soluzione ammissibile** e una **soluzione duale**, per avere i due bound;
5. si risolve il modello e si **confronta**.

Le soluzioni di queste domande, come tutte le altre del corso, stanno nel documento riservato ai docenti e non sono pubblicate.

Il primo: un deposito e i suoi furgoni

Problema 10.10 — la settimana del deposito

Un deposito rifornisce tre punti vendita per cinque giorni. Ogni giorno ogni punto vendita ha una richiesta nota, in pallet. Il deposito ha una giacenza iniziale e può ricevere pallet dalla fabbrica con un ordine giornaliero: un ordine costa una cifra fissa per l'attivazione del trasporto, più una cifra per pallet. I pallet non consegnati restano in deposito, e la giacenza a fine giornata costa. Ogni consegna a un punto vendita viaggia su un furgone che porta al più un numero dato di pallet, e ogni furgone che parte costa. In un giorno il deposito non può far partire più di un numero dato di furgoni in totale. Si vuole il piano settimanale di

costo minimo.

Domanda di modellazione 10.19 — le decisioni e le variabili

Si elenchino le decisioni del problema e, per ciascuna, il tipo di variabile che le corrisponde. Quante famiglie di variabili servono, e con quali indici?

Domanda di modellazione 10.20 — il modello

Si scriva un modello MILP completo. Si dica, per ogni famiglia di vincoli, quale frase dell'enunciato traduce e quanti vincoli sono in funzione dei dati.

Domanda di modellazione 10.21 — i legami

Nel modello compaiono almeno tre legami fra variabili del capitolo 3. Quali sono? Per ciascuno si scriva la tavola di verità e si dica se il verso mancante è imposto da un vincolo o dall'ottimalità.

Domanda di modellazione 10.22 — i bound

Si costruisca a mano una soluzione ammissibile e una soluzione duale del rilassamento LP. Si dica quale dei due valori è LB e quale UB , e perché.

Il secondo: turni e competenze

Problema 10.11 — il presidio tecnico

Un servizio di assistenza copre sette giorni. Ogni giorno serve un numero noto di tecnici presenti. I tecnici si assumono per contratti settimanali di forma fissa: un contratto stabilisce quali giorni si lavora, e ha un costo settimanale noto. Ogni tecnico ha una fra tre competenze, e ogni giorno servono almeno un tecnico per ciascuna delle tre. Due contratti sono incompatibili con la competenza più rara. Si vuole il piano di costo minimo.

Domanda di modellazione 10.23 — le decisioni e le variabili

Le variabili contano tecnici, non scelte: perché? Che differenza fa, nel modello e nel rilassamento, dichiararle intere invece che binarie?

Domanda di modellazione 10.24 — il modello

Si scriva un modello MILP completo, con gli indici sempre accompagnati dall'insieme in cui variano.

Domanda di modellazione 10.25 — un bound che si legge dai dati

Si costruisca un bound certificato senza scrivere il duale, contando le giornate-uomo coperte da un contratto. Si confronti con il valore del rilassamento LP.

Domanda di modellazione 10.26 — una disuguaglianza rafforzativa

Si trovi una disuguaglianza valida che il modello non contiene e si misuri quanto gap chiude: si risolvano i due rilassamenti, con e senza, e si riporti la differenza.

Parte III

Il corso

Organizzazione del corso

11.1 Percorso in tre parti

	Contenuto	Obiettivi di apprendimento
Parte I	Modellazione	riconoscere un legame fra variabili (attivazione, massimo, big-M, se e solo se...) e dimostrare che il modello lo impone davvero
Parte II	I problemi	applicare i legami a tre famiglie di problemi reali e ai modelli misti, dal modello al codice Gurobi
Parte III	Il corso	mettere alla prova quanto imparato con le domande di modellazione aggiuntive

11.2 Il formato di ogni esercizio (e dell'esame)

Ogni problema del corso — e ogni domanda d'esame — segue lo stesso schema in quattro quesiti:

1. **Modello.** Scrivere il MILP: variabili (con il conteggio), obiettivo, vincoli con la loro descrizione, e — se ci sono due famiglie di variabili collegate — spiegare il legame: qual è l'implicazione, e come il vincolo (o l'ottimo) la impone, nei due versi.
2. **Istanza.** Scrivere il modello per un'istanza numerica piccola.
3. **Euristica.** Progettare un algoritmo costruttivo che trovi una soluzione ammissibile dell'istanza, ed eseguirlo passo per passo per ottenere un upper bound (lower bound se il problema è di massimo).
4. **Duale.** Scrivere il duale del rilassamento LP, per il modello generale e per l'istanza, e costruire a mano una soluzione duale ammissibile per ottenere il bound dal lato opposto.

Il $LB \leq z(MILP) \leq UB$ che ne risulta è il filo conduttore del corso: un modello non si limita a scriverlo, lo si stringe da entrambi i lati prima di affidarlo al solver.

11.3 Criteri di valutazione

Dimensione	Peso
Correttezza del modello (variabili, obiettivo, vincoli)	35%
Dimostrazione del legame fra le variabili (nei due versi)	25%
Euristica costruttiva ed esecuzione corretta	20%
Duale del rilassamento e soluzione duale ammissibile	20%

11.4 Domande tipiche di discussione

- Il vincolo di link è aggregato o disaggregato? Che differenza fa sul rilassamento LP?
- Il verso opposto dell'implicazione è imposto dal vincolo o segue dall'ottimo? Come lo si dimostra?
- Qual è il big-M più piccolo che si può giustificare dai dati?
- L'euristica trova l'ottimo? Come si fa a saperlo senza il solver?
- Il duale a mano è ottimo per il rilassamento, o solo ammissibile?

11.5 Gli errori più comuni

1. Scrivere un'implicazione la cui tesi è vera comunque (controinversa vacua): prima di scriverla, verificare che antecedente e conseguente siano entrambi genuini.
2. Dimostrare un solo verso di un'implicazione «imposta dal vincolo» — servono sempre entrambi gli «if».
3. Confondere una relazione «imposta dal vincolo» con una che «segue dall'ottimo»: la seconda richiede l'argomento di scambio in sei passi, non basta dire «si vede che conviene».
4. Concludere «in ogni soluzione ottima» quando il coefficiente è solo ≥ 0 (non > 0): la conclusione corretta è più debole, «esiste un ottimo in cui...».
5. Usare un big-M enorme «per sicurezza»: peggiora il rilassamento LP senza bisogno.
6. Dimenticare che un vincolo di mutua esclusione ($A + B \leq 1$) non implica che almeno uno dei due valga 1.
7. Scrivere il duale del rilassamento sbagliando il verso o il segno di una variabile duale rispetto al verso del vincolo primale.
8. Costruire una soluzione duale ammissibile ma non verificarne l'ammissibilità su tutti i vincoli.

11.6 Riproducibilità

```
python3 -m pip install gurobipy matplotlib pandas
python3 python/esegui_tutti.py      # rigenera dati, risultati, figure e
    notebook
python3 python/verifica_numeri.py    # verifica ogni numero citato nella
    dispensa
```